

SMT FPGA Manual

Version 1.3

이 문서는 사전에 공지나 예고 없이 변경될 수 있습니다.

REVISION HISTORY

Version	Detail	Name	Date
1.0	First Version	SHMT SoC Team	2007-03-14
1.1	ADD NAND Flash Controller	SHMT SoC Team	2007-03-20
1.2	ADD AHB Master and Slave for UWB MAC and Modem	SHMT SoC Team	2007-05-04
1.3	ADD the PCI bridge	SHMT SoC Team	2007-05-17

Technical documents

Copyright © 2007 Shmt Limited. All rights reserved.

Confidentiality Status

This document is **NOT OPEN ACCESS**. The distribution is restricted to certified company.

This document is **CONFIDENTIAL**.

Feedback on this document

If you have any comments on this document, please contact your channel to our company.

Shmt Web address

<http://www.shmt.com>

Table of contents

1. Introduction	16
1.1. Overview	16
1.2. Features	17
2. ARM926EJ-S	19
2.1. Overview	19
2.1.1. Feature	19
2.1.2. Block diagram	20
2.1.3. Processor operating state	21
2.2. Switching operating state	21
2.3. Registers	21
2.3.1. ARM state 에서의 register	22
2.3.2. Thumb state 에서의 register	23
2.3.3. Status register	24
2.4. Exceptions	25
2.4.1. Exception vectors	25
2.4.2. Exception priorities	26
2.4.3. Entering an ARM exception	26
2.4.4. Leaving an ARM exception	26
2.4.5. Reset	26
2.4.6. Fast interrupt request	26
2.4.7. Interrupt request	27
2.4.8. Aborts	27
3. Programmer's model	29
3.1. Overview	29
3.2. Memory Map	29
3.2.1. Memory Map	29
3.3. Peripherals	30
3.3.1. Peripheral Memory Map	30
3.4. Boot sequence	30
3.4.1. Boot mode configuration	30
3.4.2. Direct(CS0) boot mode	31
3.4.3. NAND boot mode	31
4. Power mamangement unit	32
4.1. Overview	32
4.1.1. Feature	32
4.1.2. Block Diagram	33
4.2. Operation	33
4.2.1. Clock Control Block	33
4.2.2. Power Manage State Machine	34
4.2.3. Reset Control Block	36
4.3. Register Description	37
4.3.1. Power Management Unit Registers	37
4.3.2. Power Management Control Register	37
4.3.3. Clock Divide Register	38
4.3.4. PLL Lock Wait Register	38
4.3.5. PLL Set Register	38
5. System control	40
5.1. Overview	40
5.2. Registers	41
5.2.1. System Configuration Register	41
5.2.2. DMA Resource Share Register	41
5.2.3. Information Register	42

5.2.4. Pin Mux Register.....	42
6. Static Memory Controller	46
6.1. Overview	46
6.1.1. Feature	46
6.1.2. Block Diagram.....	46
6.2. Signal Description	47
6.3. Address Map	47
6.4. Timing	47
6.5. Interface	48
6.5.1. Interface Example	48
6.6. Register Description	50
6.6.1. Registers.....	50
6.6.2. SMC_BANKx Register	51
7. DDR-SDRAM Scheduler	52
7.1. Overview	52
7.1.1. Feature	52
7.1.2. Block Diagram.....	53
7.2. Operation	53
7.2.1. AXI BUS Interface Block	53
7.2.2. DDR SDRAM Controller Block.....	55
7.2.3. DDR SDRAM Controller Timing Description	57
7.3. Register Description	59
7.3.1. DDR SDRAM Registers.....	59
7.3.2. DDR SDRAM TIMING Control Register.....	60
7.3.3. DDR SDRAM Control Register	60
7.3.4. DDR SDRAM Power Control Register	61
7.3.5. DDR SDRAM Refresh Control Register.....	61
7.3.6. DDR SDRAM Configuration Register.....	62
8. Vectored interrupt controller	63
8.1. Overview	63
8.1.1. Features.....	63
8.1.2. Block diagram	63
8.2. Interrupt Connection	65
8.3. Operation	65
8.3.1. Register 와 Interrupt Source	65
8.3.2. Level/Edge detect logic.....	66
8.3.3. Masking & Demux logic	66
8.3.4. Arbiter	66
8.3.5. I_ISPR/F_ISPR and I_VECADDR/F_VECADDR.....	67
8.4. Programming Model	67
8.4.1. Enabling VIC.....	67
8.4.2. Disabling VIC	68
8.4.3. Enabling Each Interrupt Source(Unmasking Interrupt)	68
8.4.4. Disabling Each Interrupt Source(Masking Interrupt).....	68
8.4.5. Acknowledge Interrupt	68
8.4.6. I_ISPR/F_ISPR, I_VECADDR/F_VECADDR 레지스터 접근	69
8.4.7. Interrupt Handler Example.....	69
8.5. Register Description	70
8.5.1. Registers.....	70
8.5.2. INTCON Register	71
8.5.3. INTPND Register.....	71
8.5.4. INTMOD Register	72
8.5.5. INTMSK Register	73
8.5.6. LEVEL Register	73
8.5.7. POLARITY Register.....	74

8.5.8. PSLV Register	74
8.5.9. PMST Register.....	75
8.5.10. CSLV Register.....	76
8.5.11. CMST Register	77
8.5.12. ISPR Register	77
8.5.13. ISPC Register	78
8.5.14. VECADDR Register	78
9. Enhanced-DMA.....	80
9.1. Overview	80
9.1.1. Features.....	80
9.1.2. Block diagram	80
9.2. DMA Channel	82
9.2.1. Channel Arbitration	82
9.3. Channel Operation	83
9.3.1. Basic Operation	83
9.3.2. Descriptor Loading.....	84
9.3.3. Interrupt	84
9.3.4. Trailing Bytes.....	85
9.3.5. Memory-to-Memory Transfer	86
9.4. Software Model.....	86
9.4.1. Enabling DMA Channel	86
9.4.2. Disabling DMA Channel.....	87
9.4.3. Restriction on Register Access	87
9.5. Register Description	87
9.5.1. Registers.....	87
9.5.2. Source Address Register	88
9.5.3. Destination Address Register	89
9.5.4. Control Register	89
9.5.5. Descriptor Register	91
9.5.6. Status Register	91
10. TIMER.....	93
10.1. Overview	93
10.1.1. Features	93
10.1.2. Block diagram.....	93
10.2. Operation	94
10.2.1. Interval Mode	94
10.2.2. Match&Overflow Mode.....	94
10.3. Register Description	94
10.3.1. Registers.....	94
10.3.2. Timer Data Register (T0_TDAT/T1_TDAT/T2_TDAT/T3_TDAT).....	95
10.3.3. Timer Prescaler Register (T0_PRES/T1_PRES/T2_PRES/T3_PRES)	95
10.3.4. Timer Control Register (T0_CTRL/T1_CTRL/T2_CTRL/T3_CTRL)	96
10.3.5. Timer Counter Register (T0_CNT/T1_CNT/T2_CNT/T3_CNT).....	97
11. Watch Dog Timer	98
11.1. Overview	98
11.1.1. Feature.....	98
11.1.2. Block Diagram	99
11.2. Operation	99
11.2.1. Clock Prescaler Block	100
11.2.2. Interrupt Generation Block	100
11.2.3. Reset Output Generation	100
11.3. Register Description	101
11.3.1. WDT Registers	101
11.3.2. WDT Control Register.....	101
11.3.3. WDT Prescale Register	102

11.3.4.	WDT Load Register	102
11.3.5.	WDT Count Register	102
11.3.6.	WDT Interrupt Status Register	103
12.	I2S.....	104
12.1.	Overview	104
12.1.1.	Features	104
12.1.2.	Block diagram.....	104
12.2.	Signals.....	105
12.3.	Operation	105
12.3.1.	Digital Oscillator.....	105
12.3.2.	Master/Slave Configuration.....	107
12.3.3.	Transmit 동작 : Audio Play	108
12.3.4.	Receive 동작 : Audio Record	110
12.3.5.	Interrupt.....	113
12.3.6.	Left-justified Mode/I2S Mode Selection	113
12.3.7.	WordLength 와 FIFO data.....	114
12.4.	Register Description	115
12.4.1.	Registers.....	115
12.4.2.	I2S_CLKCTRL Register	115
12.4.3.	I2S_CTRL Register	116
12.4.4.	I2S_STATUS Register.....	117
12.4.5.	I2S_DATA Register	119
13.	I2C Master	121
13.1.	Overview	121
13.1.1.	Feature.....	121
13.1.2.	Block Diagram	121
13.2.	Operation	122
13.2.1.	Overview	122
13.3.	Signal Description	122
13.3.1.	I2C Module Signal	122
13.4.	Register Description	122
13.4.1.	Registers.....	122
13.4.2.	I2CCON (I2C Control) Register.....	122
13.4.3.	I2CSTA (I2C Status) Register.....	123
13.4.4.	I2CDAT (I2C Data) Register.....	124
13.5.	Operation Sequence.....	124
14.	UART.....	127
14.1.	Overview	127
14.1.1.	Features	127
14.2.	Register	127
14.2.1.	Master Command Register	128
14.2.2.	Status Register	129
14.2.3.	Baudrate Gen Register	131
14.2.4.	TxFIFO Write Register	131
14.2.5.	RxFIFO Read Register	131
14.2.6.	Receive Time Out Count Register.....	132
14.3.	Operation	132
14.3.1.	Baudrate Clock Generation	132
14.3.2.	Low Pass Filter.....	133
14.3.3.	DMA/Interrupt Water Level	133
14.3.4.	Receive Time Out Interrupt.....	134
14.3.5.	Loopback Mode	135
15.	GPIO	136
15.1.	Overview	136
15.1.1.	Features	136

15.2. Operation	136
15.2.1. GPIO Operation.....	136
15.2.2. Interrupt Operation	137
15.3. Register Description	138
15.3.1. Registers.....	138
15.3.2. GPIO Output Enable Register (GPIO0_OE /GPIO1_OE)	139
15.3.3. GPIO Input Register (GPIO0_IN/GPIO1_IN)	139
15.3.4. GPIO Output Register (GPIO0_OUT/GPIO1_OUT)	140
15.3.5. GPIO Interrupt Status Register (GPIO0_INTSTAT/GPIO1_INTSTAT)	140
15.3.6. GPIO Interrupt Enable Register (GPIO0_INTEN/GPIO1_INTEN)	141
15.3.7. GPIO Interrupt Level/Edge Selection Register (GPIO0_INTLEVEL/GPIO1_INTLEVEL) 142	
15.3.8. GPIO Interrupt Polarity Register (GPIO0_INTPOL/GPIO1_INTPOL)	142
15.3.9. GPIO Interrupt Both Edge Register (GPIO0_INTBEDGE/GPIO1_INTBEDGE)	143
16. SPI.....	144
16.1. Overview	144
16.1.1. Feature	144
16.1.2. Block Diagram	144
16.2. Operation	145
16.2.1. Overview	145
16.2.2. Timing	145
16.3. Signal Description	147
16.3.1. SPI BUS Signal	147
16.4. Register Description	147
16.4.1. Registers.....	147
16.4.2. SPI Control Register	148
16.4.3. SPI Prescaler Register.....	148
16.4.4. SPI Status Register.....	148
16.4.5. SPI Interrupt & DMA control Register	149
16.4.6. SPI Interrupt Status Register	150
16.4.7. SPI Tx Data Register.....	151
16.4.8. SPI RxData Register	151
17. NAND Flash Memory Controller.....	152
17.1. Overview	152
17.1.1. Feature.....	152
17.1.2. Block Diagram	152
17.2. Signal Description	153
17.3. Timing	153
17.4. Interface	154
17.5. Register Description	155
17.5.1. Registers.....	155
17.5.2. Operation Register(NFOER).....	156
17.5.3. Data Register (NFDATA)	158
17.5.4. Configuration Register(NFCONF)	159
17.5.5. Control Register(NFCTRL)	160
17.5.6. Status Register(NFSTAT)	162
17.5.7. Ecc Register	164
17.6. Programming Guide	170
17.6.1. NFOPER 레지스터의 1 st word.....	170
17.6.2. Command 전송	171
17.6.3. DATA read/write	172
17.6.4. DMA 를 이용한 Data read/write	172
18. PCI Bridge.....	175
18.1. Features	175
18.2. PCI Address Space	175

18.3. Supported Data Transfer Types	176
18.3.1. Single Transfer Type	176
18.3.2. Burst Transfer Type	176
18.3.3. Retry Operation	176
18.4. PCI Bridge Configuration	176
18.4.1. Configuration Address	176
18.4.2. Configuration Data	178
18.4.3. PCI Bridge Configuration Registers	178
18.5. PCI Interrupt Number	179
19. MultiICE	180
19.1. Signals	180
19.2. JTAG	181
19.2.1. JTAG signals	181
19.2.2. JTAG State Machine	181
19.3. Using JTAG	182
20. ARM JTAG	185
20.1. Instruction Register	185
20.1.1. EXTEST	185
20.1.2. SAMPLE/PRELOAD	185
20.1.3. SCAN_N	185
20.2. Scan Chains	185
20.2.1. Scan chain 1	185
20.2.2. Scan chain 2	186
20.2.3. Scan chain 6	187
20.2.4. Scan chain 15	187
21. EmbeddedICE-RT	188
21.1. Overview	188
21.2. Registers	189
21.2.1. Debug control register	189
21.2.2. Debug status Register	190

Table of Figure

Figure 1-1 Block diagram	16
Figure 2-1 ARM926EJ-S Block Diagram	20
Figure 2-2 ARM state에서의 register	22
Figure 2-3 Thumb state에서의 register.....	23
Figure 2-4 Status register	24
Figure 4-1 Clock Distribution Block Diagram	33
Figure 4-2 Clock Control Block Diagram	34
Figure 4-3 Power Manage State Machine Block Diagram	35
Figure 4-4 Power On Configuration & PLL Lock Time Diagram	36
Figure 4-5 Power Management Control Register Bits	37
Figure 4-6 Clock Divide Register Bits	38
Figure 4-7 PLL Lock Wait Register Bits.....	38
Figure 4-8 PLL Set Register Bits	39
Figure 6-1 External IO Timing	48
Figure 6-2 Interface 8bit memory	49
Figure 6-3 Interface 8bit memory x2	49
Figure 6-4 Interface 8bit memory x4	49
Figure 6-5 Interface 16bit memory	49
Figure 6-6 Interface (byte enable이 있는)16bit memory.....	50
Figure 6-7 Interface (byte enable이 있는)16bit memoryx2	50
Figure 6-8 SMC_BANKx Register	51
Figure 7-1 AXI Interface DDR SDRAM Controller Top Block Diagram.....	53
Figure 7-2 DDR SDRAM AXI BUS Interface Block Diagram	54
Figure 7-3 DDR SDRAM Controller Block Diagram	56
Figure 7-4 DDR SDRAM Timing Control Register Bits.....	60
Figure 7-5 DDR SDRAM Control Register Bits	60
Figure 7-6 DDR SDRAM Power Control Register Bits	61
Figure 7-7 DDR SDRAM Refresh Control Register Bits	61
Figure 7-8 DDR SDRAM Configuration Register Bits	62
Figure 8-1 VIC Block Diagram	64
Figure 8-2 Arbiter Block Diagram	64
Figure 8-3 INTCON Register	71
Figure 8-4 INTPND Register	72
Figure 8-5 INTMOD Register	72
Figure 8-6 INTMSK Register	73
Figure 8-7 LEVEL Register	73
Figure 8-8 POLARITY Register.....	74
Figure 8-9 PSLV Register.....	74
Figure 8-10 PMST Register	75
Figure 8-11 CSLV Register.....	76
Figure 8-12 CMST Register	77
Figure 8-13 ISPR Register.....	78
Figure 8-14 ISPC Register	78
Figure 8-15 VECADDR Register.....	79
Figure 9-1 DMA Controller Block Diagram	81
Figure 9-2 4 Channel DMA Controller의 내부구조.....	82
Figure 9-3 DMA Operation State Machine	83
Figure 9-4 Source Address Register	88
Figure 9-5 Destination Address Register	89
Figure 9-6 Control Register	89
Figure 9-7 Descriptor Register	91
Figure 9-8 Status Register.....	91

Figure 10-1 Block Diagram	93
Figure 10-2 Timer Data Register	95
Figure 10-3 Timer Prescaler Register	96
Figure 10-4 Timer Control Register	96
Figure 10-5 Timer Counter Register	97
Figure 11-1 Watch Dog Block Diagram	99
Figure 11-2 WDT Control Register Bits	101
Figure 11-3 WDT Prescale Register Bits	102
Figure 11-4 WDT Load Register Bits	102
Figure 11-5 WDT Count Register Bits	102
Figure 11-6 WDT Interrupt Status Register Bits	103
Figure 12-1 I2S Controller Block Diagram	105
Figure 12-2 Left(MSB)-justified Mode and I2S compatible Mode	113
Figure 12-3 8 bit/sample data format	114
Figure 12-4 16 bit/sample data format	114
Figure 12-5 24 bit/sample data format	114
Figure 12-6 32 bit/sample data format	115
Figure 12-7 I2S_CLKCTRL Register	115
Figure 12-8 I2S_CTRL Register	116
Figure 12-9 I2S_STATUS Register	118
Figure 12-10 I2S_DATA Register	119
Figure 13-1 I2C Block Diagram	121
Figure 13-2 I2CCON Register Bits	123
Figure 13-3 I2CSTA Register Bits	123
Figure 13-4 I2CDAT Register Bits	124
Figure 14-1 Block Diagram	127
Figure 14-2 Master Command Register	128
Figure 14-3 Status Register	129
Figure 14-4 Baudrate Gen Register	131
Figure 14-5 Tx FIFO Write Register	131
Figure 14-6 Rx FIFO Read Register	132
Figure 14-7 Receive Time Out Count Register	132
Figure 14-8 Low Pass Filter at Rx D	133
Figure 14-9 FIFO Water Level	134
Figure 14-10 DMA / Interrupt Generation Logic	134
Figure 15-1 GPIO Operation Block Diagram	136
Figure 15-2 Interrupt Operation Block Diagram	137
Figure 15-3 GPIO Output Enable Register	139
Figure 15-4 GPIO Input Register	139
Figure 15-5 GPIO Output Register	140
Figure 15-6 GPIO Interrupt Status Register	140
Figure 15-7 GPIO Interrupt Enable Register	141
Figure 15-8 GPIO Interrupt Level/Edge Selection Register	142
Figure 15-9 GPIO Interrupt Polarity Register	142
Figure 16-1 SPI Controller Block Diagram	144
Figure 16-2 SPI Timing (CPOL=0, CPHA=0)	145
Figure 16-3 SPI Timing (CPOL=0, CPHA=1)	145
Figure 16-4 SPI Timing (CPOL=1, CPHA=0)	145
Figure 16-5 SPI Timing (CPOL=1, CPHA=1)	146
Figure 16-6 SPI Control Register Bits	148
Figure 16-7 SPI Prescaler Register Bits	148
Figure 16-8 SPI Status Register Bits	149
Figure 16-9 SPI Interrupt & DMA Register Bits	149
Figure 16-10 SPI Interrupt Status Register Bits	150
Figure 16-11 SPI Tx Data Register Bits	151

Figure 16-12 SPI Rx Data Register Bits	151
Figure 17-1 NAND Flash Controller Block Diagram	152
Figure 17-2 Address to data load time / WE high to busy Timing.....	153
Figure 17-3 ALE,CLE Timing	154
Figure 17-4 RE, WE Timing	154
Figure 17-5 Interface example	155
Figure 17-6 1 st Operation Register bit	156
Figure 17-7 2 nd Operation Register bit	158
Figure 17-8 3 rd Operation Register bit.....	158
Figure 17-9 Data Register bit(Word Access)	159
Figure 17-10 Data Register bit(Half Word Access)	159
Figure 17-11 Data Register bit(Byte Access).....	159
Figure 17-12 Configuration Register bit	159
Figure 17-13 Control Register bit	160
Figure 17-14 Status Register bit	162
Figure 17-15 Interface 3 ECC Sector	165
Figure 17-16 ECCSECTOR0~ECCSECTOR15	165
Figure 17-17 Spare area.....	166
Figure 17-18 SECCSECTOR0~SECCSECTOR7	167
Figure 17-19 MECC Error Register bit	167
Figure 17-20 SECCERR Register bit	169
Figure 18-1 Block diagram of PCI Bridge.....	175
Figure 18-2 Format of the Configuration Address Register.....	177
Figure 18-3 Type 0 translation	177
Figure 18-4 Type 1 translation	178
Figure 18-5 PCI Bridge Configuration Register.....	179
Figure 19-1 ARM Multi-ICE Connector	180
Figure 19-2 JTAG signal standard wave	181
Figure 19-3 JTAG State Machine.....	181
Figure 19-4 JTAG interface 회로	182
Figure 19-5 ARM JTAG Waveform	183
Figure 20-1 ARM926 Scan chains 1 and 2.....	186
Figure 21-1 EmbeddedICE-RT macrocell overview	188
Figure 21-2 Debug control register format.....	189
Figure 21-3 Debus status register format.....	190
Figure 21-4 Debug control and debug status register structure	191

Tables

Table 2-1 Status register의 bit 구성	24
Table 2-2 SPR mode bit별 mode설정	25
Table 2-3 Vector base address	25
Table 2-4 Exception vector addresses	25
Table 3-1 Memory Map at CS0 Boot.....	29
Table 3-2 Memory Map at NAND Boot.....	29
Table 3-3 Peripheral Memory MAP	30
Table 3-4 Boot Mode Configuration	30
Table 4-1 Power Management Unit Registers	37
Table 4-2 Power Management Control Register Description(Offset : 0000 0000h)	37
Table 4-3 Clock Divide Register Description(Offset : 0000 0004h)	38
Table 5-1 System Configuration Register Description (Offset : 0000 0000h)	41
Table 5-2 DMA Resource Share Register Description (Offset : 0000 0004h).....	41
Table 5-3 Information Register Description(Offset : 0000 0008h)	42
Table 5-4 Function Pin-Mux Register Description(Offset : 0000 0010h).....	42
Table 5-5 GPIO0 Pin-Mux Register Description(Offset : 0000 0014h)	43
Table 5-6 GPIO1 Pin-Mux Register Description(Offset : 0000 0018h)	44
Table 6-1 ESMC Signal.....	47
Table 6-2 Address Map	47
Table 6-3 Registers	50
Table 6-4 SMC_BANKx Register (Offset : 0000 0000h~0000 000ch).....	51
Table 7-1 DDR SDRAM Registers	59
Table 7-2 DDR SDRAM Timing Control Register Description(Offset : 0000 0000h)	60
Table 7-3 DDR SDRAM Control Register Description(Offset : 0000 0004h).....	60
Table 7-4 DDR SDRAM Power Control Register Description(Offset : 0000 0008h).....	61
Table 7-5 DDR SDRAM Refresh Control Register Description(Offset : 0000 000Ch)	61
Table 7-6 DDR SDRAM Configuration Register Description(Offset : 0000 0010h)	62
Table 8-1 Interrupt Connection	65
Table 8-2 Registers	70
Table 8-3 INTCON Register Description(Offset : 0000h)	71
Table 8-4 INTPND Register Description(Offset : 0004h)	72
Table 8-5 INTMOD Register Description(Offset : 0008h).....	72
Table 8-6 INTMSK Register Description(Offset : 000Ch)	73
Table 8-7 LEVEL Register Description(Offset : 0010h)	73
Table 8-8 POLARITY Register Description(Offset : 0074h).....	74
Table 8-9 PSLV Register Description	74
Table 8-10 PMST Register Description.....	75
Table 8-11 CSLV Register Description	76
Table 8-12 CMST Register Description.....	77
Table 8-13 ISPR Register Description	78
Table 8-14 ISPC Register Description.....	78
Table 8-15 VECADDR Register Description	79
Table 9-1 Descriptor.....	84
Table 9-2 Registers	87
Table 9-3 Source Address Register Description	89
Table 9-4 Control Register Description.....	89
Table 9-5 Descriptor Register Description	91
Table 9-6 Status Register Description	91
Table 10-1 Registers	95
Table 10-2 Timer Data Register Description	95
Table 10-3 Timer Prescaler Register Description.....	96

Table 10-4 Timer Control Register Description	96
Table 10-5 Timer Counter Register Description	97
Table 11-1 WDT Registers	101
Table 11-2 WDT Control Register Description(Offset : 0000 0000h)	101
Table 11-3 WDT Prescale Register Description(Offset : 0000 0004h)	102
Table 11-4 WDT Load Register Description(Offset : 0000 0008h)	102
Table 11-5 WDT Count Register Description(Offset : 0000 000Ch)	102
Table 11-6 WDT Interrupt Status Register Description(Offset : 0000 0010h)	103
Table 12-1 External Signals	105
Table 12-2 Registers	115
Table 12-3 I2S_CLKCTRL Register Description(Offset : 0000h)	115
Table 12-4 I2S_CTRL Register Description(Offset : 0004h)	116
Table 12-5 I2S_STATUS Register Description(Offset : 0008h)	118
Table 12-6 I2S_DATA Register Description(Offset : 000Ch)	119
Table 13-1 I2C Module Signal	122
Table 13-2 Registers	122
Table 13-3 I2CCON Register Description (Offset : 0000 0000h)	123
Table 13-4 I2CSTA Register Description (Offset : 0000 0004h)	123
Table 13-5 I2CDAT Register Description (Offset : 0000 0008h)	124
Table 14-1 UART Register	127
Table 14-2 Master Command Register Description	128
Table 14-3 Status Register Description	129
Table 14-4 Baudrate Gen Register Description	131
Table 14-5 TxFIFO Write Register Description	131
Table 14-6 RxFIFO Read Register Description	132
Table 14-7 Receive Time Out Count Register	132
Table 15-1 Registers	138
Table 15-2 GPIO Output Enable Register Description	139
Table 15-3 GPIO Input Register Description	140
Table 15-4 GPIO Output Register Description	140
Table 15-5 GPIO Interrupt Status Register Description	141
Table 15-6 GPIO Interrupt Enable Register Description	141
Table 15-7 GPIO Interrupt Level/Edge Selection Register	142
Table 15-8 GPIO Interrupt Polarity Register Description	142
Table 15-9 GPIO Interrupt Both Edge Register	143
Table 15-10 GPIO Interrupt Both Edge Register Description	143
Table 16-1 SPI BUS Signal	147
Table 16-2 Registers	147
Table 16-3 SPI Control Register Description (Offset : 0000 0000h)	148
Table 16-4 SPI Precaler Register Description (Offset : 0000 0004h)	148
Table 16-5 SPI Status Register Description(Offset : 0000 0008h)	149
Table 16-6 SPI Interrupt & DMA Register Description(Offset : 0000 0008h)	149
Table 16-7 SPI Interrupt Status Register Description(Offset : 0000 0008h)	150
Table 16-8 SPI Tx Data Register Description(Offset : 0000 0008h)	151
Table 16-9 SPI Rx Data Register Description(Offset : 0000 0008h)	151
Table 17-1 NAND Flash Signal	153
Table 17-2 Registers	155
Table 17-3 1 st Operation Register Description(Offset : 0000 0000h)	156
Table 17-4 2 nd Operation Register Description(Offset : 0000 0000h)	158
Table 17-5 3 rd Operation Register Description(Offset : 0000 0000h)	158
Table 17-6 Configuration Register Description(Offset : 0000 0008h)	159
Table 17-7 Control Register Description(Offset : 0000 000Ch)	160
Table 17-8 Status Register Description(Offset : 0000 0010h)	162
Table 17-9 ECCSECTOR0~ECCSECTOR15 (0000 0014h~0000 0050h)	164
Table 17-10 SECCSECTOR0~SECCSECTOR7 (0000 0054h~0000 00070h)	165

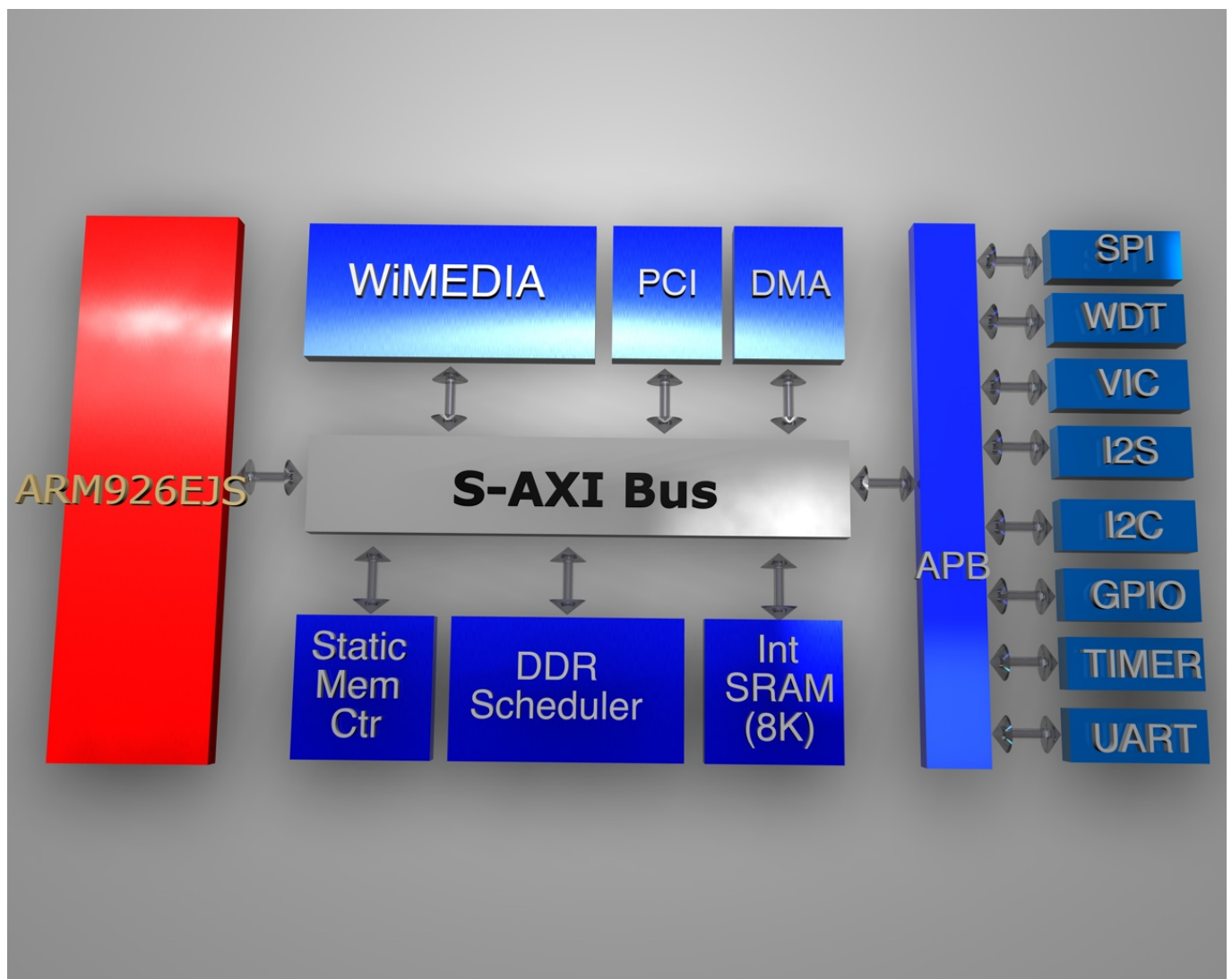
Table 17-11 MECC Error Register Description(Offset : 0000 0074h).....	167
Table 17-12 SECCERR Register Description(Offset : 0000 0078h).....	169
Table 18-1 PCI Address Space.....	175
Table 18-2 Burst Transfer Length.....	176
Table 18-3 Interrupt mapping	179
Table 19-1 Signals of the ARM Multi-ICE	180
Table 20-1 JTAG Instructions	185
Table 20-2 Scan chains	185
Table 20-3 Bit description of the ARM926 Scan chain 1.....	186
Table 20-4 Bit description of the ARM926 Scan chain 2.....	186
Table 20-5 Bit description of the ARM926 Scan chain 6.....	187
Table 21-1 EmbeddedICE-RT Registers	189
Table 21-2 Debug control register bit functions	189
Table 21-3 Bit description of the Debus status register.....	190
Table 21-4 Means of the MOE	190

1. Introduction

1.1. Overview

ARM926 을 내장하고 있으므로 다양한 어플리케이션에 효율적으로 대응을 할 수 있도록 구성되어 있다. 또한 JTAG 인터페이스를 사용하여 디버깅을 용이하도록 구성되어 있다.

Figure 1-1 Block diagram



1.2.Features

- FPGA 에서의 동작 주파수
 - System : 50MHz
 - DDR : 100MHz
 - slow peripheral : 25Mhz
 - fast peripheral : 50MhzStatic memory controller, DDR Controller, DMA
- ARM 926 내장
- Internal SRAM
 - 128 kByte
- DDR Controller
 - 128/256/512 Mbit
- I2C
 - Variable clock rate : 400kHz max.
- I2S
 - Configurable Master/Slave.
 - ADC(receive) & DAC(transmit) IF for audio Codec.
 - IF mode : Standard I₂S, MSB(=Left) justified mode(configurable).
 - 8/16/24/32bit Word lengths.
 - 16k/32k/44.1k/48kHz Sampling Frequency.
 - Each 16 word FIFO entries for Transmit & Receive.
- Timer
 - 4 channel
- Enhanced-DMA
 - 8 channel
- SPI
 - SPI clock speed up to 12MHz.
 - For transferring MP3 bit-stream.
 - 160Byte/10mS @128k Sampling.
 - Support DMA.
- UART
 - Special UART (2ch)
 - Baudrate : 115200 max
 - Channel 0 : Tx 32bytes & Rx 32bytes FIFOs.
 - Channel 1 : Tx 32bytes & Rx 8bytes FIFOs.
- WDT
- Vectored Interrupt Controller
 - 32 interrupt source
- NAND Flash memory controller
 - 32M ~ 2G Byte.
 - 512/256 byte, 1-bit ECC.
 - Command FIFO/Data FIFO.

- Support small/large block.

2. ARM926EJ-S

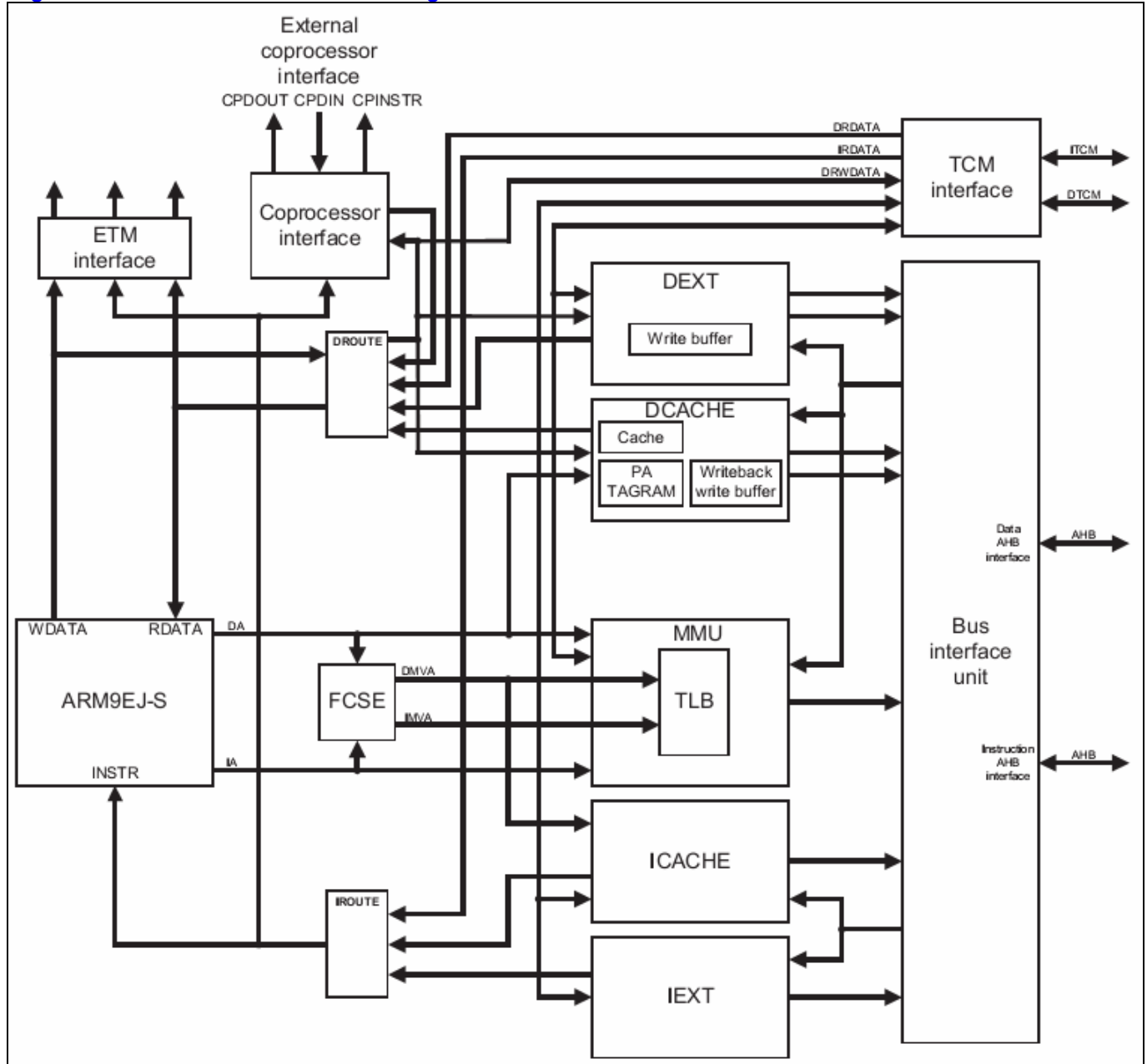
2.1. Overview

2.1.1. Feature

- Base Core : ARM9EJ-S
 - ARM architecture 5TEJ
- Little/Big endian
- Seven operation mode
 - User mode
 - Fast interrupt mode
 - Interrupt mode
 - Supervisor mode
 - Abort mode
 - System mode
 - Undefined mode
- 37 registers
 - 31 general purpose 32bit registers
 - 6 32bit status register
- Cache Size
 - I-Cache : 4way, 8Kbyte
 - D-Cache : 4way, 8Kbyte
- MMU
- AMBA AHB interface
- Three Processor operating state
 - ARM state
 - Thumb state
 - Jazelle state
- Supported Instructions
 - 32bit ARM instruction
 - 16bit Thumb instruction
 - Java byte codes
- System control coprocessor(CP15)
 - With ID code register
 - With Cache and MMU control register

2.1.2. Block diagram

Figure 2-1 ARM926EJ-S Block Diagram



ARM9EJ-S processor core 와 I/D cache, MMU, memory interface, coprocessor interface 로 구성된다.

Debug 를 위해서 ETM(Embedded Trace Macrocell) interface 를 갖고 있다.

2.1.3. Processor operating state

ARM9EJ-S 는 다음과 같이 세 가지의 operating state 를 지원한다.

- ARM state : 32bit, word aligned instruction을 수행
- Thumb state : 16bit, half word aligned instruction을 수행
- Jazelle state : variable length, byte aligned instruction을 수행

2.2. Switching operating state

- ARM ↔ Thumb : BX, BLX 명령을 이용.
- ARM ↔ Jazelle : BXJ 명령을 이용

Exception 이 발생하면 ARM state 에서 service routine 을 수행하고 원래 state 로 전환한다.

2.3. Registers

ARM926EJ-S 는 총 37 개의 32bit register 를 내장하고 있다.

Register 들은 다음과 같이 두 가지로 구분할 수 있다.

- 31개의 general purpose register
- 6개의 status register

Register 들은 processor operating state 에 따라 access 할 수 있는 register 가 다르게 구성된다.

2.3.1. ARM state에서의 register

ARM state에서는 모든 register를 각 mode에 따라 사용할 수 있다.

Figure 2-2 ARM state에서의 register

System and User	FIQ	Supervisor	Abort	IRQ	Undefined
r0	r0	r0	r0	r0	r0
r1	r1	r1	r1	r1	r1
r2	r2	r2	r2	r2	r2
r3	r3	r3	r3	r3	r3
r4	r4	r4	r4	r4	r4
r5	r5	r5	r5	r5	r5
r6	r6	r6	r6	r6	r6
r7	r7	r7	r7	r7	r7
r8	r8_fiq	r8	r8	r8	r8
r9	r9_fiq	r9	r9	r9	r9
r10	r10_fiq	r10	r10	r10	r10
r11	r11_fiq	r11	r11	r11	r11
r12	r12_fiq	r12	r12	r12	r12
r13	r13_fiq	r13_svc	r13_abt	r13_irq	r13_und
r14	r14_fiq	r14_svc	r14_abt	r14_irq	r14_und
r15 (PC)	r15 (PC)	r15 (PC)	r15 (PC)	r15 (PC)	r15 (PC)

ARM state program status registers					
CPSR	CPSR	CPSR	CPSR	CPSR	CPSR
	SPSR_fiq	SPSR_svc	SPSR_abt	SPSR_irq	SPSR_und

 Indicates that the normal register used by the User or System mode has been replaced by an alternative register specific to the exception mode (banked register).

2.3.1.1. General registers

ARM state에서는 general register 중 2 개를 특수한 용도로 사용한다.

R15는 program counter로 사용되고, R14는 link register로 사용된다.

R13은 일반적으로 stack pointer로 사용된다.

R13, R14, R15를 제외한 나머지 register는 용도가 규정되어 있지 않다.

Stack pointer와 link register는 mode에 따라 별개의 register를 사용할 수 있도록 여분의 register가 존재한다.

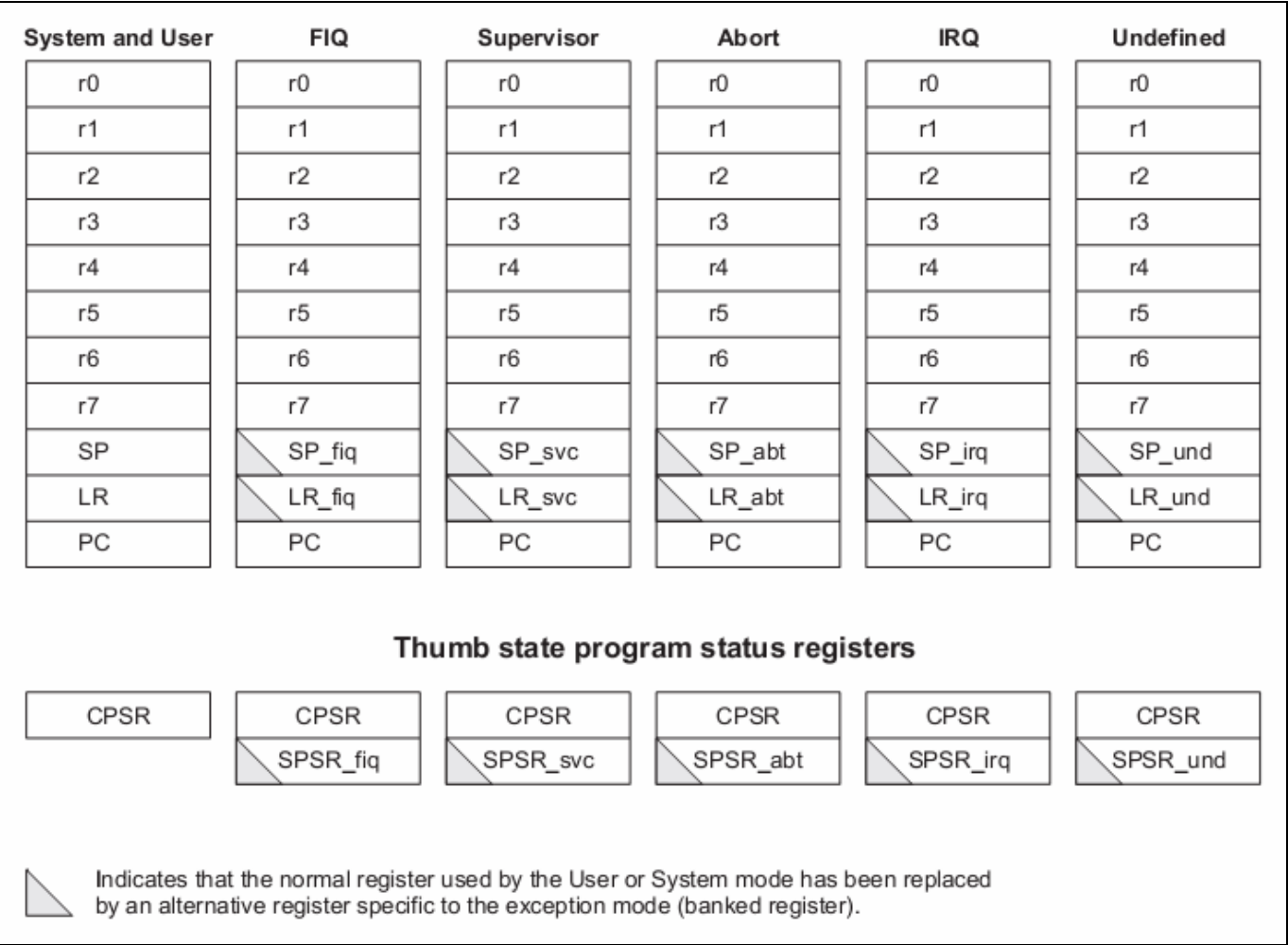
FIQ mode에서는 좀더 빠른 interrupt 처리를 위해서 별도의 R8 ~ R12를 사용하도록 한다.

2.3.1.2. Status register

ARM9EJ-S는 mode 전환시 사용하던 status register의 내용을 backup할 수 있도록 mode 별로 별도의 SPSR을 가지고 있다.

2.3.2. Thumb state에서의 register

Figure 2-3 Thumb state에서의 register



2.3.2.1. General registers

R0 ~ R7까지 총 8개의 general purpose register를 사용할 수 있다.
이 register들은 특별히 용도를 한정하지 않는다.
이 register는 ARM state에서 R0 ~ R7에 대응된다.

2.3.2.2. Special registers

- 총 4가지의 special purpose register가 있다.
- Program counter : ARM state의 R15에 대응.
 - Link register : ARM state의 R14에 대응.
 - Stack pointer : ARM state의 R13에 대응.

- Status register : 현재 mode의 status register와 각 mode에서 backup하는 SPSR로 구성.

2.3.3. Status register

Figure 2-4 Status register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
N	Z	C	V	Q	reserved	J	reserved															I	F	T	Mode						

Table 2-1 Status register의 bit 구성

Bits	Name	R / W	Description	Init. value
[31]	N	RW	Negative/Less than	0b
[30]	Z	RW	Zero	0b
[29]	C	RW	Carry/Borrow/Extend	0b
[28]	V	RW	Overflow	0b
[27]	Q	RW	Sticky overflow	0b
[25:26]			reserved	
[24]	J	R	Jazelle state bit	0b
[8:23]			reserved	
[7]	I	RW	IRQ disable	1b
[6]	F	RW	FIQ disable	1b
[5]	T	RW	Thumb state bit	0b
[4:0]	Mode	RW	Mode bits	00000b

2.3.3.1. Condition code flags

N, Z, C, V bits.
가장 최근의 연산 결과를 반영한다.
Thumb state 에서는 branch 명령만 참조한다.

2.3.3.2. Q flag

다음과 같은 certain multiply 나 fractional arithmetic 명령어에서 사용된다.

- QADD
- QDADD
- QSUB
- QDSUB
- SMLAxy
- SMLAWy

2.3.3.3. J flag

Processor operating state 가 Jazelle state 임을 나타냄.

- J = 0 : ARM or Thumb state
- J = 1 : Jazelle state

CPSR 의 J flag 는 MSR 로 변경할 수 없다.

2.3.3.4. Interrupt disable flags

IRQ 와 FIQ 를 en/disable 한다.

- I = 1 : IRQ를 disable한다.
- F = 1 : FIQ를 disable한다.

2.3.3.5. T flag

Processor operating state 를 Thumb state 로 변경한다.

- T = 1 : Processor를 Thumb state로 동작시킨다.
- T = 0 : Processor를 ARM state나 Jazelle state로 동작시킨다.

CPSR 의 T flag 는 MSR 명령으로 변경해서는 안 된다. 만일 MSR 명령어로 변경하게 되면, processor 는 예측할 수 없는 동작을 하게 된다.

2.3.3.6. Mode bits

Processor 의 mode 를 결정한다.

Table 2-2 SPR mode bit별 mode설정

M[4:0]	Mode
10000b	User
10001b	FIQ
10010b	IRQ
10011b	Supervisor
10111b	Abort
11011b	Undefined
11111b	System

2.4.Exceptions

2.4.1. Exception vectors

Exception vector 의 base address 는 CFGHIVECS signal 에 의해서 다음과 같이 결정된다.

Table 2-3 Vector base address

Value of CFGHIVECS	Vector base address
0b	0000 0000h
1b	FFFF 0000h

Table 2-4 Exception vector addresses

Exception	Offset	Mode on entry	I bit on entry	F bit on entry
Reset	00h	Supervisor	Disabled	Disabled
Undefined instruction	04h	Undefined	Disabled	Unchanged
Software interrupt	08h	Supervisor	Disabled	Unchanged
Abort(prefetch)	0Ch	Abort	Disabled	Unchanged
Abort(data)	10h	Abort	Disabled	Unchanged

Reserved	14h	Reserved	-	-
IRQ	18h	IRQ	Disabled	Unchanged
FIQ	1Ch	FIQ	Disabled	Disabled

2.4.2. Exception priorities

동시에 다수의 exception 이 발생할 경우 다음과 같은 우선 순위에 따라 exception 을 처리한다.

1. Reset(highest priority)
2. Data Abort
3. FIQ
4. IRQ
5. Prefetch Abort
6. BKPT, Undefined instruction, and SWI(lowest priority)

2.4.3. Entering an ARM exception

다음과 과정을 거쳐 exception service routine 으로 진입한다.

1. Next instruction의 주소를 LR에 저장한다.
2. CPSR를 해당 mode의 SPSR로 복사.
3. CPSR mode bit을 해당 mode로 설정.
4. PC를 exception vector의 주소로 설정하여 다음 명령어를 fetch.

Thumb state 나 Jazelle state 에서 exception 이 발생하면 자동으로 ARM state 로 전환되어 exception vector address 를 PC 로 load 한다.

2.4.4. Leaving an ARM exception

다음과 과정을 거쳐 exception service 를 종료한다.

1. LR을 PC로 복사한다.
2. S bit이 1이고, rd=r15이면, SPSR을 CPSR에 복사한다. 그리고 interrupt disable flag를 clear한다.

2.4.5. Reset

nRESET 핀이 low 로 driving 될 경우, processor 는 reset 상태에 들어가게 된다.

Reset 상태에서 nRESET 핀이 high 로 driving 되면 다음과 같은 동작을 수행하게 된다.

1. CPSR의 mode bits을 supervisor mode(10011b)로 설정한다. I와 F flag를 set하여 interrupt를 disable한다. T와 J bit을 clear하여 ARM state로 설정한다.
2. PC를 reset vector address로 설정한다.
3. ARM state에서 명령어를 수행한다.

Reset 후 PC 와 CPSR 을 제외한 다른 register 는 undefined value 를 갖는다.

2.4.6. Fast interrupt request

FIQ 를 위해서 8 개의 전용 register 를 내장하여 interrupt 응답시간을 줄인다.

Privileged mode 에서 F flag 를 set 하여 FIQ 를 disable 시킬 수 있다.

FIQ 가 발생하면, IRQ 와 FIQ 모두 disable 된다.

Fast interrupt handler 에서 본래 프로그램으로 되돌아 가려면 다음 명령을 수행하면 된다.

SUBS PC, R14_fiq, #4

2.4.7. Interrupt request

IRQ 는 FIQ 보다 우선순위가 낮다.

Privileged mode 에서 I flag 를 set 하여 IRQ 를 disable 시킬 수 있다.

IRQ 가 발생하면, IRQ 는 disable 된다.

Interrupt handler 에서 본래 프로그램으로 되돌아 가려면 다음 명령을 수행하면 된다.

SUBS PC, R14_irq, #4

2.4.8. Aborts

Abort exception 은 memory operation 은 완료되지 못 할 경우 발생한다.

Abort 는 다음과 같이 두 가지 경우에 발생하게 된다.

- Prefetch Abort
- Data Abort

Abort exception 이 발생하면 IRQ 는 disable 된다.

2.4.8.1. Prefetch Abort

Instruction fetch 의 마지막에 IABORT signal 을 검사하여, prefetched instruction 에 invalid 라고 marking 한 뒤, invalid instruction 이 execution stage 에 도착했을 때 exception 에 진입한다.

만일 branch 등에 의해서 invalid instruction 이 수행되지 않는다면, exception 은 발생하지 않는다.

Prefetch Abort exception 에서 본래 프로그램으로 되돌아 가려면 다음 명령을 수행하면 된다.

SUBS PC, R14_abt, #4

위 명령어는 PC 와 CPSR 을 복원시킨다.

2.4.8.2. Data Abort

Data abort 는 data access cycle 의 마지막에 체크되며, read 와 write 모두에서 발생할 수 있다. ARM9EJ-S 는 base restored Data Abort model 을 사용하는데 이것은 ARM7TDMI-S 의 base updated Data Abort model 과는 다르다.

base restored Data Abort model 은 memory access 명령을 수행하는 중 Data Abort 가 발생했을 때, processor 하드웨어가 base register 를 명령어가 실행되기 전의 값으로 복원시킨다. 따라서 Data Abort exception handler 가 base register 를 이전 값으로 복원시킬 필요가 없어지게 되어 handler 를 간단하게 구성할 수 있게 된다.

Abort exception 은 demand-paged virtual memory system 을 사용할 수 있도록 해준다.

Access 하는 address 의 data 가 유효하지 않다면, MMU 는 Abort 를 발생시킨다. 그러면 Abort exception handler 는 abort 의 원인을 분석하고, data 를 유효하게 한 뒤, processor 는 중단된 명령어를 재 시도한다.

Data Abort exception 에서 본래 프로그램으로 되돌아 가려면 다음 명령을 수행하면 된다.

SUBS PC, R14_abt, #8

위 명령어는 PC 와 CPSR 을 복원시킨다.

2.4.8.3. Software interrupt instruction

SWI(Software interrupt instruction)은 supervisor mode 로 진입하여 supervisor function 의 수행을 요구하는데 사용된다.

SWI handler 는 opcode 를 이용하여 SWI function 번호를 구한다.

SWI 가 발생하면, IRQ 는 disable 된다.

SWI 는 다음 명령을 수행하여 SWI 의 다음 명령어 주소로 되돌아 갈 수 있다.

MOVS PC, R14_svc

위 명령은 PC 와 CPSR 을 복원시킨다.

2.4.8.4. Undefined instruction

ARM 의 명령어를 확장하는데 이 exception 을 사용할 수 있다. 이 exception 이 발생하면 IRQ 는 disable 된다.

다음 명령을 수행하여 emulated instruction 의 다음 명령어로 되돌아 갈 수 있다.

MOVS PC, R14_und

위 명령은 CPSR 도 복원시킨다.

2.4.8.5. Breakpoint instruction(BKPT)

Breakpoint instruction 은 Prefetch Abort 를 발생시킨다. 만일 branch 등에 의해서 breakpoint instruction 이 수행되지 않는다면 Prefetch Abort 는 발생하지 않는다.

Breakpoint 에 대한 처리가 끝나면 다음 명령을 수행하여 break point 의 명령을 수행하게 된다.

SUBS PC, R14_abt, #4

위 명령어는 PC 와 CPSR 을 복원시킨다.

3. Programmer's model

3.1. Overview

3.2. Memory Map

3.2.1. Memory Map

Table 3-1 Memory Map at CS0 Boot

	Reserved	A000 0000 ~ FFFF FFFF	1.25GBytes
PCI DATA	PCI Configuration DATA	9800 0000 ~ 9FFF FFFF	128MBytes
PCI ADDR	PCI Configuration ADDR	9000 0000 ~ 97FF FFFF	128MBytes
PCI IO	PCI IO Area	8800 0000 ~ 8FFF FFFF	128MBytes
PCI Memory	PCI Memory Area	8000 0000 ~ 87FF FFFF	128MBytes
UWB Slave 1	UWB Slave 1	7800 0000 ~ 7FFF FFFF	128MBytes
UWB Slave 0	UWB Slave 0	7000 0000 ~ 77FF FFFF	128MBytes
DDR	DDR Memory	6000 0000 ~ 6FFF FFFF	256MBytes
PERI	Peripheral Area	4000 0000 ~ 5FFF FFFF	512MBytes
	Reserved	3000 0000 ~ 3FFF FFFF	256MBytes
ISMC	Internal SRAM	2000 0000 ~ 2FFF FFFF	256MBytes
	Reserved	1700 0000 ~ 1FFF FFFF	144MBytes
CS8	External Memory Chip Select 8	1600 0000 ~ 16FF FFFF	16MBytes
CS7	External Memory Chip Select 7	1500 0000 ~ 15FF FFFF	16MBytes
CS6	External Memory Chip Select 6	1400 0000 ~ 14FF FFFF	16MBytes
CS5	External Memory Chip Select 5	1300 0000 ~ 13FF FFFF	16MBytes
CS4	External Memory Chip Select 4	1200 0000 ~ 12FF FFFF	16MBytes
CS3	External Memory Chip Select 3	1100 0000 ~ 11FF FFFF	16MBytes
CS2	External Memory Chip Select 2	1000 0000 ~ 10FF FFFF	16MBytes
CS1	External Memory Chip Select 1	0800 0000 ~ 0FFF FFFF	128MBytes
CS0	External Memory Chip Select 0	0000 0000 ~ 07FF FFFF	128MBytes

Table 3-2 Memory Map at NAND Boot

	Reserved	A000 0000 ~ FFFF FFFF	1.25GBytes
PCI DATA	PCI Configuration DATA	9800 0000 ~ 9FFF FFFF	128MBytes
PCI ADDR	PCI Configuration ADDR	9000 0000 ~ 97FF FFFF	128MBytes
PCI IO	PCI IO Area	8800 0000 ~ 8FFF FFFF	128MBytes
PCI Memory	PCI Memory Area	8000 0000 ~ 87FF FFFF	128MBytes
UWB Slave 1	UWB Slave 1	7800 0000 ~ 7FFF FFFF	128MBytes
UWB Slave 0	UWB Slave 0	7000 0000 ~ 77FF FFFF	128MBytes
DDR	DDR Memory	6000 0000 ~ 6FFF FFFF	256MBytes
PERI	Peripheral Area	4000 0000 ~ 5FFF FFFF	512MBytes
	Reserved	3700 0000 ~ 3FFF FFFF	144MBytes
CS8	External Memory Chip Select 8	3600 0000 ~ 36FF FFFF	16MBytes
CS7	External Memory Chip Select 7	3500 0000 ~ 35FF FFFF	16MBytes
CS6	External Memory Chip Select 6	3400 0000 ~ 34FF FFFF	16MBytes
CS5	External Memory Chip Select 5	3300 0000 ~ 33FF FFFF	16MBytes
CS4	External Memory Chip Select 4	3200 0000 ~ 32FF FFFF	16MBytes

CS3	External Memory Chip Select 3	3100 0000 ~ 31FF FFFF	16MBytes
CS2	External Memory Chip Select 2	3000 0000 ~ 30FF FFFF	16MBytes
CS1	External Memory Chip Select 1	2800 0000 ~ 2FFF FFFF	128MBytes
CS0	External Memory Chip Select 0	2000 0000 ~ 27FF FFFF	128MBytes
	Reserved	1000 0000 ~ 1FFF FFFF	256MBytes
ISMC	Internal SRAM	0000 0000 ~ 0FFF FFFF	256MBytes

CS0 에서 직접 부팅하는 Direct boot mode 와 NAND Flash memory 에서 프로그램 코드를 복사하여 부팅하는 NAND Boot 와 External Memory Chip Select1 의 메모리에서 부팅하는 방식으로 지원된다.
FPGA Board 의 SW4 의 2 번을 ON 하면 NAND 로 부팅하고, OFF 하면 CS0 로 부팅한다.

각각의 Memory bank 는 ARM926EJS 의 MMU 설정에 의해서 Cacheable 혹은 non cacheable 영역으로 설정될 수 있다.

3.3.Peripherals

3.3.1. Peripheral Memory Map

Table 3-3 Peripheral Memory MAP

Reserved	5000 A000 ~ 5FFF FFFF
Static Memory Controller	5000 9000 ~ 5000 9FFF
SPI Controller	5000 8000 ~ 5000 8FFF
I2C Master	5000 7000 ~ 5000 7FFF
I2S Controller	5000 6000 ~ 5000 6FFF
UART	5000 5000 ~ 5000 5FFF
System Controller	5000 4000 ~ 5000 4FFF
Power Management Unit	5000 3000 ~ 5000 3FFF
GPIO	5000 2000 ~ 5000 2FFF
WatchDog Timer	5000 1000 ~ 5000 1FFF
Timer	5000 0000 ~ 5000 0FFF
Reserved	4000 5000 ~ 4FFF FFFF
Vectored Interrupt Controller	4001 4000 ~ 4001 4FFF
Reserved	4001 0000 ~ 4001 0FFF
E-DMA Controller	4000 C000 ~ 4000 CFFF
NAND Flash Memory Controller	4000 8000 ~ 4000 8FFF
DDR Memory Controller	4000 4000 ~ 4000 4FFF
Reserved	4000 0000 ~ 4000 0FFF

3.4.Boot sequence

3.4.1. Boot mode configuration

Power On Configuration 시 외부 핀의 설정에 의해서 Boot 모드를 설정할 수 있다.

Table 3-4 Boot Mode Configuration

Boot mode	Boot mode [1:0]
-----------	-----------------

Direct(CS0) boot mode	0
NAND boot mode	1
Reserved	2
Reserved	3

3.4.2. Direct(CS0) boot mode

ROM 에서 직접 부팅한다. 이 모드로 부팅하기 위해서 외부 Boot mode[1:0] pin 을 "00"으로 설정한 후 Reset 신호를 준다.

ROM 이 어드레스 0x0000 0000 으로 매핑이 되어 진행된다.

절차는 다음과 같다.

1. Boot mode pin = 0 설정
2. Reset
3. ROM 에서 부팅
4. 필요한 프로그램과 데이터를 Main Memory 로 복사

3.4.3. NAND boot mode

이 모드로 부팅하기 위해서 외부 Boot mode[1:0] pin 을 "01"으로 설정한 후 Reset 신호를 준다
NAND Flash memory 에서 지정된 크기의 데이터를 메모리로 복사한 후 부팅한다.

NAND 메모리에서 자동으로 Internal SRAM 으로 복사되는 메모리의 데이터 크기는 32Kbytes 가 된다.
복사가 완료되면 복사된 메모리에서 부팅을 하게 된다.

이때 복사되는 Internal SRAM 은 128Kbytes 의 메모리이다.

Boot mode 는 FPGA Board 의 SW4 의 1 번, 2 번 SW 에 mapping 되어 있다.

4. Power mamangement unit

4.1. Overview

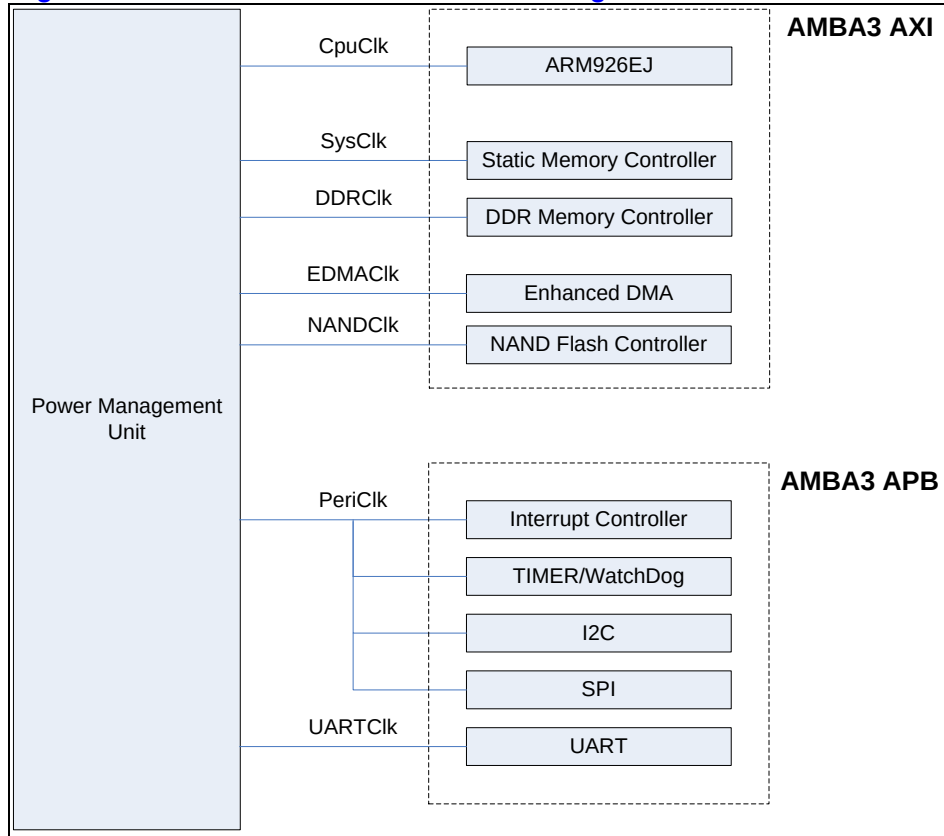
Clock Control Block 과 Power Manage State Machine, Reset Control Block 으로 나눌 수 있다.

4.1.1. Feature

- Support Slow Mode.
 - Operate by External Clock divided or not
- Support CPU Idle Mode.
 - Disconnect Only CPU Clock
 - Wake Up From Any Interrupt
- Support Power Down Mode.
 - Disconnect All Clock
 - Wake Up From External Interrupt

4.1.2. Block Diagram

Figure 4-1 Clock Distribution Block Diagram



4.2. Operation

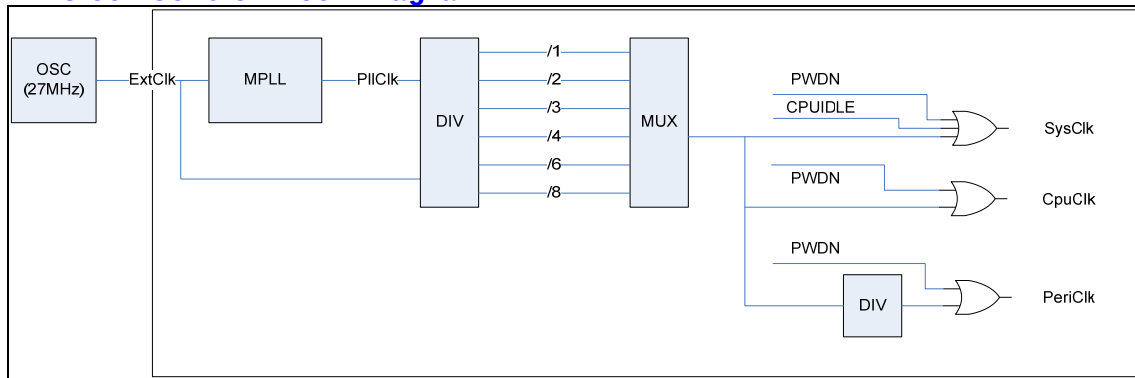
4.2.1. Clock Control Block

외부 27MHz Oscillator 의 입력을 받은 MPLL 에서 200MHz Clock 을 만들어서 CpuClk, SysClk, PeriClk 을 만든다.

정상 동작 모드의 예를 들면 200MHz Clock 은 DDR SDRAM Data Control 을 위해 사용되고, 버스나 DMA 는 100MHz 로 동작하고 Peripheral 은 50MHz 로 동작한다

FPGA 에서는 DDR 100MHz, System 50MHz, slow peripheral 25Mhz, fast peripheral 50MHz 로 동작한다.

Figure 4-2 Clock Control Block Diagram



4.2.2. Power Manage State Machine

POC State

Power On Configuration

외부 Reset(ExtResetb)이 풀리면 이 State 로 들어오게 되고, Pad 입력 쪽의 Pull-up 이나 Pull-Down Resistor 에 의해 Chip 초기 설정 Configuration 이 가능하다.

16 Cycle 이상의 POC 나 SlowEn Register 가 Set 되면 SLOW State 로 넘어 간다.

SLOW State

외부 Clock(ExtClk)에 의해 System 이 동작 하는 상태이고, PLL 이 Set 되거나(PIISetEnd) SlowEn Register 가 Clear(SlowDis)되면 LOCK State 로 넘어간다.

LOCK State

PLL 이 Locking 될 때까지 기다리는 시간이다. 이 시간은 Register 로 변경 가능하다.
PLL 이 Locking 되면 NORMAL State 로 넘어간다.

NORMAL State

System 내의 모든 Clock 이 동작하는 상태이다.

각 블록의 Clock Enable 에 의해 각각의 Power Down 이 가능하다.

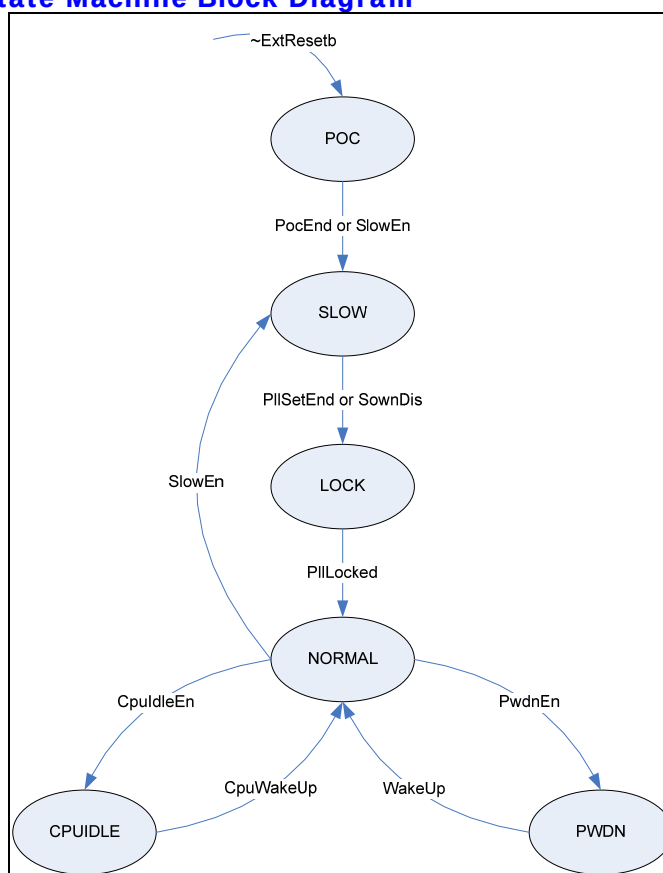
CPUIDLE State

CPU Clock 이 Disable 되는 상태이다.

PWDN State

각 블록의 Clock Enable 과 상관 없이 System 내의 모든 Clock 이 Disable 되는 상태이다.

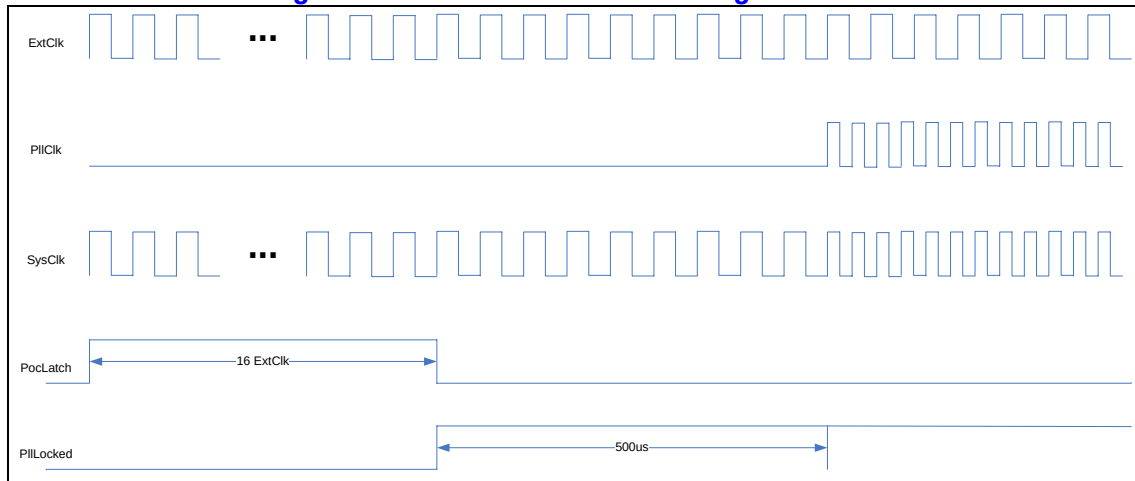
Figure 4-3 Power Manage State Machine Block Diagram



4.2.3. Reset Control Block

다음 그림과 같이 외부 클럭의 16 사이클 동안 Power On Configuration 이 이루어 지고 약 500us 의 PLL Lock Time 이 지나면 Internal 은 정상적인 PLL 클럭으로 동작 하게 된다.

Figure 4-4 Power On Configuration & PLL Lock Time Diagram



두가지의 **Reset** 방식을 사용한다. 하나는 외부 하드웨어에 의해서 **Reset** 핀을 지정된 시간 이상 **Low** 로 설정을 하면 하드웨어 리셋이 발생한다. 이 방식을 **Cold Reset** 으로 부른다.

S/W 에 의해서 **System Configuration Register** 의 비트 31 번을 '1'로 기록하면 리셋이 발생한다. 이 방식을 **Warm Reset** 으로 부른다.

Warm reset 이 된 경우에는 모든 하드웨어의 설정은 변경이 없이 **CPU** 만 다시 **Reset** 이 걸린다.

4.3. Register Description

4.3.1. Power Management Unit Registers

Table 4-1 Power Management Unit Registers

Offset	Name	R / W	Description	Init. value
0000h	PMCON	RW	Power Manage Control Register	0000 3004h
0004h	CLKDIV	RW	Clock Divide Register	0000 0008h
0008h	LOCWAIT	RW	PLL Lock Wait Register	0000 FFFFh
000Ch	PLLSET	RW	PLL Set Register	0000 1952h

4.3.2. Power Management Control Register

Figure 4-5 Power Management Control Register Bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved												PericlkEn	UARTClkEn	SEIPIClkEn	USBClkEn	Reserved	NANDClkEn	DDRCIkEn	VIFClkEn	DMClkEn	DMACIkEn	GDMACIkEn	Reserved					SlowEn	CpuIdleEn	PwdnEn	

Table 4-2 Power Management Control Register Description(Offset : 0000 0000h)

Bits	Name	R / W	Description	Init. value
[31:20]			Reserved	0
[19]	PeriClkEn	RW	TIMER/WDT/I2C/SPI Clock Enable 1 : Enable 0 : Disables	0
[18]	UARTClkEn	RW	UART Clock Enable 1 : Enable 0 : Disable	0
[17:14]			Reserved	
[13]	NANDClkEn	RW	NAND Flash Controller Clock Enable 1 : Enable 0 : Disable	1
[12]	DDRCIkEn	RW	DDR Memory Controller Clock Enable 1 : Enable 0 : Disable	1
[11:10]			Reserved	
[9]	DMACIkEn	RW	DMA Module Clock Enable 1 : Enable 0 : Disable	0
[8:3]			Reserved	
[2]	SlowEn	RW	Slow Mode Enable 1 : Enable 0 : Disable	1
[1]	CpuIdleEn	RW	CPU Idle Mode Enable 1 : Enable 0 : Disable	0
[0]	PwdnEn	RW	Power Down Enable 1 : Enable 0 : Disable	0

4.3.3. Clock Divide Register

Figure 4-6 Clock Divide Register Bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Reserved																								UClkDiv		Reserved		PClkDiv		SClkDiv			

Table 4-3 Clock Divide Register Description(Offset : 0000 0004h)

Bits	Name	R / W	Description	Init. value
[31:4]			Reserved	0
[3]	PClkDiv	RW	Peri. Clock Divide 0 : Bypass SysClk 1 : Divide By 2 SysClk	1
[2:0]	SClkDiv	RW	System Clock Divide 0 : Bypass MPIClk 1 : Divide By 2 MPIClk 2 : Divide By 3 MPIClk 3 : Divide By 4 MPIClk 4 : Divide By 6 MPIClk 5 : Divide By 8 MPIClk	0

4.3.4. PLL Lock Wait Register

Figure 4-7 PLL Lock Wait Register Bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
MPIWren	Reserved															PILocked	WaitLockCnt															

Table 5-4 PLL Lock Wait Register Description(Offset : 0000 0008h)

Bits	Name	R / W	Description	Init. value
[31]	MPIIWEn	RW	PLL Value Write Enable 이 값을 1 로 써야지만 PLL 값을 설정할 수 있다.	0
[30:17]		RW	Reserved	0
[16]	PILocked	R	PLL is Locked 0 : UnLocked 1 : Locked	0
[15:0]	WaitLockCnt	RW	PLL Lock Wait Value	ffffh

4.3.5. PLL Set Register

Figure 4-8 PLL Set Register Bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															MPIIPD	MPIIFR							MPIIRR				Reserved	MPIODR			

Table 5-5 PLL Set Register Description(Offset : 0000 000Ch)

Bits	Name	R / W	Description	Init. value
[30:17]		RW	Reserved	0
[16]	MPIIPD	RW	PLL Power Down Enable 0 : Enable 1 : Disable	0
[15:8]	MPIIFR	RW	FeedBack Divisor	25d
[7:3]	MPIIRR	RW	Input Divisor	5h
[2]			Reserved	0
[1:0]	MPIIODR	RW	Output Divisor '2' 이상의 값을 가져야 PLL 출력 클럭의 duty 가 50/50 에 근접하게 된다.	2

$$F_{REF} = F_{IN} / NR$$

$$F_{VCO} = F_{OUT} * NO$$

$F_{OUT} = F_{IN} * NF / (NR * NO)$, where F_{REF} is the comparison frequency for the PFD.
For proper operation in normal mode, the following constraints *must* be satisfied:

$$2 \text{ MHz} \leq F_{REF} \leq 8 \text{ MHz}$$

$$200 \text{ MHz} \leq F_{VCO} \leq 400 \text{ MHz}$$

$$50 \text{ MHz} \leq F_{OUT} \leq 400 \text{ MHz}$$

Example I: To synthesize a 200 MHz output clock with an input frequency $F_{IN} = 15 \text{ MHz}$.

1. Set MPIIWren = 1.
2. Set **NR** to obtain F_{REF} within the PFD comparison frequency range:
Let $F_{REF} = 5 \text{ MHz}$, Since $F_{REF} = F_{IN} / NR$,
 $NR = F_{IN} / F_{REF} = 15 \text{ MHz} / 5 \text{ MHz} = 3$
3. MPIIRR = NR = 3 = 00011₂
4. Set **NO** and ensure F_{VCO} within the VCO operating range:
Set NO = 1
 $F_{VCO} = F_{OUT} * NO$
 $F_{VCO} = 200 \text{ MHz} * 1 \square$ within the VCO operating range
5. MPIIODR = 01₂
6. Set **NF** to obtain the F_{OUT} frequency:
 $NF = F_{OUT} * NR * NO / F_{IN}$
 $NF = 200 \text{ MHz} * 3 * 1 / 15 \text{ MHz} = 40$
6. MPIIFR = NF/2 = 20 = 00010100₂

5. System control

5.1. Overview

시스템의 제어 및 관련 정보를 설정하거나 얻을 수 있는 레지스터의 모음
전원 관리등을 수행한다.

- Power On Configuration 정보.
 - External IO Width
 - Boot Mode
 - NAND Flash Configuration
- DMA Resouce Share 정보.
- System Configuration.
- Pin-Mux Control

5.2. Registers

5.2.1. System Configuration Register

Table 5-1 System Configuration Register Description (Offset : 0000 0000h)

Bits	Name	R / W	Description	Init. value
[31]	SRST	R/W	Software Reset 0 : No Effect 1 : Warm reset 1 을 기록할 경우 S/W 에 의한 리셋이 발생한다. 레지스터의 값이나 다른 값은 변동이 없이 CPU Core 의 리셋이 걸리게 된다. 해당 비트가 '1'인 경우 나머지 비트들은 영향 받지 않는다.	0000 0000h
[30:0]			Reserved	0

5.2.2. DMA Resource Share Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DMA Channel 7 Share				DMA Channel 6 Share				DMA Channel 5 Share				DMA Channel 4 Share				DMA Channel 3 Share				DMA Channel 2 Share				DMA Channel 1 Share				DMA Channel 0 Share			

Table 5-2 DMA Resource Share Register Description (Offset : 0000 0004h)

Bits	Name	R / W	Description	Init. value
[31:28]	DMA Channel 7 Share	R/W	DMA Channel 7 Input selection	0x0
[27:24]	DMA Channel 6 Share	R/W	DMA Channel 6 Input selection	0x0
[23:20]	DMA Channel 5 Share	R/W	DMA Channel 5 Input selection	0x0
[19:16]	DMA Channel 4 Share	R/W	DMA Channel 4 Input selection	0x0
[15:12]	DMA Channel 3 Share	R/W	DMA Channel 3 Input selection	0x0
[11:8]	DMA Channel 2 Share	R/W	DMA Channel 2 Input selection	0x0
[7:4]	DMA Channel 1 Share	R/W	DMA Channel 1 Input selection	0x0
[3:0]	DMA Channel 0 Share	R/W	DMA Channel 0 Input selection	0x0

각 채널의 입력 값은 아래와 같이 설정된다.

Value	Selected Input
0	Nand flash memory DMA Request
1	Reserved
2	Reserved
3	I2S Tx DMA Req
4	I2S Rx DMA Req
5	SPI 0 Tx DMA Request
6	SPI 0 Rx DMA Request
7	UART Channel 0 Tx Dma Request

8	UART Channel 0 Rx Dma Request
9	Reserved
10	Reserved

5.2.3. Information Register

Table 5-3 Information Register Description(Offset : 0000 0008h)

Bits	Name	R / W	Description	Init. value
[31:14]		R	Reserved	0000 0000h
[13]	NandOutDtmn PocCfg[5]	R	NFIO 상위 8bit 와 하위 8bit 에 동일한 Command 와 Address 를 출력함 Disable 이면 상위 8bit 는 0 하위 8bit 은 Command 또는 Address 가 출력됨 0: Disable 1: Enable	
[12:11]	NandBootCfg PocCfg[4:3]	R	부팅시 addr cycle 및 Page size 결정 00: 3 addr cycle => 1column+2row (512page) 01: 4 addr cycle => 1column+3row (512page) 10: 4 addr cycle => 2column+2row (2048page) 11: 5 addr cycle => 2column+3row (2048page)	
[10]	NandIOWidth PocCfg[2]	R	Nand Flash Controller 의 I/O 결정 0: 8bit I/O 1: 16bit I/O	
[9:8]	Boot Mode PocCfg[1:0]	R	Boot mode 0 : Direct boot mode(Wave ROM/CS0) 1 : NAND flash memory boot mode 2 : Reserved Boot mode 에 따른 설정 값이다. 외부 핀 2 개를 할당하여 Reset 시에 설정된 상태를 파악한 후 이 값이 그대로 읽혀진다	
[7:0]	MAJOR	R	Major Number 첫번째 버전은 0x01 이 된다.	0x01

5.2.4. Pin Mux Register

GPIO 및 기타 기능을 위해서 Pin 하나에 복수의 기능을 부가하였다. 부가된 기능중에 하나를 선택하기 위해서 설정되는 레지스터이다.

Table 5-4 Function Pin-Mux Register Description(Offset : 0000 0010h)

Bits	Name	R / W	Description	Init. value
[31:6]		RW	Reserved	0000 0000h
[5]		RW	0 : Reserved 1 : I2C_SDA	0
[4]		RW	0 : Reserved 1 : I2C_SCL	0

[3]		RW	0 : Reserved 1 : I2S_MCLK	0
[2]		RW	0 : Reserved 1 : I2S_BCLK	0
[1]		RW	0 : Reserved 1 : I2S_LRCLK	0
[0]		RW	0 : Reserved 1 : I2S_SDOUT	0

Table 5-5 GPIO0 Pin-Mux Register Description(Offset : 0000 0014h)

Bits	Name	R / W	Description	Init. value
[31]		RW	0 : Reserved 1 : GPIO031	0
[30]		RW	0 : Reserved 1 : GPIO030	0
[29]		RW	0 : NF_RnB1 1 : GPIO029	0
[28]		RW	0 : NF_ALE 1 : GPIO028	0
[27]		RW	0 : NF_CLE 1 : GPIO027	0
[26]		RW	0 : NF_IO7 1 : GPIO026	0
[25]		RW	0 : NF_IO6 1 : GPIO025	0
[24]		RW	0 : NF_IO5 1 : GPIO024	0
[23]		RW	0 : NF_IO3 1 : GPIO022	0
[22]		RW	0 : NF_IO3 1 : GPIO022	0
[21]		RW	0 : NF_IO2 1 : GPIO021	0
[20]		RW	0 : NF_IO1 1 : GPIO020	0
[19]		RW	0 : NF_IO0 1 : GPIO019	0
[18]		RW	0 : NF_nCE0 1 : GPIO018	0
[17]		RW	0 : NF_nCE1 1 : GPIO017	0
[16]		RW	0 : NF_nRE 1 : GPIO016	0
[15]		RW	0 : NF_nWE 1 : GPIO015	0
[14]		RW	0 : SMC_ADDR26 1 : GPIO014	0
[13]		RW	0 : SMC_ADDR25 1 : GPIO013	0
[12]		RW	0 : SMC_ADDR24 1 : GPIO012	0
[11]		RW	0 : SMC_nCS4 1 : GPIO011	0

[10]		RW	0 : SMC_nCS3 1 : GPIO010	0
[9]		RW	0 : SMC_nCS2 1 : GPIO09	0
[8]		RW	0 : SMC_nCS1 1 : GPIO08	0
[7]		RW	0 : Reserved 1 : GPIO07	0
[6]		RW	0 : Reserved 1 : GPIO06	0
[5]		RW	0 : Reserved 1 : GPIO05	0
[4]		RW	0 : Reserved 1 : GPIO04	0
[3]		RW	0 : Reserved 1 : GPIO03	0
[2]		RW	0 : Reserved 1 : GPIO02	0
[1]		RW	0 : Reserved 1 : GPIO01	0
[0]		RW	0 : Reserved 1 : GPIO00	0

Table 5-6 GPIO1 Pin-Mux Register Description(Offset : 0000 0018h)

Bits	Name	R / W	Description	Init. value
[31:30]			Reserved	
[29]		RW	0 : Reserved 1 : GPIO129	0
[28]		RW	0 : UART0_TXD 1 : GPIO128	0
[27]		RW	0 : SPI_CLK 1 : GPIO127	0
[26]		RW	0 : SPI_MOSI 1 : GPIO126	0
[25]		RW	0 : SPI_MISO 1 : GPIO125	0
[24]		RW	0 : SPI_nSS 1 : GPIO124	0
[23]		RW	0 : Reserved 1 : GPIO123	0
[22]		RW	0 : Reserved 1 : GPIO122	0
[21]		RW	0 : Reserved 1 : GPIO121	0
[20]		RW	0 : Reserved 1 : GPIO120	0
[19]		RW	0 : Reserved 1 : GPIO119	0
[18]		RW	0 : Reserved 1 : GPIO118	0
[17]		RW	0 : Reserved 1 : GPIO117	0
[16]		RW	0 : Reserved	0

			1 : GPIO116	
[15]		RW	0 : Reserved 1 : GPIO115	0
[14]		RW	0 : Reserved 1 : GPIO114	0
[13]		RW	0 : Reserved 1 : GPIO113	0
[12]		RW	0 : Reserved 1 : GPIO112	0
[11]		RW	0 : Reserved 1 : GPIO111	0
[10]		RW	0 : Reserved 1 : GPIO110	0
[9]		RW	0 : Reserved 1 : GPIO19	0
[8]		RW	0 : Reserved 1 : GPIO18	0
[7]		RW	0 : Reserved 1 : GPIO17	0
[6]		RW	0 : Reserved 1 : GPIO16	0
[5]		RW	0 : Reserved 1 : GPIO15	0
[4]		RW	0 : Reserved 1 : GPIO14	0
[3]		RW	0 : Reserved 1 : GPIO13	0
[2]		RW	0 : Reserved 1 : GPIO12	0
[1]		RW	0 : Reserved 1 : GPIO11	0
[0]		RW	0 : Reserved 1 : GPIO10	0

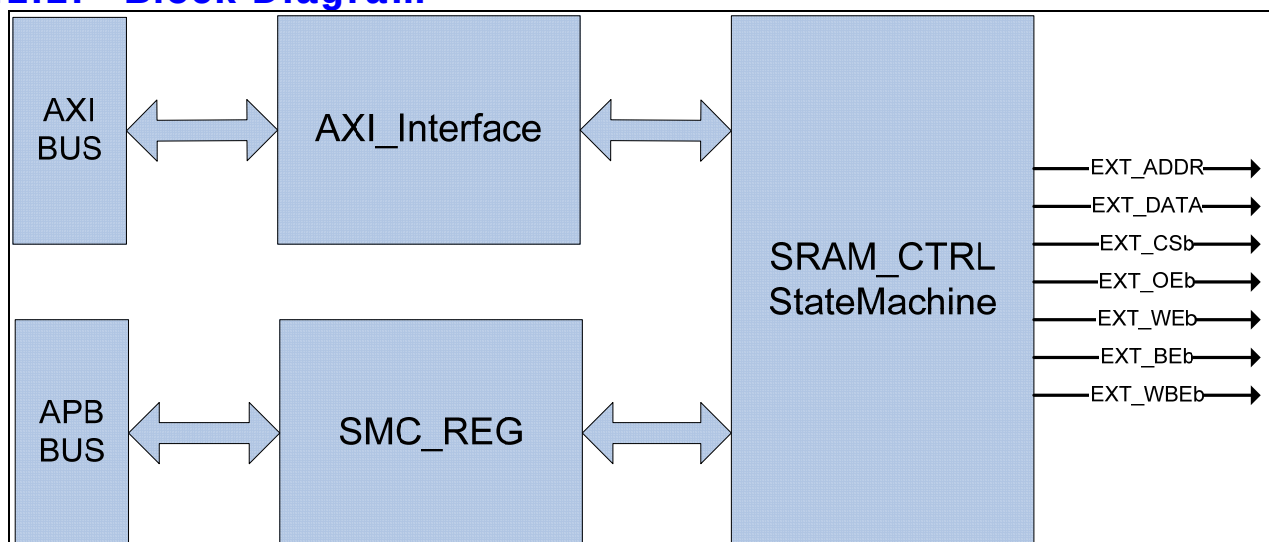
6. Static Memory Controller

6.1. Overview

6.1.1. Feature

- AXI Slave Interface
- Support 8-BANK
- Programmable access cycle
- Programmable device bus Width(8/16/32)
 - ◆ Support 8 Bit External Bus Width at CT500.
- Programmable address shift(1bit/2bit)

6.1.2. Block Diagram



6.2. Signal Description

Table 6-1 ESMC Signal

Signal Name	Direction	Description	Init. value
EXT_ADDR[26:0]	Output	Address signal	
EXT_DATA[7:0]	InOut	8bit Data Input/Output signal	
EXT_CSb[8:0]	Output	Chip select (active low)	
EXT_OEb	Output	Output Enable signal(active low)	
EXT_Web	Output	Write Enable signal(active low)	
EXT_BEb[3:0]	Output	Byte Enable signal(active low)	
EXT_WBEb[3:0]	Output	Write byte enable signal(active low)	

EXT_BEb[0]은 EXT_DATA[7:0],EXT_BEb[1]은 EXT_DATA[15:8],EXT_BEb[2]는 EXT_DATA[23:16],EXT_BEb[4]는 EXT_DATA[31:24]을 Enable/Disable 한다.
EXT_BEb 는 ChipSelect 신호(EXT_CSb)와 같은 timing 을 가지고 EXT_WBEb 는 WriteEanble 신호 (EXT_Web)와 같은 timing 을 가진다.

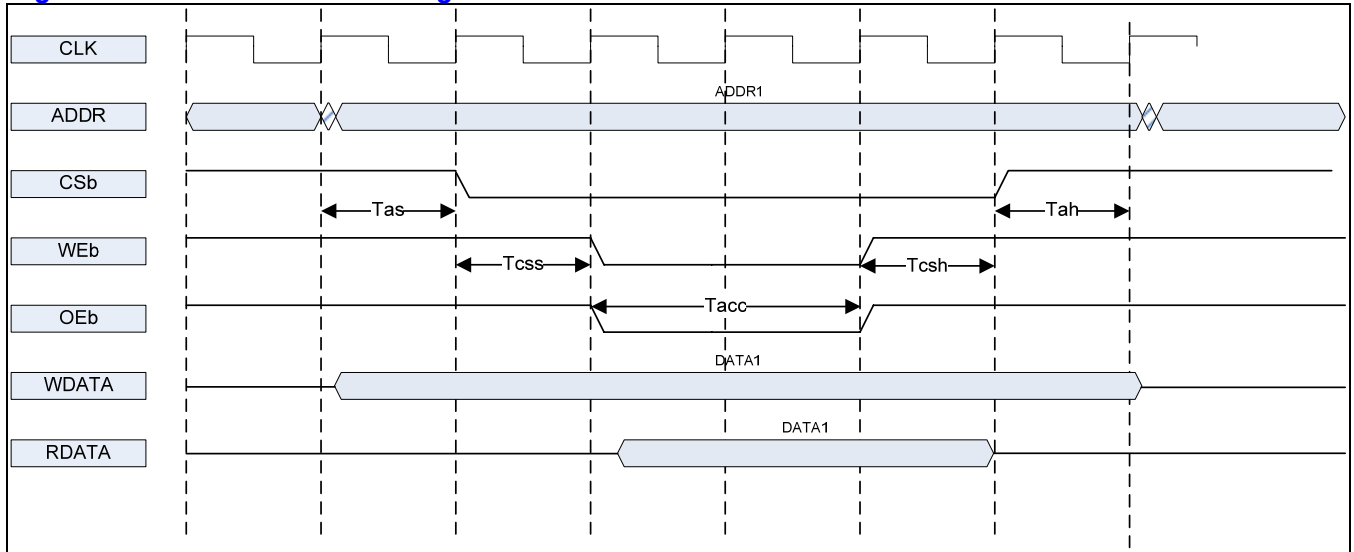
6.3. Address Map

Table 6-2 Address Map

Name	Start Address	End Address
External SRAM(CS0:128M) WaveROM 과 공유	0x00000000	0x07FFFFFF
External SRAM(CS1:128M)	0x08000000	0x0FFFFFFF
External SRAM(CS2:128M)	0x10000000	0x17FFFFFF
External SRAM(CS3:64M)	0x18000000	0x1BFFFFFF
External SRAM(CS4:64M)	0x1C000000	0x1FFFFFFF

6.4. Timing

Figure 6-1 External IO Timing



Timing diagram 에 있는 time 정보에 대한 약자는 다음과 같다.

Tas : address setup time
 Tcss : chip select setup time
 Tacc : access time
 Tcsh : chip select hold time
 Tah : address hold time

기본적으로 address 출력이 제일 먼저 이루어 진다. Address setup time 이 지난 다음에 CSb 가 low 가 되고 그 다음 Chip select setup time 이 지나면 Output Enable 혹은 Write Enable 신호가 low 가 된다.

Access time 이 지나면 RDATA 를 latch 하게 되고 그 다음 chip select hold time, address hold time 이 지나면 모든 access 가 끝나게 된다.

Tas, Tcss, Tacc, Tcsh, Tah 를 system clock(정확하게는 AXI interface clock) cycle 수로 정할 수 있는 register 가 각 bank 별로 존재한다.

6.5. Interface

32bit SRAM 의 경우 EXT_DATA 의 32bit 을 모두 사용하면 되고, 16bit SRAM 의 경우 EXT_DATA[15:0]만, 8 bit SRAM 의 경우 EXT_DATA[7:0]만 연결하여 사용한다.

EXT_ADDR 은 0 번 bit 부터 연결한 후에 각 bank 별 Address shift register 를 조정하여도 되고 아니면 SRAM 의 bit width 에 맞추어 하위 1bit 혹은 2bit 를 빼고 연결해도 된다.

EXT_CSb 와 EXT_OEb, EXT_WEB 는 SRAM 의 각 pin 에 연결하면 되고, SRAM 이 byte enable signal 을 가지고 있으면 EXT_BEb 에 연결한다.

여기서 주의할 사항이 하나 있는데, 외부의 device 가 ByteEnable pin 이 없는 device 라면 Write Enable pin 에 EXT_WEB 를 연결하지 말고 EXT_WBEb 를 연결해야 한다. 보통의 8bit SRAM 도 byte enable pin 이 없으므로 동일하다. AXI Write data channel 에서 WSTRB 가 모두 0 으로 된 signal 이 들어올 수 있는데 그런 경우에는 EXT_WEB 는 low 로 떨어지는데 EXT_WBEb 는 high 로 유지가 되기 때문이다.

6.5.1. Interface Example

EXT_ADDR[26:0]은 메모리 크기에 따라 붙여준다 아래 그림에서는 A[15:0]인 메모리를 예로 들었다.

Figure 6-2 Interface 8bit memory

EXT_ADDR0	A0	D0	EXT_DATA0
EXT_ADDR1	A1	D1	EXT_DATA1
EXT_ADDR2	A2	D2	EXT_DATA2
EXT_ADDR3	A3	D3	EXT_DATA3
EXT_ADDR4	A4	D4	EXT_DATA4
EXT_ADDR5	A5	D5	EXT_DATA5
EXT_ADDR6	A6	D6	EXT_DATA6
EXT_ADDR7	A7	D7	EXT_DATA7
EXT_ADDR8	A8		
EXT_ADDR9	A9		
EXT_ADDR10	A10		
EXT_ADDR11	A11		
EXT_ADDR12	A12	WE	EXT_Web
EXT_ADDR13	A13	OE	EXT_OEb
EXT_ADDR14	A14	CS	EXT_CEb
EXT_ADDR15	A15		

Figure 6-3 Interface 8bit memory x2

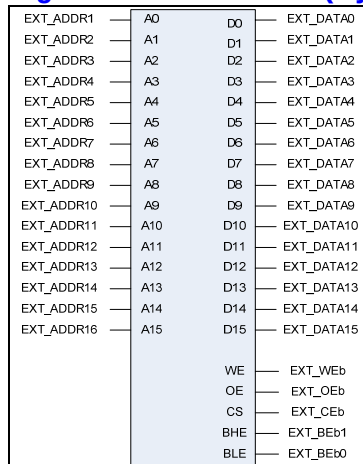
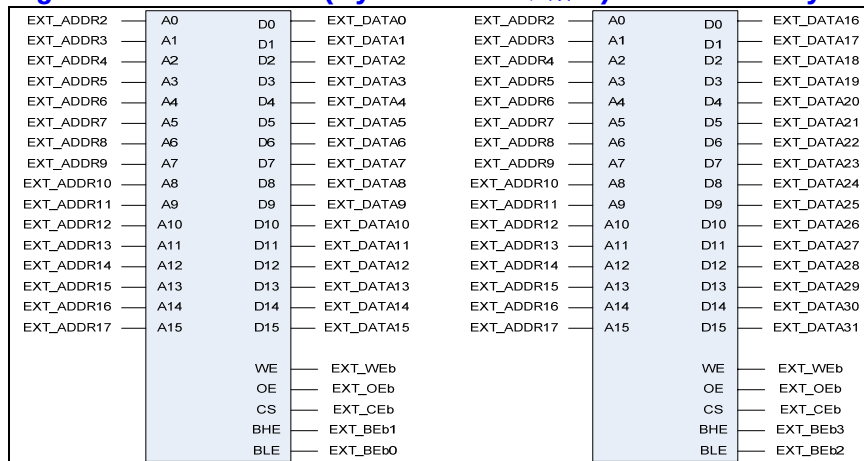
EXT_ADDR1	A0	D0	EXT_DATA0	EXT_ADDR1	A0	D0	EXT_DATA8
EXT_ADDR2	A1	D1	EXT_DATA1	EXT_ADDR2	A1	D1	EXT_DATA9
EXT_ADDR3	A2	D2	EXT_DATA2	EXT_ADDR3	A2	D2	EXT_DATA10
EXT_ADDR4	A3	D3	EXT_DATA3	EXT_ADDR4	A3	D3	EXT_DATA11
EXT_ADDR5	A4	D4	EXT_DATA4	EXT_ADDR5	A4	D4	EXT_DATA12
EXT_ADDR6	A5	D5	EXT_DATA5	EXT_ADDR6	A5	D5	EXT_DATA13
EXT_ADDR7	A6	D6	EXT_DATA6	EXT_ADDR7	A6	D6	EXT_DATA14
EXT_ADDR8	A7	D7	EXT_DATA7	EXT_ADDR8	A7	D7	EXT_DATA15
EXT_ADDR9	A8			EXT_ADDR9	A8		
EXT_ADDR10	A9			EXT_ADDR10	A9		
EXT_ADDR11	A10			EXT_ADDR11	A10		
EXT_ADDR12	A11			EXT_ADDR12	A11		
EXT_ADDR13	A12	WE	EXT_WBE0	EXT_ADDR13	A12	WE	EXT_WBE1
EXT_ADDR14	A13	OE	EXT_OEb	EXT_ADDR14	A13	OE	EXT_OEb
EXT_ADDR15	A14	CS	EXT_CEb	EXT_ADDR15	A14	CS	EXT_CEb
EXT_ADDR16	A15			EXT_ADDR16	A15		

Figure 6-4 Interface 8bit memory x4

EXT_ADDR2	A0	D0	EXT_DATA0	EXT_ADDR2	A0	D0	EXT_DATA8	EXT_ADDR2	A0	D0	EXT_DATA16	EXT_ADDR2	A0	D0	EXT_DATA24
EXT_ADDR3	A1	D1	EXT_DATA1	EXT_ADDR3	A1	D1	EXT_DATA9	EXT_ADDR3	A1	D1	EXT_DATA17	EXT_ADDR3	A1	D1	EXT_DATA25
EXT_ADDR4	A2	D2	EXT_DATA2	EXT_ADDR4	A2	D2	EXT_DATA10	EXT_ADDR4	A2	D2	EXT_DATA18	EXT_ADDR4	A2	D2	EXT_DATA26
EXT_ADDR5	A3	D3	EXT_DATA3	EXT_ADDR5	A3	D3	EXT_DATA11	EXT_ADDR5	A3	D3	EXT_DATA19	EXT_ADDR5	A3	D3	EXT_DATA27
EXT_ADDR6	A4	D4	EXT_DATA4	EXT_ADDR6	A4	D4	EXT_DATA12	EXT_ADDR6	A4	D4	EXT_DATA20	EXT_ADDR6	A4	D4	EXT_DATA28
EXT_ADDR7	A5	D5	EXT_DATA5	EXT_ADDR7	A5	D5	EXT_DATA13	EXT_ADDR7	A5	D5	EXT_DATA21	EXT_ADDR7	A5	D5	EXT_DATA29
EXT_ADDR8	A6	D6	EXT_DATA6	EXT_ADDR8	A6	D6	EXT_DATA14	EXT_ADDR8	A6	D6	EXT_DATA22	EXT_ADDR8	A6	D6	EXT_DATA30
EXT_ADDR9	A7	D7	EXT_DATA7	EXT_ADDR9	A7	D7	EXT_DATA15	EXT_ADDR9	A7	D7	EXT_DATA23	EXT_ADDR9	A7	D7	EXT_DATA31
EXT_ADDR10	A8			EXT_ADDR10	A8			EXT_ADDR10	A8			EXT_ADDR10	A8		
EXT_ADDR11	A9			EXT_ADDR11	A9			EXT_ADDR11	A9			EXT_ADDR11	A9		
EXT_ADDR12	A10			EXT_ADDR12	A10			EXT_ADDR12	A10			EXT_ADDR12	A10		
EXT_ADDR13	A11			EXT_ADDR13	A11			EXT_ADDR13	A11			EXT_ADDR13	A11		
EXT_ADDR14	A12	WE	EXT_WBE0	EXT_ADDR14	A12	WE	EXT_WBE1	EXT_ADDR14	A12	WE	EXT_WBE2	EXT_ADDR14	A12	WE	EXT_WBE3
EXT_ADDR15	A13	OE	EXT_OEb	EXT_ADDR15	A13	OE	EXT_OEb	EXT_ADDR15	A13	OE	EXT_OEb	EXT_ADDR15	A13	OE	EXT_OEb
EXT_ADDR16	A14	CS	EXT_CEb	EXT_ADDR16	A14	CS	EXT_CEb	EXT_ADDR16	A14	CS	EXT_CEb	EXT_ADDR16	A14	CS	EXT_CEb
EXT_ADDR17	A15			EXT_ADDR17	A15			EXT_ADDR17	A15			EXT_ADDR17	A15		

Figure 6-5 Interface 16bit memory

EXT_ADDR1	A0	D0	EXT_DATA0
EXT_ADDR2	A1	D1	EXT_DATA1
EXT_ADDR3	A2	D2	EXT_DATA2
EXT_ADDR4	A3	D3	EXT_DATA3
EXT_ADDR5	A4	D4	EXT_DATA4
EXT_ADDR6	A5	D5	EXT_DATA5
EXT_ADDR7	A6	D6	EXT_DATA6
EXT_ADDR8	A7	D7	EXT_DATA7
EXT_ADDR9	A8	D8	EXT_DATA8
EXT_ADDR10	A9	D9	EXT_DATA9
EXT_ADDR11	A10	D10	EXT_DATA10
EXT_ADDR12	A11	D11	EXT_DATA11
EXT_ADDR13	A12	D12	EXT_DATA12
EXT_ADDR14	A13	D13	EXT_DATA13
EXT_ADDR15	A14	D14	EXT_DATA14
EXT_ADDR16	A15	D15	EXT_DATA15
		WE	EXT_Web
		OE	EXT_OEb
		CS	EXT_CEb

Figure 6-6 Interface (byte enable이 있는)16bit memory**Figure 6-7 Interface (byte enable이 있는)16bit memoryx2**

6.6. Register Description

6.6.1. Registers

External Sram Controller 는 총 4 개의 BANK 를 가지고 있으며 각 BANK 별로 Timing 과 Bus Width, Address Shift 여부를 셋팅 할수 있다.

Table 6-3 Registers

Offset	Name	R / W	Description	Init. value
0000h	SMC_BANK0	R/W	BANK0 Control	
0004h	SMC_BANK1	R/W	BANK1Control	
0008h	SMC_BANK2	R/W	BANK2Control	
000Ch	SMC_BANK3	R/W	BANK3Control	
0010h	SMC_BANK4	R/W	BANK4Control	

6.6.2. SMC_BANKx Register

각각의 BANK Register 는 아래와 같은 구성을 가진다.

Figure 6-8 SMC_BANKx Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved												Tas		Tcss		Tacc			Tcsh			Tah			BusWidth		AddrShift				

Table 6-4 SMC_BANKx Register (Offset : 0000 0000h~0000 000ch)

Bits	Name	R / W	Description	Init. value
[19:17]	Tacs	R/W	Address set-up time before csb 000 = 0clock 001 = 1clock 010 = 2clock 011 = 3clock 100 = 4clock 101 = 5clock 110 = 6clock 111 = 7clock	
[16:14]	Tcos	R/W	Chip selection set-up time before oeb, web 000 = 0clock 001 = 1clock 010 = 2clock 011 = 3clock 100 = 4clock 101 = 5clock 110 = 6clock 111 = 7clock	
[13:10]	Tacc	R/W	Access cycle 0000=1clock 0001=2clock 0010=3clock 0011=4clock 0100=5clock 0101=6clock 0110=7clock 0111=8clock 1000=9clock 1001=10clock 1010=11clock 1011=12clock 1100=13clock 1101=14clock 1110=15clock 1111=16clock	
[9:7]	Tcoh	R/W	Chip selection hold time after oeb, web 000 = 0clock 001 = 1clock 010 = 2clock 011 = 3clock 100 = 4clock 101 = 5clock 110 = 6clock 111 = 7clock	
[6:4]	Tcah	R/W	Address hold time after csb 000 = 0clock 001 = 1clock 010 = 2clock 011 = 3clock 100 = 4clock 101 = 5clock 110 = 6clock 111 = 7clock	
[3 : 2]	BusWidth	R/W	SRAM Bus Width 00=8bit 01=16bit 10=32 bit 11= reserved	
[1 : 0]	AddrShift	R/W	Address shift by right 00=0bit 01=1bit 10=2bit 11= reserved	

7. DDR-SDRAM Scheduler

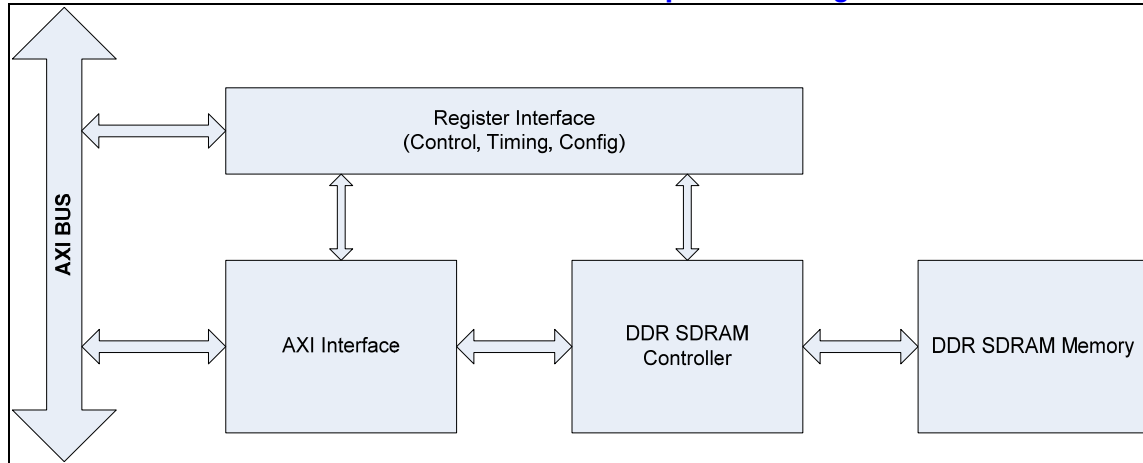
7.1. Overview

7.1.1. Feature

- Support Maximum 16 Wrap/Increment Burst.
- Support Up to 4 Physical Memory Banks
- Up to 4 Simultaneous Open Banks
- Self-Refresh, DDR SDRAM Power Save Feature
- Programmable Timing latencies and refresh rates.
- Support Up to 128MBytes DDR SDRAM.
- Support 16 Bit DDR SDRAM Interface.
- Configurable Data FIFO Depth.
- Maximum 80% Data Bandwidth
 - Maximum 425MBytes/Sec, if Full Bandwidth is 532MBytes/Sec at 133MHz Memory Clock and 32Bit Memory Width
 - Condition is Consecutive Incremental Read Burst 8
 - If Random Read or Write Applied then Data Bandwidth is Maximum 60%

7.1.2. Block Diagram

Figure 7-1 AXI Interface DDR SDRAM Controller Top Block Diagram



7.2. Operation

AXI Interface DDR SDRAM Controller 는 3 개의 Block 으로 나눌 수 있다.

DDR SDRAM 의 Configuration, Timing 및 Control 에 필요한 Register 인터페이스 블록과 AXI BUS 인터페이스 블록 및 실제 Memory 의 Timing 에 맞게 Read 나 Write 할 Data, Address 및 Command 를 생성하는 블록이다.

7.2.1. AXI BUS Interface Block

AMBA3 AXI BUS 는 Write Command Channel, Write Data Channel, Write Response Channel, Read Command Channel, Read Data Channel 등의 5 채널로 나눌 수 있다.

DDR SDRAM 메모리는 Read 나 Write 를 동시에 처리할 수 없으므로 이 블록에서 Read 와 Write 에 대한 Command 를 Arbitration 해야 한다.

Write Control FSM

Memory Controller 에 보낼 Write Command 와 Address, Data 및 부가 정보를 Control 하는 메인 Finite State Machine 이다.

Read Control FSM

Memory Controller 에 보낼 Read Command 와 Address 및 부가 정보를 Control 하는 메인 Finite State Machine 이다.

Address/Control Mux

Read/Write 각 FSM 에 의해 생성된 Command, Address 및 부가 정보를 Multiplexing 하여 DDR SDRAM Memory Controller 에 전달하는 역할을 한다.

Write Data Control

Write Control FSM 과 DDR SDRAM Memory Controller 에서 생성된 Memory Write Data Valid 에 의해 Write Buffer 의 Data Management 를 담당한다.

Write Buffer

FIFO 구조이고 Configuration 가능하다.

Read Data Control

DDR SDRAM Memory Controller 에서 생성된 Memory Read Data Valid 에 의해 Read Buffer 의 Data Management 를 담당한다.

Read Buffer

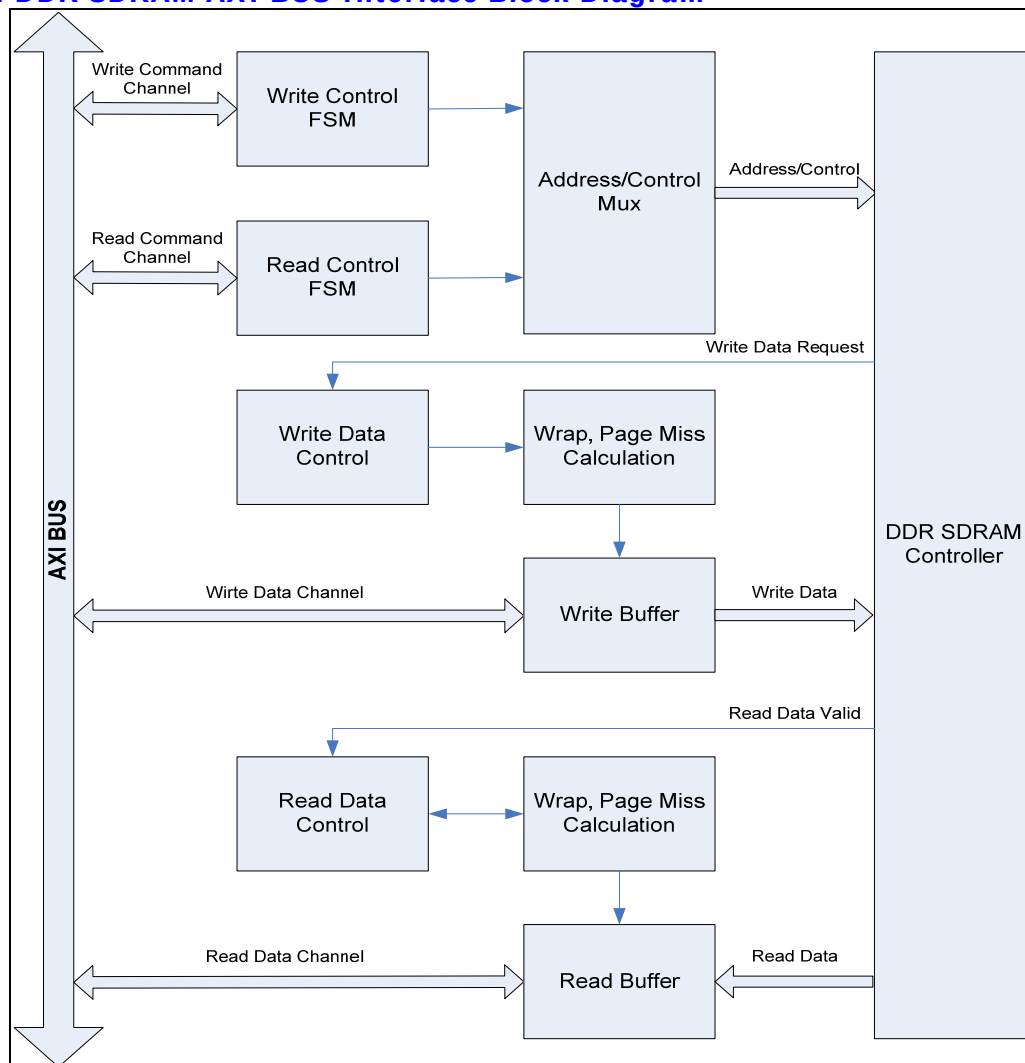
FIFO 구조이고 Configuration 가능하다.

Wrap, Increment Miss Calculation

DDR SDRAM 의 Wrap Burst Mode 를 사용하므로 Data 의 Increment Burst 에서 Data 의 Miss 의 처리를 담당한다. 즉, DDR SDRAM 의 Wrap Burst Boundary 를 넘으면, Burst 를 Split 시켜준다.

다음 그림은 AXI BUS Interface Block 을 나타낸다. AXI Write Channel 과 Read Channel 을 Arbitration 하여 메모리에 전달할 Address 와 Control 및 부가정보를 만들고, Data 는 FIFO 에 저장하여 처리하여 AXI BUS 의 Wrap Burst 처리가 가능하며, FIFO Depth 를 늘릴 수 있어 Latency 가 긴 DDR SDRAM 메모리의 효율을 높일 수 있다.

Figure 7-2 DDR SDRAM AXI BUS Interface Block Diagram



7.2.2. DDR SDRAM Controller Block

Initialize & Refresh & Power Down FSM

DDR SDRAM의 초기화에 필요한 Timing을 생성하고, Refresh Count에 의해 Auto Refresh Command Timing을 만들며, Power Down을 위한 Self Refresh Mode 진입에 필요한 Timing을 만드는 역할을 한다.

Master FSM

DDR SDRAM Controller의 Main State Machine으로써 DDR SDRAM 동작의 Timing을 제어한다.

Bank/Time FSM

DDR SDRAM의 Timing에 맞게 각 Bank의 Row를 Open하는 역할과 Data Read/Write Timing을 생성한다.

RAS Machine

AXI BUS Interface에서 보내온 Row Address를 FSM에서 생성한 적절한 Timing에 내 놓는다.

Cas Machine

AXI BUS Interface에서 보내온 Column Address를 FSM에서 생성한 적절한 Timing에 내 놓는다.

Write Data

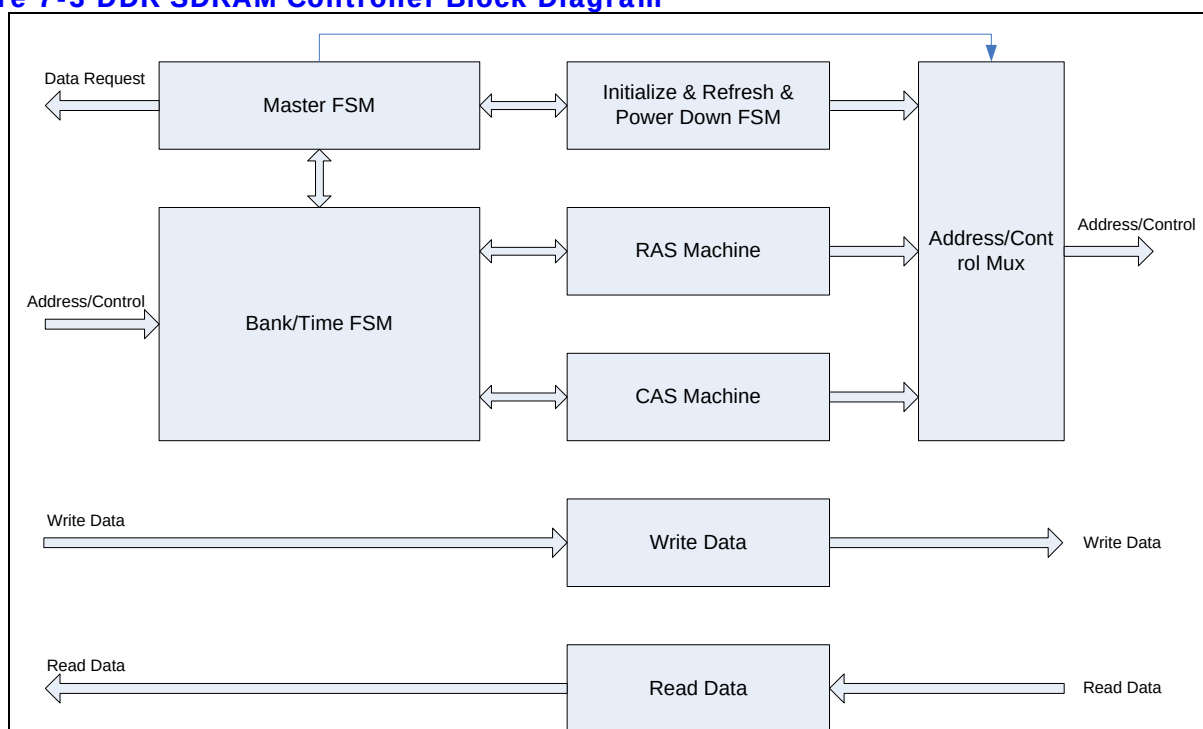
AXI BUS Interface에서 보내온 Data를 FSM에서 생성한 적절한 Timing에 내 놓는다.

Read Data

DDR SDRAM Memory에서 보내온 Data를 FSM에서 생성한 적절한 Timing에 AXI BUS Interface에 보낸다.

DDR SDRAM Controller는 다음 그림과 같이 Initialize & Refresh & Power Down FSM, Master FSM, Bank/Time FSM, RAS Machine, Cas Machine, Address/Control Mux. 및 Read/Write Data 부분으로 나눌 수 있다.

Figure 7-3 DDR SDRAM Controller Block Diagram



7.2.3. DDR SDRAM Controller Timing Description

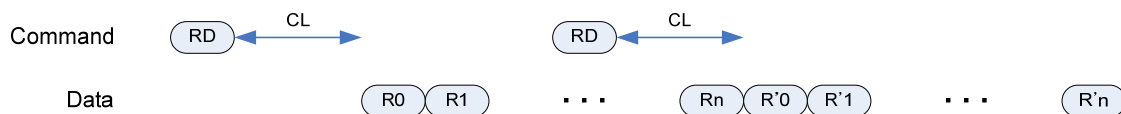
그림에서 Row/Bank Open 은 생략되어 있고, RAS Buffer 가 채워져 있으면 꺼내서 미리 Row 와 Bank 를 열어 놓는 구조이므로 일단 열린 Row/Bank(Same Row at Different Bank 나 Different Row at Same Bank)에서 연속적으로 Read/Write 가 가능하다. Row/Bank 를 Open 하는 시간과 FIFO 상황과 Auto Refresh Time 을 고려하면, 80% 이상의 Memory BandWidth 가 가능하다.

DDR SDRAM 은 Burst 8 Mode 와 Read/Write Without Auto Precharge Mode 를 사용한다.

다음에서 $n=7$, 즉 DDR SDRAM 의 Burst 8 을 사용할 경우의 예제이다.

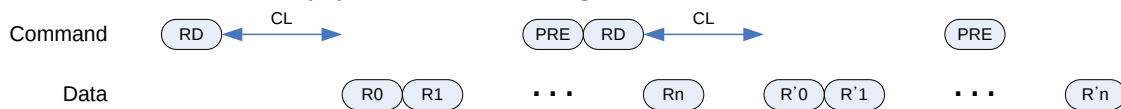
Read to Read

Same Row at Different Bank or Different Row at Same Bank



Same Row at Same Bank Read → PreCharge

Ras/Cas Queue Empty Read → PreCharge All Bank



Read to Write

Same Row at Different Bank or Different Row at Same Bank

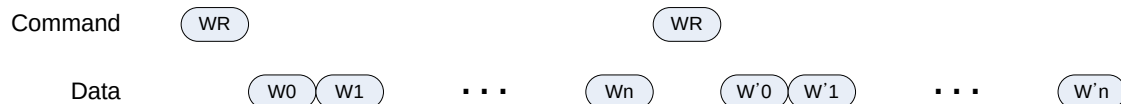


Same Row at Same Bank Read → PreCharge
 Ras/Cas Queue Empty Read → PreCharge All Bank



Write to Write

Same Row at Different Bank or Different Row at Same Bank

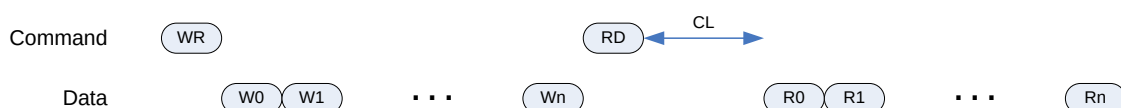


Same Row at Same Bank Write → PreCharge All Bank after tWR
 Ras/Cas Queue Empty Write → PreCharge All Bank after tWR

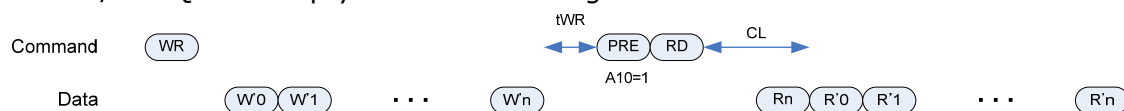


Write to Read

Same Row at Different Bank or Different Row at Same Bank



Same Row at Same Bank Write → PreCharge All Bank after tWR
 Ras/Cas Queue Empty Write → PreCharge All Bank after tWR



7.3. Register Description

7.3.1. DDR SDRAM Registers

Table 7-1 DDR SDRAM Registers

Offset	Name	R / W	Description	Init. value
0000h	DDRTCON	RW	DDR SDRAM Timing Control Register	0003 B93Ah
0004h	DDRCON	RW	DDR SDRAM Control Register	0000 0000h
0008h	DDRPCON	RW	DDR SDRAM Power Control Register	0000 FFFFh
000Ch	DDRREF	RW	DDR SDRAM Refresh Control Register	0000 0410h
0010h	DDRCFG	RW	DDR SDRAM Configuration Register	0000 0024h

7.3.2. DDR SDRAM TIMING Control Register

Figure 7-4 DDR SDRAM Timing Control Register Bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved													tCLs			tRC			tRAS			Reserved	tCL			tRCD		tRP			

Table 7-2 DDR SDRAM Timing Control Register Description(Offset : 0000 0000h)

Bits	Name	R / W	Description	Init. value
[31:16]			Reserved	0
[18:16]	tCLs	RW	Cas Latency For Read State Machine	3h
[15:12]	tRC	RW	Row Cycle Time	Bh
[11:8]	tRAS	RW	Minimum Row Active Time	8h
			Reserved	
[6:4]	tCL	RW	Cas Latency For Read Data	3h
[3:2]	tRCD	RW	Ras to Cas Delay	2h
[1:0]	tRP	RW	Precharge Time	2h

7.3.3. DDR SDRAM Control Register

Figure 7-5 DDR SDRAM Control Register Bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Dly1Sel								Dly0Sel								Reserved								RdDataPol	Reserved	ColAddrSiz	Reserved	AddrSwapDis	DDRen		

Table 7-3 DDR SDRAM Control Register Description(Offset : 0000 0004h)

Bits	Name	R / W	Description	Init. Value
[31:24]	Dly1Sel	RW	Upper Read Data Latch Delay Select	0
[23:16]	Dly0Sel	RW	Lower Read Data Latch Delay Select	0
[15:8]			Reserved	
[7]	RdDataPol	RW	Read Data Latch Polarity 0 : Latch at Rising Edge 1 : Latch at Falling Edge	0
[6]		RW	Reserved	0
[5:4]	ColAddrSiz	RW	Column Address Size 0 : 8 Bit, 16MB 1 : 9 Bit, 32MB 2 : 10 Bit, 64MB 3 : 11 Bit, 128MB	0
[3:2]			Reserved	0
[1]	AddrSwapDis	RW	Address Swap Disable 0 : Swap : RA-BA-CA 1 : Normal : BA-RA-CA	0
[0]	DDREn	RW	DDR SDRAM Enable 0 : Disable	0

			1 : Enable	
--	--	--	------------	--

7.3.4. DDR SDRAM Power Control Register

Figure 7-6 DDR SDRAM Power Control Register Bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SelfRefEn		Reserved														PwDnEn	PwDnRef														

Table 7-4 DDR SDRAM Power Control Register Description(Offset : 0000 0008h)

Bits	Name	R / W	Description	Init. value
[31]	SelfRefEn	RW	DDR SDRAM Self Refresh Enable 0 : Disable 1 : Enable	0
[30:17]			Reserved	0
[16]	PwDnEn	RW	Auto Power Down Enable 0 : Disable 1 : Enable	0
[15:0]	PwDnRef	RW	Auto Power Down Count PwDnEn 이 Enable 되었고, Master 에서 메모리 요구가 없을 때부터 설정된 값만큼 Down Counting 이 시작 되며, 이 Zero 가 되면 자동으로 DDR 의 Power Down Mode 로 진입한다.	ffffh

7.3.5. DDR SDRAM Refresh Control Register

Figure 7-7 DDR SDRAM Refresh Control Register Bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved												RefNo				RefCount															

Table 7-5 DDR SDRAM Refresh Control Register Description(Offset : 0000 000Ch)

Bits	Name	R / W	Description	Init. value
[30:20]		RW	Reserved	0
[19:16]	RefNo	RW	Refresh Number Refresh cycle 마다 수행되는 Refresh 개수	0
[15:0]	RefCount	RW	Refresh Count 7.8usec Refresh time RefNo 가 7 일 경우에는 2080h 값을 넣어 준다. 즉, 8320 cycle 마다 8 번의 Refresh 를 한꺼번에 한다는 의미이다.	0410h

7.3.6. DDR SDRAM Configuration Register

Figure 7-8 DDR SDRAM Configuration Register Bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																								tCLp		Reserved	ResetDLL	ReducedDrv	DisableDLL		

Table 7-6 DDR SDRAM Configuration Register Description(Offset : 0000 0010h)

Bits	Name	R / W	Description	Init. value
[30:7]		RW	Reserved	0
[6:4]	tCLp	W	2 : DDR SDRAM CL2 설정 6 : DDR SDRAM CL2.5 설정	2
[3]			Reserved	
[2]	ResetDLL	W	DDR SDRAM DLL Reset Enable	1
[1]	ReducedDrv	W	DDR SDRAM Reduced Drive Mode Enable	0
[0]	DisableDLL	W	DDR SDRAM DLL Disable	0

8. Vectored interrupt controller

8.1. Overview

ARM Processor 는 기본적으로 IRQ 와 FIQ, 두 개의 interrupt line 을 가지고 있을 뿐이고, 여러 개의 interrupt source 에 대한 처리는 Interrupt Controller 를 사용하여야 한다. 간단한 Interrupt Controller 의 경우 Pending 된 Interrupt bit 를 나타내는 레지스터만 있어, interrupt handle software 에서 해당 bit 를 loop 을 돌며 어떤 bit 가 pending 되어 있는지를 찾아야 한다. Pending 된 interrupt 를 찾는 순서는 software 가 결정하고 그 순서에 따라서 interrupt source 에 대한 priority 가 달라지게 된다. 하지만 이렇게 software 가 loop 을 돌며 interrupt 를 arbitration 하는 것은 상당한 시간이 걸리는 일로, 이로 인해 interrupt latency 가 증가하여 real time system 의 제약 조건이 될 수 있다.

Vectored Interrupt Controller(이하 VIC)는 여러 개의 interrupt source 에 대한 arbitration 을 하드웨어로 처리하는 구조를 가지고 있어, software 가 별도로 priority 에 따른 arbitration 을 수행할 필요가 없어 interrupt handle software 의 overhead 를 줄여준다.

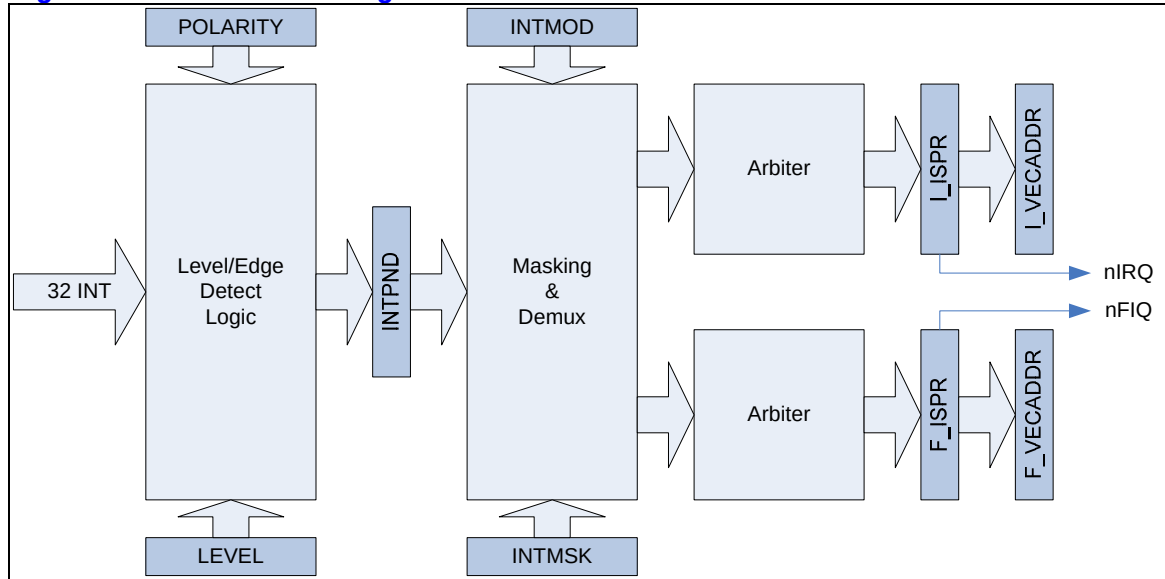
8.1.1. Features

- 32 interrupt source
- Interrupt arbitration logic
 - Fixed priority arbitration mode
 - Round-robin arbitration mode
- Interrupt vector table offset address generation
- Each interrupt source can be mapped to either IRQ or FIQ
- Programmable Level/edge triggered mode for each interrupt source
- Programmable polarity for each interrupt source

8.1.2. Block diagram

전체적인 하드웨어 구성은 다음 그림과 같다.

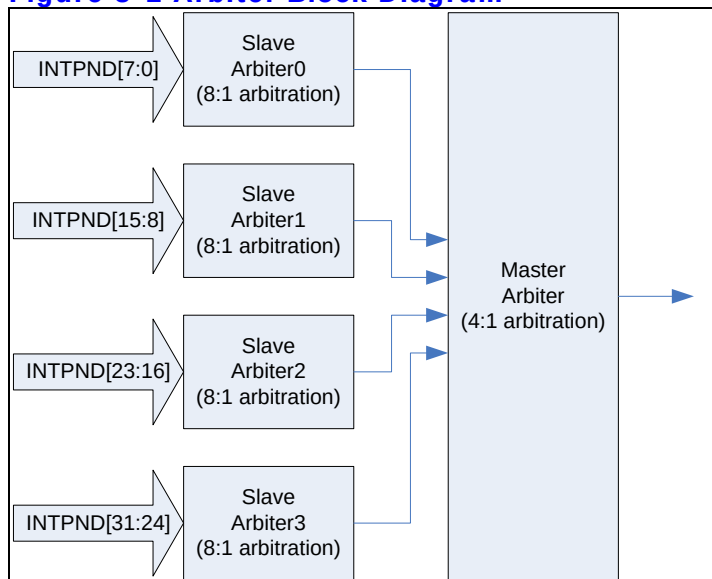
Figure 8-1 VIC Block Diagram



총 32 개의 interrupt line 을 가지고 있다. 32 개의 interrupt line 은 Level/Edge detect logic 을 지나 INTPND 레지스터에 저장된다. 그 이후 INTMSK 레지스터에 의해 masking 되고 INTMOD 에 따라서 FIQ 와 IRQ 를 demux 하여 각각 별도의 arbiter 을 통해 하나의 interrupt 가 선택되어 최종적으로 I_ISPR/F_ISPR 레지스터에 저장되고 I_VECADDR/F_VECADDR 에 address 가 저장된다.

Arbiter block 은 FIQ/IRQ 마다 하나씩 존재하고 다음과 같은 구조를 가지고 있다.

Figure 8-2 Arbiter Block Diagram



전체 32 개의 pending interrupt source 를 8 개씩 묶어 4 개의 slave arbiter 를 통과하여 8 개 중에 가장 높은 priority 를 가지는 interrupt 를 선택한 다음, 선택된 4 개의 interrupt 를 master arbiter 가 selection 을 하도록 되어 있다.

8.2. Interrupt Connection

Interrupt 연결은 다음과 같다.

Table 8-1 Interrupt Connection

Interrupt Line	Device	Level/Edge
0	UART Channel 0	Active Low Level
1	MacIntSrc 0	Active High Edge
2	MacIntSrc 1	Active High Edge
3	Watch Dog Timer	Active High Edge
4	Reserved	
5	Reserved	
6	DMA Channel 0	Active High Level
7	DMA Channel 1	Active High Level
8	DMA Channel 2	Active High Level
9	DMA Channel 3	Active High Level
10	DMA Channel 4	Active High Level
11	DMA Channel 5	Active High Level
12	DMA Channel 6	Active High Level
13	DMA Channel 7	Active High Level
14	Reserved	
15	Reserved	
16	Timer Channel 0	Active High Edge
17	Timer Channel 1	Active High Edge
18	Timer Channel 2	Active High Edge
19	Timer Channel 3	Active High Edge
20	Reserved	
21	UART Channel 1	Active Low Level
22	Reserved	
23	SPI	Active High Level
24	I2S Controller	Active High Level
25	I2C Controller	Active High Level
26	Reserved	
27	NAND Flash Controller	Active High Edge
28	PCI INT A	Active High Level
29	PCI INT B	Active High Level
30	PCI INT C	Active High Level
31	PCI INT D	Active High Level

8.3. Operation

8.3.1. Register와 Interrupt Source

모든 설명에 앞서 다음의 레지스터의 bit 는 해당 interrupt source 에 영향을 주게 된다.

INTPND/INTMOD/INTMSK/LEVEL/I_ISPR/I_ISPC/F_ISPR/F_ISPC/POLARITY 등의 레지스터는 모두 32bit 레지스터로 각 bit 는 32 개의 interrupt source 에 1:1 대응한다. 즉, INTPND[3]는 interrupt source 3 번이 pending 되어 있음을 나타낸다.

위의 Block diagram 에서 보았듯이 VIC 내부는 일정부분 IRQ 쪽 path 와 FIQ 쪽 path 가 분리되어 있고, 독립적으로 동작하게 된다. 레지스터 중에 "I_"라는 접두어로 시작되는 레지스터는 IRQ 쪽에 해당하고 "F_"라는 접두어로 시작되는 레지스터는 FIQ 쪽에 해당한다.

8.3.2. Level/Edge detect logic

Level/Edge detect logic 은 LEVEL 레지스터와 POLARITY 레지스터에 의해 영향을 받는다.

LEVEL 레지스터는 interrupt source 가 level trigger 인지 edge trigger 인지 선택하는 레지스터이다. POLARITY 레지스터는 interrupt source 가 active high 인지 active low 인지 선택하는 레지스터이다.

Level/Edge detect logic 은 32 개의 interrupt source signal 을 모니터링하며 LEVEL/POLARITY 레지스터에 의해서 선택된 형태의 signal 이 들어오면 INTPND 레지스터에 해당 bit 를 1 로 set 한다.

8.3.3. Masking & Demux logic

Level/Edge Detect logic 의 출력인 INTPND 레지스터는 masking 되지 않은 interrupt 상태를 알려준다. Masking logic 은 단순히 INTMSK 레지스터를 이용하여 INTPND 레지스터의 값을 masking 하여 나머지 logic 에 전달하는 일을 수행한다.

Demux logic 은 masking 된 interrupt 를 INTMOD 레지스터 세팅에 따라 IRQ 쪽 path 와 FIQ 쪽 path 로 분리하여 전달하는 일을 수행한다.

8.3.4. Arbiter

이미 알아본 대로 Arbiter 는 IRQ/FIQ path 각각 하나씩 존재한다. 단일 arbiter 는 8 개의 interrupt 입력을 받는 slave arbiter 4 개와 slave arbiter 의 결과를 arbitration 하는 master arbiter 하나로 구성된다. Master arbiter, 4 개의 slave arbiter 각각은 round robin mode 혹은 fixed priority mode 로 동작할 수 있다.

Fixed priority mode 에서는 각 interrupt line 의 priority 가 결정되어 있고 그 priority 가 변하지 않은 상태에서 arbitration 이 일어나게 된다. Slave arbiter 각각에 대해서 8 개의 interrupt

line 의 priority 는 PSLV 레지스터를 이용하여 programming 할 수 있고, master arbiter 가 입력 받는 slave arbitration 결과 4 line 에 대해서 PMST 레지스터를 통해 priority 를 programming 할 수 있다.

Master arbiter 와 slave arbiter 4 개가 cascading 으로 연결되어 있으므로 master arbiter 에 programming 된 slave arbiter 4 개의 priority 가 각각의 slave arbiter 내부 priority 보다 우선시 된다. 예를 들어 slave arbiter0 에 연결되어 있는 interrupt line 0 의 priority 가 slave arbiter1 에 연결된 interrupt line 8 보다 높다고 하더라도 slave arbiter0 의 priority 가 slave arbiter1 의 priority 보다 낮으면 interrupt line 8 이 선택된다. 즉 slave arbiter 내부의 priority setting 은 다른 slave arbiter 와는 아무런 연관 관계를 지니지 않는다.

Round robin mode 에서는 매 arbitration 마다 priority 가 update 되고 현재 priority 값은 CSLV/CMST 레지스터를 통해서 읽을 수 있다.

8.3.5. I_ISPR/F_ISPR and I_VECADDR/F_VECADDR

I_ISPR 과 F_ISPR 은 각각 IRQ/FIQ 에 해당하는 interrupt 가 detect 되고 arbitration 된 결과를 나타낸다. INTPND 와 달리 masking 되어 있어 enable 되지 않은 interrupt source 는 표시되지 않고, 또한 단 하나의 bit 만 1 로 세팅된다. 어떤 bit 가 1 로 세팅 되었는지 체크할 필요는 없고 그 대신 I_VECADDR/F_VECADDR 레지스터의 값을 읽으면 해당 bit 가 어떤 bit 인지 알 수 있도록 되어 있다.

8.4. Programming Model

8.4.1. Enabling VIC

VIC 가 ARM processor 로 nIRQ/nFIQ 신호를 전달할 수 있도록 하기 위해서는 INTCON 레지스터의 GIE, nENBIRQ, nENBFIQ 가 올바른 값으로 세팅 되어야 한다. GIE bit 가 1 이면 INTPND 레지스터가 clear 된 상태를 유지하고 따라서 nIRQ/nFIQ 도 high 인 상태를 유지한다. nENBIRQ bit 가 1 이면 nIRQ 가 high 로 유지되고 nENBFIQ bit 가 1 이면 nFIQ signal 이 high 로 유지되어 ARM processor 의 IRQ/FIQ exception 이 발생하지 않는다. GIE, nENBIRQ, nENBFIQ 가 모두 0 으로 세팅 되더라도 각 interrupt source 를 enable 시키기 전에는 해당

interrupt 가 발생하지 않는다. 각 interrupt source 를 enable 시키는 방법은 아래에 별도의 section 을 참조하라.

Priority based arbiter 의 동작을 변경하는 것은 VIC 를 enable 하고 수행해도 되지만, enable 시키기 전에 수행하는 것을 권장한다.

참고로 VIC 가 enable 되어 있고 nIRQ/nFIQ 신호가 low 로 떨어져 있다고 하더라도 ARM processor 의 CPSR 레지스터의 세팅이 IRQ/FIQ disable mode 라면 IRQ/FIQ exception 이 발생되지 않음을 유념하라.

8.4.2. Disabling VIC

VIC 를 disable 시키기 위해서는 INTCON 레지스터의 GIE, nENBIRQ, nENBFIQ 를 모두 1 로 세팅하면 된다. VIC 를 다시 enable 시키는 경우를 대비하여 INTMSK 레지스터의 모든 bit 를 1 로 write 해 두는 것을 권장한다.

8.4.3. Enabling Each Interrupt Source(Unmasking Interrupt)

각 interrupt source 를 enable 시키기 위해서는 우선 해당 interrupt source 에서 발생시키는 interrupt signal 이 level trigger 인지 edge trigger 인지, active high 인지 low 인지를 알아야 한다. 그 값에 따라서 LEVEL 레지스터와 POLARITY 레지스터의 해당 bit 를 알맞게 programming 해 준다. 또한 그 interrupt source 를 nIRQ 에 연결할지 아니면 nFIQ 에 연결할지 결정하여 INTMOD 레지스터의 해당 bit 를 세팅을 해 준다. 그런 다음 INTMSK 레지스터의 해당 bit 를 0 으로 clear 해 주면 된다.

모든 interrupt 가 on chip device 에서 발생되면 모든 interrupt signal 의 level/edge 특성 및 active high/low 특성을 알고 있으므로 VIC 를 enable 시킬 때 LEVEL 레지스터와 POLARITY 레지스터를 미리 setting 할 수도 있다.

8.4.4. Disabling Each Interrupt Source(Masking Interrupt)

각 interrupt source 를 disable 시키기 위해서는 INTMSK 의 해당 bit 를 1 로 세팅하기만 하면 된다.

8.4.5. Acknowledge Interrupt

IRQ 나 FIQ 가 발생하면 handler software 는 VIC 의 INTPND 의 해당 bit 를 clear 해 줄 필요가 있다. Edge trigger mode interrupt 는 INTPND 의 해당 bit 가 clear 되지 않으면 VIC hardware 는 해당 interrupt 가 처리된 것인지 알 수 없기 때문이다. 이것을 수행하는 방법은

간단하다. I_ISPC/F_ISPC 레지스터에 I_ISPR/F_ISPR 레지스터의 값을 읽어서 쓰면 된다. I_ISPR/F_ISPR 레지스터는 단 하나의 bit 만 1 인데, I_ISPC/F_ISPC 에 값을 쓰면 INTPND 레지스터의 해당 bit 가 clear 된다.

Level trigger mode interrupt 의 경우는 INTPND 의 해당 bit 를 clear 하더라도 device 에서 계속 interrupt signal 을 high 나 low 로 보내고 있으므로 acknowledge 를 하는 것이 의미가 없다. 따라서 acknowledge 를 하지 말아야 하고, 해당 device 의 interrupt handler 에서 device 가 더 이상 interrupt signal 을 보내지 않도록 하는 것으로 대신할 수 있다.

8.4.6. I_ISPR/F_ISPR, I_VECADDR/F_VECADDR 레지스터 접근

Arbiter 가 들어온 interrupt source 중에 priority 가 가장 높은 것을 선정하고, 그것을 I_ISPR/F_ISPR 레지스터에 write 하게 되면, I_ISPR/F_ISPR 의 값은 새로운 arbitration 이 이루어지기 전까지는 변경되지 않는다. 새로운 arbitration 이 이루어지는 때는 I_ISPR 혹은 F_ISPR 레지스터의 값과 INTPND 레지스터, INTMSK 레지스터를 bit-wise AND 하여 0 이 되는 경우이다.

이렇게 될 때는 사용자가 I_ISPC/F_ISPC 에 I_ISPR/F_ISPR 의 값을 write 하여 INTPND 의 해당 bit 를 clear 하는 경우 혹은 해당 interrupt source 를 masking 하는 경우, 그리고 해당 interrupt source 가 level trigger interrupt signal 을 assert 하고 있다가 deassert 하는 경우이다. 즉 acknowledge 를 하거나 아니면 masking 을 할 때 새로운 arbitration 이 발생하게 된다.

이렇게 새로운 arbitration 이 발생하면 I_ISPR/F_ISPR 의 값이 update 되므로 읽으면 새로운 interrupt source 에 해당하는 값이 나오게 된다. I_VECADDR/F_VECADDR 레지스터의 경우도 마찬가지이다. 따라서 I_ISPR/F_ISPR 또는 I_VECADDR/F_VECADDR 레지스터의 값이 추후에 필요하면 새로운 arbitration 이 일어나기 전에 읽어 둔 값을 저장하였다가 사용하여야 한다.

8.4.7. Interrupt Handler Example

IRQ 에 해당하는 Interrupt Handler 를 다음과 같이 작성할 수 있다.

```

void AckIRQ(int irq)
{
    if(edge triggered)    // DO NOT Ack if level triggered
        I_ISPC = I_ISPR;
}

void InterruptHandler(void)
{
    int irq;
    while(1)
    {
        if(I_ISPR == 0)
            break;
        irq = I_VECADDR>>2;
        // MUST DO AckIRQ before MaskIRQ
        AckIRQ(irq);    // write I_ISPR to I_ISPC only if edge triggered
        MaskIRQ(irq); // write 1 to correspondent bit of INTMSK
        // DO NOT read I_ISPR/I_VECADDR any more after AckIRQ or MaskIRQ
        Call correspondent irq service routine;
        UnmaskIRQ(irq); // write 0 to correspondent bit of INTMSK
    }
}

```

8.5. Register Description

8.5.1. Registers

Table 8-2 Registers

Offset	Name	R / W	Description	Init. value
0000h	INTCON	RW	Interrupt Control Register	0000 000Bh
0004h	INTPND	R	Interrupt Pending Register	0000 0000h
0008h	INTMOD	RW	Interrupt Mode Register	0000 0000h
000Ch	INTMSK	RW	Interrupt Mask Register	FFFF FFFFh
0010h	LEVEL	RW	Interrupt Level/Edge Selection Register	0000 0000h
0014h	I_PSLV0	RW	IRQ Slave Arbiter0 Priority Register	00FA C688h
0018h	I_PSLV1	RW	IRQ Slave Arbiter1 Priority Register	00FA C688h
001Ch	I_PSLV2	RW	IRQ Slave Arbiter2 Priority Register	00FA C688h
0020h	I_PSLV3	RW	IRQ Slave Arbiter3 Priority Register	00FA C688h
0024h	I_PMST	RW	IRQ Master Arbiter Priority Register	0000 1FE4h
0028h	I_CSLV0	R	Current IRQ Slave Arbiter0 Priority Register	00FA C688h
002Ch	I_CSLV1	R	Current IRQ Slave Arbiter1 Priority Register	00FA C688h
0030h	I_CSLV2	R	Current IRQ Slave Arbiter2 Priority Register	00FA C688h
0034h	I_CSLV3	R	Current IRQ Slave Arbiter3 Priority Register	00FA C688h
0038h	I_CMST	R	Current IRQ Master Arbiter Priority Register	0000 1FE4h
003Ch	I_ISPR	R	IRQ Interrupt Service Pending Register	0000 0000h
0040h	I_ISPC	W	IRQ Interrupt Service Clear Register	N/A
0044h	F_PSLV0	RW	FIQ Slave Arbiter0 Priority Register	00FA C688h
0048h	F_PSLV1	RW	FIQ Slave Arbiter1 Priority Register	00FA C688h
004Ch	F_PSLV2	RW	FIQ Slave Arbiter2 Priority Register	00FA C688h
0050h	F_PSLV3	RW	FIQ Slave Arbiter3 Priority Register	00FA C688h

0054h	F_PMST	RW	FIQ Master Arbiter Priority Register	0000 1FE4h
0058h	F_CSLV0	R	Current FIQ Slave Arbiter0 Priority Register	00FA C688h
005Ch	F_CSLV1	R	Current FIQ Slave Arbiter1 Priority Register	00FA C688h
0060h	F_CSLV2	R	Current FIQ Slave Arbiter2 Priority Register	00FA C688h
0064h	F_CSLV3	R	Current FIQ Slave Arbiter3 Priority Register	00FA C688h
0068h	F_CMST	R	Current FIQ Master Arbiter Priority Register	0000 1FE4h
006Ch	F_ISPR	R	FIQ Interrupt Service Pending Register	0000 0000h
0070h	F_ISPC	W	FIQ Interrupt Service Clear Register	N/A
0074h	POLARITY	RW	Interrupt Polarity Register	0000 0000h
0078h	I_VECADDR	R	IRQ Vector Address Register	0000 0000h
007Ch	F_VECADDR	R	FIQ Vector Address Register	0000 0000h

8.5.2. INTCON Register

INTCON 레지스터는 VIC 의 전체 동작을 enable/disable 시키기 위한 레지스터이다.

Figure 8-3 INTCON Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															

Table 8-3 INTCON Register Description(Offset : 0000h)

Bits	Name	R / W	Description	Init. value
[3]	GIE	RW	Global Interrupt Enable 0 : VIC Enable 1 : VIC Disable	1b
[1]	nENBIRQ	RW	nIRQ Enable 0 : nIRQ generation enable 1 : nIRQ generation disable	1b
[0]	nENBFIQ	RW	nFIQ Enable 0 : nFIQ generation enable 1 : nFIQ generation disable	1b

VIC 를 enable 시키기 위해서는 GIE bit 를 '0'으로 세팅해야 한다. GIE bit 가 '1'이면 INTPND 가 0 으로 clear 되고 더 이상 interrupt 가 발생하지 않게 된다. nIRQ/nFIQ 에 대해서 각각 별도의 enable bit 인 nENBIRQ, nENBFIQ 가 있다.

8.5.3. INTPND Register

INTPND 레지스터는 Read Only 레지스터로 현재 Pending 되어 있는 Interrupt 상황을 알 수 있다. INTPND 는 masking 이 이루어지기 전의 결과로, read 하여 사용하는 것을 권장하지 않는다.

Pending 된 interrupt 를 확인 하기 위해서는 I_ISPR/F_ISPR 혹은 I_VECADDR/F_VECADDR 레지스터를 읽는 것을 권장한다.

I_ISPC/F_ISPC 에 값을 write 하여 INTPND 의 특정 bit 를 clear 할 수 있다.

Figure 8-4 INTPND Register

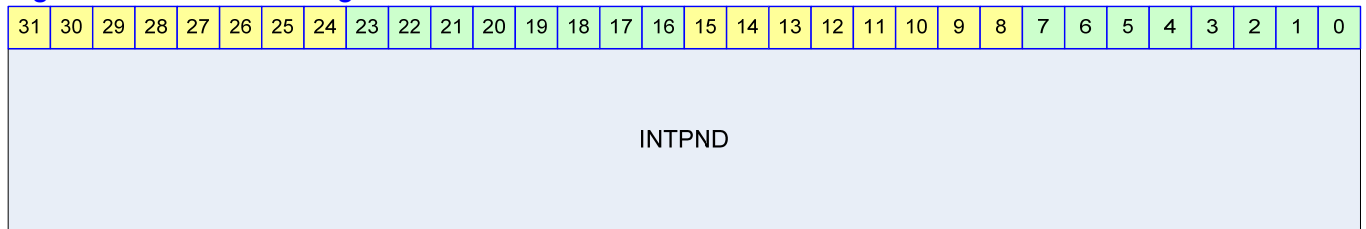


Table 8-4 INTPND Register Description(Offset : 0004h)

Bits	Name	R / W	Description	Init. value
[31:0]	INTPND	R	Interrupt Pending Status 0 : no interrupt pended 1 : interrupt pended 각 bit 는 각 interrupt source 에 대응한다.	0000 0000h

8.5.4. INTMOD Register

INTMOD 레지스터는 각 interrupt source 를 nIRQ 에 연결할지 nFIQ 에 연결할지를 나타내는 레지스터이다.

Figure 8-5 INTMOD Register

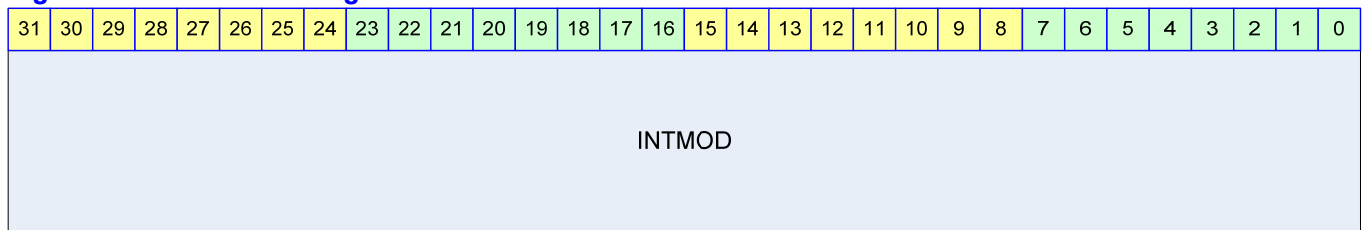


Table 8-5 INTMOD Register Description(Offset : 0008h)

Bits	Name	R / W	Description	Init. Value
[31:0]	INTMOD	RW	Interrupt Mode Register 0 : Interrupt 를 nIRQ 로 연결 1 : Interrupt 를 nFIQ 로 연결 각 bit 는 각 interrupt source 에 대응한다.	0000 0000h

8.5.5. INTMSK Register

INTMSK 레지스터는 각 Interrupt 를 masking 하기 위해서 사용한다.

Figure 8-6 INTMSK Register

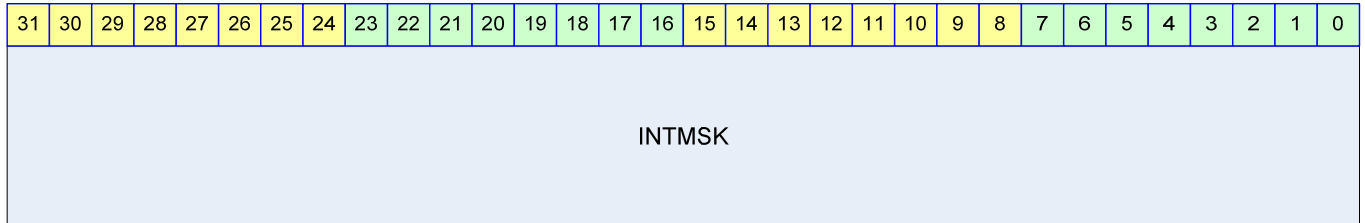


Table 8-6 INTMSK Register Description(Offset : 000Ch)

Bits	Name	R / W	Description	Init. Value
[31:0]	INTMSK	RW	Interrupt Mask Register 0 : Interrupt Unmasked(enabled) 1 : Interrupt Masked 각 bit 는 각 interrupt source 에 대응한다.	FFFF FFFFh

8.5.6. LEVEL Register

LEVEL 레지스터는 각 interrupt source 가 level trigger interrupt signal 을 사용하는지 아니면 edge trigger interrupt signal 을 사용하는지 결정한다.

Figure 8-7 LEVEL Register

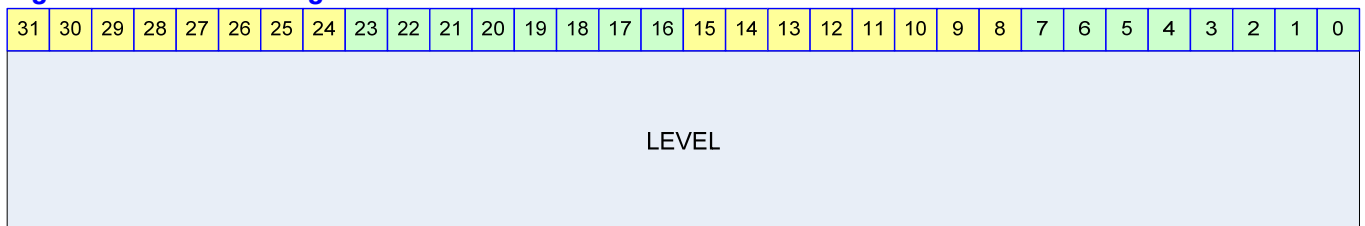


Table 8-7 LEVEL Register Description(Offset : 0010h)

Bits	Name	R / W	Description	Init. Value
[31:0]	LEVEL	RW	Interrupt Level Register 0 : Interrupt signal is edge triggered 1 : Interrupt signal is level triggered 각 bit 는 각 interrupt source 에 대응한다.	0000 0000h

8.5.7. POLARITY Register

POLARITY 레지스터는 Interrupt signal 의 polarity 를 설정하는 레지스터이다.

Figure 8-8 POLARITY Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
POLARITY																															

Table 8-8 POLARITY Register Description(Offset : 0074h)

Bits	Name	R / W	Description	Init. Value
[31:0]	POLARITY	RW	Interrupt Polarity Register 0 : Interrupt signal is active high 1 : Interrupt signal is active low 각 bit 는 각 interrupt source 에 대응한다.	0000 0000h

8.5.8. PSLV Register

PSLV 레지스터는 Slave Arbiter 에 대한 priority setting 을 위한 레지스터이다. IRQ path 에 총 4 개의 slave arbiter 가 있으므로, I_PSLV0, I_PSLV1, I_PSLV2, I_PSLV3 레지스터가 존재하고, FIQ 쪽도 F_PSLV0, F_PSLV1, F_PSLV2, F_PSLV3 레지스터가 존재한다.

Figure 8-9 PSLV Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								PR7		PR6		PR5		PR4		PR3		PR2		PR1		PR0									

Table 8-9 PSLV Register Description

Bits	Name	R / W	Description	Init. Value
[23:21]	PR7	RW	Interrupt Priority for Interrupt Source 7	111b
[20:18]	PR6	RW	Interrupt Priority for Interrupt Source 6	110b
[17:15]	PR5	RW	Interrupt Priority for Interrupt Source 5	101b
[14:12]	PR4	RW	Interrupt Priority for Interrupt Source 4	100b
[11:9]	PR3	RW	Interrupt Priority for Interrupt Source 3	011b
[8:6]	PR2	RW	Interrupt Priority for Interrupt Source 2	010b
[5:3]	PR1	RW	Interrupt Priority for Interrupt Source 1	001b
[2:0]	PR0	RW	Interrupt Priority for Interrupt Source 0	000b

			여기서 Interrupt Source 0 의 의미는 I_PSLV0/F_PSLV0 는 Interrupt Source 0 가 32 개의 interrupt source 중 0 번을 의미하고, I_PSLV1/F_PSLV1 은 interrupt source 8 번, I_PSLV2/F_PSLV2 는 interrupt source 16 번, I_PSLV3/F_PSLV3 은 interrupt source 24 번을 의미한다. Priority Setting 은 가장 높은 priority 는 000b 이고 111b 가 가장 낮은 priority 를 의미한다.	
--	--	--	---	--

PR0 에서 PR7 까지 8 개의 priority 는 모두 다른 값으로 세팅되어야 한다. 그렇지 않으면 VIC 가 오동작을 하게 된다.

8.5.9. PMST Register

PMST 레지스터는 Master arbiter 에 대한 priority 를 세팅하고 arbitration mode 를 바꾸기 위한 레지스터이다. IRQ/FIQ 에 대해 레지스터가 각각 존재하고 I_PMST/F_PMST 이다.

Figure 8-10 PMST Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																				MMode	SMode3	SMode2	SMode1	SMode0	PR3	PR2	PR1	PR0			

Table 8-10 PMST Register Description

Bits	Name	R / W	Description	Init. Value
[12]	MMode	RW	Master Arbiter Mode 0 : Round robin arbitration 1 : Fixed priority arbitration	1b
[11]	SMode3	RW	Slave Arbiter 3 Mode 0 : Round robin arbitration 1 : Fixed priority arbitration	1b
[10]	SMode2	RW	Slave Arbiter 2 Mode 0 : Round robin arbitration 1 : Fixed priority arbitration	1b
[9]	SMode1	RW	Slave Arbiter 1 Mode 0 : Round robin arbitration 1 : Fixed priority arbitration	1b
[8]	SMode0	RW	Slave Arbiter 0 Mode 0 : Round robin arbitration 1 : Fixed priority arbitration	1b
[7:6]	PR3	RW	Priority for Slave Arbitration Result 3	11b
[5:4]	PR2	RW	Priority for Slave Arbitration Result 2	10b
[3:2]	PR1	RW	Priority for Slave Arbitration Result 1	01b

[1:0]	PR0	RW	Priority for Slave Arbitration Result 0 Master Arbiter 는 4 개의 Slave Arbiter 의 결과에 대한 arbitration 을 수행하는데 각 Slave Arbiter 의 결과에 대해 priority 를 setting 할 수 있다. Priority Setting 은 가장 높은 priority 는 00b 이고 11b 가 가장 낮은 priority 를 의미한다.	00b
-------	-----	----	--	-----

PR0 에서 PR3 까지 4 개의 priority 는 모두 다른 값으로 세팅되어야 한다. 그렇지 않으면 VIC 가 오동작을 하게 된다.

8.5.10. CSLV Register

CSLV 레지스터는 Slave Arbiter 에 대한 priority 가 어떻게 변하는지 볼 수 있는 Read Only 레지스터이다. Arbitration mode 가 round robin 인 경우 arbitration 마다 priority 가 변하게 되는데, 이것을 monitoring 할 수 있는 레지스터이다. Fixed priority arbitration 을 선택한 경우 CSLV 의 값은 PSLV 와 같은 값으로 변하지 않게 된다. 또한 PSLV 레지스터에 값을 쓰게 되면 CSLV 는 해당하는 값으로 자동으로 update 된다.

IRQ path 에 총 4 개의 slave arbiter 가 있으므로, I_CSLV0, I_CSLV1, I_CSLV2, I_CSLV3 레지스터가 존재하고, FIQ 쪽도 F_CSLV0, F_CSLV1, F_CSLV2, F_CSLV3 레지스터가 존재한다.

Figure 8-11 CSLV Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								PR7		PR6		PR5		PR4		PR3		PR2		PR1		PR0									

Table 8-11 CSLV Register Description

Bits	Name	R / W	Description	Init. Value
[23:21]	PR7	R	Interrupt Priority for Interrupt Source 7	111b
[20:18]	PR6	R	Interrupt Priority for Interrupt Source 6	110b
[17:15]	PR5	R	Interrupt Priority for Interrupt Source 5	101b
[14:12]	PR4	R	Interrupt Priority for Interrupt Source 4	100b
[11:9]	PR3	R	Interrupt Priority for Interrupt Source 3	011b
[8:6]	PR2	R	Interrupt Priority for Interrupt Source 2	010b
[5:3]	PR1	R	Interrupt Priority for Interrupt Source 1	001b
[2:0]	PR0	R	Interrupt Priority for Interrupt Source 0 여기서 Interrupt Source 0 의 의미는 I_PSLV0/F_PSLV0 는 Interrupt Source 0 가 32 개의 interrupt source 중 0 번을 의미하고, I_PSLV1/F_PSLV1 은 interrupt source 8 번, I_PSLV2/F_PSLV2 는 interrupt source 16 번,	000b

			I_PSLV3/F_PSLV3 은 interrupt source 24 번을 의미한다. Priority Setting 은 가장 높은 priority 는 000b 이고 111b 가 가장 낮은 priority 를 의미한다.	
--	--	--	---	--

8.5.11. CMST Register

Master Arbiter 가 round robin mode 로 세팅되어 있는 경우에는 매 arbitration 마다 priority 가 변하게 되어 있는데 그것을 monitoring 할 수 있는 Read Only 레지스터이다. IRQ/FIQ 에 대해 레지스터가 각각 존재하고 I_CMST/F_CMST 이다.

Figure 8-12 CMST Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																								PR3		PR2		PR1		PR0	

Table 8-12 CMST Register Description

Bits	Name	R / W	Description	Init. Value
[7:6]	PR3	R	Priority for Slave Arbitration Result 3	11b
[5:4]	PR2	R	Priority for Slave Arbitration Result 2	10b
[3:2]	PR1	R	Priority for Slave Arbitration Result 1	01b
[1:0]	PR0	R	Priority for Slave Arbitration Result 0 Master Arbiter 는 4 개의 Slave Arbiter 의 결과에 대한 arbitration 을 수행하는데 각 Slave Arbiter 의 결과에 대해 priority 를 setting 할 수 있다. Priority Setting 은 가장 높은 priority 는 00b 이고 11b 가 가장 낮은 priority 를 의미한다.	00b

8.5.12. ISPR Register

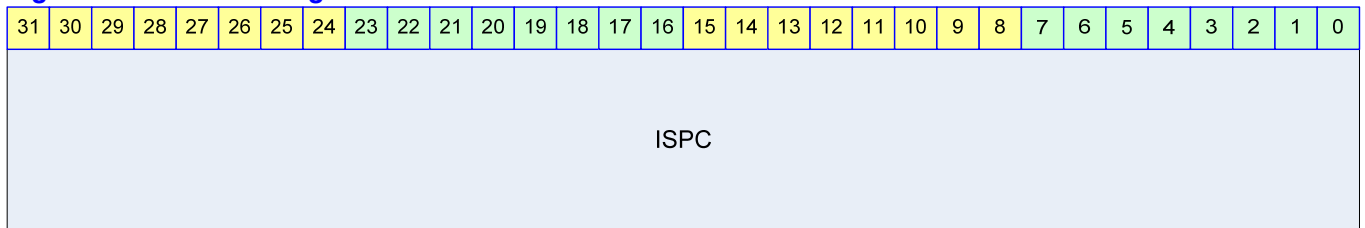
ISPR 레지스터는 arbitration 결과 선택된 interrupt source bit 를 알 수 있는 Read Only 레지스터이다. 현재 pending 되어 있는 interrupt 중에 priority 가 가장 높은 interrupt 가 선택되어 한 bit 만 set 되게 된다. 하지만 이 레지스터를 이용하여 interrupt source 번호를 찾을 필요는 없고 interrupt 를 acknowledge 하기 위해서 사용된다. IRQ/FIQ 각각에 대해서 I_ISPR/F_ISPR 이 존재한다.

Figure 8-13 ISPR Register**Table 8-13 ISPR Register Description**

Bits	Name	R / W	Description	Init. Value
[31:0]	ISPR	R	Interrupt Service Pending Register 0 : Interrupt is not pended 1 : Interrupt is pended 각 bit 는 각 interrupt source 에 대응하고 단 하나의 bit 만 set 된다.	0000 0000h

8.5.13. ISPC Register

ISPC 레지스터는 pending 된 interrupt 를 clear 하기 위해서 사용하는 Write Only 레지스터이다. ISPC 에 값을 쓰면 1 값을 가지는 bit 위치마다 INTPND 레지스터가 clear 된다. 현재 service 할 interrupt source 에 acknowledge 를 하기 위해서 사용하고 ISPR 레지스터의 값을 ISPC 에 write 하는 것이 보통이다. 현재 pending 된 interrupt 가 INTPND 에서 clear 되면 새로운 arbitration 이 시작된다. IRQ/FIQ 각각 I_ISPC/F_ISPC 레지스터로 존재한다.

Figure 8-14 ISPC Register**Table 8-14 ISPC Register Description**

Bits	Name	R / W	Description	Init. Value
[31:0]	ISPC	W	Interrupt Service Pending Clear Register Write 0 : No meaning Write 1 : correspondent bit of INTPND cleared 각 bit 는 각 interrupt source 에 대응한다.	N/A

8.5.14. VECADDR Register

VECADDR 레지스터는 현재 arbitration 된 결과 pending 된 interrupt 의 vector address 를 찾기 위한 Read Only 레지스터이다. ISPR 에 한 bit 가 set 되어도 software 가 해당 bit 를 찾기

위해서는 loop 을 돌거나 많은 instruction 을 수행해야 한다. – 참고로 ARM 의 clz(counting leading zero)와 같은 instruction 을 수행하면 쉽게 할 수 있기는 하다. Software 는 ISPR 레지스터를 읽어서 현재 pending 된 interrupt 의 번호를 체크할 필요가 없이 VECADDR 레지스터를 읽어서 interrupt 의 번호를 알 수 있다. 또한 interrupt service routine table address 에 VECADDR 에서 읽은 값을 더하기만 하더라도 해당 interrupt service routine 을 찾아낼 수 있다.

IRQ/FIQ 각각 I_VECADDR/F_VECADDR 레지스터로 존재한다. 단, ISPR 의 값이 0 이면 VECADDR 에서 읽은 값은 의미가 없음을 유의하라.

Figure 8-15 VECADDR Register

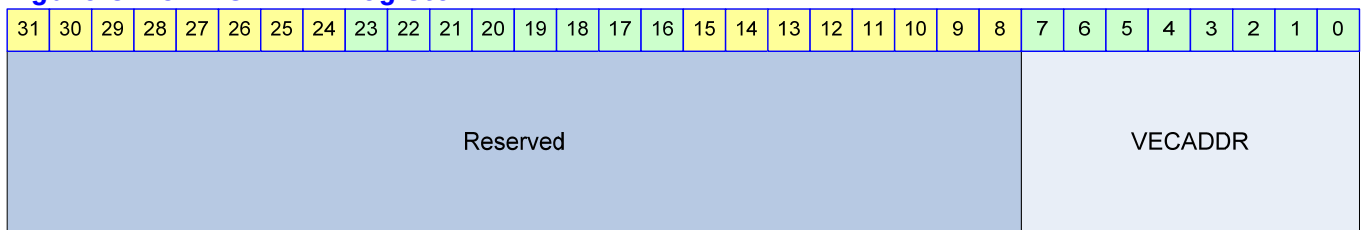


Table 8-15 VECADDR Register Description

Bits	Name	R / W	Description	Init. Value
[7:0]	VECADDR	R	Pending Interrupt Vector Address Interrupt Source 0 이 Pending 이면 0x00, Interrupt Source 1 이 Pending 이면 0x04, Interrupt Source 2 이 Pending 이면 0x08, Interrupt Source 3 이 Pending 이면 0x0C, ... Interrupt Source 31 이 Pending 이면 0x7C 의 값을 가진다.	00h

9. Enhanced-DMA

9.1. Overview

DMA(Direct Memory Access) Controller 는 기본적으로 Memory 와 onchip Peripheral 사이의 data 전송을 관장한다. Onchip peripheral 은 data 를 load 할 필요가 있거나 store 해야할 필요가 있는 경우에 DMA Controller 에 request 를 생성하고 DMA Controller 는 해당 request 를 받으면 memory 와 peripheral 사이의 data 전송을 service 하게 된다. 설계된 DMA Controller 는 총 8 개의 channel 이 존재하고, 4 개의 Channel 을 가지는 DMA Engine 이 두 개 들어 있는 구조를 가지고 있어 2 개의 독립적인 data 전송이 가능하다.

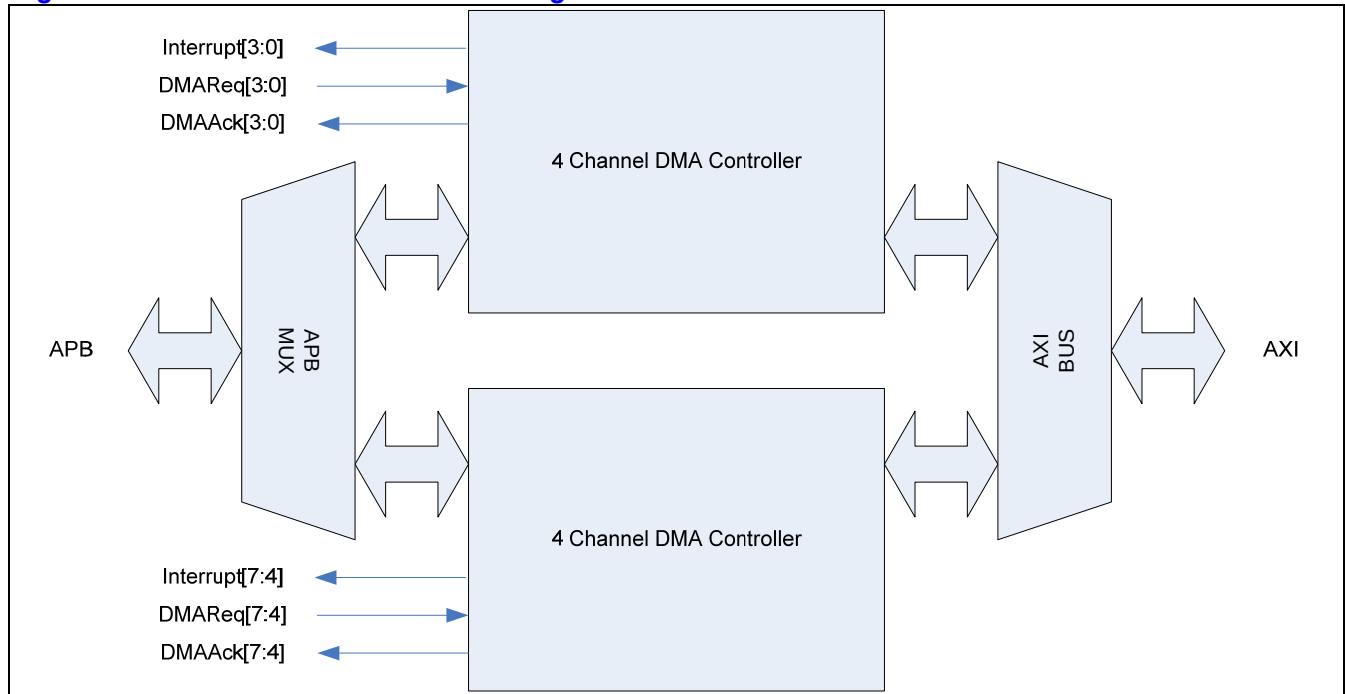
9.1.1. Features

- 8 Channel Support
 - 2 x 4 Channel DMA Engine
 - 2 independent data transfer at a time
- Round-robin arbitration
- Source BUS width : BYTE/HWORD/WORD support
- Destination BUS width : BYTE/HWORD/WORD support
- Byte 단위의 transfer count
- Automatic Pack/Unpack
- Unaligned address support
 - 단, Destination address가 unaligned인 경우에는 ByteEnable신호를 받을 수 있는 Slave이어야 한다.
- Descriptor를 이용한 Linked list scatter/gather DMA support
- Memory-to-Memory copy support
- Programmable data-burst size(1/2/4/8/16/32 byte)
- Up to (1M -1) bytes of data transfer per descriptor
- Only little endian support

9.1.2. Block diagram

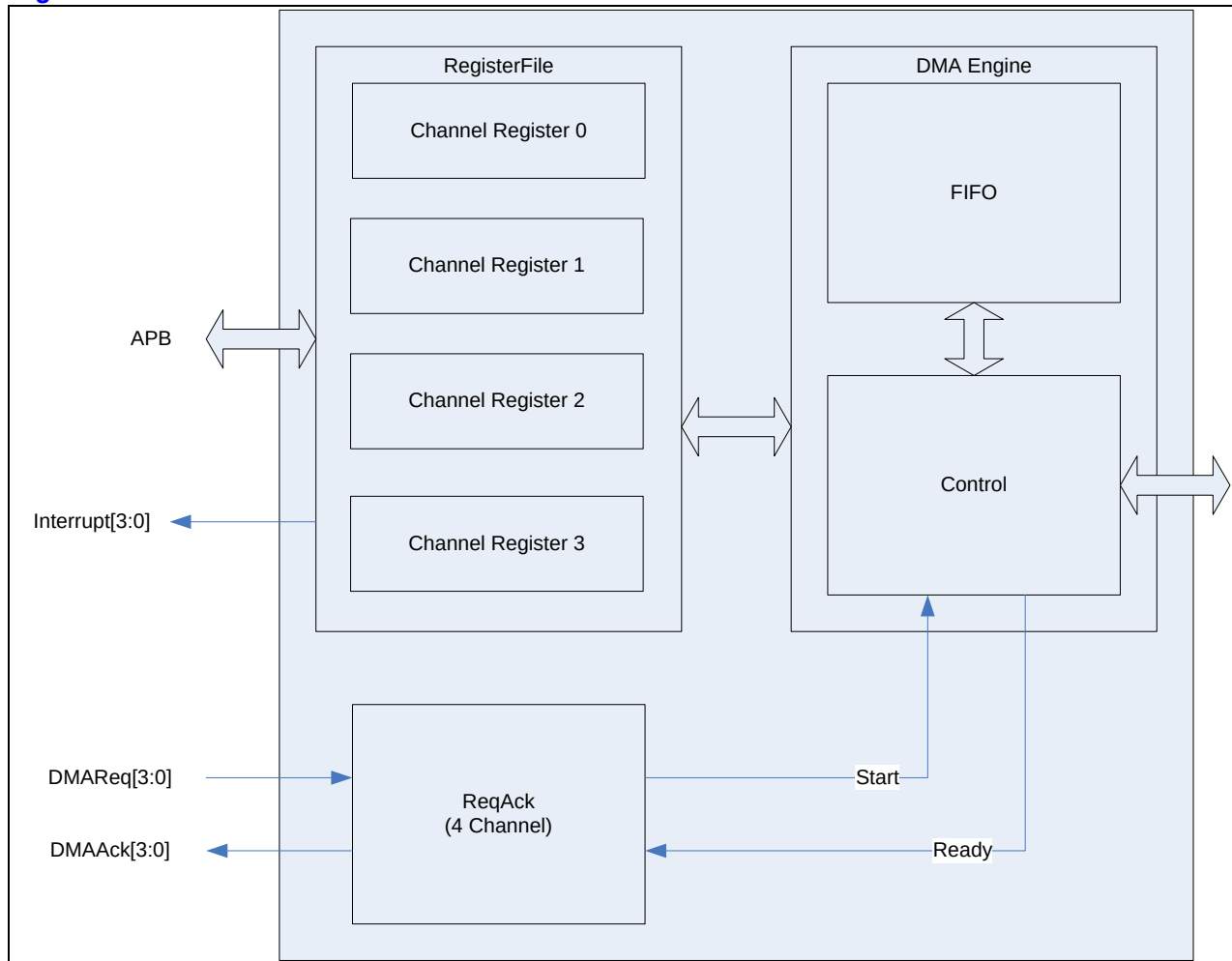
전체적인 하드웨어 구성은 다음 그림과 같다.

Figure 9-1 DMA Controller Block Diagram



4 Channel DMA Controller 가 두 개 들어 있고, 그것이 BUS MUX 로 연결되어 있다. 각 Channel 별로 DMA Request 신호를 받고 DMA service 의 시작과 끝을 알리는 Acknowledge 신호가 peripheral 로 출력된다. 각 channel 의 동작의 시작과 끝 혹은 error 상황을 알려주기 위해서 8 개의 interrupt line 이 출력된다. 4 Channel DMA Controller 의 내부 구조는 다음과 같다.

Figure 9-2 4 Channel DMA Controller의 내부구조



4 Channel DMA Controller 는 4 개의 channel 마다 register set 가 존재하고, DMA Request 시에는 하나의 Channel register 가 enable 되어 DMA Engine 이 동작하도록 되어있다. 각 channel 별로 DMA Engine 를 가지고 있는 것보다 하드웨어 복잡도가 낮아지지만 4 Channel 의 DMA 가 동시에 enable 될 수는 없다.

9.2. DMA Channel

각 channel 에 할당 된 onchip Peripheral 은 System Register 의 DMA Channel Mux. 부분을 참조 하면 된다.

9.2.1. Channel Arbitration

하나의 DMA Engine 에 물려 있는 4 개의 Channel 에 동시에 request 가 오는 경우에는 round-robin scheme 을 이용하여 arbitration 하게 된다. 즉, request 에 해당 하는 service 가 끝난 DMA channel 이 가장 low priority 를 가지게 된다.

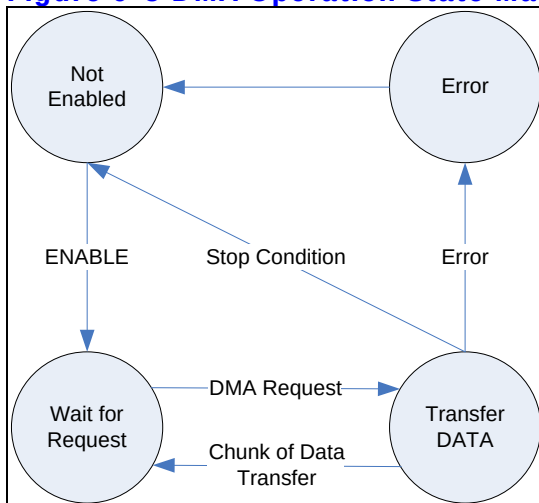
Channel 0/1/2/3 를 처리하는 DMA Engine 과 Channel 4/5/6/7 을 처리하는 DMA Engine 사이에는 항상 Channel 0/1/2/3 의 DMA Engine 이 높은 우선 순위를 가지고 BUS 를 점유하게 된다. 하지만 DMA Controller 가 AXI BUS 의 out-of-order transfer 를 지원하기 때문에 BUS 점유가 data transfer 의 순서와 큰 연관 관계를 맺지 않고 효율적인 data transfer 가 일어나게 된다.

9.3. Channel Operation

9.3.1. Basic Operation

각 Channel 의 기본적인 동작은 다음과 같은 state machine 으로 설명할 수 있다.

Figure 9-3 DMA Operation State Machine



- Not Enabled State

초기 상태에서는 DMA 가 Enable 되지 않는다.

- Wait for Request State

DMA 가 enable 되면 외부의 DMA Request 를 기다리게 된다.

- Transfer Data State

DMA request 가 들어오면 Source Address 레지스터가 지정하고 있는 address 에서 일정한 양의 data 를 읽어 Destination Address 레지스터가 지정하고 있는 address 에 복사를 한다. 복사후에

stop condition 이 발생하면 Stop Interrupt 신호가 생성되고 Not Enabled State 로 돌아가게 되고, 아니면 Wait for Request State 로 돌아가게 된다. Data 의 transfer 는 source address 에서 destination address 로 일어나게 되며 한번에 전송되는 data size 는 control 레지스터의 Size field 에 의해 결정된다.

- Error State

레지스터 세팅이나 Data 전송 시에 오류가 발생하면 Error State 가 되어 interrupt 가 발생하고 Not Enabled State 로 돌아가게 된다.

Channel Enable 을 enable 시키기 위해서는 Status 레지스터의 enable bit 를 '1'로 setting 해 주고 Pending interrupt 를 모두 clear 해 주면 된다.

DMA Channel 의 stop condition 은 Control register 의 TransferCount field 가 0 이 되고 Descriptor 레지스터의 DescriptorEnd bit 가 '1'인 경우이다.

9.3.2. Descriptor Loading

Transfer Data State 에서 Control 레지스터의 TransferLength field 가 0 이 되면 일단 DMA Request 에 대응하는 data transfer 는 종료된다. Descriptor 레지스터의 DescriptorEnd bit 가 '0'인 경우에는 종료된 후 새롭게 DMA Request 신호가 들어오면 Descriptor 가 loading 이 일어나고 새로운 data transfer 가 발생하게 된다. Descriptor Loading 이란 Memory 에서 4 개의 Word 를 읽어서 Source Address 레지스터, Destination Address 레지스터, Control 레지스터, Descriptor 레지스터를 update 하는 과정을 의미한다. 새로운 Descriptor 의 위치는 Descriptor 레지스터의 상위 30bit 로 addressing 되는데, Descriptor 의 내용은 다음과 같은 4 Word 로 구성된다.

Table 9-1 Descriptor

Word	Name	Description
1 번째 Word	SourceAddr	Source Address 레지스터를 update 할 내용이다.
2 번째 Word	DestAddr	Destination Address 레지스터를 update 할 내용이다.
3 번째 Word	Control	Control 레지스터를 update 할 내용이다. 참고로 TransferLength 가 0 이 되도록 하면 안된다.
4 번째 Word	Descriptor	Descriptor 레지스터를 update 할 내용이다.

이렇게 새로운 Descriptor 를 계속 Loading 하여 사용자는 Scatter/gather DMA 를 할 수 있다.

9.3.3. Interrupt

DMA Controller 의 interrupt line 은 각 channel 별로 존재하고 Active high level sensitive interrupt signal 을 생성한다. 각 channel 별로 발생하는 interrupt 는 다음과 같은 4 가지 상황에서 발생하게 된다.

(a) Error Interrupt

BUS Error 를 detect 하거나 사용자의 레지스터 세팅에 오류가 있는 경우, Status 레지스터의 ErrInt bit 가 '1'로 set 되고 interrupt 가 발생한다. Error Interrupt 가 발생하면 해당 채널의 state 는 Not Enabled State 로 변경되고 더 이상 data transfer 가 일어나지 않는다. Not Enabled State 로 변경되더라도 Status 레지스터의 Enable bit 은 '0'으로 clear 되지 않음을 유의하라.

(b) Stop Interrupt

Control 레지스터의 TransferLength field 가 0 이 되고 더 이상 load 할 descriptor 가 없는 경우(즉, Descriptor 레지스터의 DescriptorEnd bit 가 '1'인 경우) Stop Interrupt 가 발생한다. 이 interrupt 는 Status 레지스터의 StopIntEn bit 로 masking 을 할 수 있는데 masking 이 되어도 Stop Interrupt 상황이면 Status 레지스터의 StopInt bit 가 '1'로 set 되고 해당 채널의 state 는 Not Enabled State 로 변경되어 더 이상 data transfer 가 일어나지 않는다. Not Enabled State 로 변경되더라도 Status 레지스터의 Enable bit 은 '0'으로 clear 되지 않음을 유의하라.

(c) End Interrupt

Control 레지스터의 TransferLength field 가 0 이 되면 End Interrupt 가 발생한다. 이 interrupt 는 Control 레지스터의 EndIntEn bit 로 masking 이 가능하다. End Interrupt 가 발생하면 Status 레지스터의 EndInt bit 이 '1'로 set 되지만 해당 채널의 data transfer 는 계속 진행된다.

(d) Start Interrupt

새로운 Descriptor 가 성공적으로 Loading 된 경우 Start Interrupt 가 발생한다. 이 interrupt 는 Control 레지스터의 StartIntEn bit 로 masking 이 가능하다. Start Interrupt 가 발생하면 Status 레지스터의 StartInt bit 가 '1'로 set 되지만 해당 채널의 data transfer 는 계속 진행된다.

9.3.4. Trailing Bytes

Control 레지스터의 TransferLength field 는 현재의 Descriptor 를 이용하여 전송할 총 byte 수를 세팅하게 되어 있다. DMA Engine 은 매 전송마다 TransferLength field 를 감소시키고, TransferLength field 를 현재의 descriptor 에서 전송을 기다리는 byte 수로 인식하게 된다. 또한 Control 레지스터의 Size field 를 이용하여 한번에 전송할 byte 수를 결정할 수 있다. 어떤

시점에서 Size field 로 결정된 byte 수보다 TransferLength field 로 결정된 byte 수가 작으면 TransferLength field 로 결정된 byte 수 만큼 전송하게 된다.

예를 들어 Size field 로 한번의 전송으로 32byte 씩 전송하도록 세팅되어 있고, TransferLength field 의 값이 15 byte 라고 하면 15 byte 만 전송한다.

9.3.5. Memory-to-Memory Transfer

DMA Controller 를 이용하여 memory 에 있는 data 를 memory 로 빠르게 전송할 수 있다. 이런 기능을 하기 위해서는 Control 레지스터의 M2M bit 를 '1'로 세팅하면 된다. 이 경우 peripheral 에서 오는 DMA request 는 무시되고, DMA Request 가 항상 assert 된 것처럼 동작하게 된다.

9.4. Software Model

9.4.1. Enabling DMA Channel

DMA Channel 을 enable 하기 위해서는 Status 레지스터의 Enable bit 를 '1'로 세팅하고 StopInt bit 과 ErrInt bit 을 '0'으로 clear 해 주면 된다. 당연히 해당 DMA Channel 이 올바르게 동작하려면 Source Address 레지스터, Destination Address 레지스터, Control 레지스터, Descriptor 레지스터가 올바르게 세팅된 후 enable 하여야 한다.

9.4.1.1. Enabling DMA Channel(No Descriptor Mode)

Descriptor 를 사용하지 않는 경우 다음과 같이 레지스터를 세팅하면 된다.

- (a) Source Address 레지스터/Destination Address 레지스터를 원하는 값으로 세팅한다.
- (b) Control 레지스터를 세팅하는데 TransferLength 는 반드시 0 보다 큰 값을 가져야 한다.
- (c) Descriptor 레지스터의 DescriptorEnd bit 를 '1'로 세팅한다.
- (d) Status 레지스터를 세팅하여 해당 channel 을 enable 시킨다.

9.4.1.2. Enabling DMA Channel(Using Descriptor)

Descriptor 를 사용하여 DMA 를 동작시키기 위해서는 다음과 같이 레지스터를 세팅할 수 있다.

- (a) memory 의 특정 위치에 Descriptor 를 Linked list 로 연결되도록 4 word 씩 write 한다.
- (b) Control 레지스터의 TransferLength 를 0 으로 write 한다. Memory-to-Memory 전송이라면 M2M bit 를 '1'로 세팅한다.

- (c) Descriptor 레지스터에 첫 번째 Descriptor 가 write 된 address 를 세팅한다. 이때 DescriptorEnd bit 는 '0'으로 세팅한다.
- (d) Status 레지스터를 세팅하여 해당 channel 을 enable 시킨다.

당연히 첫 번째 Descriptor 를 memory 에 write 하지 않고 직접 레지스터에 write 해도 된다.
이때는 두 번째 Descriptor 의 address 를 Descript 레지스터에 세팅하면 된다.

9.4.2. Disabling DMA Channel

위에서 언급한 대로 Error Interrupt 나 Stop Interrupt 가 발생하면 해당 channel 의 state 는 Not Enabled State 로 변경되고 자동으로 해당 채널이 Disable 된다. 사용자가 강제로 동작하고 있는 DMA channel 을 disable 하기 위해서는 다음과 같은 절차를 거쳐야 한다.

- (a) Status 레지스터의 Enable bit 을 '0'으로 clear 해 준다.
- (b) Status 레지스터의 Active bit 을 polling 하여 '0'이 될 때까지 기다린다.
- Active bit 은 현재 data transfer 가 진행중임을 나타내는 bit 로, data transfer 중간에 다른 레지스터를 access 하면 안되기 때문에 polling 을 하여 data transfer 가 끝났는지 detect 하는 것이다.

9.4.3. Restriction on Register Access

Status 레지스터를 제외하고는 Channel 이 enable 된 상태에서 레지스터에 write 를 시도하면 안된다. Data transfer 가 일어나는 도중 레지스터를 write 하게 되면 DMA Controller 가 오동작 할 수 있기 때문이다.

9.5. Register Description

9.5.1. Registers

Table 9-2 Registers

Offset	Name	R / W	Description	Init. value
0000h	SRC_CH0	RW	Source Address(Channel 0)	0000 0000h
0004h	DEST_CH0	RW	Destination Address(Channel 0)	0000 0000h
0008h	CTRL_CH0	RW	Control(Channel 0)	0000 0000h
000Ch	DESC_CH0	RW	Descriptor(Channel 0)	0000 0000h
0010h	STAT_CH0	RW	Status(Channel 0)	0000 0000h
0020h	SRC_CH1	RW	Source Address(Channel 1)	0000 0000h
0024h	DEST_CH1	RW	Destination Address(Channel 1)	0000 0000h
0028h	CTRL_CH1	RW	Control(Channel 1)	0000 0000h
002Ch	DESC_CH1	RW	Descriptor(Channel 1)	0000 0000h

0030h	STAT_CH1	RW	Status(Channel 1)	0000 0000h
0040h	SRC_CH2	RW	Source Address(Channel 2)	0000 0000h
0044h	DEST_CH2	RW	Destination Address(Channel 2)	0000 0000h
0048h	CTRL_CH2	RW	Control(Channel 2)	0000 0000h
004Ch	DESC_CH2	RW	Descriptor(Channel 2)	0000 0000h
0050h	STAT_CH2	RW	Status(Channel 2)	0000 0000h
0060h	SRC_CH3	RW	Source Address(Channel 3)	0000 0000h
0064h	DEST_CH3	RW	Destination Address(Channel 3)	0000 0000h
0068h	CTRL_CH3	RW	Control(Channel 3)	0000 0000h
006Ch	DESC_CH3	RW	Descriptor(Channel 3)	0000 0000h
0070h	STAT_CH3	RW	Status(Channel 3)	0000 0000h
0080h	SRC_CH4	RW	Source Address(Channel 4)	0000 0000h
0084h	DEST_CH4	RW	Destination Address(Channel 4)	0000 0000h
0088h	CTRL_CH4	RW	Control(Channel 4)	0000 0000h
008Ch	DESC_CH4	RW	Descriptor(Channel 4)	0000 0000h
0090h	STAT_CH4	RW	Status(Channel 4)	0000 0000h
00A0h	SRC_CH5	RW	Source Address(Channel 5)	0000 0000h
00A4h	DEST_CH5	RW	Destination Address(Channel 5)	0000 0000h
00A8h	CTRL_CH5	RW	Control(Channel 5)	0000 0000h
00ACh	DESC_CH5	RW	Descriptor(Channel 5)	0000 0000h
00B0h	STAT_CH5	RW	Status(Channel 5)	0000 0000h
00C0h	SRC_CH6	RW	Source Address(Channel 6)	0000 0000h
00C4h	DEST_CH6	RW	Destination Address(Channel 6)	0000 0000h
00C8h	CTRL_CH6	RW	Control(Channel 6)	0000 0000h
00CCh	DESC_CH6	RW	Descriptor(Channel 6)	0000 0000h
00D0h	STAT_CH6	RW	Status(Channel 6)	0000 0000h
00E0h	SRC_CH7	RW	Source Address(Channel 7)	0000 0000h
00E4h	DEST_CH7	RW	Destination Address(Channel 7)	0000 0000h
00E8h	CTRL_CH7	RW	Control(Channel 7)	0000 0000h
00ECh	DESC_CH7	RW	Descriptor(Channel 7)	0000 0000h
00F0h	STAT_CH7	RW	Status(Channel 7)	0000 0000h

8 개 Channel 전체 레지스터의 map 은 위의 그림과 같다. 각 Channel 별 레지스터에 대해서 살펴보자.

9.5.2. Source Address Register

Source Address 레지스터는 말 그대로 복사가 일어날 Source 의 address 를 지칭하기 위한 레지스터이다.

Figure 9-4 Source Address Register

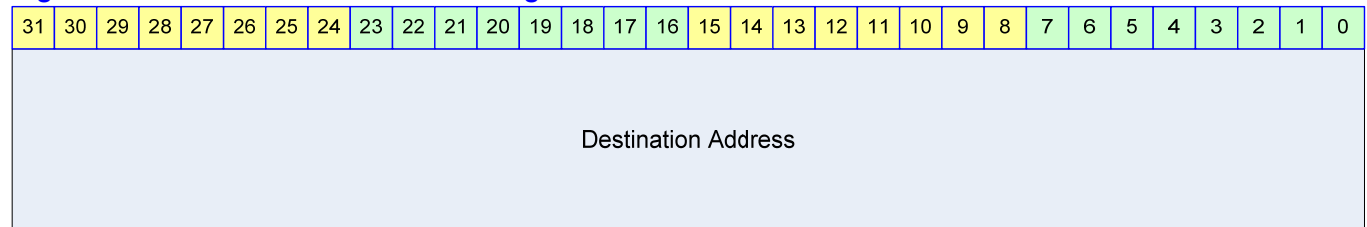


Table 9-3 Source Address Register Description

Bits	Name	R / W	Description	Init. value
[31:0]	Source Address	RW	32bit Source Address Control 레지스터의 SrcIncr bit 가 '1'이면 매 전송된 byte 수 만큼 증가한다.	0000 0000h

9.5.3. Destination Address Register

Destination Address 레지스터는 말 그대로 복사가 일어날 Destination Address 를 지칭하기 위한 레지스터이다.

Figure 9-5 Destination Address Register**Table 9-6 Destination Address Register Description**

Bits	Name	R / W	Description	Init. value
[31:0]	Destination Address	RW	32bit Destination Address Control 레지스터의 DstIncr bit 가 '1'이면 매 전송된 byte 수 만큼 증가한다.	0000 0000h

9.5.4. Control Register

Control 레지스터는 현재 전송하고 있는 Descriptor 의 data transfer 의 동작을 규정하는 레지스터이다.

Figure 9-6 Control Register**Table 9-4 Control Register Description**

Bits	Name	R / W	Description	Init. Value
------	------	-------	-------------	-------------

[31]	StartInEn	RW	Start Interrupt Enable Flag 0 : Start Interrupt 를 masking 함 1 : Start Interrupt 를 발생시킴	0b
[30]	EndIntEn	RW	End Interrupt Enable Flag 0 : End Interrupt 를 masking 함 1 : End Interrupt 를 발생시킴 1	0b
[29]	M2M	RW	Memory-to-Memory Transfer Flag 0 : DMA Request 신호로 DMA 동작 1 : DMA Request 신호 없이 DMA 동작 Memory 에서 memory 로 data 를 전송하는 경우 외부의 DMA Request 신호와 관계 없이 전송이 계속된다. M2M bit 가 '1'인 경우 외부 DMA Request 신호를 무시하고 DMA Request 가 항상 assert 되어 있는 것과 동일하게 DMA Controller 가 동작하게 된다.	0b
[28]	SrcIncr	RW	Source Address Increment Flag 1 : 매 transfer 마다 Source Address 증가시킴 0 : 매 transfer 마다 Source Address 증가안함	0b
[27:26]	SrcWidth	RW	Source Device 의 BUS Width 00 : BYTE(8bit) 01 : HWORD(16bit) 10 : WORD(32bit) 11 : reserved Source Device 가 memory 라면 반드시 WORD 로 세팅해야 한다.	00b
[25]	DstIncr	RW	Destination Address Increment Flag 1 : 매 transfer 마다 Destination Address 증가시킴 0 : 매 transfer 마다 Destination Address 증가안함	0b
[24:23]	DstWidth	RW	Destination Device 의 BUS Width 00 : BYTE(8bit) 01 : HWORD(16bit) 10 : WORD(32bit) 11 : reserved Destination Device 가 memory 라면 반드시 WORD 로 세팅해야 한다.	00b
[22:20]	Size	RW	Transfer Size 매 DMA Request 마다 한번에 전송할 byte 수 000 : 1 byte 001 : 2 byte 010 : 4 byte 011 : 8 byte 100 : 16 byte 101 : 32 byte Other : reserved	000b
[19:0]	TransferLength	RW	전체 전송할 byte 수/현재 남아 있는 byte 수 20 bit 이므로 하나의 Descriptor 에서 1MB-1Byte 까지 전송할 수 있다. 매 data transfer 마다 감소하게 되어 "현재 남아있는 전송할 byte 수"를 지칭하게 된다.	00000h

9.5.5. Descriptor Register

Descriptor 레지스터는 새로 Loading 될 Descriptor 의 address 를 담고 있는 레지스터이다.

Figure 9-7 Descriptor Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Descriptor Address																												Reserved	DescriptorEnd		

Table 9-5 Descriptor Register Description

Bits	Name	R / W	Description	Init. Value
[31:2]	Descriptor Address	RW	Load 할 Descriptor Address 의 상위 32bit Descriptor Address 는 32bit align 되어야 함. DescriptorEnd bit 가 '1'이면 어떠한 값이라도 관계 없다.	0
[0]	DescriptorEnd	RW	Descriptor End Flag 0 : Descriptor Address 가 VALID 함 1 : Descriptor Address 가 VALID 하지 않음. '1'로 세팅된 경우 현재 처리중인 Descriptor 가 마지막 Descriptor 임을 의미한다.	0b

9.5.6. Status Register

Status 레지스터는 현재 DMA Channel 의 동작 상황을 알려준다.

Figure 9-8 Status Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Enable	Reserved				Active	Req	Reserved																				StopIntEn	StopInt	StartInt	EndInt	ErrInt

Table 9-6 Status Register Description

Bits	Name	R / W	Description	Init. Value
[31]	Enable	RW	Enable Flag 0 : DMA Channel Disabled 1 : DMA Channel Enabled Channel 이 Stop Interrupt 나 Error Interrupt 에 의해 disable 되더라도 '0'으로 clear 되지 않는다.	0b

[27]	Active	R	Active Flag 0 : 현재 data transfer 를 진행중이지 않음 1 : 현재 data transfer 를 진행중임	N/A
[26]	Req	R	DMA Request Status 1 : Device 가 DMA Request 를 assert 하고 있음 0 : Device 가 DMA Request 를 assert 하지 않음	N/A
[4]	StopIntEn	RW	Stop Interrupt Enable Flag 0 : Stop Interrupt 발생을 disable 함 1 : Stop Interrupt 발생을 enable 함	0b
[3]	StopInt	RW	Stop Interrupt Status 0 : Stop Interrupt 가 발생하지 않았음 1 : Stop Interrupt 가 발생하였음 StopIntEn 과 상관 없이 Stop interrupt 상황에서 '1'로 set 된다. StopIntEn 이 '0'이면 이 bit 의 값과 관계 없이 interrupt 가 발생하지 않는다. 이 bit 가 '1'일 때는 Channel 의 동작이 멈추게 되므로 Channel 을 enable 시키기 위해서는 이 bit 를 '0'으로 clear 해 주어야 한다. '1'을 write 하면 clear 되고 '0'을 write 하는 것은 아무런 영향을 미치지 않는다.	0b
[2]	StartInt	RW	Start Interrupt Status 0 : Start Interrupt 가 발생하지 않았음 1 : Start Interrupt 가 발생하였음 Control 레지스터의 StartIntEn 이 '0'이면 이 bit 는 '1'로 변하지 않는다. '1'을 write 하면 clear 되고 '0'을 write 하는 것은 아무런 영향을 미치지 않는다.	0b
[1]	EndInt	RW	End Interrupt Status 0 : End Interrupt 가 발생하지 않았음 1 : End Interrupt 가 발생하였음 Control 레지스터의 EndIntEn 이 '0'이면 이 bit 는 '1'로 변하지 않는다. '1'을 write 하면 clear 되고 '0'을 write 하는 것은 아무런 영향을 미치지 않는다.	0b
[0]	ErrInt	RW	Error Interrupt Status 0 : Error Interrupt 가 발생하지 않았음 1 : Error Interrupt 가 발생하였음 이 bit 가 '1'일 때는 Channel 의 동작이 멈추게 되므로 Channel 을 enable 시키기 위해서는 이 bit 를 '0'으로 clear 해 주어야 한다. '1'을 write 하면 clear 되고 '0'을 write 하는 것은 아무런 영향을 미치지 않는다.	0b

10. TIMER

10.1. Overview

총 4 개의 32bit Timer 를 제공한다.

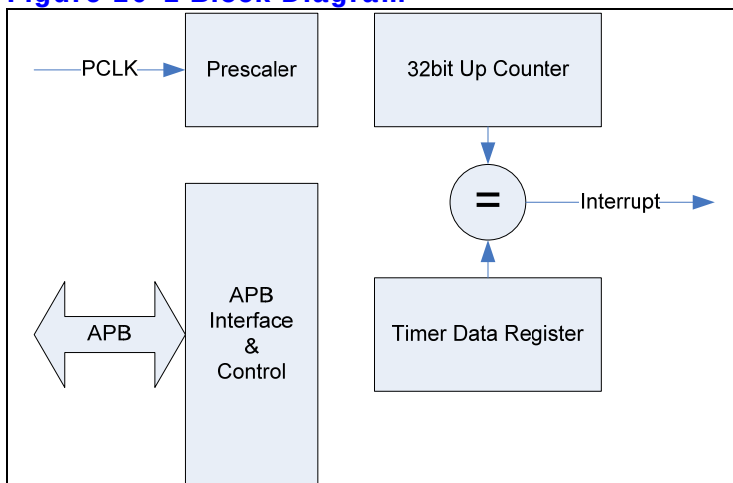
10.1.1. Features

- Maximum period : $\text{PCLK duration} * 2048 * (2^{32}-1)$
- Minimum period : $\text{PCLK duration} * 2$

10.1.2. Block diagram

하나의 Timer 의 하드웨어 구성은 다음과 같다.

Figure 10-1 Block Diagram



기본적으로 timer 는 레지스터를 읽고 쓰기 위해서 APB Bus interface 를 가지고 있다. APB BUS 의 clock 인 PCLK 을 Prescaler 를 이용하여 분주하게 되고 그 분주된 clock 으로 32bit up counter 가 동작하게 된다. 32 bit up counter 의 값이 Timer Data 레지스터의 같은 값이 되면 positive edge 형식의 interrupt 가 발생하게 되고 그 interrupt 는 interrupt controller 에 연결된다.

위와 같은 구조를 가지는 독립된 timer 4 개가 병렬로 연결되어 있다.

10.2. Operation

다음과 같은 2 가지 모드로 동작이 가능하다.

10.2.1. Interval Mode

Interval Mode에서는 counter가 증가하여 timer data 레지스터에 저장된 값과 같아지면 interrupt가 생성되고 counter의 값은 자동으로 0으로 clear된다. 일정한 시간마다 interrupt의 발생을 원하면 이 mode를 사용하면 된다.

10.2.2. Match&Overflow Mode

Match&Overflow Mode에서는 counter가 증가하여 timer data 레지스터에 저장된 값과 같아지면 interval mode에서와 마찬가지로 interrupt가 생성된다. 하지만 counter값은 0으로 clear되지 않고, 계속 증가하게 된다. 증가가 계속되어 0xffff가 되면 다시 0으로 바뀌게 된다. Interval mode를 사용하는 경우 interrupt의 발생 주기가 interrupt service routine의 service latency보다 작은 경우 interrupt가 여러 번 발생하였는데도 한번으로 인식할 가능성이 생기게 된다. 이런 경우에는 Match&Overflow mode를 사용하게 되면 interrupt service routine이 interrupt 발생 시점에서 얼마간의 시간이 흘렀는지 detect할 수 있고, Interrupt service routine에서 timer data 레지스터 값을 새로 write함으로써 다음에 발생할 interrupt의 시점을 제어할 수 있다.

10.3. Register Description

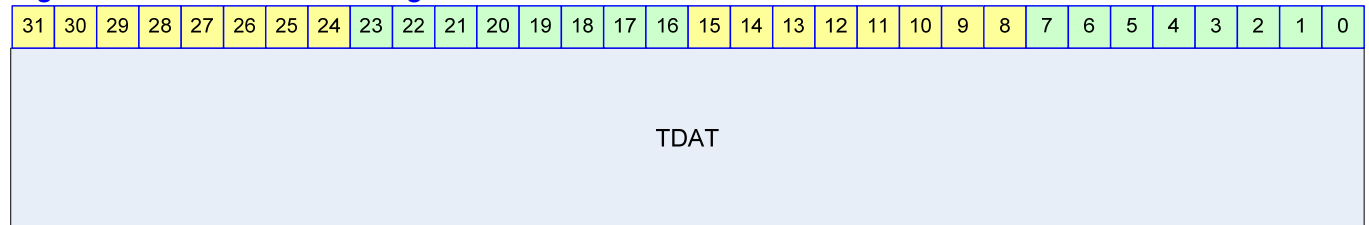
10.3.1. Registers

Table 10-1 Registers

Offset	Name	R / W	Description	Init. value
0000h	T0_TDAT	RW	(Timer0) Timer Data Register	FFFF FFFFh
0004h	T0_PRES	RW	(Timer0) Prescaler Register	XXXX X3FFh
0008h	T0_CTRL	RW	(Timer0) Control Register	XXXX XXX0h
000Ch	T0_CNT	R	(Timer0) Counter Register	0000 0000h
0010h	T1_TDAT	RW	(Timer1) Timer Data Register	FFFF FFFFh
0014h	T1_PRES	RW	(Timer1) Prescaler Register	XXXX X3FFh
0018h	T1_CTRL	RW	(Timer1) Control Register	XXXX XXX0h
001Ch	T1_CNT	R	(Timer1) Counter Register	0000 0000h
0020h	T2_TDAT	RW	(Timer2) Timer Data Register	FFFF FFFFh
0024h	T2_PRES	RW	(Timer2) Prescaler Register	XXXX X3FFh
0028h	T2_CTRL	RW	(Timer2) Control Register	XXXX XXX0h
002Ch	T2_CNT	R	(Timer2) Counter Register	0000 0000h
0030h	T3_TDAT	RW	(Timer3) Timer Data Register	FFFF FFFFh
0034h	T3_PRES	RW	(Timer3) Prescaler Register	XXXX X3FFh
0038h	T3_CTRL	RW	(Timer3) Control Register	XXXX XXX0h
003Ch	T3_CNT	R	(Timer3) Counter Register	0000 0000h

10.3.2. Timer Data Register (T0_TDAT/T1_TDAT/T2_TDAT/T3_TDAT)

Timer Data 레지스터는 counter 의 값과 비교를 하기 위한 레지스터이다.

Figure 10-2 Timer Data Register**Table 10-2 Timer Data Register Description**

Bits	Name	R / W	Description	Init. value
[31:0]	TDAT	RW	Timer Data 레지스터 TDAT 과 32bit Counter 값이 같으면 interrupt 가 발생한다.	FFFFFFFFh

10.3.3. Timer Prescaler Register (T0_PRES/T1_PRES/T2_PRES/T3_PRES)

Timer Prescaler Register 는 32bit Counter 에 들어가는 clock 의 속도를 조정하는 레지스터이다.

Figure 10-3 Timer Prescaler Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0						
Reserved																						Prescale															

Table 10-3 Timer Prescaler Register Description

Bits	Name	R / W	Description	Init. value
[9:0]	Prescale	RW	32bit Counter 이 들어가는 clock 의 속도를 결정	3FFh

32bit Counter 에 들어가는 Clock 의 속도는 다음과 같이 결정된다.

$$\text{Clock} = \text{PCLK}/(\text{Prescale}+1)$$

매 네번의 PCLK rise 마다 32bit Counter 가 1 씩 증가하도록 만들고 싶으면 Prescale 의 값을 3 으로 정하면 된다.

10.3.4. Timer Control Register (T0_CTRL/T1_CTRL/T2_CTRL/T3_CTRL)

Timer Control 레지스터는 Timer 의 동작 여부와 동작 mode 를 결정하는 레지스터이다.

Figure 10-4 Timer Control Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																											Mode	Clear	Enable		

Table 10-4 Timer Control Register Description

Bits	Name	R / W	Description	Init. Value
[2]	Mode	RW	동작 mode 0 : Interval mode 1 : Match&Overflow Mode	0b
[1]	Clear	RW	Counter Clear 0 : 의미 없음 1 : Counter 의 값을 0 으로 초기화함 Clear bit 가 1 로 세팅되면 32 bit Counter 의 값을 0 으로 초기화하고 Timer Data 레지스터와 Timer Prescaler 레지스터의 값을 Reset 값으로 바꾸게 된다. 1 로 write 되고 나면 timer 가 동작하지 않으므로 0 으로 반드시 다시 써 주어야 한다.	0b

[0]	Enable	R/W	Timer Enable bit 0 : Not Enabled 1 : Enabled	0b
-----	--------	-----	--	----

10.3.5. Timer Counter Register (T0_CNT/T1_CNT/T2_CNT/T3_CNT)

Timer Counter 레지스터는 Read only 레지스터로 32bit Counter 의 현재 값을 읽을 수 있는 레지스터이다.

Figure 10-5 Timer Counter Register

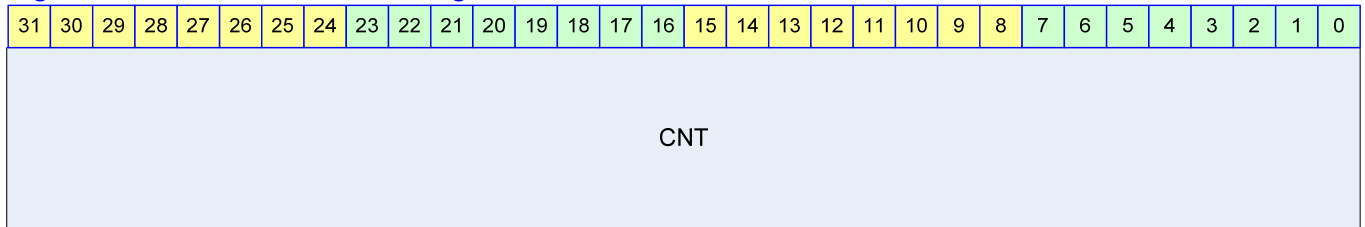


Table 10-5 Timer Counter Register Description

Bits	Name	R / W	Description	Init. Value
[31:0]	CNT	R	32bit Counter 의 현재 값	00000000h

11. Watch Dog Timer

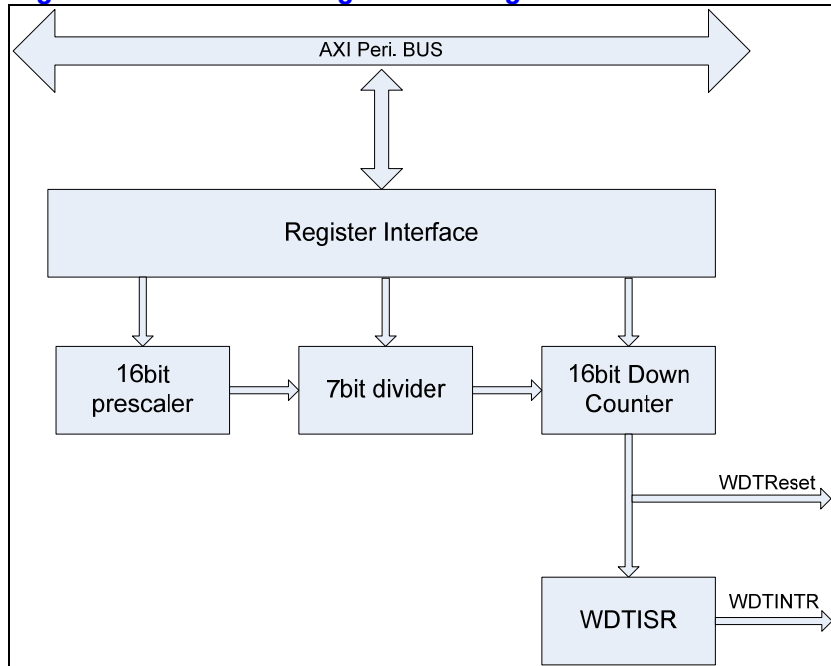
11.1. Overview

11.1.1. Feature

- Support 16 bit Programmable interval down counter.
- Support clock division factor 16,32,64, and 128.
- Support 16 bit prescale counter.
- Support Interrupt Request
- Generation of Reset Output Signal

11.1.2. Block Diagram

Figure 11-1 Watch Dog Block Diagram



11.2. Operation

Chip 이 소프트웨어나 다른 환경에 의해 오동작할 경우 전체 동작을 **Reset** 해주는 역할을 한다. 즉, 카운터 값이 설정된 값에 도달할 경우 **Interrupt** 를 생성하게 되는데, 정상적인 상태에 있는 칩이라면 **Interrupt** 발생시 마다 카운터 값을 **Reload** 하여 전체 동작에 **Reset** 를 걸지 않게 할 수 있지만, 반대의 경우라면 칩에 **Reset** 이 걸리게 되어 있다.

11.2.1. Clock Prescaler Block

16bit Prescaler Count 와 7Bit Clock Divider 를 이용하여 다양한 PCLK Clock Divide 가 가능하다.

11.2.2. Interrupt Generation Block

설정된 Prescaler Count 와 7Bit Divide 값이 16bit Down Count 값에 도달하면 Interrupt 를 생성하게 된다.

Interrupt Status 가 Clear 되기전까지 High Level 을 유지한다.

Interrupt 가 생성되기 위해서는 Control Register 의 Interrupt Enable Bit 를 Active 시켜야 한다.

11.2.3. Reset Output Generation

설정된 Prescaler Count 와 7Bit Divide 값이 16bit Down Count 값에 도달하면 Interrupt 와 함께 Watch Dog Reset 을 생성하게 된다.

Reset 이 생성되기 위해서는 Control Register 의 Reset Enable Bit 를 Active 시켜야 한다

11.3. Register Description

11.3.1. WDT Registers

Table 11-1 WDT Registers

Offset	Name	R / W	Description	Init. value
0000h	WDTCN	RW	WDT Control Register	0000 0030h
0004h	WDTPSR	RW	WDT Prescale Register	0000 FFFFh
0008h	WDTLDR	RW	WDT Load Register	0000 FFFFh
000Ch	WDTCNT	RW	WDT Count Register	0000 FFFFh
0010h	WDTISR	RW	WDT Interrupt Status Register	0000 0000h

11.3.2. WDT Control Register

Figure 11-2 WDT Control Register Bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																										DIV	ClkSel	IntEn	RSTEn	WDTEn	

Table 11-2 WDT Control Register Description(Offset : 0000 0000h)

Bits	Name	R / W	Description	Init. value
[31:6]			Reserved	0
[5:4]	DIV	RW	0b00 : 16 0b01 : 32 0b10 : 64 0b11 : 128	11b
[3]	ClkSel	RW	0 : Both Prescale and Divide Clock is used for Counting 1 : Only Prescal Clock is user for Counting	0
[2]	IntEn	RW	0 : WDT Interrupt Disable 1 : WDT Interrupt Enable	0
[1]	RSTEn	RW	0 : Reset Output Disabled 1 : Reset Output Enabled	0
[0]	WDTEn	RW	WDT Enable 0 : WDT Disable 1 : WDT Enable	0

11.3.3. WDT Prescale Register

Figure 11-3 WDT Prescale Register Bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																WDTPSR															

Table 11-3 WDT Prescale Register Description(Offset : 0000 0004h)

Bits	Name	R / W	Description	Init. value
[31:16]			Reserved	0
[15:0]	WDTPSR	RW	Prescaler Value	ffffh

11.3.4. WDT Load Register

Figure 11-4 WDT Load Register Bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																WDTLDR															

Table 11-4 WDT Load Register Description(Offset : 0000 0008h)

Bits	Name	R / W	Description	Init. value
[31:16]			Reserved	0
[15:0]	WDTLDR	RW	Reload Value	ffffh

11.3.5. WDT Count Register

Figure 11-5 WDT Count Register Bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																WDCNT															

Table 11-5 WDT Count Register Description(Offset : 0000 000Ch)

Bits	Name	R / W	Description	Init. value
[31:16]			Reserved	0
[15:0]	WDCNT	R	Current WDT Count Value	ffffh

11.3.6. WDT Interrupt Status Register

Figure 11-6 WDT Interrupt Status Register Bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Reserved																															WDTCCLR	WDTISR

Table 11-6 WDT Interrupt Status Register Description(Offset : 0000 0010h)

Bits	Name	R / W	Description	Init. value
[31:2]			Reserved	0
[1]	WDTCCLR	W	Write 1, Cleared Interrupt	
[0]	WDTISR	R	0 : Interrupt Is asserted 0 : Interrupt Is not asserted	

12. I2S

12.1. Overview

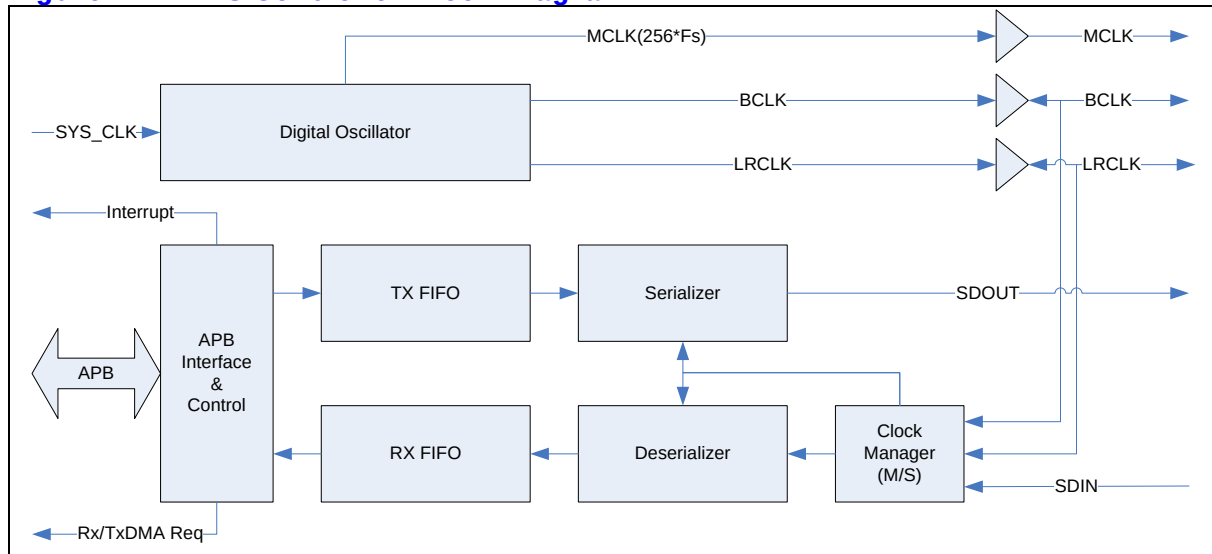
I2S Protocol 은 Philips 에서 개발한 stereo audio data 를 전송하는 규약으로 많은 audio codec 이 I2S protocol 을 사용하고 있다. I2S protocol 은 BCLK(bit clock), LRCLK(left/right channel detect signal), SD(Serial Audio Data) signal 을 사용하여 audio data 를 전송한다. I2S controller 는 I2S protocol 을 사용하여 외부의 audio codec 으로 audio data 를 전송하거나 audio codec 으로부터 audio data 를 수신하는 기능을 한다. 즉, 사용자는 audio data 를 play 하거나 외부의 analog audio 를 audio codec 을 거쳐 record 할 수 있다.

12.1.1. Features

- 8/16/24/32bit per sample support
- No audio PLL need
- Integrated Digital Oscillator
 - Fully programmable sampling rate(determined by SYS_CLK frequency)
 - MCLK(256*sampling rate) generation
 - ◆ Not support 384*sampling rate, 512*sampling rate
- Support normal I2S mode and MSB(=Left) justified mode
- Programmable LRCLK polarity
- Maximum 32x32bit FIFO for each transmission/receive
 - DMA request generation
- Master/slave configuration
 - BCLK output support only 64*sampling rate

12.1.2. Block diagram

전체적인 하드웨어 구성은 다음 그림과 같다.

Figure 12-1 I2S Controller Block Diagram

칩 외부 pin 으로 I2S Protocol signal 인 BCLK/LRCLK/SDOUT/SDIN 이 연결되어 있고, 256 배의 sampling rate 를 출력하는 MCLK pin 도 존재한다. 칩 내부로는 기본적으로 APB BUS interface 를 이용하여 연결되고, 그 외로 Digital Oscillator 의 입력 clock 으로 SYS_CLK 을 입력 받고, DMA request 와 Interrupt request 를 출력한다.

12.2. Signals

칩 외부로 연결되는 신호들은 다음과 같다.

Table 12-1 External Signals

Signal Name	Direction	Description	Init. value
MCLK	Output	256 배의 sampling rate 의 clock 을 외부로 출력	N/A
BCLK	InOut	Bit Clock Master 로 configuration 되면 output 이 되고 Slave 로 configuration 되면 input 이 된다.	N/A
LRCLK	InOut	Left/Right Channel Indicator Master 로 configuration 되면 output 이 되고 Slave 로 configuration 되면 input 이 된다.	N/A
SDOUT	Output	Serial Data output	N/A
SDIN	Input	Serial Data Input	N/A

12.3. Operation

12.3.1. Digital Oscillator

Digital Oscillator 는 SYS_CLK 입력을 바탕으로 MCLK/BCLK/LRCLK 을 생성하는 동작을 한다. MCLK 은 Master Clock 으로 대부분의 I2S Codec 이 요구하는 sampling rate 의 256 배에 해당되는 clock 을 말한다. 이 MCLK 의 생성을 위해서 보통의 경우에는 Audio PLL 을 사용하지만 설계된 I2S Controller 는 Audio PLL 을 사용하지 않고 SYS_CLK 을 분주하여 사용하여 생성하도록 되어있다. SYS_CLK 을 단순히 정수배 분주를 하게 되면 정확한 주파수의 MCLK 을 생성할 수 없기 때문에 DDS(또는 DTO) 알고리즘을 사용하도록 구성하였다.

Digital Oscillator 의 동작은 I2S_CLKCTRL 레지스터의 MClkOn field 에 의해 On/Off 할 수 있다. 생성할 MCLK 의 주파수는 I2S_CLKCTRL 레지스터의 DTORATIO field 에 의해서 결정된다. DTORATIO field 에서는 SYS_CLK 의 주파수와 생성할 MCLK 의 주파수의 비율을 적어주면 되는데 다음과 같은 식을 사용하면 된다.

$$DTORATIO = \text{RoundToNearInteger}[2^{18} * (\text{MCLK 의 주파수} / \text{SYS_CLK 의 주파수})]$$

예를 들어 SYS_CLK 으로 100MHz 의 clock 을 가하고 생성하고 싶은 MCLK 이

12.288MHz(256*48KHz)라면 다음과 같은 식에 의해 DTORATIO 에 32212 를 적어주면 된다.

$$\begin{aligned} DTORATIO &= \text{RoundToNearInteger}[2^{18} * (12.288\text{MHz} / 100\text{MHz})] \\ &= \text{RoundToNearInteger}[262144 * 0.12288] \\ &= \text{RoundToNearInteger}[32212.25472] \\ &= 32212 \end{aligned}$$

이렇게 생성된 MCLK 은 비교적 정교한 clock 이지만 Audio PLL 을 사용하여 생성하는 clock 에 비해 몇가지 오차가 발생한다.

12.3.1.1. 주파수 오차

DTORATIO 값이 정수 값이므로 finite bit-length 로 인한 주파수 오차가 발생한다. 또한 clock cycle time 이 매 clock 동일하게 유지되지 않는다. 우선 finite bit-length 로 인한 주파수 오차는 18bit 의 DTORATIO 를 사용하여 0.1%보다 작도록 설계되었으므로 큰 문제가 되지 않는다. 매 clock 달라질 수 있는 clock cycle time 은 다음과 같이 계산할 수 있다.

우선 FrequencyRatio 라는 값은 다음과 같이 정의하자.

$$\text{FrequencyRatio} = 2^{18} / \text{DTORATIO}$$

FrequencyRatio 의 값은 'SYS_CLK 의 주파수 / MCLK 의 주파수'와 finite bit-length 로 인한 오차를 제외하고는 동일한 값을 가지게 된다. FrequencyRatio 를 rounding 하여 다음 두 값을 계산한다.

$$\text{FrequencyRatio_Higher} = \text{FrequencyRatio 보다 작지 않은 가장 작은 integer}$$

$$\text{FrequencyRatio_Lower} = \text{FrequencyRatio 보다 크지 않은 가장 큰 integer}$$

이 값으로 다음과 같은 값을 계산할 수 있다.

$$\text{Minimum Cycle Time} = \text{FrequencyRatio_Lower} * \text{SYS_CLK 의 주기}$$

Maximum Cycle Time = FrequencyRatio_Higher*SYS_CLK 의주기

예를 들어 SYS_CLK 이 100MHz(주기는 10ns)이고 MCLK 이 12.288MHz 라면 위에서 계산한대로 DTORATIO 는 32212 가 되므로 FrequencyRatio = 8.183... 이 된다.

Minimum Cycle Time 은 80ns(8*10ns)가 되고 Maximum Cycle Time 은 90ns(9*10ns)가 된다. 즉 특정 cycle 은 80 ns 가 되고 특정 cycle 은 90 ns 가 된다.

12.3.1.2. duty cycle

MCLK 의 duty cycle 이 정확하게 50%를 가지지 않는다. Duty cycle 이 정확하게 50%가 되려면 FrequencyRatio 의 값이 짝수 Integer 값을 가지면 되지만 일반적인 경우 그럴 가능성은 매우 낮다. 이런 경우 Worst duty cycle 은 다음과 같이 계산할 수 있다.

FrequencyRatio_Odd = FrequencyRatio 와 가장 가까운 홀수 integer

Worst Duty Cycle(%) = $100 * \text{RoundToNearInteger}[\text{FrequencyRatio_Odd}/2] / \text{FrequencyRatio_Odd}$

동일한 예로 SYS_CLK 이 100 MHz 이고 MCLK 이 12.288MHz 라면 위에서 계산한 대로 FrequencyRatio 는 8.183...이 되고 FrequencyRatio_Odd 는 9 가 된다. 따라서 Worst Duty Cycle 은 다음과 같다.

Worst Duty Cycle(%) = $100 * 4 / 9 = 44.44(\%)$

일반적인 Audio Codec 은 MCLK 의 duty cycle 을 40%~60%를 요구한다.(CirrusLogic CS42L51 과 같은 칩은 45%~55%를 요구하기도 한다.) 40%를 만족시키기 위해서는 SYS_CLK 이 MCLK 에 비해 적어도 5 배 이상 동작해야 한다.(45%를 만족시키기 위해서는 11 배 이상으로 동작해야 한다.)

BCLK 과 LRCLK 도 MCLK 과 마찬가지로 주파수 오차와 duty cycle 오차가 생기게 된다. 이 경우도 동일한 방법으로 계산할 수 있다. 하지만 BCLK/LRCLK 은 MCLK 에 비해 훨씬 느리게 동작하므로 보통의 경우 그러한 오차가 큰 의미를 지니지는 않는다.

12.3.2. Master / Slave Configuration

I2S protocol 에서 master 라고 하면 BCLK 과 LRCLK 을 출력하는 device 를 의미하고, slave 는 BCLK 과 LRCLK 을 생성하지 않고 받기만 하는 device 를 의미한다. I2S_CLKCTRL 레지스터의 ClkMS bit 를 '1'로 하면 I2S Controller 가 master 로 동작하게 되고, '0'으로 하면 slave 로 동작하게 된다. Master 로 동작하는 경우에는 반드시 I2S_CLKCTRL 레지스터의 MClkOn 0 '1'로 세팅되어야 한다.

만약 I2S Controller 가 MCLK 을 생성하여 외부의 I2S Codec 에 MCLK 을 공급하면서 I2S protocol 상에서는 slave 로 동작하고 싶은 경우에는 MClkOn 을 '1', MClkOE 를 '1'로 하고 ClkMS 를 '0'으로 세팅하면 된다.

12.3.3. Transmit 동작 : Audio Play

외부의 I2S DAC 으로 PCM data 를 transmit 하는 동작은 serializer block 이 수행하게 되는데 I2S_CTRL 레지스터의 하위 16bit 에 의해 control 된다. Serializer block 은 enable 되면 TX FIFO(Transmit FIFO)에서 data 를 읽어서 SDOUT pin 으로 PCM data 를 serial 로 전송을 하게 된다.

12.3.3.1. Enabling Sequence

Transmit 을 enable 시키기 위해서는 단순히 I2S_CTRL 레지스터의 하위 16bit 을 원하는 값으로 세팅하면 초기화는 끝나게 된다. TX FIFO 에 데이터가 없는 상황에서 TxEn bit 가 '1'로 세팅된 경우 TX FIFO UnderRun Interrupt 가 발생하게 된다. 초기에 이것을 피하기 위해서는 TX FIFO 에 어느 정도 PCM data 를 채운 상태에서 TxEn bit 을 '1'로 세팅할 수도 있다.

12.3.3.2. Transmit data의 공급

외부 I2S DAC 으로 전송할 PCM data 는 TX FIFO 에 채워져야 하는데 TX FIFO 에 data 를 채우는 데는 두 가지 방법을 사용할 수 있다. 우선 CPU 가 직접 공급하는 방법이 있다. CPU 가 polling 하여 data 를 공급할 수 있는데 I2S_STATUS register 에서 TxFifoLevel field 를 보고 TX FIFO 가 full 이 아니라면 계속 TX FIFO 에 값을 채울 수 있다. Interrupt 를 사용하고 싶을 때는 I2S_CTRL 레지스터의 TxDMAIntEn bit 를 '1'로 세팅해 주고 TxFifoTH field 의 값을 적절히 세팅하면 TX FIFO 의 Level 이 (TxFifoTH+1)보다 작으면 interrupt 가 발생하게 된다. 그 때 마다 CPU 가 TX FIFO 에 data 를 write 하면 된다.

또 한가지 방법은 DMA Controller 를 사용하는 방법이다. I2S Controller 는 TX FIFO 의 Level 이 (TxFifoTH+1)보다 작으면 무조건 Tx DMA Request 신호를 생성하게 된다. DMA Controller 가 해당 signal 을 받으면 DMA 가 동작하도록 configuration 해 두면 DMA Controller 가 Memory 에서 data 를 읽어 TX FIFO 에 자동으로 채워주게 된다.

참고로 DMA Controller 를 사용하는 방법과 CPU 가 직접 data 를 TX FIFO 에 채워주는 방법을 동시에 사용하는 것은 권장되지 않는다. TxDMAIntEn bit 가 '1'로 세팅된 경우 Tx DMA Request 상황(즉, TX FIFO 의 Level 이 TxFifoTH+1 보다 작은 경우)에서 Interrupt 가 발생하고 I2S_STATUS 레지스터의 TxDMA bit 가 set 되고 DMA Controller 가 동작하게 된다. DMA Controller 가 Service 를 마치게 되면 TX FIFO 의 Level 이 TxFifoTH+1 보다 작아져 다시

TxDMA bit 가 '0'으로 clear 된다. 이러한 상황이 발생하면 CPU 는 interrupt 를 받지만 interrupt handler 가 I2S_STATUS 레지스터를 확인할 때는 이미 TxDMA bit 가 clear 될 수 있기 때문에 interrupt 를 놓칠 수 있다. 따라서 DMA Controller 를 사용하는 경우에는 TxDMAIntEn bit 를 '0'으로 하는 것을 권장한다.

12.3.3.3. SDOUT 출력

TX FIFO 에서 읽은 데이터가 SDOUT 으로 차례로 출력되며 각 채널의 data 가 모두 전송되고 BCLK 의 toggle 이 계속되는 경우에는 하위 bit 를 '0'으로 출력하게 된다.

12.3.3.4. TX FIFO UnderRun Error

Serializer block 은 I2S protocol 상에서 Left Channel 의 전송이 시작될 때 TX FIFO 에서 data 를 읽어오게 되는데, 이 때 TX FIFO 에 data 가 없으면 TX FIFO UnderRun Error 가 발생되고, TX FIFO 가 다시 채워질 때까지 SDOUT pin 으로는 마지막으로 전송된 sample 이 전송되게 된다. 만약 I2S_CTRL 레지스터의 TxWLen field 가 24bit/32bit 으로 세팅된 경우에는 TX FIFO 에 적어도 두 개 이상의 32bit data 가 있어야 UnderRun Error 가 발생하지 않으며, 16bit/8bit 의 경우에는 TX FIFO 에 하나 이상의 data 가 있을 때 UnderRun Error 가 발생하지 않는다.

UnderRun Error 가 발생하고 I2S_CTRL 레지스터의 TxFifoURIntEn bit 이 '1'이고, I2S_STATUS 레지스터의 EnTxFifoURInt bit 이 '1'이면, I2S_STATUS 레지스터의 TxFifoURInt bit 이 '1'로 set 되면서 interrupt 가 발생한다. I2S_STATUS 레지스터의 EnTxFifoURInt bit 은 hardware state 에 따라서 변하게 되는데, 다음과 같은 때 변하게 된다.

- Reset 상태에서는 0의 값으로 초기화 된다.
- TxReset이 assert될 때 0의 값으로 초기화된다.
- EnTxFifoURInt가 0이고 TxFIFO에 2 word 이상의 data가 들어 있으면 1로 바뀐다.
- TxFifoURInt가 '1'로 바뀌면 0으로 reset된다.

EnTxFifoURInt 가 위와 같이 변하기 때문에 Transmit 이 맨 처음 enable 되고 Tx FIFO 에 어떠한 data 도 존재하지 않는 경우에는 TX FIFO UnderRun Error 상황이지만 Tx FIFO 에 두 word 이상의 data 를 채울 때 까지는 TxFifoURInt interrupt 가 발생하지 않는다. 또한 TxFifoURInt interrupt 가 일단 발생되면, TxFifoURInt bit 을 clear 하더라도 TxEn 이 '1'인 경우 계속 TX FIFO UnderRun Error 상황인데 Tx FIFO 에 2 개 이상의 data 를 write 하는 시점까지는 TxFifoURInt interrupt 의 발생이 연기된다.

12.3.3.5. Disabling Transmit

Transmit 동작을 disable 시키고 싶으면 I2S_CTRL 레지스터의 TxEn bit 를 '0'으로 clear 해 주면 된다. TxEn bit 가 '0'이 되면 더 이상 DMA Request 는 생성되지는 않지만, TX FIFO 에 남아 있는 data 는 계속 출력이 되고 TX FIFO 에 data 가 모두 소진 될 때 동작이 끝나게 된다. 만약 I2S_CTRL 레지스터의 TxWLEN 이 24bit/32bit 이고 TX FIFO 에 data 가 1 개만 남아 있는 경우에는 TX FIFO 가 소진되 않고 TX FIFO UnderRun Error 가 발생된다. 이 경우 Transmit 을 강제로 disable 시키기 위해서는 I2S_CTRL 레지스터의 TxReset bit 를 '1'로 write 하면 된다. 이러한 경우를 피하기 위해서 DMA Controller 는 항상 짝수개의 word 단위로 FIFO 를 채우도록 setting 하는 것을 권장한다.

Transmit 이 disable 되면 맨 마지막으로 출력된 sample 이 계속 SDOUT pin 으로 출력된다. 참고로 TxEn bit 가 '0'이 되더라도 I2S_STATUS 의 TxFifoURInt bit 은 clear 되지 않는다. 정리하면 다음과 같은 절차로 transmit 을 disable 시킬 수 있다.

- (a) I2S_CTRL 레지스터의 TxEn bit 를 '0'으로 clear
- (b) I2S_STATUS 레지스터의 TxFifoLevel field 를 보고 empty 가 될 때까지 대기
- (c) TxWLen 이 24bit/32bit 이고 TxFifoLevel field 가 1 에서 계속 변하지 않으면 TxReset bit 를 '1'로 write 하여 강제로 종료

12.3.3.6. TxReset

Transmit block 의 state 를 초기화하기 위해서 I2S_CTRL 레지스터의 TxReset bit 를 '1'로 write 해 주면 된다. TxReset bit 를 '1'로 write 하면 TX FIFO 의 level 이 0 이 되고 TX FIFO 에 들어있는 data 는 flushing 된다. 또한 맨 마지막으로 전송된 Left/Right sample 은 모두 0 으로 초기화된다. Transmit 을 enable 하기 전에 TxReset bit 을 '1'로 write 하여 state 를 초기화해 주는 것을 권장한다.

12.3.4. Receive 동작 : Audio Record

외부의 I2S ADC 를 통해서 audio data 를 받는 일은 Deserializer block 이 수행한다.

Deserializer block 은 enable 되면 SDIN pin 으로 들어오는 serial audio data 를 RX FIFO 에 채우는 동작을 수행하게 된다.

12.3.4.1. Enabling Sequence

Receive 동작을 enable 시키기 위해서는 I2S_CTRL 레지스터의 상위 16bit 를 세팅해주면 된다. RxEn bit 가 '1'이 되고 Left channel data 입력이 시작되면 동작을 시작하게 된다. RxEn bit 를 '1'로 세팅하기 전에 I2S_STATUS 의 RxFifoORInt bit 을 '0'으로 clear 해 주는 것을 권장한다.

12.3.4.2. Receive data

외부 I2S ADC 에서 전송한 PCM data 는 Deserializer block 이 받아 RX FIFO 에 채워 넣게 되고, 그것을 읽어가는데는 두 가지 방법이 사용될 수 있다. 우선 CPU 가 직접 읽어가는 방법이 있다. CPU 가 polling 을 통해 data 를 읽어갈 수 있는데 I2S_STATUS 레지스터의 RxFifoLevel field 의 값을 보고 RX FIFO 가 empty 가 아니라면 계속 RX FIFO 에서 data 를 읽어갈 수 있다.

Interrupt 를 사용할 수도 있는데 I2S_CTRL 레지스터의 RxDMAIntEn bit 를 '1'로 세팅해 주고 RxFifoTh 의 값을 적절히 세팅하면 RX FIFO 의 Level 이 RxFifoTH 보다 커지면 interrupt 가 발생하게 된다. 그 interrupt 를 받을 때 마다 CPU 가 RX FIFO 에서 data 를 read 하면 된다.

또 한가지 방법은 DMA Controller 를 사용하는 방법이다. I2S Controller 는 RX FIFO 의 Level 이 RxFifoTH 보다 커지면 Rx DMA Request 신호를 생성하게 된다. DMA Controller 가 해상 신호를 받으면 DMA 가 동작하도록 configuration 하면 DMA Controller 가 RX FIFO 에서 data 를 읽어 Memory 로 전송하게 된다.

참고로 DMA Controller 를 사용하는 방법과 CPU 가 직접 data 를 RX FIFO 에서 읽어가는 방법을 동시에 사용하는 것은 권장되지 않는다. RxDMAIntEn bit 가 '1'로 세팅된 경우 Rx DMA Request 상황(즉 RX FIFO 의 Level 이 RxFifoTH 보다 큰 경우)에서 Interrupt 가 발생하여

I2S_STATUS 레지스터의 RxDMAInt bit 이 '1'로 set 되는데, DMA Controller 가 DMA service 를 완료하여 RX FIFO 의 Level 이 RxFifoTH 보다 작아지게 되면 자동으로 RxDMAInt bit 이 '0'으로 clear 된다. 따라서 Interrupt 가 발생하고 CPU 가 interrupt 를 check 하기 위해서 I2S_STATUS 레지스터를 읽었을 때 RxDMAInt bit 를 올바르게 읽지 못할 가능성이 존재한다. 따라서 DMA Controller 를 사용하는 경우에는 RxDMAIntEn bit 를 '0'으로 하는 것을 권장한다.

12.3.4.3. RX FIFO OverRun Error

Deserializer block 이 SDIN pin 으로 serial audio data 를 읽어왔는데 RX FIFO 에 더 이상 저장할 공간이 남아 있지 않은 경우 RX FIFO OverRun Error 가 발생한다. RX FIFO OverRun 이 발생하면 입력 받은 sample 을 저장하지 못하고 잃어버리게 되고 RX FIFO 에 저장 공간이 생길 때까지 입력 받은 sample 을 모두 잃어버리게 된다. I2S_CTRL 레지스터의 RxWLen 이 24bit/32bit 으로 configuration 되어 있는 경우에는 RX FIFO 에 저장할 수 있는 공간이 1 word 가 남아 있더라도 left/right channel sample 을 모두 저장할 수 없으므로 OverRun Error 가 발생하게 된다.

RX FIFO OverRun Error 상황이고 I2S_CTRL 레지스터의 RxFifoORIntEn bit 과 I2S_STATUS 레지스터의 EnRxFifoORInt bit 이 모두 1 인 경우 I2S_STATUS 레지스터의 RxFifoORInt bit 이 '1'로 set 되며 interrupt 가 발생된다. EnRxFifoORInt bit 은 hardware state 를 나타내는 Read Only bit 인데 이 hardware state 는 다음과 같이 변하게 된다.

- Reset 상태에서는 1의 값으로 초기화 된다.
- RxReset이 assert될 때 1의 값으로 초기화된다.
- RxFifoORInt가 1로 바뀌면 0으로 reset된다.
- EnRxFifoORInt가 0이고 RxFIFO에 2 word 이상의 data를 저장할 여유가 있으면 1로 바뀐다.

EnRxFifoORInt bit 이 위와 같이 변함으로 인해, RX FIFO OverRun Interrupt 가 발생하게 되었을 때, 사용자가 RxFifoORInt 를 0 으로 clear 하면 CPU 나 DMA Controller 가 RX FIFO 에서 두 개 이상의 data 를 읽어갈 때까지 Rx FIFO OverRun Interrupt 가 추가로 발생하는 것을 막아주게 된다.

12.3.4.4. Disabling Receive

Receive 동작을 disable 시키기 위해서는 I2S_CTRL 레지스터의 RxEn bit 을 '0'으로 clear 해 주면 된다. RxEn bit 가 '0'이 되면 더 이상 Rx DMA request 가 발생하지 않으며, 입력 받은 sample 을 더 이상 FIFO 에 채우지 않게 된다. 따라서 RX FIFO 에 남아 있는 data 는 CPU 가 직접 읽어가야 한다. 이 때 주의할 점은 RxEn bit 가 '0'이 되기 전에 RX DMA Request 가 발생하여 DMA Controller 가 동작을 시작하였을 때는 DMA Controller 의 동작이 끝날 때까지 기다렸다가 FIFO 에서 data 를 읽어가야 한다는 점이다. 정리하면 다음과 같은 절차로 Receive 동작을 disable 시킬 수 있다.

- (a) I2S_CTRL 레지스터의 RxEn bit 를 '0'으로 clear 함
- (b) DMA Controller 를 사용하고 있었으면 DMA Controller 를 disable 함
- (c) RX FIFO 의 내용을 CPU 가 읽어서 memory 에 저장함

참고로 RxWLen 이 24bit/32bit 으로 configuration 되었을 때는 맨 마지막 Left channel sample 만 FIFO 에 저장되고 Right channel sample 이 저장되지 않은 상황에서 receive 동작이 중지 될 수도 있으므로 유의해야 한다.

12.3.4.5. RxReset

Receive block 의 state 를 초기화하기 위해서는 I2S_CTRL 레지스터의 RxReset bit 를 '1'로 write 해 주면 된다. RxReset bit 을 '1'로 write 하면 RX FIFO 의 level 이 0 이 되고 RX FIFO 에 들어 있는 data 는 flushing 된다. Receive 를 enable 하기 전에 RxReset bit 을 '1'로 write 하여 state 를 초기화해 주는 것을 권장한다.

12.3.5. Interrupt

I2S Controller 는 Active high level sensitive interrupt signal 을 다음과 같은 경우에 생성한다.

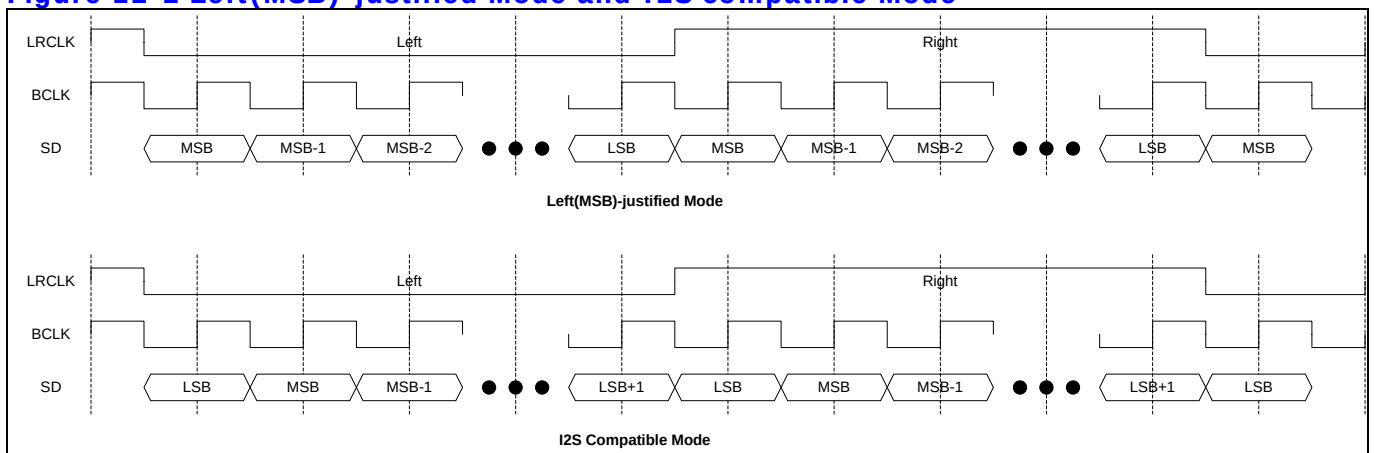
- TxDMAIntEn이 '1'이고 TX DMA Request 상황
- RxDMAIntEn이 '1'이고 RX DMA Request 상황
- TxFifoURIntEn이 '1'이고 EnTxFifoURInt가 '1'이고 TX FIFO UnderRun Error 발생
- RxFifoORIntEn이 '1'이고 EnRxFifoORInt가 '1'이고 RX FIFO OverRun Error 발생

단, DMA Interrupt(TxDMAInt 와 RxDMAInt)의 경우 DMA Controller 가 동작할 때는 자동으로 interrupt 가 low 로 떨어지게 되므로 그것을 피하려면 TxDMAIntEn 과 RxDMAIntEn 을 '0'으로 setting 해 주어야 한다.

12.3.6. Left-justified Mode / I2S Mode Selection

Transmit/Receive 각각 Left(MSB)-justified mode 와 I2S compatible mode 를 선택할 수 있다. Left-justified mode 와 I2S compatible mode 는 기본적으로 동일하고 Channel Selection Signal(LRCLK)이 transition 한 다음에 처음에 전송되는 data 가 어떤 channel 에 속하는 지만 차이가 있다. (아래 그림 참조)

Figure 12-2 Left(MSB)-justified Mode and I2S compatible Mode



I2S_CTRL 레지스터의 TxLJust bit, RxLJust bit 에 의해 Transmit/Receive 동작이 configuration 된다. 또한 I2S_CLKCTRL 레지스터의 LRClkInv bit 를 변경하여 LRCLK 의 polarity 를 변경할 수 있다.

12.3.7. WordLength와 FIFO data

Transmit/Receive 각각 I2S_CTRL 레지스터의 TxWLen, RxWLen field 에 의해 sample 하나의 sample 의 bit width 를 8bit/16bit/24bit/32bit 으로 변경할 수 있다. FIFO 는 32bit word 단위를 사용하므로 각 sample 이 어떤 방식으로 FIFO 에 저장되는지를 알아보면 다음과 같다.

12.3.7.1. 8 bit/sample

8 bit/sample 의 경우 다음과 같이 packing 되어 32bit FIFO 에 저장된다.

Figure 12-3 8 bit/sample data format

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RightSample1								LeftSample1								RightSample0								LeftSample0							

LeftSample/RigthSample 은 각각 Left Channel Sample, Right Channel Sample 을 의미하며, LeftSample0/RightSample0 가 LeftSample1/RigthSample1 에 비해 먼저 전송되고 Receive 의 경우에는 먼저 받은 sample 이 LeftSample0/RightSample0 가 된다. 당연히 TX FIFO 에 먼저 저장된 data 부터 전송이 되고, Receive 의 경우 먼저 받은 data 들이 RX FIFO 에 먼저 채워지게 된다.

12.3.7.2. 16 bit/sample

16 bit/sample 의 경우 다음과 같이 packing 되어 32bit FIFO 에 저장된다.

Figure 12-4 16 bit/sample data format

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RightSample																LeftSample															

하위 16bit 가 Left channel sample 이며 상위 16bit 가 Right channel sample 이다.

12.3.7.3. 24 bit/sample

24 bit/sample 의 경우 다음과 같이 32bit FIFO 에 저장된다.

Figure 12-5 24 bit/sample data format

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								LeftSample																							
Reserved								RightSample																							

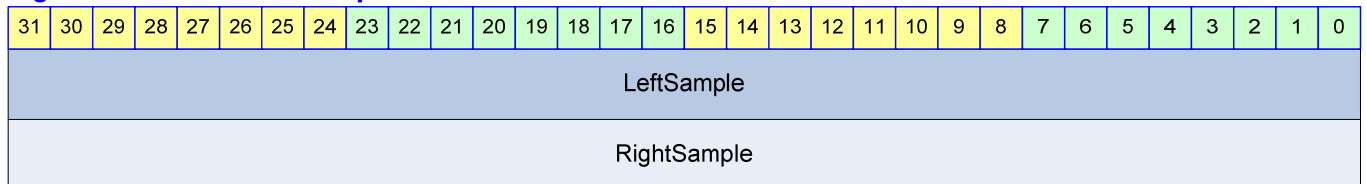
FIFO 에는 Left Channel sample 이 먼저 저장되고 Right Channel sample 이 나중에 저장된다. Transmit 의 경우 상위 8bit reserved 영역은 의미가 없고, Receive 의 경우 Reserved 영역이

자동으로 sign extension 되어 저장된다. 즉 Reserved 영역 8bit 가 23 번 bit 와 같은 값으로 저장된다.

12.3.7.4. 32 bit/sample

32 bit/sample 의 경우 다음과 같이 32bit FIFO 에 저장된다.

Figure 12-6 32 bit/sample data format



FIFO 에는 32bit Left Channel sample 이 먼저 저장되고 Right Channel sample 이 나중에 저장된다.

12.4. Register Description

12.4.1. Registers

Table 12-2 Registers

Offset	Name	R / W	Description	Init. value
0000h	I2S_CLKCTRL	RW	Clock Control Register	0000 0000h
0004h	I2S_CTRL	RW	Control Register	0201 0201h
0008h	I2S_STATUS	RW	Status Register	4500 0500h
000Ch	I2S_DATA	RW	FIFO	N/A

12.4.2. I2S_CLKCTRL Register

I2S_CLKCTRL 레지스터는 Digital Oscillator 의 동작과 Master/Slave configuration 을 control 하는 레지스터이다.

Figure 12-7 I2S_CLKCTRL Register

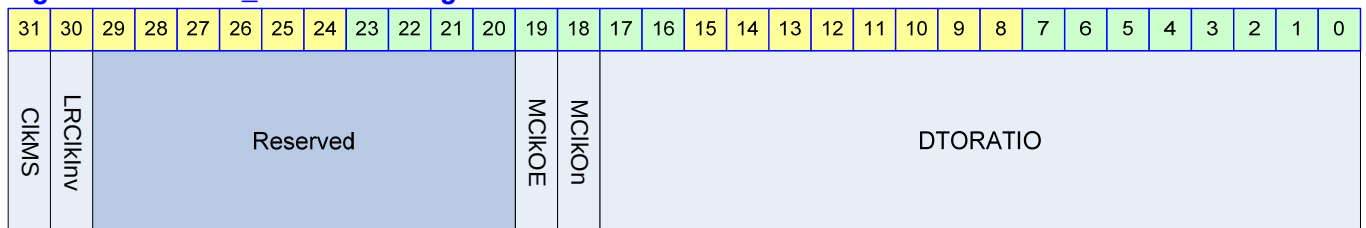


Table 12-3 I2S_CLKCTRL Register Description(Offset : 0000h)

Bits	Name	R / W	Description	Init. value
------	------	-------	-------------	-------------

[31]	ClkMS	RW	LRCLK/BCLK 의 master/slave mode 결정 0 : LRCLK/BCLK 의 slave 모드로 동작 1 : LRCLK/BCLK 의 master 모드로 동작	0b
[30]	LRClkInv	RW	LRCLK 의 polarity 0 : LRCLK 정상(Left 0, Right 1) 1 : LRCLK Inverting(Left 1, Right 0)	0b
[19]	MClkOE	RW	MCLK 의 Output Enable 신호 0 : MCLK 을 output pin 으로 출력하지 않음 1 : MCLK 을 output pin 으로 출력함 MCLK output pin 이 chip 전체의 GPIO 와 mux 될 수 있으므로 이 field 는 변경될 가능성이 크다.	0b
[18]	MClkOn	RW	Digital Oscillator 의 동작을 On/Off 시킴 0 : Digital Oscillator 가 동작하지 않음 1 : Digital Oscillator 가 동작함	0b
[17:0]	DTORATIO	RW	18bit DTO 비율 ' $2^{18} \times \text{MCLK}$ 의 주파수 / SYS_CLK 의 주파수'를 계산한 후 정수로 반올림한 값을 적어준다. MCLK 을 생성하고 싶지 않으면 0 을 write 한다.	0

ClkMS bit 이 '1'이면 MClkOn bit 은 반드시 '1'이어야 하고, DTORATIO field 가 올바르게 setting 되어야 한다. 참고로 MClkOn 이 '1'이고 MClkOE 가 '0'이면 MCLK 을 출력하지는 않고, BCLK 과 LRCLK 을 생성하기 위해서만 사용하게 된다.

12.4.3. I2S_CTRL Register

I2S_CTRL 레지스터는 Serializer/Deserializer Block 의 동작을 제어한다.

Figure 12-8 I2S_CTRL Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0							
RxEn		RxReset		RxDMAIntEn		RxFifoORIntEn		Reserved		RxFifoTH							TxEn		TxReset		TxDMAIntEn		Reserved		TxWLen		TxLJust		Reserved		TxFifoTH							

Table 12-4 I2S_CTRL Register Description(Offset : 0004h)

Bits	Name	R / W	Description	Init. value
[31]	RxEn	RW	Receive 동작의 Enable/Disable 0 : disable 1 : enable	0b
[30]	RxReset	W	Receive hardware reset 1 을 write 하면 Receive 동작 관련 hardware 를 초기화함 0 을 write 하는 것은 의미 없음	0b
[29]	RxDMAIntEn	RW	RX DMA request 에 Interrupt 발생을 enable 함 0 : DMA Request Interrupt disable	0b

			1 : DMA Request Interrupt enable	
[28]	RxFifoORIntEn	RW	RX FIFO OverRun Interrupt 발생을 enable 함 0 : RX FIFO OverRun Interrupt disable 1 : RX FIFO OverRun Interrupt enable	0b
[26:25]	RxWLen	RW	Receive 동작시 Word Length 00 : 8 bit/sample 01 : 16bit/sample 10 : 24bit/sample 11 : 32bit/sample	01b
[24]	RxLJust	RW	Receive 동작의 Left-justified mode 와 I2S Mode 선택 0 : I2S compatible mode 1 : Left(MSB)-justified mode	0b
[21:16]	RxFifoTH	RW	RX FIFO 의 DMA request threshold RX FIFO 의 level 이 RxFifoTH 보다 크면 DMA Request 가 생성된다.	1
[15]	TxEn	RW	Transmit 동작의 Enable/Disable 0 : disable 1 : enable	0b
[14]	TxReset	W	Transmit hardware reset 1 을 write 하면 Receive 동작 관련 hardware 를 초기화함 0 을 write 하는 것은 의미 없음	0b
[13]	TxDMAIntEn	RW	TX DMA request 에 Interrupt 발생을 enable 함 0 : DMA Request Interrupt disable 1 : DMA Request Interrupt enable	0b
[12]	TxFifoURIntEn	RW	TX FIFO UnderRun Interrupt 발생을 enable 함 0 : TX FIFO UnderRun Interrupt disable 1 : TX FIFO UnderRun Interrupt enable	0b
[10:9]	TxWLen	RW	Transmit 동작시 Word Length 00 : 8 bit/sample 01 : 16bit/sample 10 : 24bit/sample 11 : 32bit/sample	01b
[8]	TxLJust	RW	Transmit 동작의 Left-justified mode 와 I2S Mode 선택 0 : I2S compatible mode 1 : Left(MSB)-justified mode	0b
[5:0]	TxFifoTH	RW	TX FIFO 의 DMA request threshold TX FIFO 의 level 이 TxFifoTH 보다 작거나 같으면 DMA Request 가 생성된다.	1

12.4.4. I2S_STATUS Register

I2S_STATUS 레지스터는 I2S Controller 의 동작 상황을 알려주기 위한 레지스터이다.

Figure 12-9 I2S_STATUS Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Reserved		EnRx Fifo ORInt		Rx FifoDepth				Reserved	Rx FifoLevel							Reserved	EnTx Fifo URInt		TxDMAInt	Tx Fifo URInt	Tx FifoDepth				Reserved	Tx FifoLevel						

Table 12-5 I2S_STATUS Register Description(Offset : 0008h)

Bits	Name	R / W	Description	Init. Value
[30]	EnRx Fifo ORInt	R	Rx Fifo OverRun Interrupt Enable H/W State 0 : Rx Fifo OverRun Interrupt 발생 불가능 1 : Rx Fifo OverRun Interrupt 발생 가능 I2S_CTRL 레지스터의 RxReset 에 '1'을 write 하면 1로 바뀐다.	1b
[29]	RxDMAInt	R	Receive DMA Request Interrupt 0 : DMA Request 를 하고 있지 않음 1 : DMA Request 를 하고 있음 Receive DMA Request 를 하고 있으면 '1'이 되고 DMA Controller 나 CPU 가 RX FIFO 를 읽어가면 자동으로 '0'으로 clear 된다. I2S_CTRL 의 RxDMAIntEn 의 값과 관계 없이 변하고, RxDMAIntEn 이 '1'이면 CPU 로 interrupt 를 보내게 된다.	0b
[28]	Rx Fifo ORInt	RW	Receive FIFO OverRun Interrupt 발생 0 : OverRun 상황 없었음 1 : OverRun 상황 있었음 이 bit 는 I2S_CTRL 레지스터의 Rx Fifo ORIntEn 이 '1'일 때만 1로 set 되고 이 bit 가 1 이면 CPU 로 interrupt 를 보내게 된다. CPU 가 '1'을 write 하여 clear 할 수 있고, clear 하기 전까지는 '0'으로 reset 되지 않는다.	0b
[27:24]	Rx FifoDepth	R	Receive FIFO 의 Depth Maximum 6 까지의 값을 가진다. Receive FIFO 는 $2^{\text{Rx FifoDepth}}$ 개의 Entry 를 가지게 된다.	5h
[22:16]	Rx FifoLevel	R	현재 RX FIFO 의 level 로 0 에서 $2^{\text{Rx FifoDepth}}$ 의 값을 가진다. I2S_CTRL 레지스터의 RxReset 에 '1'을 write 하여 강제로 0 으로 초기화할 수 있다.	0
[14]	EnTx Fifo URInt	R	Tx Fifo UnderRun Interrupt Enable H/W State 0 : Tx Fifo UnderRun Interrupt 발생 불가능 1 : Tx Fifo UnderRun Interrupt 발생 가능 I2S_CTRL 레지스터의 TxReset 에 '1'을 write 하면 0로 바뀐다.	0b
[13]	TxDMAInt	R	Transmit DMA Request Interrupt 0 : DMA Request 를 하고 있지 않음 1 : DMA Request 를 하고 있음 Transmit DMA Request 를 하고 있으면 '1'이 되고 DMA Controller 나 CPU 가 TX FIFO 에	0b

			data 를 채우면 자동으로 '0'으로 clear 된다. I2S_CTRL 의 TxDMAIntEn 의 값과 관계 없이 변하고, TxDMAIntEn 이 '1'이면 CPU 로 interrupt 를 보내게 된다.	
[12]	TxFifoURInt	RW	Transmit FIFO UnderRun Interrupt 발생 0 : UnderRun 상황 없었음 1 : UnderRun 상황 있었음 이 bit 는 I2S_CTRL 레지스터의 TxFifoURIntEn 이 '1'일 때만 1 로 set 되고 이 bit 가 1 이면 CPU 로 interrupt 를 보내게 된다. CPU 가 '1'을 write 하여 clear 할 수 있고, clear 하기 전까지는 '0'으로 reset 되지 않는다.	0b
[11:8]	TxFifoDepth	R	Transmit FIFO 의 Depth Maximum 6 까지의 값을 가진다. Transmit FIFO 는 $2^{\text{TxFifoDepth}}$ 개의 Entry 를 가지게 된다.	5h
[6:0]	TxFifoLevel	R	현재 TX FIFO 의 level 로 0 에서 $2^{\text{TxFifoDepth}}$ 의 값을 가진다. I2S_CTRL 레지스터의 TxReset 에 '1'을 write 하여 강제로 0 으로 초기화할 수 있다.	0

RxDMAInt 와 TxDMAInt 는 DMA Controller 를 사용하는 경우 CPU 의 동작과는 무관하게

toggle 되므로 RxDMAIntEn, TxDMAIntEn 을 모두 '0'으로 setting 하여 CPU 로 interrupt 가
전달되지 않도록 하여야 한다.

RxFifoLevel 의 maximum 값은 $2^{\text{RxFifoDepth}}$ 이고, 이 때 Receive FIFO 가 full 인 상태가 된다.

TxFifoLevel 의 maximum 값은 $2^{\text{TxFifoDepth}}$ 이고, 이 때 Transmit FIFO 가 full 인 상태가
된다. RxFifoLevel 과 TxFifoLevel 이 0 인 경우 각 FIFO 가 empty 상황임을 나타낸다.

12.4.5. I2S_DATA Register

I2S_DATA 레지스터는 TX FIFO 와 RX FIFO 를 access 할 수 있는 레지스터로 Word(32bit)
단위로 access 해야 한다.

Figure 12-10 I2S_DATA Register

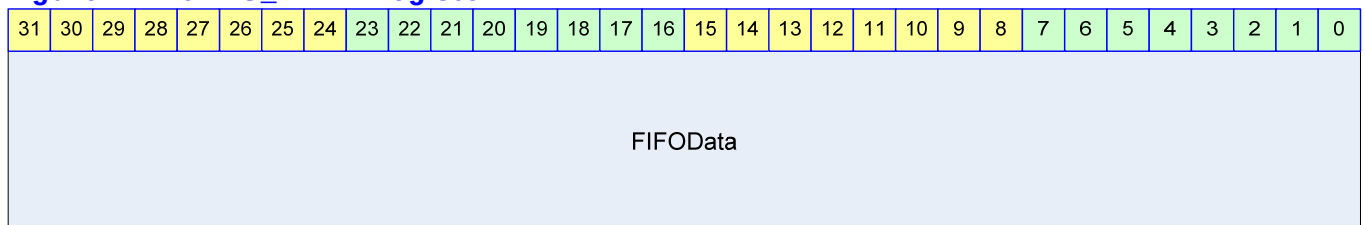


Table 12-6 I2S_DATA Register Description(Offset : 000Ch)

Bits	Name	R / W	Description	Init. Value
[31:0]	FIFOData	RW	Read 의 경우 RX FIFO 에서 data 를 읽어서 return 하고 Write 의 경우 TX FIFO 에 data 를	N/A

			write 하게 된다.	
--	--	--	--------------	--

TX FIFO 가 full 일 때 I2S_DATA 에 data 를 write 하게 되면 write 한 data 는 FIFO 에 write 되지 않는다. RX FIFO 가 empty 일 때 I2S_DATA 를 read 하게 되면 0 을 리턴한다.

13. I2C Master

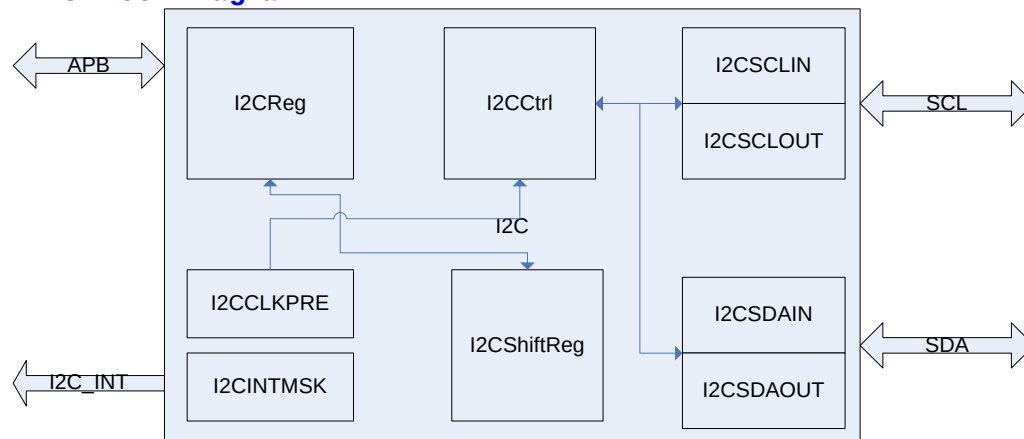
13.1. Overview

13.1.1. Feature

- APB Interface
- 7/10bit Addressing
- HS-Mode Support
- Multi-Master Support
- Interrupt Signal Support

13.1.2. Block Diagram

Figure 13-1 I2C Block Diagram



13.2. Operation

13.2.1. Overview

- 7/10bit addressing
- Clock Synchronization
- Arbitration
- HS-Mode

13.3. Signal Description

13.3.1. I2C Module Signal

Table 13-1 I2C Module Signal

Signal Name	Direction	Description	Init. Value
SDAo	Output	SDA line Output	
SDAi	Input	SDA line Input	
SCLo	Output	SCL line Output	
SCLi	Input	SCL line Input	
n_SDAEn	Output	SDA 3-State Buffer Enable	
n_SCLEn	Output	SCL 3-State Buffer Enable	
I2CInt	Output	I2C Interrupt Signal	

13.4. Register Description

13.4.1. Registers

Table 13-2 Registers

Offset	Name	R / W	Description	Init. Value
0000h	I2CCON	RW	I2C Control Register	0000 0000h
0004h	I2CSTA	RW	I2C Status Register	0000 0000h
0010h	I2CDAT	RW	I2C Data Register	0000 0000h

13.4.2. I2CCON (I2C Control) Register

Figure 13-2 I2CCON Register Bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																			TxPre									IntPendFlag	OpMode	AckEn	TxRxIntEn

Table 13-3 I2CCON Register Description (Offset : 0000 0000h)

Bits	Name	R / W	Description	Init. Value
[12:4]	TxPre	R/W	I2C Clock Prescaler Value Bus Clock Divide	111111111
[3]	IntPendFlag	R/C	1: Pending Status (READ) Pending Clear (Write) 0: Not Pending (READ)	0
[2]	OpMode	R/W	Operation Mode 1: Master Tx 0: Master Rx	1
[1]	AckEn	R/W	Acknowledge Enable 1: Send ACK 0: Send NACK	0
[0]	TxRxIntEn	R/W	Interrupt Enable 1: Interrupt Enable 0: Interrupt Disable	0

13.4.3. I2CSTA (I2C Status) Register

Figure 13-3 I2CSTA Register Bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																															
BusError																															
AckValue																															
ArbitSta																															
BusBusy																															
StartStop																															
OutputEn																															

Table 13-4 I2CSTA Register Description (Offset : 0000 0004h)

Bits	Name	R / W	Description	Init. Value
[5]	OutputEn	R/W	//Reserved 1: Output Enable 0: Output Disable	0
[4]	StartStop	W	Start/ Stop 1: Start 0: Stop 세팅되고 실행되면 자동으로 clear 되는 구조	?
[3]	BusBusy	R	1: Bus Busy 0: Bus not Busy 전송을 개시할 경우 이 bit 를 보고 다른 mater 가 사용을 하고 있는지 알 수 있다.	0
[2]	ArbitSta	R	Arbitration Status 1: Arbitration Lost 0: Arbitration OK	0

[1]	AckValue	R	Received Ack Value (Tx Mode 일 경우에 유효) 1: Receive NACK 0: Receive ACK	0
[0]	BusError	R	Bus Error Status 1: Bus Error (다른 사용 중에 start 시켰을 경우) 0: Bus no Error	0

13.4.4. I2CDAT (I2C Data) Register

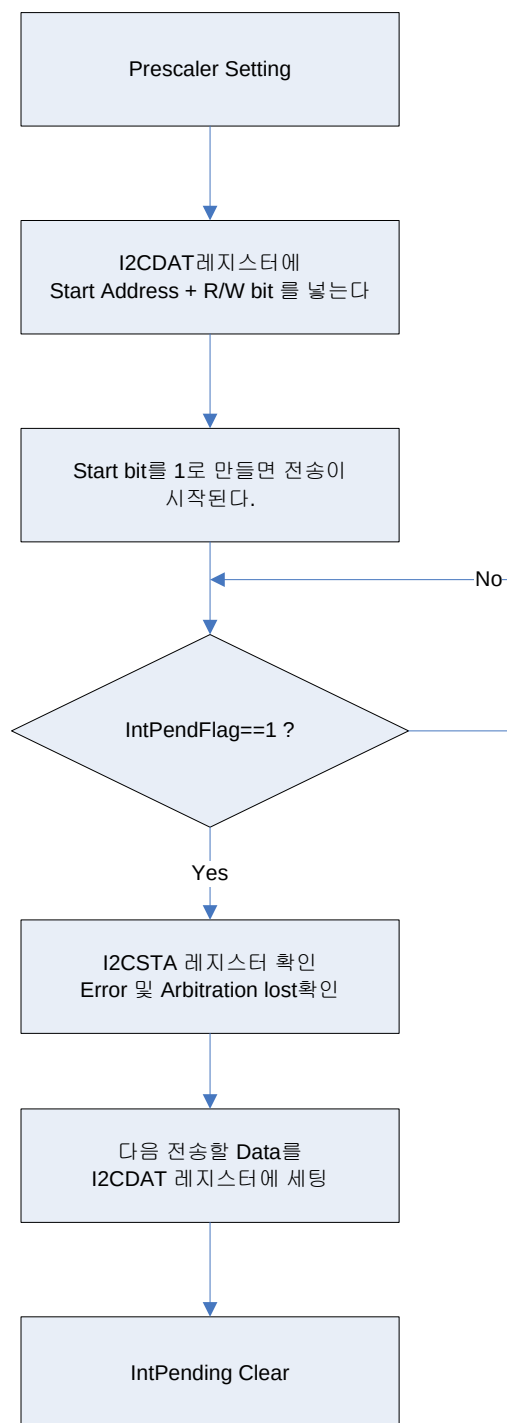
Figure 13-4 I2CDAT Register Bits

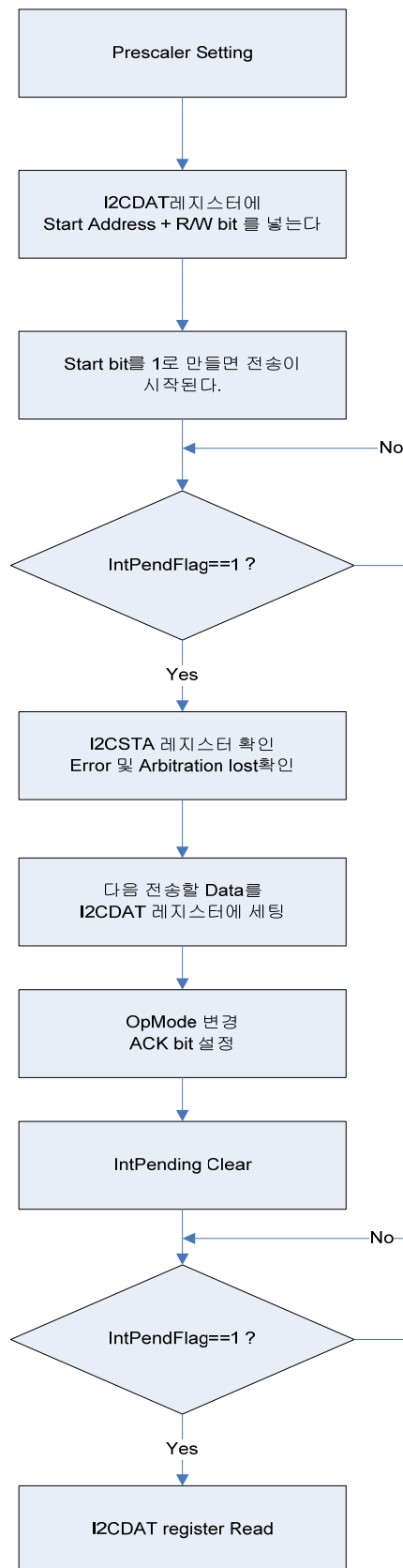
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																								DATA							

Table 13-5 I2CDAT Register Description (Offset : 0000 0008h)

Bits	Name	R / W	Description	Init. Value
[7:0]	I2C_DATA	RW	Send or Received Data Tx 로 사용될 때는 보낼 데이터를 쓰고 Rx 로 사용할 때는 받은 데이터를 읽는 용도로 사용된다. Rx 모드가 아닐 때의 값은 유효하지 않음	0

13.5. Operation Sequence





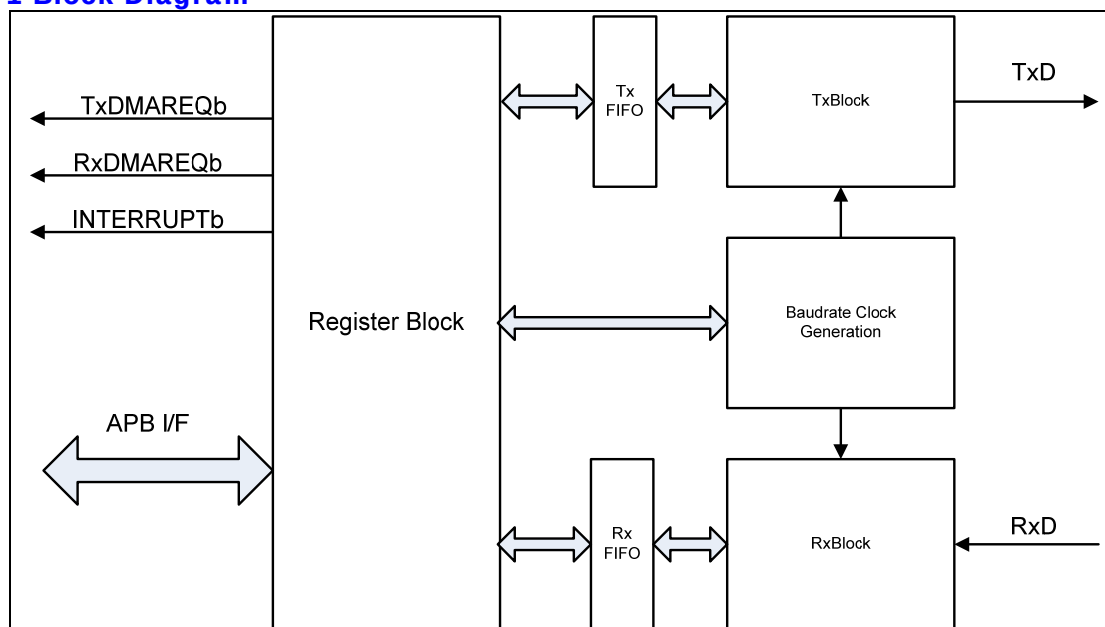
14. UART

14.1. Overview

14.1.1. Features

- Support 2 Channel
- Baudrate : up to 115200 bps
- 32 Byte FIFO for Tx/Rx(Channel 0)
- 8 Byte FIFO for Tx/Rx(Channel 1)
- Digital Frequency Synthesizer
- Receive Time Out
- Included Low pass filter at Rx D Signal
- DMA Support for Channel 0

Figure 14-1 Block Diagram



14.2. Register

Table 14-1 UART Register

Offset	Name	R / W	Description	Init. value
--------	------	-------	-------------	-------------

0x0000	UART0CMD	RW	UART 0 Master command register	0000 0000h
0x0004	UART0STS	RW	UART 0 Status register	0000 0000h
0x0008	UART0BAUD	RW	UART 0 Buadrate Gen Register	0000 0000h
0x000C	UART0TXDAT	W	UART 0 TxFIFO Write register	0000 0000h
0x0010	UART0RXDAT	R	UART 0 Rx FIFO Read register	0000 0000h
0x0014	UART0RX TMOUT	RW	UART 0 Receive time out register	0000 0000h
0x0040	UART1CMD	RW	UART 1 Master command register	0000 0000h
0x0044	UART1STS	RW	UART 1 Status register	0000 0000h
0x0048	UART1BAUD	RW	UART 1 Buadrate Gen Register	0000 0000h
0x004C	UART1TXDAT	W	UART 1 Tx FIFO Write register	0000 0000h
0x0050	UART1RXDAT	R	UART 1 Rx FIFO Read register	0000 0000h
0x0054	UART1RX TMOUT	RW	UART 1 Receive time out register	0000 0000h

2 Channel 의 UART 는 동일한 기능을 제공한다. 각 Register 설명은 한 Channel 에 대해서만 설명하기로 한다.

14.2.1. Master Command Register

Figure 14-2 Master Command Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
UARTEN	INTEN	RTINTEN	SWRST	DMAREQEN	PARITY		DATA	STOP	LPBACKEN	Reserved							TXFIFOINTEN	Reserved	TxWaterLevel					RXFIFOINTEN	Reserved	RxWaterLevel					

UART 의 Master Command 레지스터이다. UART 의 enable 및 각종 제어 부분을 설정한다.

Table 14-2 Master Command Register Description

Bits	Name	R / W	Description	Init. Value
31	Enable UART	R/W	UART 동작의 enable/disable 0 : disable 1 : enable	0b
30	Enable interrupt generation	R/W	UART 의 인터럽트 발생을 Enable 혹은 Disable 한다. 0 : Disable 1 : Enable	0b
29	Enable RT Out Interrupt	R/W	UART 의 Receive timeout interrupt 를 Enable 혹은 Disable 한다. 0 : Disable 1 : Enable	0b
28	Software Reset	W	UART 의 모든 상태를 Clear 한다. 0 : no effect 1 : Software reset(auto cleared) Software reset 이 assert 되면 Status 레지스터의 parity/frame/overrun/receive time out error bit 과 interrupt status bit 이 clear 되고 TX Status 가 idle 로 돌아가며 TX FIFO 가 Flush 된다.	N/A

27	Enable DMA Request	R/W	DMA Request 를 Enable 혹은 Disable 한다. 0 : Disable 1 : Enable	0b
26:25	Parity	R/W	UART 의 Parity mode 를 설정한다. 0x : None 10 : Even parity 11 : Odd parity	00b
24	Data bit	R/W	UART 의 Databit 를 설정한다. 0 : 7 bit 1 : 8 bit	0b
23	Stop bit	R/W	UART 의 Stop bit 를 설정한다. 0 : 1 Stop bit 1 : 2 Stop bit	0b
22	Enable Loopback	R/W	UART 의 Loopback mode 를 설정한다. 0 : Disable 1 : Enable	0b
15	TxFIFOINTEN	R/W	TxFIFO 의 Interrupt 를 Enable 여부를 설정한다. 0 : Disable 1 : Enable TxFIFOINTEN 이 1 이고 TxFIFO 의 Level 이 TxWaterLevel 보다 적거나 같으면 Interrupt 가 발생하고 Status Register TxFIFOINT 가 '1'로 set 된다.	0b
13:8	Tx Water Level	R/W	TX FIFO 의 Water Level 을 설정한다. TxFIFO 에 Tx Water Level 보다 많거나 같은 데이터가 있으면 인터럽트 혹은 DMA 요청이 설정에 따라서 발생한다.	000000b
7	RxFIFOINTEN	R/W	RxFIFO 의 Interrupt 를 Enable 여부를 설정한다. 0 : Disable 1 : Enable RxFIFOINTEN 이 1 이고 RxFIFO 의 Level 이 RxWaterLevel 보다 크면 Interrupt 가 발생하고 Status Register RxFIFOINT 가 '1'로 set 된다.	0b
5:0	Rx Water Level	R/W	Rx FIFO 의 Water Level 을 설정한다. RxFIFO 에 Rx Water level 보다 적은 데이터가 있으면 인터럽트 혹은 DMA 요청이 설정에 따라서 발생한다.	000000b

주의 사항 : Software reset bit 가 '1'이 값을 Master command register 에 write 하면 다른 field 는 변경되지 않는다.

14.2.2. Status Register

Figure 14-3 Status Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
INTERRUPT	MAX FIFO Depth				TXBUSY	RXBUSY	PERITYINT	FRAMEINT	OVERRUNINT	RTOUTINT	Reserved						TxFIFOINT	TXDMAREQ	TxFIFOCNT						RxFIFOINT	RXDMAREQ	RxFIFOCNT					

Table 14-3 Status Register Description

Bits	Name	R / W	Description	Init. Value
------	------	-------	-------------	-------------

31	INTERRUPT	R	UART 에서 인터럽트가 발생하였는지 여부를 알려주는 레지스터 0 : None 1 : Interrupt Asserted	0b
30:28	MAX FIFO Depth	R	UART 의 FIFO 의 최대 Depth 를 알려준다. 버전별로 틀리므로 이 레지스터를 읽어서 크기를 확인할 수 있다. 000 : FIFO None 001 : 8 depth 010 : 16 depth 011 : 32 depth Others : Reserved	CH0:011b CH1:001b
27	TxBUSY	R	TxModule 의 동작이 현재 전송 동작 중이라는 것을 알려준다. 0 : IDLE or wait tx fifo fill 1 : Transmit busy	0b
26	RxBUSY	R	Rx Module 의 동작이 현재 수신 동작 중이라는 것을 알려준다. 0 : IDLE or wait state 1 : Receive Busy	0b
25	Parity Error interrupt	R/W	Rx Buffer 에 수신된 데이터에 Parity Error 가 발생하였음을 알려준다. 0 : none parity error 1 : parity error 1 을 write 하면 clear 된다.	0b
24	Frame error interrupt	R/W	RxBuffer 에 수신된 데이터에 Frame Error 가 있음을 알려준다. 0 : none frame error 1 : frame error 1 을 write 하면 clear 된다.	0b
23	Overrun error	R/W	Rx FIFO 가 Overrun 되었음을 알려준다. 0 : None overrun error 1 : Overrun error 1 을 write 하면 clear 된다.	0b
22	Receive time out	R	Receive time out 이 발생하였음을 알려준다. 0 : none time out 1 : Time out 이 bit 만 clear 시키고 싶으면 master command register 29 번 bit 를 clear 하면 된다.	0b
15	TxFIFOINT	R	TxFIFO Interrupt 발생 여부를 알려준다. 0 : Tx FIFO Interrupt 발생 안함 1 : Tx FIFO Interrupt 발생	0b
14	TxFIFO DMA Request	R	TxFIFO water level DMA request 0 : None request 1 : DMA Request	0b
13:8	TxFIFO count	R	Tx Buffer 의 FIFO 에 남아 있는 데이터의 개수를 확인할 수 있다.	0
7	RxFIFOINT		RxFIFO Interrupt 발생 여부를 알려준다.	

			0 : RxFIFO Interrupt 발생 안함 1 : RxFIFO Interrupt 발생	
6	RxFIFO Count	R	RxFIFO water level DMA request 0 : None request 1 : DMA Request	0b
5:0	RxFIFO Count		Rx Buffer 의 FIFO 에 수신된 데이터의 Count 의 개수를 확인 할 수 있다.	0

14.2.3. Baudrate Gen Register

Figure 14-4 Baudrate Gen Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																BaudRate GEN															

Table 14-4 Baudrate Gen Register Description

Bits	Name	R / W	Description	Init. Value
31:16	Reserved			
15:0	Baudrate Gen	R/W	Baudrate generation clock value	0000h

14.2.4. TxFIFO Write Register

Figure 14-5 TxFIFO Write Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																								TxFIFO Data							

Table 14-5 TxFIFO Write Register Description

Bits	Name	R / W	Description	Init. Value
31:8	Reserved			N/A
7:0	TxFIFO Data	W	TxFIFO Data	N/A

14.2.5. RxFIFO Read Register

Figure 14-6 RxFIFO Read Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																								Rx FIFO Data							

Table 14-6 RxFIFO Read Register Description

Bits	Name	R / W	Description	Init. Value
31:8	Reserved			N/A
7:0	RxFIFO Data	R	RxFIFO Data	00h

14.2.6. Receive Time Out Count Register

Figure 14-7 Receive Time Out Count Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved												RTIMEOUT																			

Table 14-7 Receive Time Out Count Register

Bits	Name	R / W	Description	Init. Value
31:20	Reserved			N/A
19:0	RTIMEOUT	R/W	Receive Time Out	00000h

14.3. Operation

14.3.1. Baudrate Clock Generation

UART 는 PCLK 을 Digital Synthesizer 로 분주하여 Rx/Tx 의 속도를 결정하게 되는데 다음과 같이 계산된 값을 Baudrate Gen 레지스터에 적어 주어야 한다.

$$\text{Baudrate Gen Value} = \text{RoundtoNearInteger}[2^{16} * (\text{Baudrate} * 16) / 50\text{MHz}]$$

예를 들어 115200 bps 를 만들고 싶으면

$$\text{Baudrate Gen Value} = \text{RoundtoNearInteger}[2^{16} * (115200\text{Hz} * 16) / 50\text{MHz}]$$

$$= \text{RoundToNearInteger}[65536 * 115200 * 16 / 50000000]$$

$$= \text{RoundToNearInteger}[2415.9191]$$

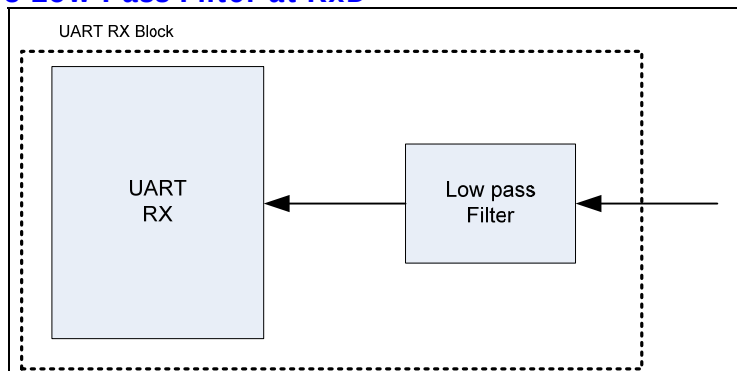
$$= 2416$$

2416 의 값을 Baudrate Gen 레지스터에 적어주면 된다.

14.3.2. Low Pass Filter

RXD 입력단에 Low pass Filter 를 내장하여 외부 신호의 Noise 에 강한 특성을 가지고 있다.

Figure 14-8 Low Pass Filter at Rx D



Startbit 에서 noise 가 발생하여도 bit time 의 23.5%까지는 noise 로 인식을 하게 된다. 그 이상의 비율로 signal 이 detect 되면 그것은 올바른 signal 로 인식하게 된다.

내부의 start signal 판별 회로에 의해서 한번 더 filtering 됨으로써 전체적으로 안정적인 동작을 수행할 수 있다.

14.3.3. DMA / Interrupt Water Level

Channel 0 은 32 Depth 의 FIFO 을 Rx/Tx 각각 가지고 있고, Channel 1 은 8 Depth 의 FIFO 를 Rx/Tx 각각 가지고 있다. FIFO 에 저장된 data 의 개수에 의해서 DMA Request 가 발생하거나 Interrupt 가 발생하게 된다. Rx/Tx 각각 FIFO Interrupt enable bit(RxFIFOINTEN/TxFIFOINTEN)을 가지고 있고, Interrupt 발생 여부를 알 수 있는 flag(RxFIFOINT/TxFIFOINT)가 있다. DMA mode 로 동작시킬 때(Enable DMA Request 가 1 이고 DMA Controller 를 activation 시킨 경우)는 Rx/Tx FIFO 의 level 이 DMA 상황에 따라서 변하므로 RxFIFOINTEN 과 TxFIFOINTEN 을 '1'로 세팅하지 말아야 한다.

Figure 14-9 FIFO Water Level

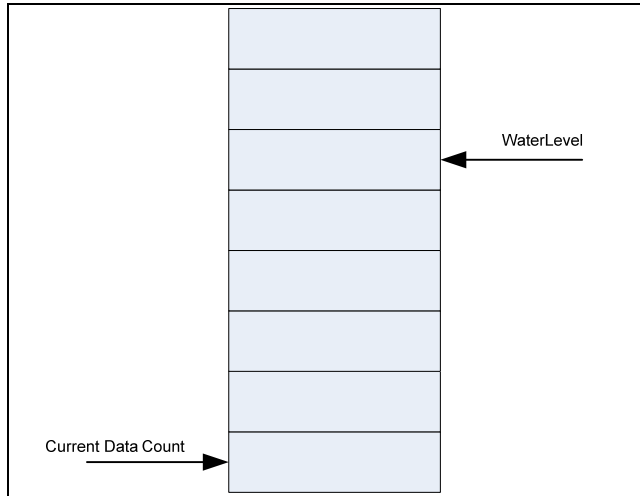
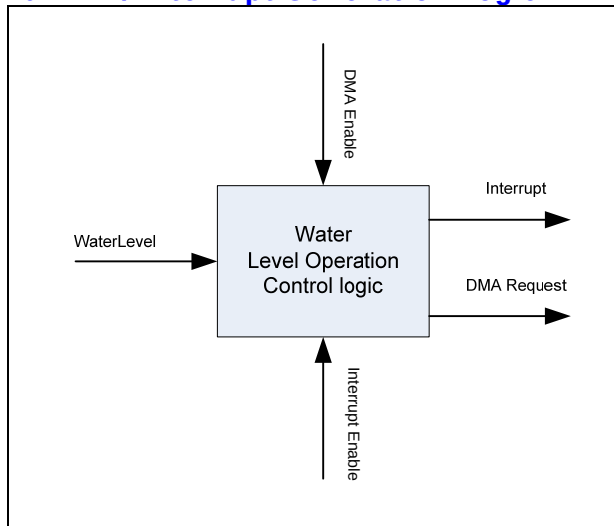


Figure 14-10 DMA/ Interrupt Generation Logic



Tx의 경우 Water Level 보다 FIFO의 data 개수가 적거나 같으면 event가 발생하는데, 이 event는 DMA가 활성화되어 있는 경우 DMA Request를 생성하고 그렇지 않으면 interrupt를 발생시킨다.

Rx의 경우 Water Level 보다 FIFO의 data 개수가 많아지면 event가 발생하는데, 이 event는 DMA가 활성화되어 있는 경우 DMA Request를 생성하고 그렇지 않으면 interrupt를 발생시킨다. Interrupt 신호는 Active High Level triggered signal을 사용한다.

14.3.4. Receive Time Out Interrupt

Receive Time Out은 RXD 핀으로 데이터가 일정 시간 동안 수신되지 않는 경우 발생하게 된다. 이 기능을 활성화시키기 위해서는 Master command 레지스터의 29번 bit를 1로 세팅하면 된다.

Time Out 이 걸리는 시간은 Receive Time Out Count 레지스터의 값에 의해 결정되는데, 일단 활성화되면 수신이 되지 않은 시간을 16 배의 baudrate clock 으로 count 하게 된다.

이 counter 의 값이 Receive Time Out Count 레지스터의 값과 같아지면 Receive Time Out Interrupt 가 발생한다.

14.3.5. Loopback Mode

UART 의 기능을 테스트하기 위해서 송신 데이터가 그대로 수신 데이터로 수신되는 모드이다. 이 기능은 Master command 레지스터의 22 번 bit 를 1 로 세팅하여 활성화시킬 수 있다. 단 loopback mode 에서도 송신데이터는 외부 핀으로 전송된다.

15. GPIO

15.1. Overview

32bit GPIO port 를 가지는 GPIO Controller 를 두 개 제공한다.

15.1.1. Features

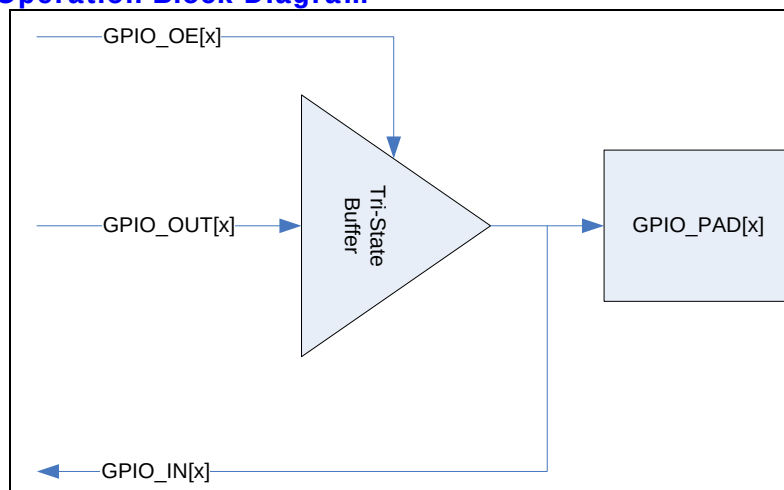
- 32bit GPIO x 2
- Interrupt Generation
 - Active high/low level interrupt
 - Positive/Negative edge interrupt
 - Both edge interrupt

15.2. Operation

15.2.1. GPIO Operation

GPIO 동작은 다음과 같은 block diagram 으로 이해할 수 있다. 사용자는 출력하고 싶은 data 가 있으면 GPIO_OUT 레지스터에 값을 쓰고, GPIO_OE 레지스터에 출력 pin 에 해당 하는 bit 를 '1'로 write 해 주면 tri-state buffer 가 enable 되어 외부 PAD 로 출력이 이루어지게 된다. GPIO port 로 입력 받은 signal 은 GPIO_IN 레지스터를 읽음으로써 detect 할 수 있다.

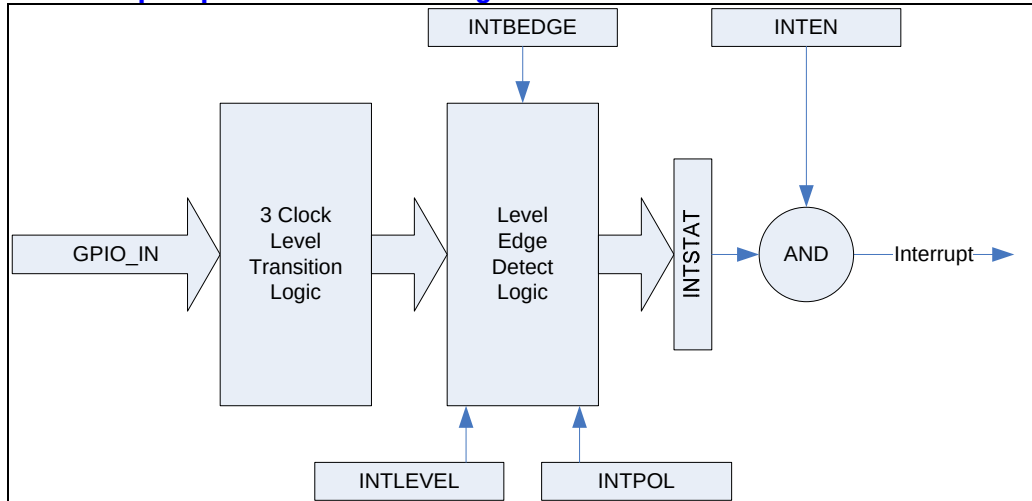
Figure 15-1 GPIO Operation Block Diagram



15.2.2. Interrupt Operation

외부에서 PAD 에서 입력되는 신호를 detect 하여 Interrupt Controller 로 interrupt 를 발생시킬 수 있다.

Figure 15-2 Interrupt Operation Block Diagram



위의 그림과 같이 GPIO PAD 에서 오는 GPIO_IN 신호는 3 Clock Level Transition Logic 을 지나게 된다. 3 Clock Level Transition Logic 은 PAD 에서 들어오는 노이즈를 걸러내는 일을 하는데, 총 3 APB Clock Cycle 동안 GPIO_IN 이 stable 한 경우에 그것을 올바른 신호로 인식하고, GPIO_IN 이 3 cycle 보다 작은 시간 동안 변하는 경우에는 노이즈로 인식하여 입력 신호가 바뀌었다고 판단하지 않게 된다. 처리가 끝난 신호는 Level/Edge Detect Logic 에 인가 되어 Level/Edge 의 detect 결과가 INTSTAT 레지스터로 출력 된다. 그 후에 INTSTAT 레지스터와 INTEN 레지스터를 bitwise AND 하고 그 결과값이 0 이 아니면 active high level interrupt 가 생성되게 된다. Level/Edge Detect Logic 은 INTLEVEL/INTPOL/INTBEDGE 레지스터에 의해서 control 되는데 우선 INTLEVEL 은 Level interrupt 인지 edge interrupt 인지를 결정하게 된다. INTLEVEL[x]가 1 이면 GPIO[x]에 입력되는 signal 은 level interrupt 로 처리되고 0 이면 edge interrupt 로 처리된다. INTBEDGE 는 interrupt 가 both edge 에서 동작하도록 하는 레지스터이다. INTBEDGE[x]가 1 이면 GPIO[x]에 입력되는 signal 의 rising edge 와 polling edge 양쪽에서 interrupt 가 생성된다. 단 이때는 INTLEVEL[x]가 0 이어야 하고 INTLEVEL[x]가 1 이면 INTBEDGE[x] 값은 무시된다. INTPOL 는 GPIO 에 인가되는 interrupt signal 의 polarity 를 정하게 된다. INTPOL[x]가 1 이면 active high(edge 의 경우 rising edge)가 되고 0 이면 active low(edge 의 경우 polling edge)가 된다. INTLEVEL[x]가 0 이고 INTBEDGE[x]가 1 이면 both edge 를 detect 하는 것이므로 INTPOL[x]는 무시된다.

15.2.2.1. Interrupt Operation Example

- GPIO PAD[x]에서 받는 신호가 high level에서 interrupt 생성을 원하는 경우

INTLEVEL[x] : 1 로 세팅

INTPOL[x] : 1 로 세팅

INTEN[x] : 1 로 세팅

- GPIO PAD[x]에서 받는 신호의 falling edge에서 interrupt 생성을 원하는 경우

INTLEVEL[x] : 0 로 세팅

INTPOL[x] : 0 로 세팅

INTBEDGE[x] : 0 로 세팅

INTSTAT[x] : 0 으로 clear 함

INTEN[x] : 1 로 세팅

- GPIO PAD[x]에서 받는 신호의 rising/falling edge 모두에서 interrupt 생성을 원하는 경우

INTLEVEL[x] : 0 로 세팅

INTBEDGE[x] : 1 로 세팅

INTSTAT[x] : 0 으로 clear 함

INTEN[x] : 1 로 세팅

15.3. Register Description

15.3.1. Registers

Table 15-1 Registers

Offset	Name	R / W	Description	Init. value
0000h	GPIO0_OE	RW	(GPIO0) Output Enable Register	0000 0000h
0004h	GPIO0_IN	R	(GPIO0) Input Register	N/A
0008h	GPIO0_OUT	RW	(GPIO0) Output Register	1234 5678h
000Ch	GPIO0_INTSTAT	RW	(GPIO0) Interrupt Status Register	N/A
0010h	GPIO0_INTEN	RW	(GPIO0) Interrupt Enable Register	0000 0000h
0014h	GPIO0_INTLEVEL	RW	(GPIO0) Interrupt Level/Edge Selection Register	0000 0000h
0018h	GPIO0_INTPOL	RW	(GPIO0) Interrupt Polarity Register	0000 0000h
001Ch	GPIO0_INTBEDGE	RW	(GPIO0) Interrupt Both Edge Register	0000 0000h
0020h	GPIO1_OE	RW	(GPIO1) Output Enable Register	0000 0000h

0024h	GPIO1_IN	R	(GPIO1) Input Register	N/A
0028h	GPIO1_OUT	RW	(GPIO1) Output Register	1234 5678h
002Ch	GPIO1_INTSTAT	RW	(GPIO1) Interrupt Status Register	N/A
0030h	GPIO1_INTEN	RW	(GPIO1) Interrupt Enable Register	0000 0000h
0034h	GPIO1_INTLEVEL	RW	(GPIO1) Interrupt Level/Edge Selection Register	0000 0000h
0038h	GPIO1_INTPOL	RW	(GPIO1) Interrupt Polarity Register	0000 0000h
003Ch	GPIO1_INTBEDGE	RW	(GPIO1) Interrupt Both Edge Register	0000 0000h

15.3.2. GPIO Output Enable Register (GPIO0_OE / GPIO1_OE)

GPIO Output Enable 레지스터는 GPIO PAD 의 tristate buffer 를 enable 하기 위한 레지스터이다.

Figure 15-3 GPIO Output Enable Register

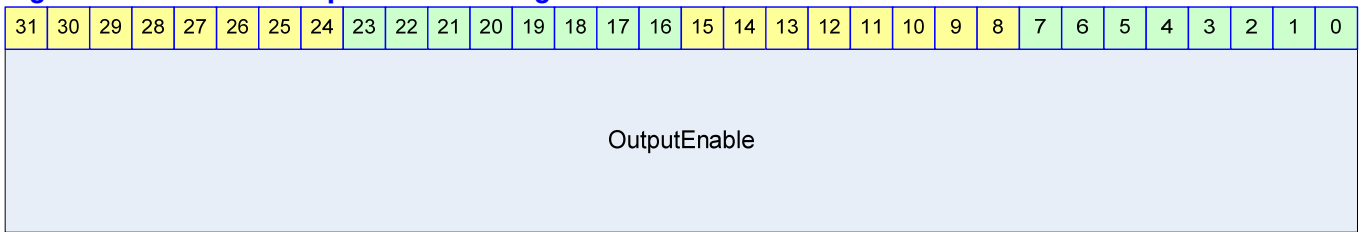


Table 15-2 GPIO Output Enable Register Description

Bits	Name	R / W	Description	Init. value
[31:0]	OutputEnable	RW	GPIO Output Enable 각 bit 가 1 이면 해당 GPIO PAD 에 GPIO_OUT 레지스터의 해당 bit 가 출력된다. 0 이면 해당 GPIO PAD 가 tristate 상태가 된다.	0000 0000h

15.3.3. GPIO Input Register (GPIO0_IN / GPIO1_IN)

GPIO Input 레지스터는 GPIO PAD 에 인가된 signal 을 읽을 수 있는 레지스터이다.

Figure 15-4 GPIO Input Register

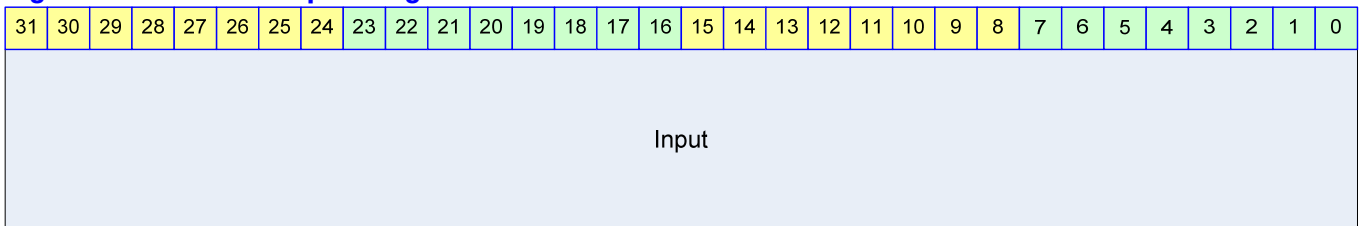
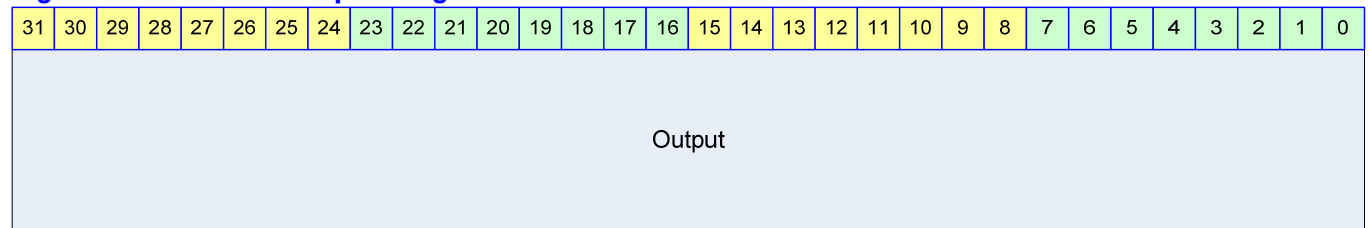


Table 15-3 GPIO Input Register Description

Bits	Name	R / W	Description	Init. value
[31:0]	Input	R	GPIO Input GPIO PAD 에 인가된 값을 읽을 수 있다. Input[x]가 1 이면 해당 GPIO PAD 에 high level 의 voltage 가 인가되었음을 나타낸다.	N/A

15.3.4. GPIO Output Register (GPIO0_OUT/GPIO1_OUT)

GPIO Output 레지스터는 GPIO PAD 에 출력될 값을 저장하고 있는 레지스터이다.

Figure 15-5 GPIO Output Register**Table 15-4 GPIO Output Register Description**

Bits	Name	R / W	Description	Init. Value
[31:0]	Output	RW	GPIO Output OutputEnable[x]가 1 이고 Output[x]가 1 이면 해당 GPIO PAD 에 high level 에 해당하는 전압이 인가된다. OutputEnable[x]가 1 이고 Output[x]가 0 이면 해당 GPIO PAD 에 low level 에 해당하는 전압이 인가된다. OutputEnable[x]가 0 이면 Output[x]는 아무런 의미가 없다.	0b

15.3.5. GPIO Interrupt Status Register (GPIO0_INTSTAT/GPIO1_INTSTAT)

GPIO PAD 로 입력 받은 interrupt status 를 의미하는 레지스터이다.

Figure 15-6 GPIO Interrupt Status Register

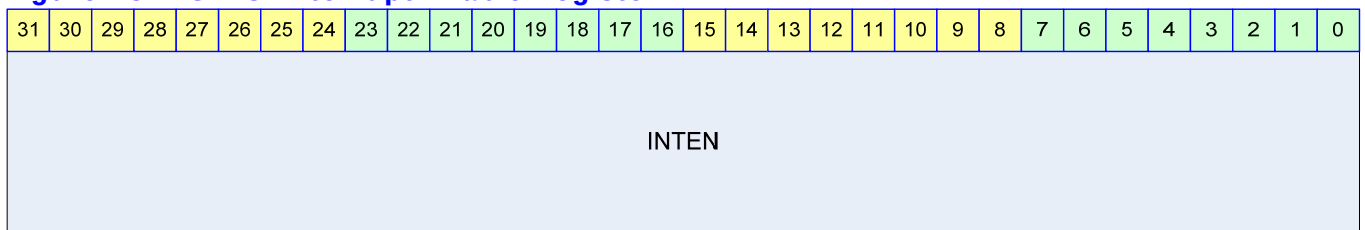
Table 15-5 GPIO Interrupt Status Register Description

Bits	Name	R / W	Description	Init. Value
[31:0]	INTSTAT	RW	GPIO Interrupt Status 1 : Level or Edge detected 0 : Level or Edge not detected Level/Edge detect 상황이 반영되는 register 이다. 1 을 write 하면 해당 bit 가 0 으로 clear 되고 0 을 write 하는 것은 의미가 없다.	0000 0000h

위의 동작설명에서 보았듯이 INTSTAT 의 bit 는 INTEN 와는 아무런 상관 없이 변하게 된다. 즉 GPIO PAD 에 인가되고 있는 signal 에 변화가 생기게 되면 해당 bit 의 INTSTAT 이 변하게 된다. Interrupt 는 INTSTAT 과 INTEN 의 bitwise AND 의 결과 32bit 값이 0 이 아니면 발생하게 된다. 어떤 bit 때문에 interrupt 가 생성되었는지 알고 싶으면 사용자는 INTEN 와 INTSTAT 을 bitwise AND 한 32bit data 에 대해서 0 이 아닌 bit 를 찾아야 한다. Edge interrupt 의 경우 interrupt service routine 에서 해당 bit 를 0 으로 clear 하는 것으로 Interrupt Acknowledge 를 하며 된다. Level interrupt 는 별도의 interrupt acknowledge 를 할 필요는 없고 interrupt 를 assert 한 외부 device 에서 해당 interrupt 를 clear 하는 것으로 모든 처리가 끝나게 된다.

15.3.6. GPIO Interrupt Enable Register (GPIO0_INTEN/GPIO1_INTEN)

GPIO PAD 로 입력 받은 interrupt status 를 의미하는 레지스터이다.

Figure 15-7 GPIO Interrupt Enable Register**Table 15-6 GPIO Interrupt Enable Register Description**

Bits	Name	R / W	Description	Init. Value
[31:0]	INTEN	RW	GPIO Interrupt Enable 1 : Interrupt Enable 0 : Interrupt Disable 어떤 bit 를 1 이면 INTSTAT 의 해당 bit 가 1 로 바뀔 때 interrupt 가 생성된다.	0000 0000h

15.3.7. GPIO Interrupt Level/Edge Selection Register (GPIO0_INTLEVEL/GPIO1_INTLEVEL)

GPIO PAD 로 입력 받을 interrupt 신호의 형태가 level trigger 인지 edge trigger 인지 선택하는 레지스터이다.

Figure 15-8 GPIO Interrupt Level/Edge Selection Register



Table 15-7 GPIO Interrupt Level/Edge Selection Register

Bits	Name	R / W	Description	Init. Value
[31:0]	INTLEVEL	RW	GPIO Interrupt Level/Edge Selection 1 : Level trigger mode 0 : Edge trigger mode	0000 0000h

15.3.8. GPIO Interrupt Polarity Register (GPIO0_INTPOL/GPIO1_INTPOL)

GPIO PAD 에 가해지는 signal 로 interrupt 를 detect 하는 경우 그 interrupt 신호가 active high 인지 active low 인지 결정하는 레지스터이다.

Figure 15-9 GPIO Interrupt Polarity Register

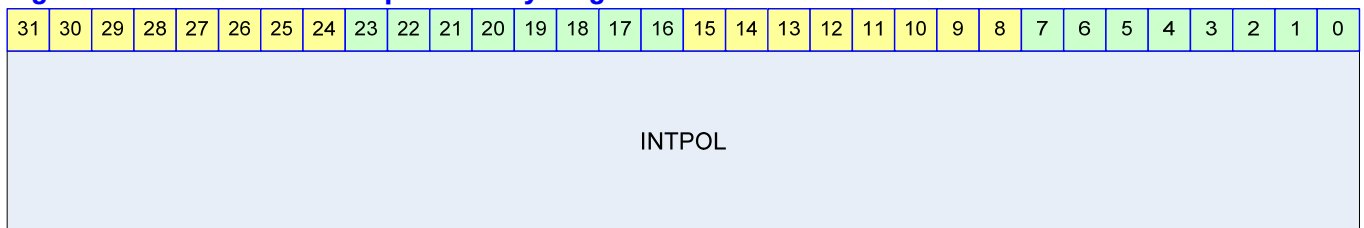


Table 15-8 GPIO Interrupt Polarity Register Description

Bits	Name	R / W	Description	Init. Value
[31:0]	INTPOL	RW	GPIO Interrupt Polarity Selection 1 : active high(rising edge) 0 : active low(falling edge)	0000 0000h

15.3.9. GPIO Interrupt Both Edge Register (GPIO0_INTBEDGE/GPIO1_INTBEDGE)

GPIO PAD 에 인가되는 signal 의 양쪽 edge(즉 rising edge, falling edge 모두)에서 interrupt 를 생성할 것인지 결정하는 레지스터이다.

Table 15-9 GPIO Interrupt Both Edge Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
INTBEDGE																															

Table 15-10 GPIO Interrupt Both Edge Register Description

Bits	Name	R / W	Description	Init. Value
[31:0]	INTBEDGE	RW	GPIO Interrupt Both Edge 1 : interrupt when both edge 0 : interrupt when single edge INTLEVEL 의 특정 bit 위치의 값이 1 인경우 해당 위치의 INTBEDGE bit 는 아무런 의미가 없다.	0000 0000h

16. SPI

16.1. Overview

Serial Peripheral Interface

Serial 통신을 하는 방법의 하나로 Master 에서 나오는 SPI CLOCK 과 MOSI (Master Output Slave Input), MISO(Master Input Slave Output)과 SS(Slave Select)신호로 구성이 된다. 통신은 Master 가 Clock 을 내보내는 것으로 송수신이 이루어진다

Master

Clock 을 내보내는 쪽을 지칭한다.

Slave

Master 에서 보내온 Clock 에 따라 Data 를 송수신 하는 모듈을 말한다. Master 는 여러 Slave 중에서 하나의 Slave 를 선택하기 위해서 SS 핀을 사용하기도 한다.

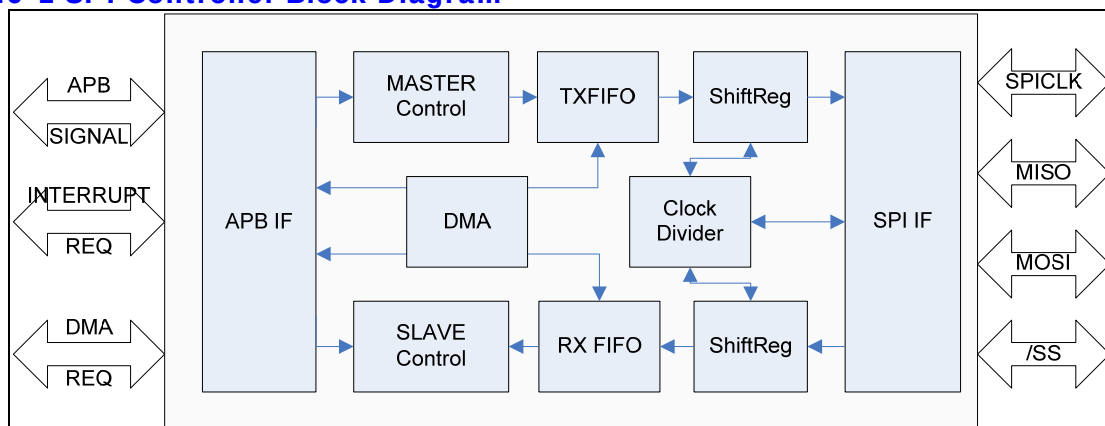
정확한 송수신을 위해서 Master 와 Slave 간의 Clock Polarity 와 Clock Phase 가 서로 같아야만 한다. 그렇지 않으면 엉뚱한 데이터 값을 읽고 쓰게 된다

16.1.1. Feature

- 1 Channel SPI (SPI0)
- Master/Slave Mode Support
- Receive Time Out Interrupt Support for DMA Remain Value
- DMA Support (Tx 8depth, Rx 8depth)
- SPI0 is Support DMA and FIFO Feature

16.1.2. Block Diagram

Figure 16-1 SPI Controller Block Diagram



SPI Channel 는 Rx FIFO 와 Tx FIFO 는 각각 8 depth 를 가지고 있다. Rx 와 Tx 각각 DMA Unit 으로 DMA Request 를 보낼 수 있다. DMA 는 FIFO Level 에 따라 Setting 이 이루어 진다. Rx FIFO 에 Data 를 일정 Cycle 이상 읽어가지 않을 경우 Timeout Interrupt 를 발생시킬 수도 있다.

16.2. Operation

16.2.1. Overview

APB BUS 를 사용하고 있으며 Master Mode 및 Slave Mode 를 지원한다.

SPI0 는 DMA 를 지원하며 지원하는 DMA Level 을 선택할 수 있다. 각 DMA 및 Interrupt 는 FIFO Level Setting 으로 조절 가능하다.

16.2.2. Timing

다음은 SPI Mode Setting 과 Wave Form 을 나타내었다.

Figure 16-2 SPI Timing (CPOL=0, CPHA=0)

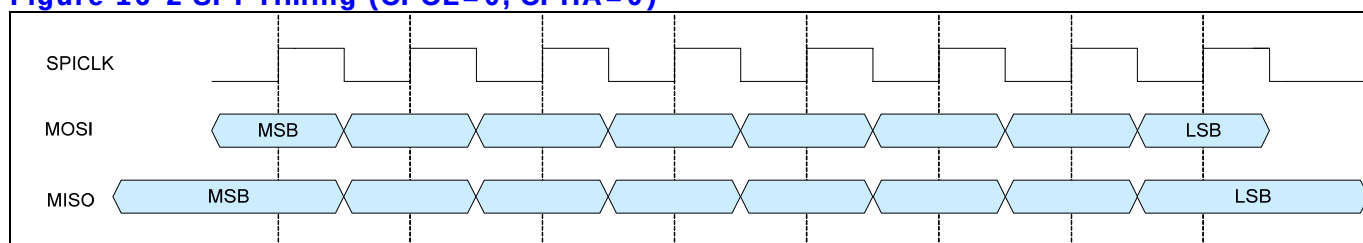


Figure 16-3 SPI Timing (CPOL=0, CPHA=1)

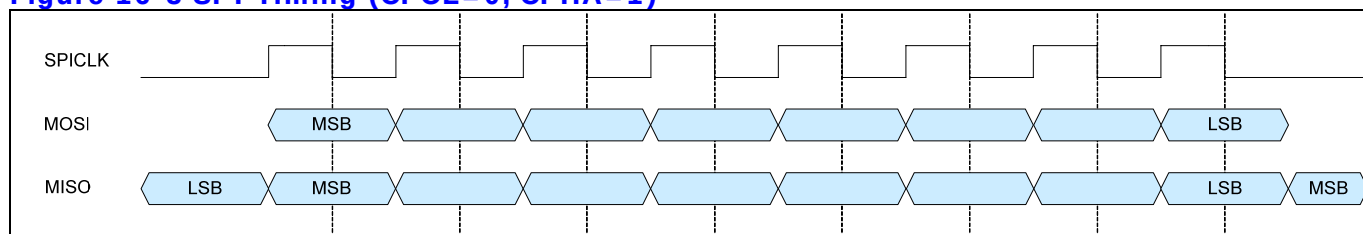


Figure 16-4 SPI Timing (CPOL=1, CPHA=0)

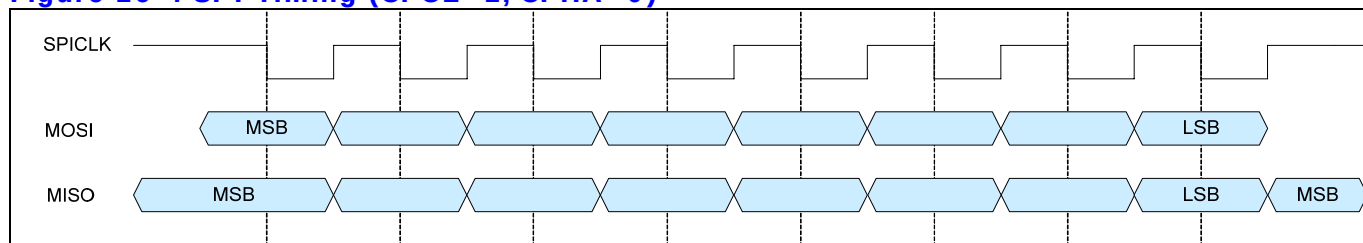
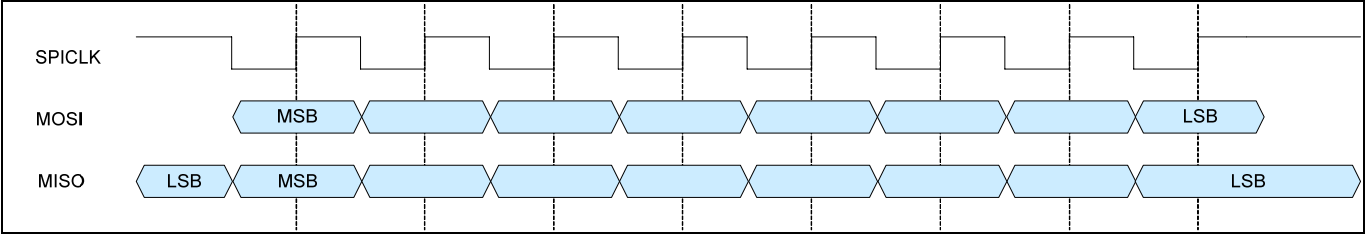


Figure 16-5 SPI Timing (CPOL=1, CPHA=1)



16.3. Signal Description

16.3.1. SPI BUS Signal

Table 16-1 SPI BUS Signal

Signal Name	Direction	Description	Init. Value
MISO	InOut	Master Input Slave Output	
MOSI	InOut	Master Output Slave Input	
CLK	InOut	Master Output Clock Slave Input Clock	
/SS	InOut	Slave Select	

16.4. Register Description

16.4.1. Registers

Table 16-2 Registers

Offset	Name	R / W	Description	Init. Value
0000h	SPICON	RW	SPI Control register	0000 0000h
0004h	SPIPRE	RW	SPI Prescaler register	FFFF FFFFh
0008h	SPISTA	RW	SPI Status Register	
000Ch	SPIINTDMA	RW	SPI Interrupt DMA setting Register	
0010h	SPIINTSTA	RW	SPI Interrupt Status Register	
0014h	SPITxDAT	RW	SPI Tx Data register	0000 0000h
0018h	SPIRxDAT	R	SPI Rx Data Register	

16.4.2. SPI Control Register

Figure 16-6 SPI Control Register Bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																												Enable	MasterSlave	CPOL	CPHA

Table 16-3 SPI Control Register Description (Offset : 0000 0000h)

Bits	Name	R / W	Description	Init. Value
[3]	Enable	RW	Enable 0: SPI Disable 1: SPI Enable	0
[2]	MasterSlave	RW	Master or Slave Select 0: Master 1: Slave	0
[1]	CPOL	RW	Clock Polarity Setting 0: 1:	0
[0]	CPHA	RW	Clock Phase Setting 0: 1:	0

16.4.3. SPI Prescaler Register

Figure 16-7 SPI Prescaler Register Bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																DivVal								PreScaVal							

Table 16-4 SPI Precaler Register Description (Offset : 0000 0004h)

Bits	Name	R / W	Description	Init. Value
[15:8]	DivVal	RW		
[7:0]	PreScaVal	RW	Prescaler Setting $PCLK/(DIV*(PreScaVal+1))$	1111b

16.4.4. SPI Status Register

Figure 16-8 SPI Status Register Bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																		TxFIFOLevel				RxFIFOLevel				TxFIFOFull	TxFIFOEmpty	RxFIFOFull	RxFIFOEmpty	Busy	Reserved

Table 16-5 SPI Status Register Description(Offset : 0000 0008h)

Bits	Name	R / W	Description	Init. Value
[13:10]	TxFIFOLevel	R	Present Tx FIFO Level (현재 Tx FIFO 내에 남아있는 데이터양을 나타낸다.)	
[9:6]	RxFIFOLevel	R	Present Rx FIFO Level (현재 Rx FIFO 내에 남아있는 데이터양을 나타낸다.)	
[5]	TxFIFOFull	R	1: Tx FIFO Full 0: Tx FIFO Not Full	
[4]	TxFIFOEmpty	R	1: Tx FIFO Empty 0: Tx FIFO Not Empty	
[3]	RxFIFOFull	R	1: Rx FIFO Full 0: Rx FIFO Not Full	
[2]	RxFIFOEmpty	R	1: Rx FIFO Empty 0: Rx FIFO Not Empty	
[1]	Busy	R	1: SPI Bus Transferring 0: SPI Bus Not Transferring	

16.4.5. SPI Interrupt & DMA control Register

Figure 16-9 SPI Interrupt & DMA Register Bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
Reserved																			TxFIFOInt	RxFIFOInt	RxTimeOutInt	RxOverrunInt	TxDMAEnable	TxDMALevel		RxDMAEnable	RxDMALevel								

Table 16-6 SPI Interrupt & DMA Register Description(Offset : 0000 0008h)

Bits	Name	R / W	Description	Init. Value
[13]	TxFIFOIntEn	RW	TxFIFO Interrupt Enable 1: Enable 0: Disable (이 Interrupt 는 TxFIFOFillLevel 이 TxDMALevel 설정 값 이하 일 경우 발생한다.)	0
[12]	RxFIFOIntEn	RW	RxFIFO Interrupt Enable 1: Enable 0: Disable	0

			(이 Interrupt 는 RxFIFOFillLevel 이 RxDMALevel 에 설정한 값 이상 일 경우 발생한다.)	
[11]	RxTimeOutIntEn	RW	RxTimeOut Interrupt Enable 1: Enable 0: Disable (Receive FIFO 에 Data 가 있는 상태에서 일정 time 이내에 Data 를 읽어가지 않을 경우 발생한다)	0
[10]	RxOverrunIntEn	RW	RxOverrun Interrupt Enable 1: Enable 0: Disable	0
[9]	TxDMAEnable	RW	Tx DMA Request Enable 1: Tx DMA Enable 0: Rx DMA Disable	0
[8:5]	TxDMALevel	RW	Tx DMA Request Level (DMA request 를 내보낼 FIFO 용량을 설정한다. 미리 전송하고자 하는 data 양을 알고 있기에 FIFO 의 남은 Level 을 보고 request 를 보내면 된다)	1
[4]	RxDMAEnable	RW	Rx DMA Request Enable 1: Rx DMA Enable 0: Rx DMA Disable	0
[3:0]	RxDMALevel	RW	Rx DMA Request Level (Rx 의 경우 FIFO 가 일정 Level 이상 data 가 있을 때 DMA request 를 보내므로 FIFO Remain Data 처리에 유의할 필요가 있다.)	1

16.4.6. SPI Interrupt Status Register

Figure 16-10 SPI Interrupt Status Register Bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																										RToutIntClr	ROverIntClr	TxFIFOIntSta	RxFIFOIntSta	RxTimeOutIntSta	RxOverrunIntSta

Table 16-7 SPI Interrupt Status Register Description(Offset : 0000 0008h)

Bits	Name	R / W	Description	Init. Value
[5]	RToutIntClr	C	Receive Time out Interrupt Clear 1: Receive Time Out Interrupt 가 clear 됨 0: 아무 영향 없음	
[4]	ROverIntClr	C	Receive Overrun Interrupt Clear 1: Receive Overrun Interrupt 가 clear 됨 0: 아무 영향 없음	
[3]	TxFIFOIntSta	R	TxFIFO Interrupt Status 1: Interrupt Generated 0: Interrupt Not Generated (이 Interrupt 는 TxFIFOCnt 값이 TxDMALevel 이하 일 경우 발생한다.)	0
[2]	RxFIFOIntSta	R	RxFIFO Interrupt Status	0

			1: Interrupt Generated 0: Interrupt Not Generated (이 Interrupt 는 RxFIFOCnt 값이 TxDMALevel 이상 일 경우에 발생한다.)	
[1]	RxTimeOutIntSta	R	RxTimeOut Interrupt Status 1: Interrupt Generated 0: Interrupt Not Generated	0
[0]	RxOverrunIntSta	R	RxOverrun Interrupt Status 1: Interrupt Generated 0: Interrupt Not Generated	0

16.4.7. SPI Tx Data Register

Figure 16-11 SPI Tx Data Register Bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																TxDAT															

Table 16-8 SPI Tx Data Register Description(Offset : 0000 0008h)

Bits	Name	R / W	Description	Init. Value
[15:0]	TxDAT	RW	SPI Tx Data (읽기의 경우 제일 마지막에 Write 한 값이 읽힌다)	

16.4.8. SPI RxData Register

Figure 16-12 SPI Rx Data Register Bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																RxDAT															

Table 16-9 SPI Rx Data Register Description(Offset : 0000 0008h)

Bits	Name	R / W	Description	Init. Value
[15:0]	RxDAT	R	SPI Rx Data	

17. NAND Flash Memory Controller

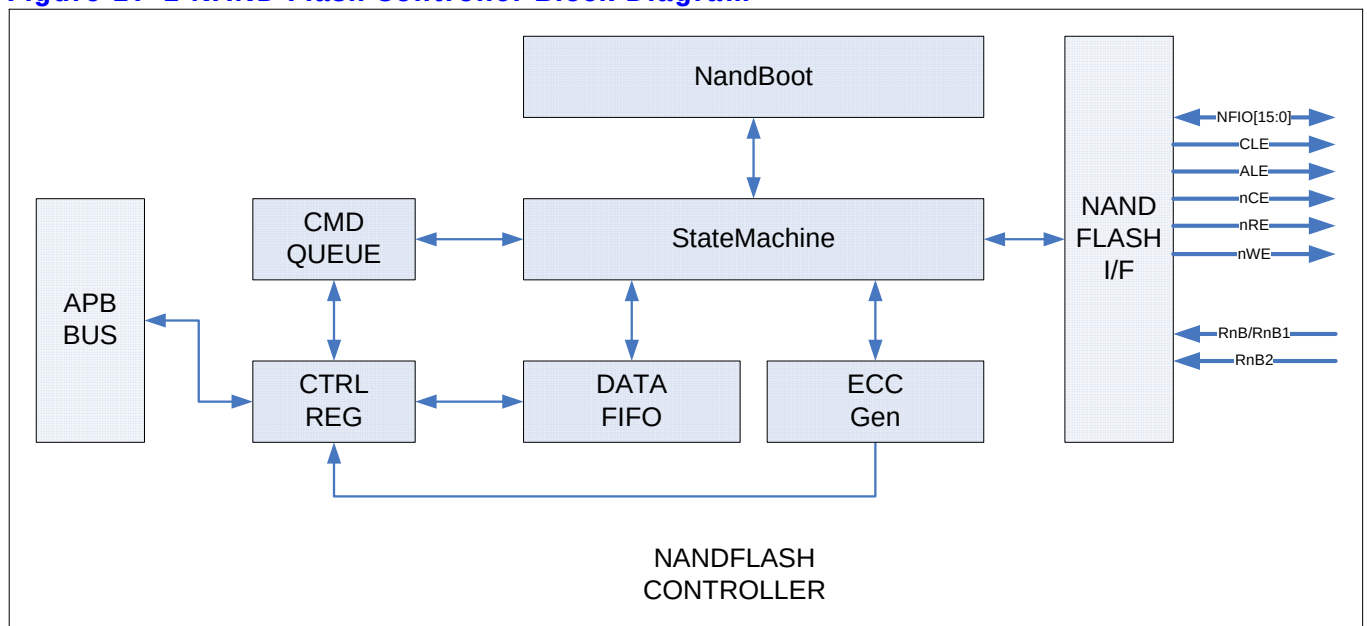
17.1. Overview

17.1.1. Feature

- APB Interface.
- NAND Boot support(up to 8KB)
- 8bit NAND Flash Interface.
- 512 byte/2048byte Page support
- 256 byte ECC support
- Read/Erase/Program support
- 128Mbit~2Gbit Flash memory support
- DMA support
- Using Command FIFO,DATA FIFO

17.1.2. Block Diagram

Figure 17-1 NAND Flash Controller Block Diagram



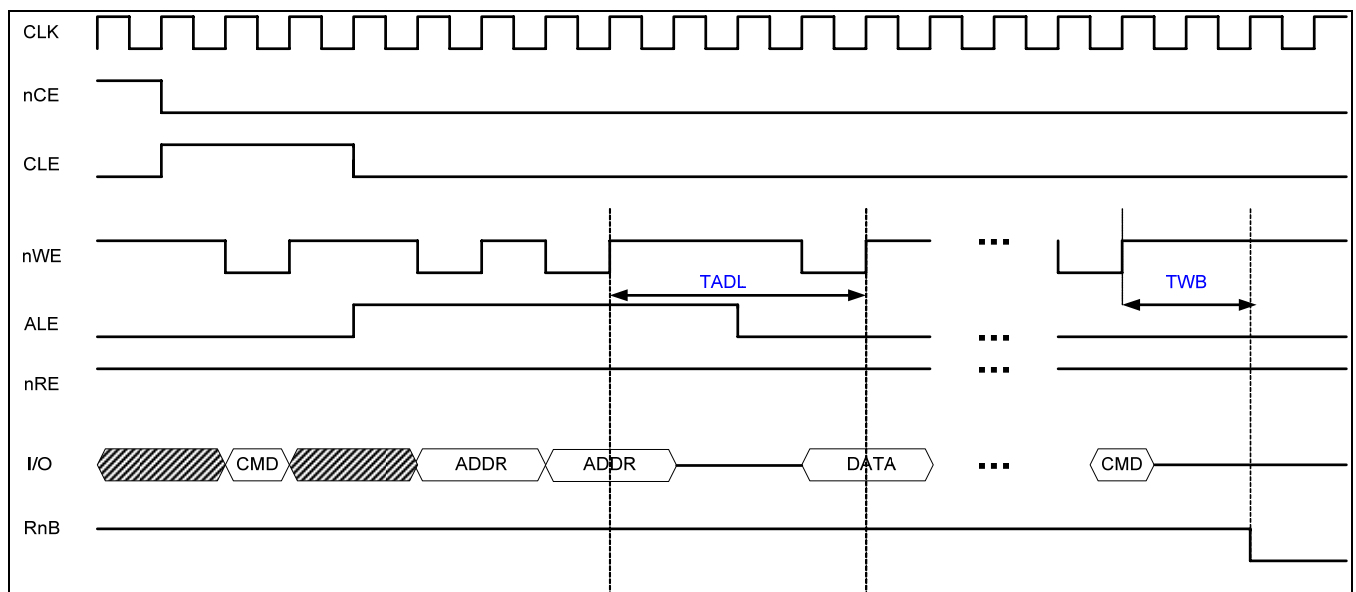
17.2. Signal Description

Table 17-1 NAND Flash Signal

Signal Name	Direction	Description	Init. value
NFIO0 ~ NFIO15	InOut	Inout pin Address, data, command 입출력 핀	
CLE	Output	Command latch enable CLE 가 High 이고 nWE 이 rising edge 일 때 command 가 래치됨	1
ALE	Output	Address latch enable ALE 가 High 이고 nWE 이 rising edge 일 때 address 가 래치됨	1
nNFCE	Output	Chip enable	1
nNFRE	Output	Read enable	1
nNFW	Output	Write enable	1
RnB	Input	Ready & Busy	1

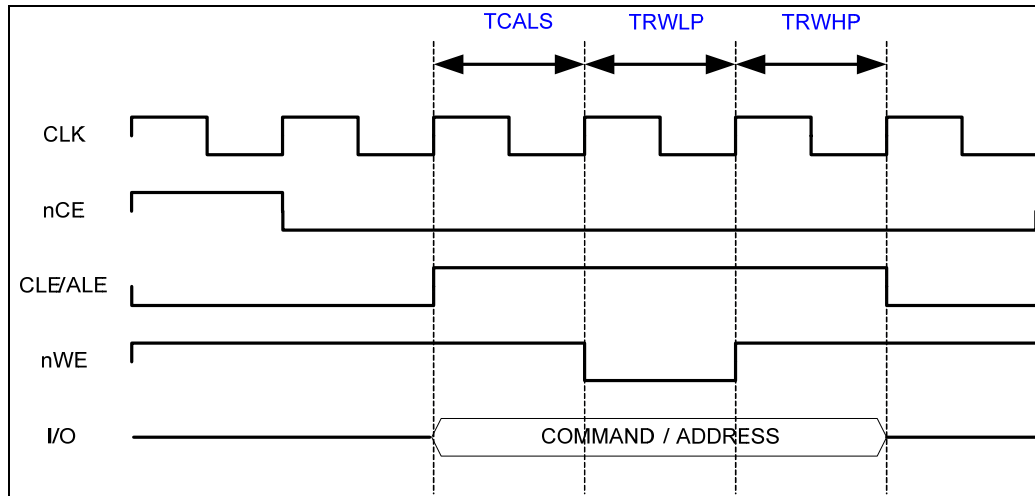
17.3. Timing

Figure 17-2 Address to data load time / WE high to busy Timing



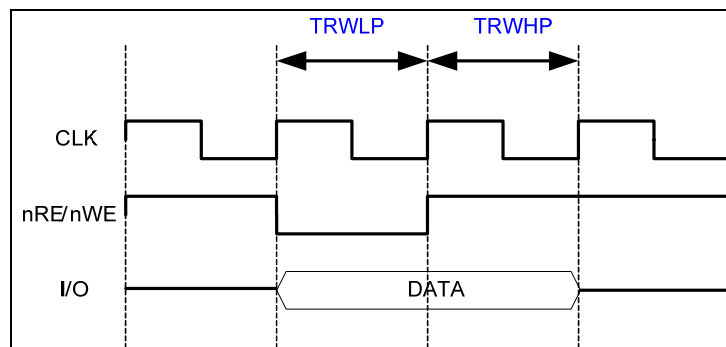
NFCONF Register 에 TADLTWB 값으로 TADL, TWB 타이밍 조정이 가능함.

Figure 17-3 ALE,CLE Timing



NFCONF Register 에 TCALS, TRWLP, TRWHP 값으로 ALE, CLE 타이밍 조정이 가능함.

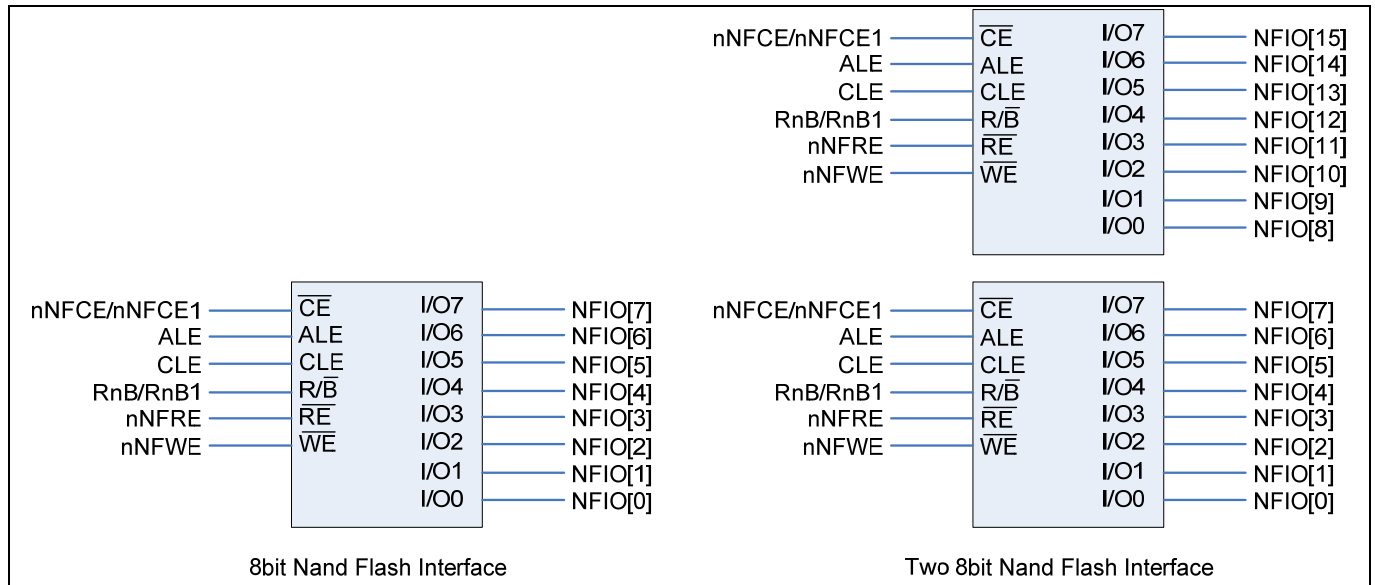
Figure 17-4 RE, WE Timing



NFCONF Register 에 TRWLP, TRWHP 값으로 RE, WE 의 타이밍 조정이 가능함.

17.4. Interface

Figure 17-5 Interface example



17.5. Register Description

17.5.1. Registers

Table 17-2 Registers

Offset	Name	R / W	Description	Init. value
0000h	NFOPER	R	Nand Flash Operation register	0000 0000h
0004h	NFDATA	RW	Nand Flash Data register	0000 0000h
0008h	NFCONF	RW	Nand Flash Configuration register	0000 1e92h
000Ch	NFCTRL	RW	Nand Flash Control register	0000 0000h
0010h	NFSTAT	RW	Nand Flash Status register	0000 0003h
0014h	ECCSECTOR0	R	Nand Flash Ecc sector0 register	0000 0000h
0018h	ECCSECTOR1	R	Nand Flash Ecc sector1 register	0000 0000h
001Ch	ECCSECTOR2	R	Nand Flash Ecc sector2 register	0000 0000h
0020h	ECCSECTOR3	R	Nand Flash Ecc sector3 register	0000 0000h
0024h	ECCSECTOR4	R	Nand Flash Ecc sector4 register	0000 0000h
0028h	ECCSECTOR5	R	Nand Flash Ecc sector5 register	0000 0000h
002ch	ECCSECTOR6	R	Nand Flash Ecc sector6 register	0000 0000h
0030h	ECCSECTOR7	R	Nand Flash Ecc sector7 register	0000 0000h
0034h	ECCSECTOR8	R	Nand Flash Ecc sector8 register	0000 0000h
0038h	ECCSECTOR9	R	Nand Flash Ecc sector9 register	0000 0000h
003ch	ECCSECTOR10	R	Nand Flash Ecc sector10 register	0000 0000h

0040h	ECCSECTOR11	R	Nand Flash Ecc sector11 register	0000 0000h
0044h	ECCSECTOR12	R	Nand Flash Ecc sector12 register	0000 0000h
0048h	ECCSECTOR13	R	Nand Flash Ecc sector13 register	0000 0000h
004ch	ECCSECTOR14	R	Nand Flash Ecc sector14 register	0000 0000h
0050h	ECCSECTOR15	R	Nand Flash Ecc sector15 register	0000 0000h
0054h	SECCSECTOR0	R	Nand Flash spare Ecc sector0 register	0000 0000h
0058h	SECCSECTOR1	R	Nand Flash spare Ecc sector1 register	0000 0000h
005ch	SECCSECTOR2	R	Nand Flash spare Ecc sector2 register	0000 0000h
0060h	SECCSECTOR3	R	Nand Flash spare Ecc sector3 register	0000 0000h
0064h	SECCSECTOR4	R	Nand Flash spare Ecc sector4 register	0000 0000h
0068h	SECCSECTOR5	R	Nand Flash spare Ecc sector5 register	0000 0000h
006ch	SECCSECTOR6	R	Nand Flash spare Ecc sector6 register	0000 0000h
0070h	SECCSECTOR7	R	Nand Flash spare Ecc sector7 register	0000 0000h
0074h	MECCERR	R	Nand Flash Main Ecc Error register	0000 0000h
0078h	SECCERR	R	Nand Flash Spare Ecc Error register	0000 0000h

17.5.2. Operation Register(NFOER)

Operation Register 는 Nand flash 에 전송할 command 또는 address 각 command 가 어떤 동작을 하게 할것인지를 설정해주는 Register 로 하나의 command(ex. Page read, Page write, Block Erase)를 수행하는데 최대 3Word 로 구성 할 수 있다 1st word 는 Operation 의 Information 을 가지는 Word 이고 2nd, 3th Word 는 command 와 address 의 조합이다. 레지스터 구성은 아래 그림과 같다.

Figure 17-6 1st Operation Register bit

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ChipSel	RnBWait	AutoRdStat	DataSize												OpMode	TransSize	CmdAddrTransByte				CmdAddrFlag										

Table 17-3 1st Operation Register Description(Offset : 0000 0000h)

Bits	Name	R / W	Description	Init. value
[31]	Reserved			0
[30:29]	Option	WO	00: no Option 01: RnBWait	00

			10: AutoRdStat 11: Continue RnBWait:command Queue 의 다음 command set 이 RnB 의 Rising edge 에서 시작된다 AutoRdStat:RnB 가 High 가 된 후 read status Command 를 전송하고 status 를 읽어온다. Continue : command Queue 의 다음 command set 을 load 하기 전까지 CE 을 유지한다.	
[28:17]	DataSize	WO	한 page 에서 R/W 할 Data Size 지정한다. IOWidth(8bit)이면 byte 단위,IOWidth(16bit)이면 halfword 단위가 기본 전송단위가 된다.	0
[16:14]	OpMode	WO	000: read data(page read 시 선택) 001: read status 010: read ID 011: write data 100,101,110 : reserved 111: NOP Read data => page read 시 사용됨 Write data => nand flash 에 data 를 write 할 때 Read status => status read 시 사용 (Read status: TransSize 는 Byte 단위만 가능) Read ID => ID read 시 사용 (Read ID: TransSize 는 Byte 단위만 가능) Nand flash 와 Nand flash controller 간에 data 전송이 없으면 NOP	0
[13:12]	TransSize	WO	NFDATA 를 읽거나 쓸 때 TransferSize IOWidth 16bit 이면 Half word 이거나 Word 만 가능 00: Byte transfer 10: Half Word transfer 10: Word transfer 11: Reserved	0
[11:8]	CmdAddrTransByte	WO	전송할 command byte 와 address byte 의 수 최대값은 8(1000b)	0
[7:0]	CmdAddrFlag	WO	Command 인지 Address 인지를 나타내는 Flag 임.	0

			CmdAddrFlag[0]~CmdAddrFlag[7]의 순서로 전송된다. 0: address flag 1: command flag	
--	--	--	--	--

Figure 17-7 2nd Operation Register bit

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
4th CMD or ADDR byte								3rd CMD or ADDR byte								2nd CMD or ADDR byte								1st CMD or ADDR byte							

Table 17-4 2nd Operation Register Description(Offset : 0000 0000h)

Bits	Name	R / W	Description	Init. value
[31:24]	CMDADDR4	WO	Command or address 4 th byte 를 써준다.	0
[23:16]	CMDADDR3	WO	Command or address 3 rd byte 를 써준다.	0
[15:8]	CMDADDR2	WO	Command or address 2 nd byte 를 써준다.	0
[7:0]	CMDADDR1	WO	Command or address 1 st byte 를 써준다.	0

Figure 17-8 3rd Operation Register bit

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
8th CMD or ADDR byte								7th CMD or ADDR byte								6th CMD or ADDR byte								5th CMD or ADDR byte							

Table 17-5 3rd Operation Register Description(Offset : 0000 0000h)

Bits	Name	R / W	Description	Init. value
[31:24]	CMDADDR8	WO	Command or address 8 th byte 를 써준다.	0
[23:16]	CMDADDR7	WO	Command or address 7 th byte 를 써준다.	0
[15:8]	CMDADDR6	WO	Command or address 6 th byte 를 써준다.	0
[7:0]	CMDADDR5	WO	Command or address 5 th byte 를 써준다.	0

17.5.3. Data Register (NFDATA)

Nand Flash 에 data 를 read/write 할 때 access 하는 32bit 레지스터이다.

Operation 레지스터에서 설정한 TransSize 에 따른 NFDATA 레지스터의 Access 는 아래 그림과 같다. (ex. operation 레지스터에서 word transfer 로 셋팅을 하게 되면 4byte 가 유효한 값을

가지고 halfword transfer 로 셋팅 하면 하위 2byte 만 유효한 값을 가지며 byte transfer 로 셋팅 하면 최하위 byte 만 유효한 값을 가진다.)

Figure 17-9 Data Register bit(Word Access)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
4th I/O[7:0]								3rd I/O[7:0]								2nd I/O[7:0]								1st I/O[7:0]							

Figure 17-10 Data Register bit(Half Word Access)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Invalid								Invalid								2nd I/O[7:0]								1st I/O[7:0]							

Figure 17-11 Data Register bit(Byte Access)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Invalid								Invalid								Invalid								1st I/O[7:0]							

17.5.4. Configuration Register(NFCONF)

Nand 부팅시 설정과 Timing 을 설정하는 레지스터이다.

NandBootEn = nand 부팅을 enable 할지 disable 할지 선택함

IOWidth = NFIO 를 8bit 으로 할지 16bit 으로 할지 선택함

BootCfg = Nand 부팅시 Nand Flash 의 address cycle 과 page size 를 설정함

타이밍 설정에 대해서는 1-3 Timing 을 참조

Figure 17-12 Configuration Register bit

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved													NandBootEn	IOWidth	NandWidth	BootCfg	OutDmn	TADLTWB				TCALS		TRWLP			TRWHP				

Table 17-6 Configuration Register Description(Offset : 0000 0008h)

Bits	Name	R / W	Description	Init. value
[31:19]	Reserved			
[18]	NandBootEn	RO	0: normal mode 1: nand boot enable	NFBootPin

[17]	IOWidth	RO	Nand flash controller 의 I/O select 0: 8bit I/O 1: 16bit I/O	IOWidthPin
[16]	Reserved	RO	Reserved	0
[15:14]	BootCfg	RO	00: 3 addr cycle = 1column+2row(512page) 01: 4 addr cycle = 1column+3row(512page) 10: 4 addr cycle = 2column+2row(2048page) 11: 5 addr cycle = 2column+3row(2048page)	BootCfgPin
[13]	Reserved	RO	Reserved	0
[12:9]	TADLTWB	RW	Duration = CLK x (TADLTWB+1)	1111b
[8:6]	TCALS	RW	Duration = CLK x (TACLH+1)	10b
[5:3]	TRWLP	RW	Duration = CLK x (TWEW+1)	10b
[2:0]	TRWHP	RW	Duration = CLK x (TREW+1)	10b

17.5.5. Control Register(NFCTRL)

Figure 17-13 Control Register bit

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																		NFCtrlRst	FIFOLevel		NFIDByte	AutoEccWr	DMAEn	WEndIntEn	RdEndIntEn	EccErrIntEn	FIFOIntEn	RnBIntEn1	RnBIntEn0		

Table 17-7 Control Register Description(Offset : 0000 000ch)

Bits	Name	R / W	Description	Init. value
[31:14]	Reserved	RW	Reserved	0
[13]	NFCtrlRst	RW	Nand flash controller reset 1 을 쓰면 reset 되며 auto clear 된다.	0
[12:10]	FIFOLevel	RW	FIFOLevel 셋팅 Value = FIFOLevel + 1	111
[9]	Reserved		reserved	0
[8]	Ecc512byteEn		0: 256byte Ecc 1: 512byte Ecc	0

[7]	AutoEccWr	RW	한 페이지 모두 write 하면 spare area 의 Ecc 영역에 계산된 ECC 를 write 한다. 0: Disable 1: Enable	0
[6]	DMAEn	RW	DMA request enable 총 전송사이즈가 FIFOLevel 의 배수여야 한다. Read : Enable 시 FIFO 가 FIFOLevel 에서 정해진 크기 만큼 데이터가 차면 DMAReq 신호 발생 Write : FIFO 가 empty 이면 DMAReq 신호 발생 0: Disable 1:Enable	1
[5]	WrEndIntEn	RW	Write end interrupt enable Trans size 까지 모두 write 되었을때 INT 발생 만약 AutoEccWr 을 enable 하면 status 를 NFStatus 에 로드한후 인터럽트가 발생한다. 0: Disable 1: Enable	0
[4]	RdEndIntEn	RW	Read end interrupt enable Trans size 까지 모두 read 되었을때 INT 발생 : Disable 1: Enable	0
[3]	EccErrIntEn	RW	Ecc Error interrupt enable 전체 페이지를 read 하였을때 Ecc Error 가 생기면 인터럽트가 발생한다 0: Disable 1: Enable	0
[2]	FIFOIntEn	RW	FIFO level interrupt enable Read : FIFO 가 지정된 레벨까지 차게 되면 인터럽트 발생 Write : FIFO 가 empty 면 인터럽트 발생 0: Disable 1:Enable	0
[1]	Reserved		Reserved	0
[0]	RnbIntEn0	RW	Rnb0 interrupt enable enable 되면 Rnb rising edge 에서 인터럽트 발생 0: Disable 1:Enable	0

NFCtrlRst 을 셋팅하게 되면 내부 컨트롤 StatMachine, FIFO 및 ECC 레지스터를 초기화 함

17.5.6. Status Register(NFSTAT)

Figure 17-14 Status Register bit

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								EccErr	NFQLevel				NFCtrlBusy	NFIDValid	NFStatValid	NFSTATUS							WfEnd	RdEnd	WfFIFOReady	RdFIFOReady	RnBDetect1	RnBDetect0	RnBStat1	RnBStat0	

Table 17-8 Status Register Description(Offset : 0000 0010h)

Bits	Name	R / W	Description	Init. value
[31:23]	Reserved	RW	Reserved	0
[24]	MEccErr	RW	0: MEccErr 발생 하지 않았음 1: MEccErr 발생 하였음 EccErrIntEn 을 1 로 셋팅 하는 것과 상관없이 Main Ecc 에러가 발생하면 1 로 변경 된다. 만약 EccErrIntEn 을 Enable 하면 인터럽트가 발생하며 clear 해줘야만 다음 Ecc Error 인터럽트를 받을수 있다 반드시 clear 해야 함 1 을 write 하면 clear	0
[23]	SEccErr	RW	0: SEccErr 발생 하지 않았음 1: SEccErr 발생 하였음 EccErrIntEn 을 1 로 셋팅 하는 것과 상관없이 Spare Ecc 에러가 발생하면 1 로 변경 된다. 만약 EccErrIntEn 을 Enable 하면 인터럽트가 발생하며 clear 해줘야만 다음 Ecc Error 인터럽트를 받을수 있다 반드시 clear 해야 함 1 을 write 하면 clear	
[22:19]	NFQueLevel	RW	내부 command queue 에 레벨값 Max 1000(8)	0
[18]	NFCtrlBusy	RW	0 : Controller Idle 1 : Controller busy	0
[17]	NFIDValid	RW	Nand flash 의 read ID 를 수행하여 내부 fifo 에 채운후 high 가됨. NFSTAT 를 읽어가면 auto clear 됨.	0

			0: not valid 1: valid	
[16]	NFStatVaild	RW	Nand flash 의 read status 를 수행한후 High 가됨 NFSTAT 를 읽어가면 auto clear 됨. 0: not valid 1: valid	0
[15:8]	NFStatus	RW	Nand flash 의 status 를 나타내며 NFSTAT 를 읽어가면 clear 된다	00000000
[7]	WrEnd	RW	Trans size 까지 모두 write 되면 HIGH 가 되며 AutoRdStatus 를 enable 하게 되면 Status 를 NFStatus 로 로드한후에 HIGH 가 된다. WrEndIntEn 의 enable 여부에 상관없이 HIGH 가 된다. 만약 인터럽트가 enable 되어 있으면 WrEnd 가 High 일 때 인터럽트 발생 0: write not end 1: write end 1 을 Write 하면 clear 됨	0
[6]	RdEnd	RW	Trans size 까지 모두 read 되면(FIFO 에서 모두 읽어 가게 되면) HIGH 가 된다. RdEndIntEn 의 enable 여부에 상관없이 HIGH 가 된다 만약 인터럽트가 enable 되어 있으면 RdEnd 가 High 일 때 인터럽트 발생 0: read not end 1: read end 1 을 Write 하면 clear 됨	0
[5]	WrFIFOReady	RW	Data Write 시 FIFO 가 empty 일 때 HIGH 가됨. 인터럽트 enable 여부에 상관없이 HIGH 가 된다 만약 인터럽트가 enable 되어 있으면 WrFIFOReadyr 가 High 일 때 인터럽트 발생함 반드시 clear 해야만 다음 인터럽트가 발생한다. 0: FIFO not empty 1: FIFO empty 1 을 Write 하면 clear 됨	0
[4]	RdFIFOReady	RW	Data Read 시 FIFO 가 FIFOLevel 에서 정해진 크기 만큼 차거나 OpRlen 에서 지정한 데이터의 마지막이 되면 HIGH 가 된다.	0

			인터럽트 enable 여부에 상관없이 HIGH 가 된다 만약 인터럽트가 enable 되어 있으면 RdFIFOReadyr 가 High 일 때 인터럽트 발생함 반드시 clear 해야만 다음 인터럽트가 발생한다 0: FIFO not Ready 1: FIFO Ready(interrupt enable 시 인터럽트 발생했음) 1 을 Write 하면 clear 됨	
[3]	Reserved			0
[2]	RnBDetect0	RW	RnB 의 Rising edge 에서 HIGH 가 됨. RnbIntEn0 가 enable 되어 있으면 RnBDetect0 가 High 일 때 interrupt 발생함. 만약 auto read status 를 enable 하면 read status 를 수행 하여 Nstatus 에 nand flash status 를 load 한 후 interrupt 가 발생한다. 1 을 Write 하면 clear 됨	0
[1]	Reserved			
[0]	RnBSTAT0	RO	Nand Flash 의 RnB0 pin 의 상태를 나타냄.	1

17.5.7. Ecc Register

총 16 개의 256Byte ECC register(ECCSECTOR0~ECCSECTOR15) 와 8 개의 spare area ECC register 를 가진다(SECCSECTOR0 ~ SECCSECTOR15) 각 레지스터의 구성은 아래와 같다.

17.5.7.1. Ecc 구성

Table 17-9 ECCSECTOR0~ECCSECTOR15 (0000 0014h~0000 0050h)

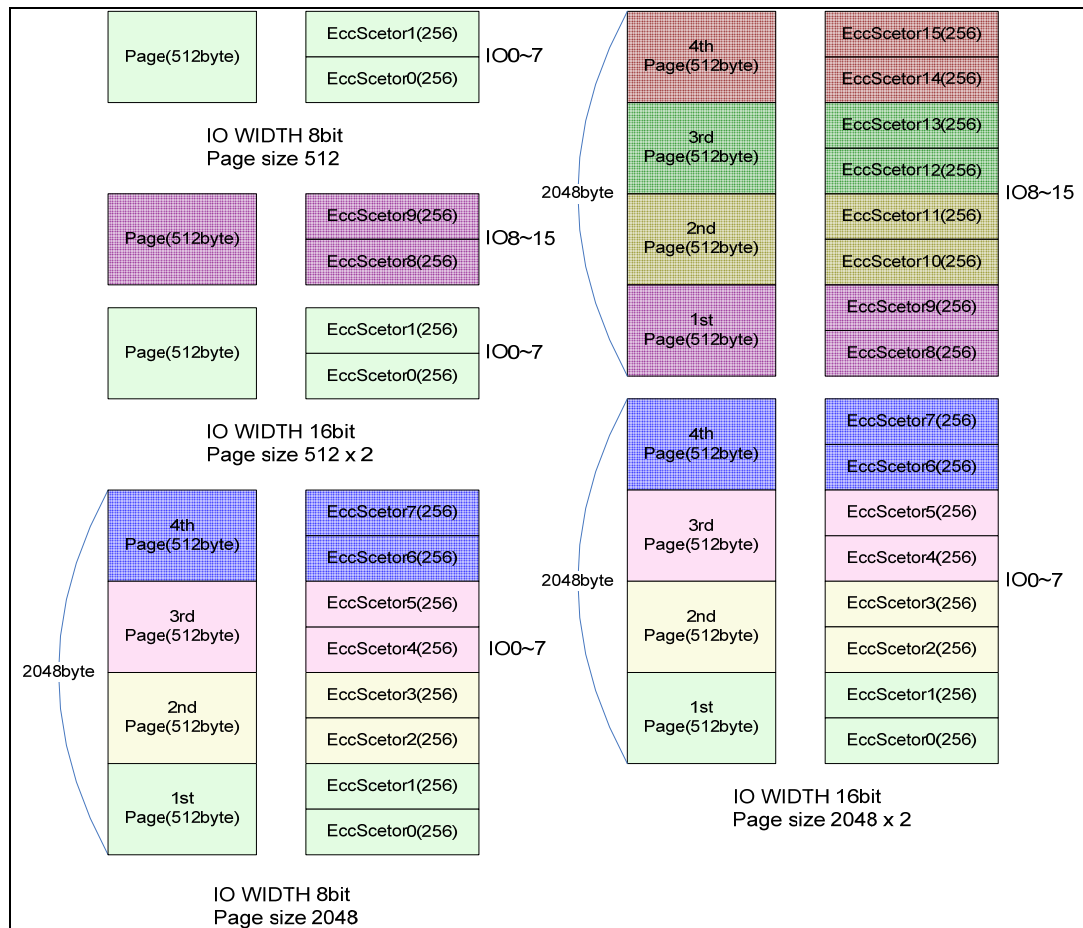
	D7	D6	D5	D4	D3	D2	D1	D0
ECC2	P4	P4'	P2	P2'	P1	P1'	1	1
ECC1	P1024	P1024'	P512	P512'	P256	P256'	P128	P128'
ECC0	P64	P64'	P32	P32'	P16	P16'	P8	P8'

Table 17-10 SECCSECTOR0~SECCSECTOR7 (0000 0054h~0000 00070h)

	D7	D6	D5	D4	D3	D2	D1	D0
SECC1	1	1	1	1	1	1	P4	P4'
SECC0	P2	P2'	P1	P1'	P16	P16'	P8	P8'

17.5.7.2. Main Ecc Sector Register(ECCSECTOR0~ECCSECTOR15)

EccSector Register 는 EccSector0 에서 EccSector15 까지 총 16 개로 구성된다
 각 interface 별 EccSector 의 구성은 그림 과 같다.

Figure 17-15 Interface별 Ecc Sector**Figure 17-16 ECCSECTOR0~ECCSECTOR15**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								ECC2 (sector15~sector0)								ECC1 (sector15~sector0)								ECC0 (sector15~sector0)							

17.5.7.3. Spare Ecc Sector Register (SECCSECTOR0~SECCSECTOR7)

Spare EccSector Register 는 SEccSector0 에서 SEccSector7 까지 총 8 개로 구성된다
SEcc 는 spare area 의 LSN0,LSN1,LSN2 의 값으로 생성된다. Spare Area 는 아래 그림과 같은 구조로 되어있다.

Figure 17-17 Spare area

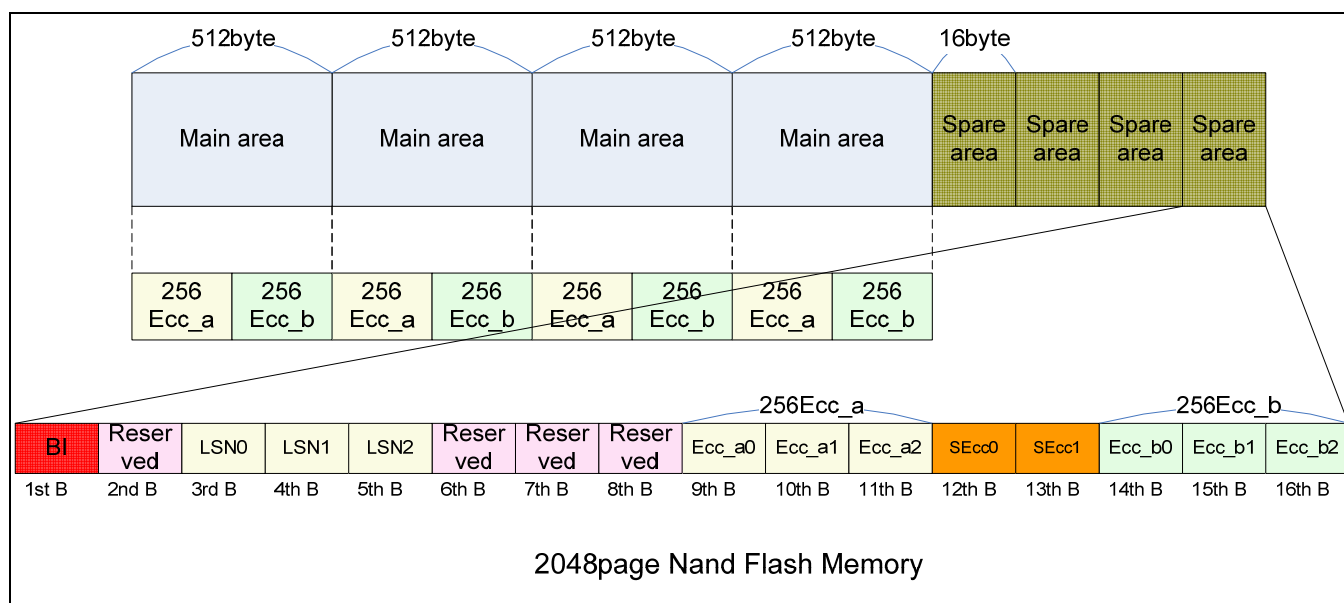
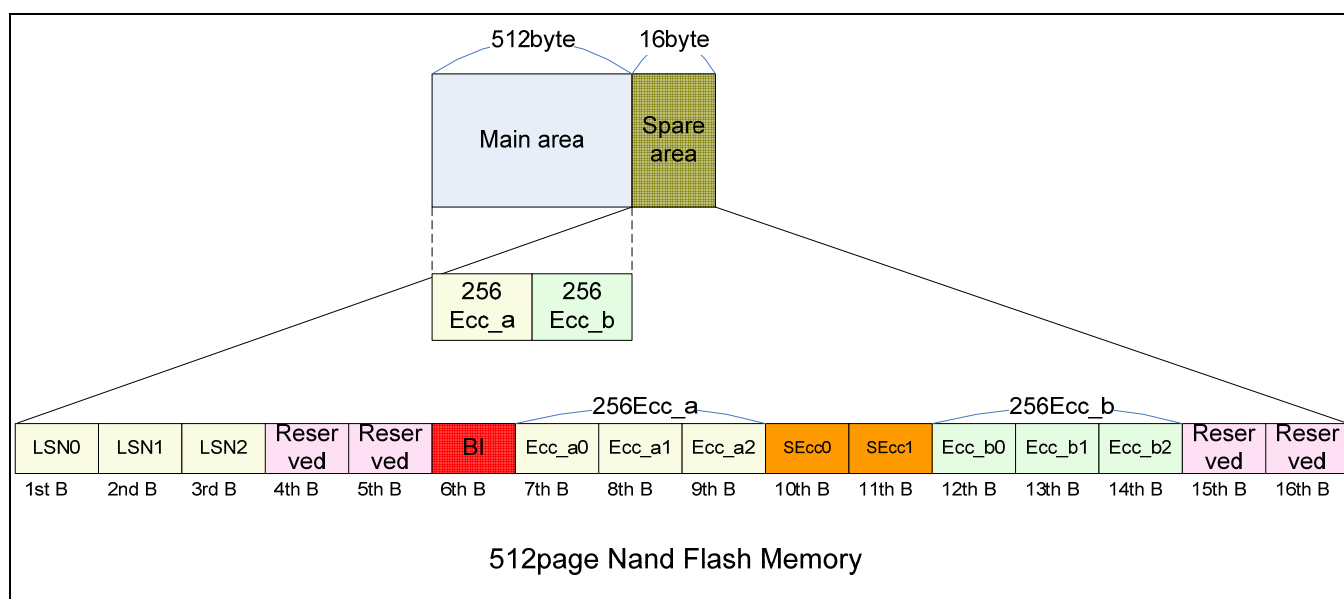


Figure 17-18 SECCSECTOR0~SECCSECTOR7

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								Reserved								SECC1 (sector15~sector0)								SECC0 (sector15~sector0)							

17.5.7.4. Ecc Error Register

ECC ERROR 를 hardware 에서 체크한다. 에러의 발생 유무만 판단하게 되며 정정 가능한 에러일 때 에러의 위치는 소프트웨어로 계산해야 한다. 단 페이지 전체를 read 해야만 유효한 값을 가진다.

Figure 17-19 MECC Error Register bit

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EccErrSect15	EccErrSect14	EccErrSect13	EccErrSect12	EccErrSect11	EccErrSect10	EccErrSect9	EccErrSect8	EccErrSect7	EccErrSect6	EccErrSect5	EccErrSect4	EccErrSect3	EccErrSect2	EccErrSect1	EccErrSect0																

Table 17-11 MECC Error Register Description(Offset : 0000 0074h)

Bits	Name	R / W	Description	Init. value
[31:30]	EccErrSect15	RO	00: no Error 01: correctable error 10: uncorrectable error 11: not used	00b
[29:28]	EccErrSect14	RO	00: no Error 01: correctable error 10: uncorrectable error 11: not used	00b
[27:26]	EccErrSect13	RO	00: no Error 01: correctable error 10: uncorrectable error 11: not used	00b
[25:24]	EccErrSect12	RO	00: no Error 01: correctable error 10: uncorrectable error 11: not used	00b
[23:22]	EccErrSect11	RO	00: no Error 01: correctable error	00b

			10: uncorrectable error 11: not used	
[21:20]	EccErrSect10	RO	00: no Error 01: correctable error 10: uncorrectable error 11: not used	00b
[19:18]	EccErrSect9	RO	00: no Error 01: correctable error 10: uncorrectable error 11: not used	00b
[17:16]	EccErrSect8	RO	00: no Error 01: correctable error 10: uncorrectable error 11: not used	00b
[15:14]	EccErrSect7	RO	00: no Error 01: correctable error 10: uncorrectable error	00b
[13:12]	EccErrSect6	RO	00: no Error 01: correctable error 10: uncorrectable error 11: not used	00b
[11:10]	EccErrSect5	RO	00: no Error 01: correctable error 10: uncorrectable error 11: not used	00b
[9:8]	EccErrSect4	RO	00: no Error 01: correctable error 10: uncorrectable error 11: not used	00b
[7:6]	EccErrSect3	RO	00: no Error 01: correctable error 10: uncorrectable error 11: not used	00b
[5:4]	EccErrSect2	RO	00: no Error 01: correctable error 10: uncorrectable error	00b

			11: not used	
[3:2]	EccErrSect1	RO	00: no Error 01: correctable error 10: uncorrectable error 11: not used	00b
[1:0]	EccErrSect0	RO	00: no Error 01: correctable error 10: uncorrectable error 11: not used	00b

Figure 17-20 SECCERR Register bit

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																SEccErrSect7	SEccErrSect6	SEccErrSect5	SEccErrSect4	SEccErrSect3	SEccErrSect2	SEccErrSect1	SEccErrSect0								

Table 17-12 SECCERR Register Description(Offset : 0000 0078h)

Bits	Name	R / W	Description	Init. value
[31:16]	Reserved		reserved	
[15:14]	SeccErrSect7	RO	00: no Error 01: correctable error 10: uncorrectable error 11: not used	00b
[13:12]	SeccErrSect6	RO	00: no Error 01: correctable error 10: uncorrectable error 11: not used	00b
[11:10]	SeccErrSect5	RO	00: no Error 01: correctable error 10: uncorrectable error 11: not used	00b
[9:8]	SeccErrSect4	RO	00: no Error 01: correctable error 10: uncorrectable error 11: not used	00b

[7:6]	SeccErrSect3	RO	00: no Error 01: correctable error 10: uncorrectable error 11: not used	00b
[5:4]	SeccErrSect2	RO	00: no Error 01: correctable error 10: uncorrectable error 11: not used	00b
[3:2]	SeccErrSect1	RO	00: no Error 01: correctable error 10: uncorrectable error 11: not used	00b
[1:0]	SeccErrSect0	RO	00: no Error 01: correctable error 10: uncorrectable error 11: not used	00b

17.6. Programming Guide

17.6.1. NFOPER 레지스터의 1st word

17.6.1.1. NFOER의 Option[30:29]

RnBWait : 현재 Command 를 수행하고 RnB 가 Low 에서 High 로 변한 후 Next Command 가 수행되기 원할때 사용된다. (Current Commnad -> RnB Low -> RnB High -> next Commnad)

AutoRdStat : option 으로 AutoRdStat 를 선택하게 되면 Ready/Busy 가 Low 에서 High 로 변한 뒤에 readStatus Commad 전송하여 read status 를 읽을 수 있다.

(page program ,Erase 시 사용)

Ex. Page program 시 address 5cycle, page program command (80h,10h), Read Status Command(70h)일 때

1st NFOPER = AutoRdStat Enable(data size,등등 셋팅)

2nd NFOPER = addr3 | addr 2 | addr1 | cmd(80h)

3rd NFOPER = cmd70h | cmd10h | addr5 | addr4

위와 같이 구성하면 Command(80h),Address(5cycle) -> Data -> Command(10h) -> RnB Low -> RnB High -> Command(70h) -> read status 순으로 진행된다.

Continue : 현재 Command 를 수행을 완료하고 Next Command 를 Load 하기 전까지 CE 이 Low 인 상태로 유지된다. 하나의 Command 가 수행이 완료되면 CE 은 High 가 되지만 이 option 을 써주게 되면 CE 이 Low 로 유지된다.

17.6.1.2. NFOPER의 OpMode

read data : Nand Flash 의 Data 를 read 할 때 사용된다.(Page read 시) Command 를 전송하지 않고 data 만 read 하고 싶다면 CmdAddrTransByte 를 0 으로 셋팅하고 OpMode 는 read data 로 설정하고 DataSize 를 정해주면 지정된 사이즈만큼 RE 신호가 전송된다.

read status : Nand Flash 의 status 를 read 할 때 사용됨

read ID : Nand Flash 의 ID 를 read 할 때 사용됨

write data : Nand Flash 에 Data 를 Write 할 때 사용된다.(page program)

Command 를 전송하지 않고 data 만 write 하고 싶다면 CmdAddrTransByte 를 0 으로 셋팅하고 OpMode 는 write data 로 설정하고 DataSize 를 정해주면 지정된 사이즈만큼 WE 신호가 전송된다.

NOP : Nand Flash 로 Data 의 이동 없이 Command,Address 만 전송 할 때 사용됨

17.6.2. Command 전송

Command 전송을 위한 레지스터 셋팅은 다음과 같다.

1st Operation register 에 Option, DataSize, OpMode, CmdAddrTransbyte, CmdAddrFlag 를 셋팅 해준다

2nd, 3rd Operation register 에 전송하고자 하는 Command 나 Address 를 써주면 원하는 Command 가 전송된다. 아래 Page program Command 전송의 예를 보았다

Ex. Page program

Nand Flash page size : 2048 byte

address cycle : 5 cycle

write 할 사이즈 : 2048 byte

전송 단위 : word

Page write command : 80h,10h

WriteSize = $2048 < 17$

WRITEDATA = $0x3 < 14$

CmdAddrTransByte = $0x7 < 8$ (7byte 전송=command 2byte,address 5byte)

CmdAddrFlag = $0x41$

addr1 = 00h ,addr2 = 00h, addr3 = 00h, addr4 = 00h, addr5 = 00h

1st NFOPER = WriteSize|WRITEDATA|CmdAddrTransByte|CmdAddrFlag

2nd NFOPER = (addr3<<24) | (addr 2 <<16) | (addr1<<8) | (cmd(80h)

3rd NFOPER = (cmd(10h) <<16) | (addr5<<8) | (addr4)

위와 같이 NFOPER 을 써주게되면 Command Queue 에 순서대로 쓰여지고 Controller 에서 State 에 따라 Command Queue 를 read 하여 Nand Flash 로 Command/Address 를 전송하게 된다.

17.6.3. DATA read / write

17.6.3.1. Data write

위에서 살펴본 것처럼 command 전송을 후 NFSTAT 의 WrFIFOReady flag 를 참조하여 NFDATA 레지스터에 전송 사이즈 만큼 data(위의 예에서는 2048byte)를 써주게 되면 Data write 가 완료된다.(DATA FIFO Level 을 8 로 셋팅 했다면 WrFIFOReady Flag 가 High 일 때 8 번 wirte 한다.)

주의 WrFIFOReady Flag 는 반드시 clear 해줘야 한다.

17.6.3.2. Data read

Page read 의 경우 역시 command 전송 방법은 page Program 과 동일하며 NFSTAT 의 dFIFOReady flag 를 참조하여 NFDATA 레지스터를 read 하면 Nand Flash 의 data 를 읽어오게 된다.(DATA FIFO Level 을 8 로 셋팅했다면 RdFIFOReady Flag 가 High 가 됐을 때 8 번 read 한다.)

주의 RdFIFOReady Flag 는 반드시 clear 해줘야 한다.

17.6.4. DMA를 이용한 Data read / write

DMA 로 Data 를 전송하기 위해서는 NFCTRL register 의 DMAEn 을 1 로 셋팅하고 Read/Write Command 를 전송하면 Read 시에는 DATAFIFO 가 설정된 FIFO size 만큼 차게 되면 DMA request 가 발생하여 DMA Controller 에 설정된 값에 의해 data read 가 수행되고 Write 시에는 DATAFIFO 가 empty 일 때 DMA request 가 발생하여 DMA Controller 에 설정된 값에 의해 data write 가 수행된다.

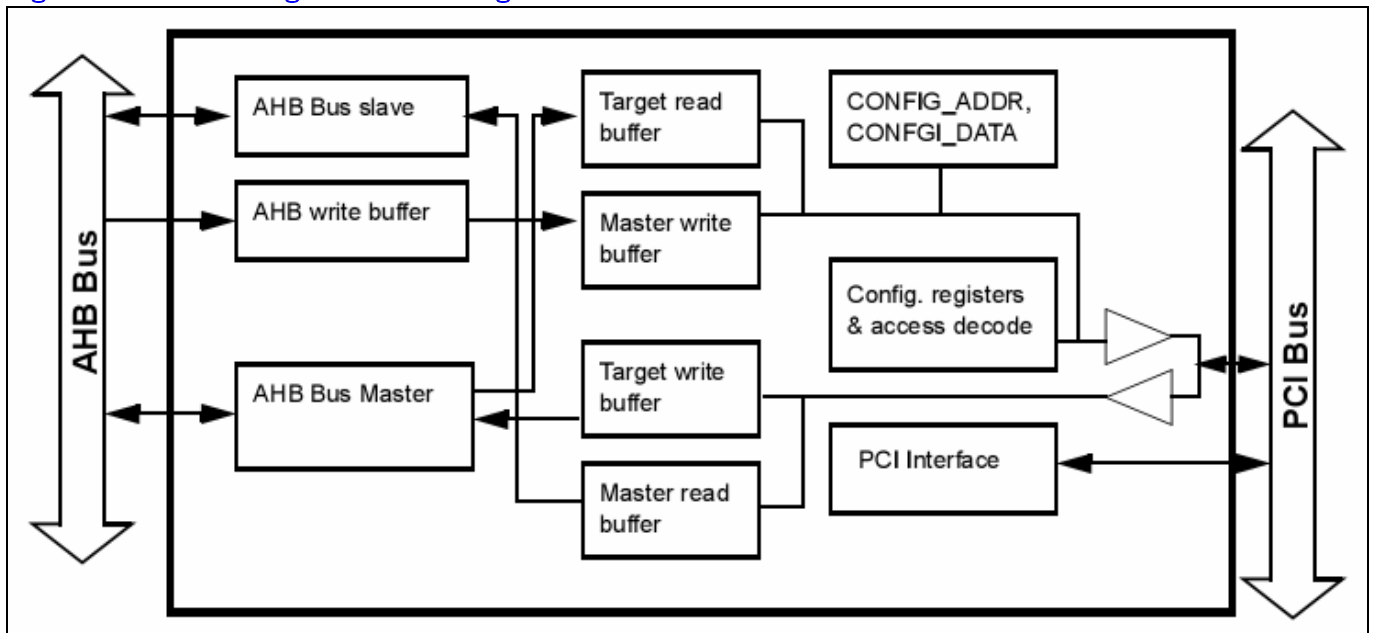
DMA 를 이용한 데이터 Read/Write 역시 위에서 살펴본 바와 같이 Command 전송방법은 동일하며 단지 Data 를 read/write 하는 것만 DMA 에서 해주게 된다.

18. PCI Bridge

18.1. Features

- PCI bus bridge with PCI bus arbiter
- Supports PCI specification 2.2 to 3.0
- Downstream and upstream access allows
- Total six write buffers.
- Supports burst transfers 4, 8, 16 data words
- Generates standard PCI type 0 and type 1 configuration accesses
- Supports 4 interrupts
- PCI memory and PCI IO range are 128Mbytes each.
- Supports 4 PCI cards

Figure 18-1 Block diagram of PCI Bridge



18.2. PCI Address Space

PCI 에 할당된 address range 는 0x80000000~0x9FFFFFFF 이다.
PCI 에서는 이 영역을 아래 Table 과 같이 분할하여 사용한다.

Table 18-1 PCI Address Space

Area	Start Address	End Address
PCI Memory Area	0x80000000	0x87FFFFFF
PCI IO Area	0x88000000	0x8FFFFFFF
Configuration ADDR	0x90000000	0x97FFFFFF
Configuration DATA	0x98000000	0x9FFFFFFF

PCI Memory Area 는 memory 영역을 사용하는 PCI device 가 할당될 영역이다.
PCI IO Area 는 IO 영역을 사용하는 PCI device 가 할당될 영역이다.

Configuration ADDR 와 configuration DATA 는 PCI device 를 설정하기 위한 각 device 의 configuration register 를 access 하기 위한 영역이다. 이 영역은 address 의 하위 bit 가 모두 무시된다. 즉 128Mbyte 영역 모두가 하나의 32bit 영역으로 overlap 된다.

18.3. Supported Data Transfer Types

18.3.1. Single Transfer Type

Single transfer 는 3 가지 data size 를 지원한다.

- Byte
- Half word
- Word

18.3.2. Burst Transfer Type

Burst 는 항상 32bit size 의 data 만 사용 가능하다. 8bit burst 나 16bit burst 는 사용할 수 없다. Burst type 은 incrementing burst 만 지원한다. 사용 가능한 burst length 는 다음 Table 과 같다.

Table 18-2 Burst Transfer Length

Burst Length	Description
SINGLE	Single transfer
INCR4	4 beat incrementing burst
INCR8	8 beat incrementing burst
INCR16	16 beat incrementing burst

18.3.3. Retry Operation

PCI bridge 는 operation 이 오래 걸릴 것으로 예상되는 경우 retry operation 을 발생시킨다. Retry operation 은 AXI to AHB bridge 에서 처리한다.

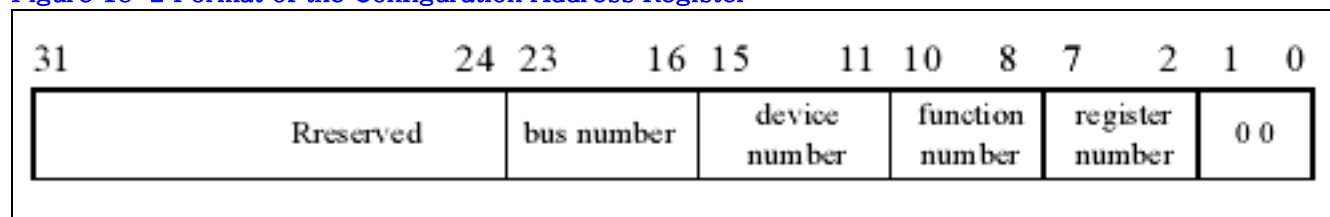
18.4. PCI Bridge Configuration

PCI Bridge Configuration 은 다른 PCI device 와 마찬가지로 configuration register 를 access 하여 이루어진다. Configuration register 를 access 하기 위해서는 Configuration Address 와 Configuration Data 를 이용해야 한다.

18.4.1. Configuration Address

PCI 의 Address Area 중 Configuration ADDR Area 에 write 하여 setting 한다. Configuration ADDR Area 는 모두 하나의 Configuration ADDR register 에 mapping 되어 있어, cpu 의 address[26:0]는 무시된다. Register format 은 다음 그림과 같다.

Figure 18-2 Format of the Configuration Address Register



Configuration Address 는 bus number 에 의해서 다음 그림과 같이 두 가지 type 중 하나로 변환된다.

- Bus number = 0 (local bus) : Type 0
- Other Bus number : Type 1

Figure 18-3 Type 0 translation

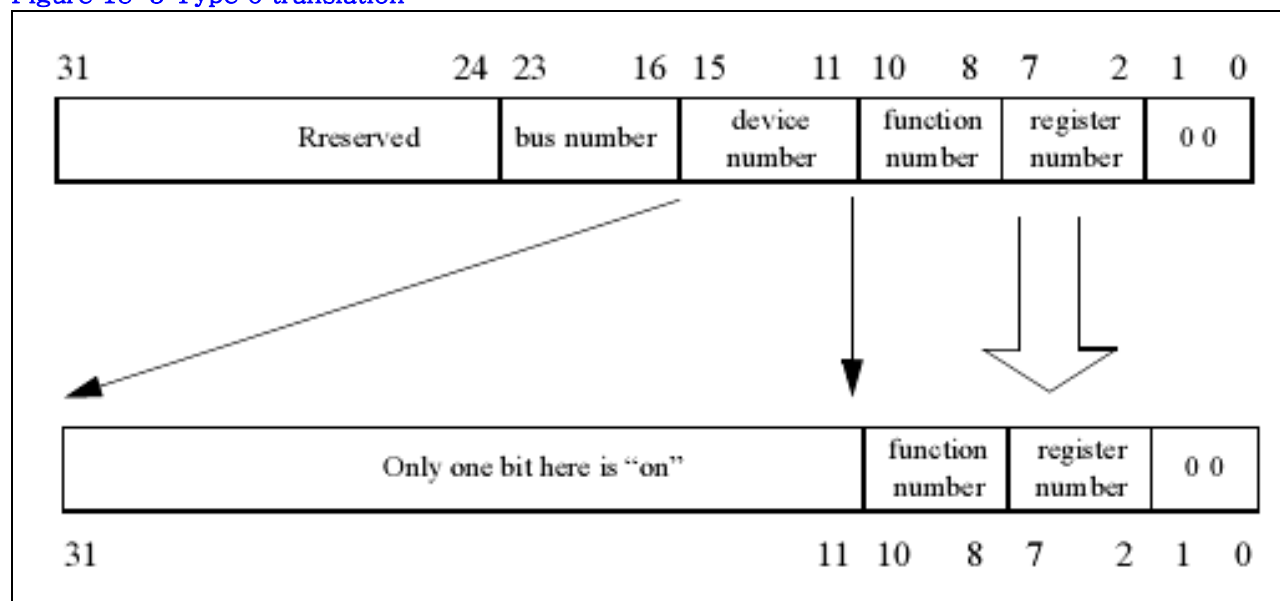
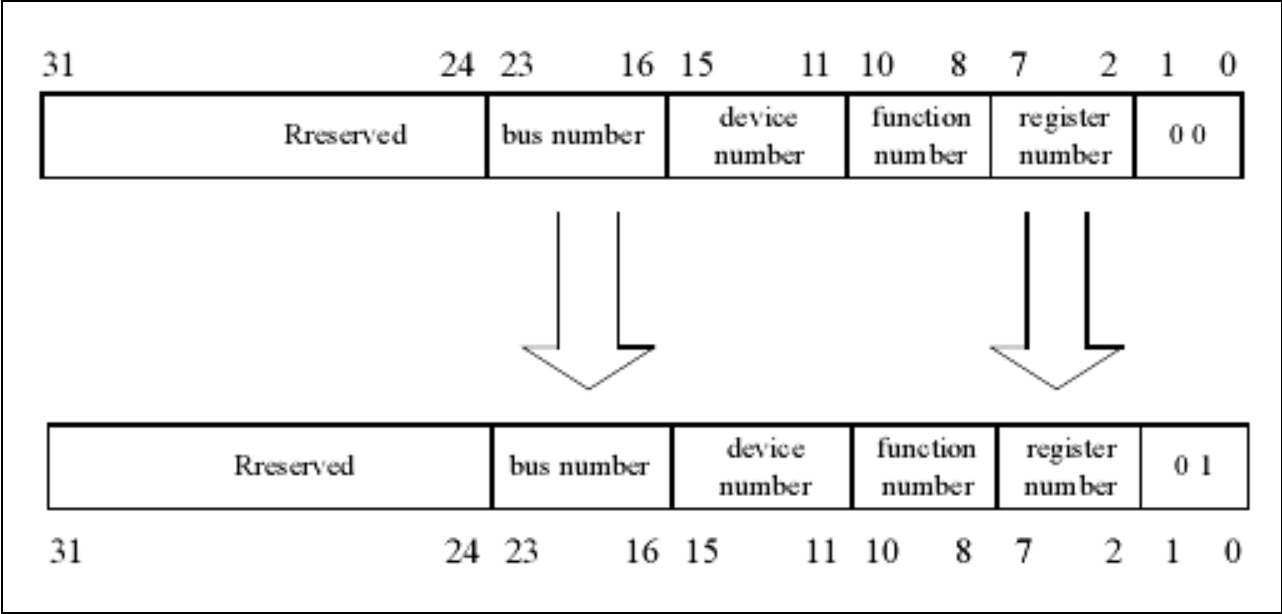


Figure 18-4 Type 1 translation



18.4.2. Configuration Data

Configuration register 에 write 할 data 나 read 한 data.

18.4.3. PCI Bridge Configuration Registers

PCI Configuration Register 는 다음 그림과 같다.(ep454b datasheet 참조)

Figure 18-5 PCI Bridge Configuration Register

Device ID		Vendor ID		00h
Status		Command		04h
Class Code			Revision ID	08h
BIST	Header	Lat. timer	Cache size	0Ch
Base Address Registers 0				10h
Base Address Registers 1				14h
Base Address Registers 2				18h
Base Address Registers 3				1Ch
Base Address Registers 4				20h
Base Address Registers 5				24h
Cardbus CIS pointer				28h
Subsystem ID		Subsystem Vendor ID		2Ch
Expansion ROM				30h
Reserved				34h
Reserved				38h
Max_Lat	Min_Gnt	Int. pin	Int. line	3Ch
Retry over-flow flag	Counter enable	Retry counter		40h

18.5. PCI Interrupt Number

Interrupt 는 총 4 개이며, 다음 Table 과 같이 할당되어 있다.
 Interrupt Number 는 VIC 에서의 interrupt 번호입니다.

Table 18-3 Interrupt mapping

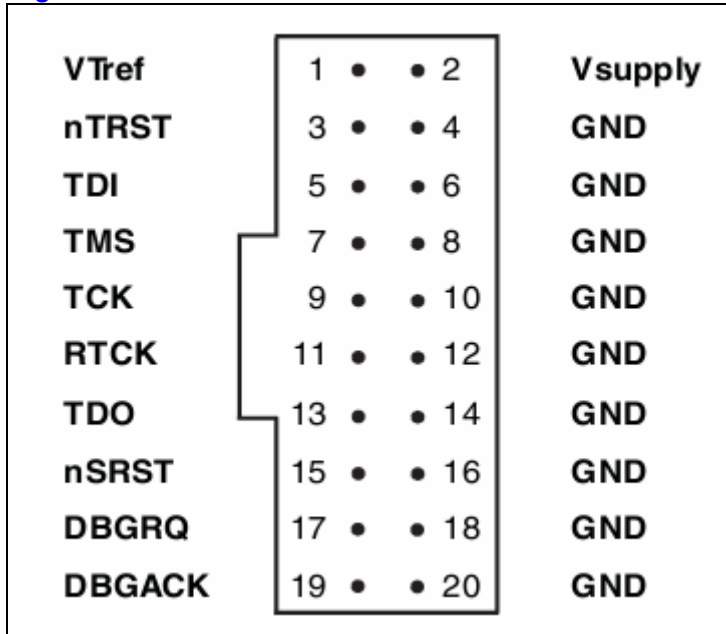
PCI Interrupt	Interrupt Number
PCI INT A	28
PCI INT B	29
PCI INT C	30
PCI INT D	31

각 interrupt 는 각 PCI slot 마다 다른 pin 에 연결되어 있다.

19. MultiICE

MultiICE pin connections

Figure 19-1 ARM Multi-ICE Connector



19.1. Signals

Table 19-1 Signals of the ARM Multi-ICE

Signal Name	I / O	Description
VTref	I	Target reference voltage. RTCK 와 TDO 의 logic-level 의 기준 전압을 알려준다.
Vsupply	I	MultiICE 의 target 보드쪽에서의 공급 전원.
nTRST	O	JTAG reset
TDI	O	JTAG data output to target cpu
TMS	O	JTAG test mode signal
TCK	O	JTAG JTAG test clock
RTCK	I	Return test clock
TDO	I	JTAG data input from target cpu
nSRST	I/O	All system is reset. 이 신호는 option 으로 사용비중은 높으나 필수 신호는 아니다.
DBGRQ	O	Debug request. Not used in the MultiICE.
DBGACK	I	Debug acknowledge. Not used in the MultiICE.

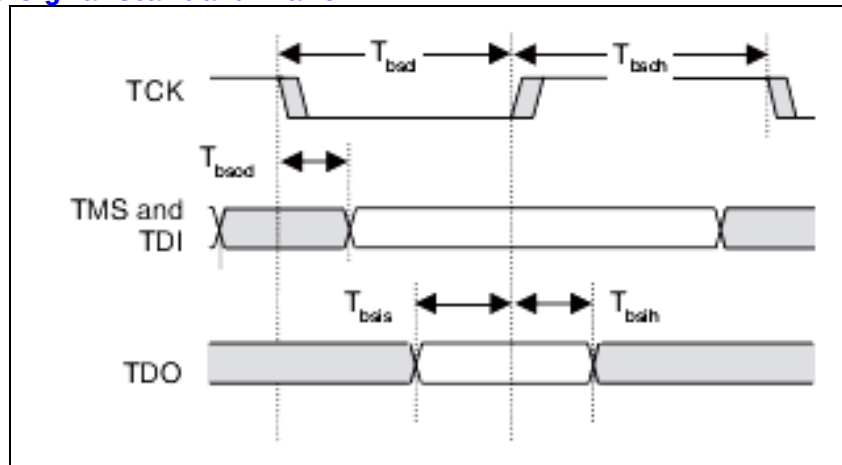
19.2. JTAG

19.2.1. JTAG signals

- TRST : Jtag reset. Negative active
- TCK : Jtag clock. Rising edge에서 TDI와 TDO가 유효하다.
- TDI : data input
- TDO : data output

Figure 18-1 은 JTAG 의 wave form 을 보여준다. TDI 나 TDO 는 TCK 의 rising edge 에서 유효하다.

Figure 19-2 JTAG signal standard wave



19.2.2. JTAG State Machine

Figure 19-3 JTAG State Machine

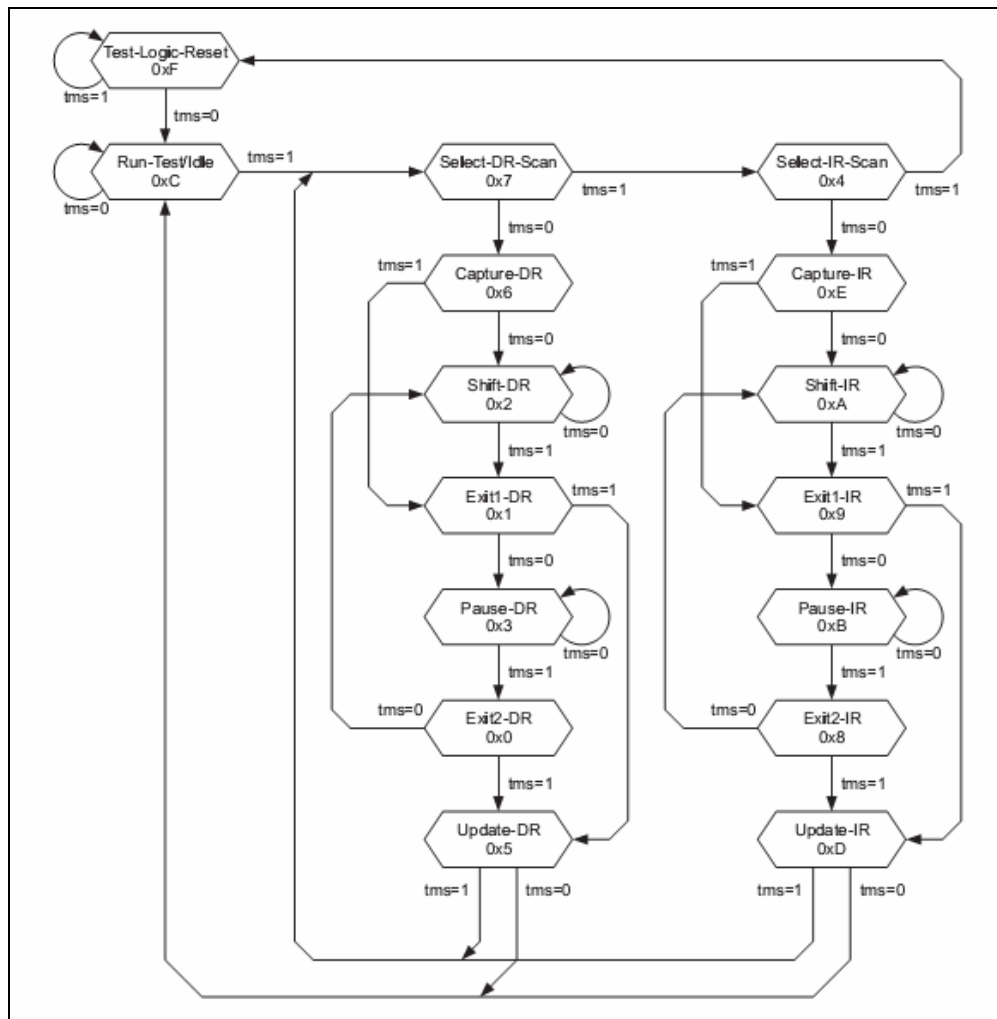


Figure 18-2 는 tap controller 의 state diagram 이다.

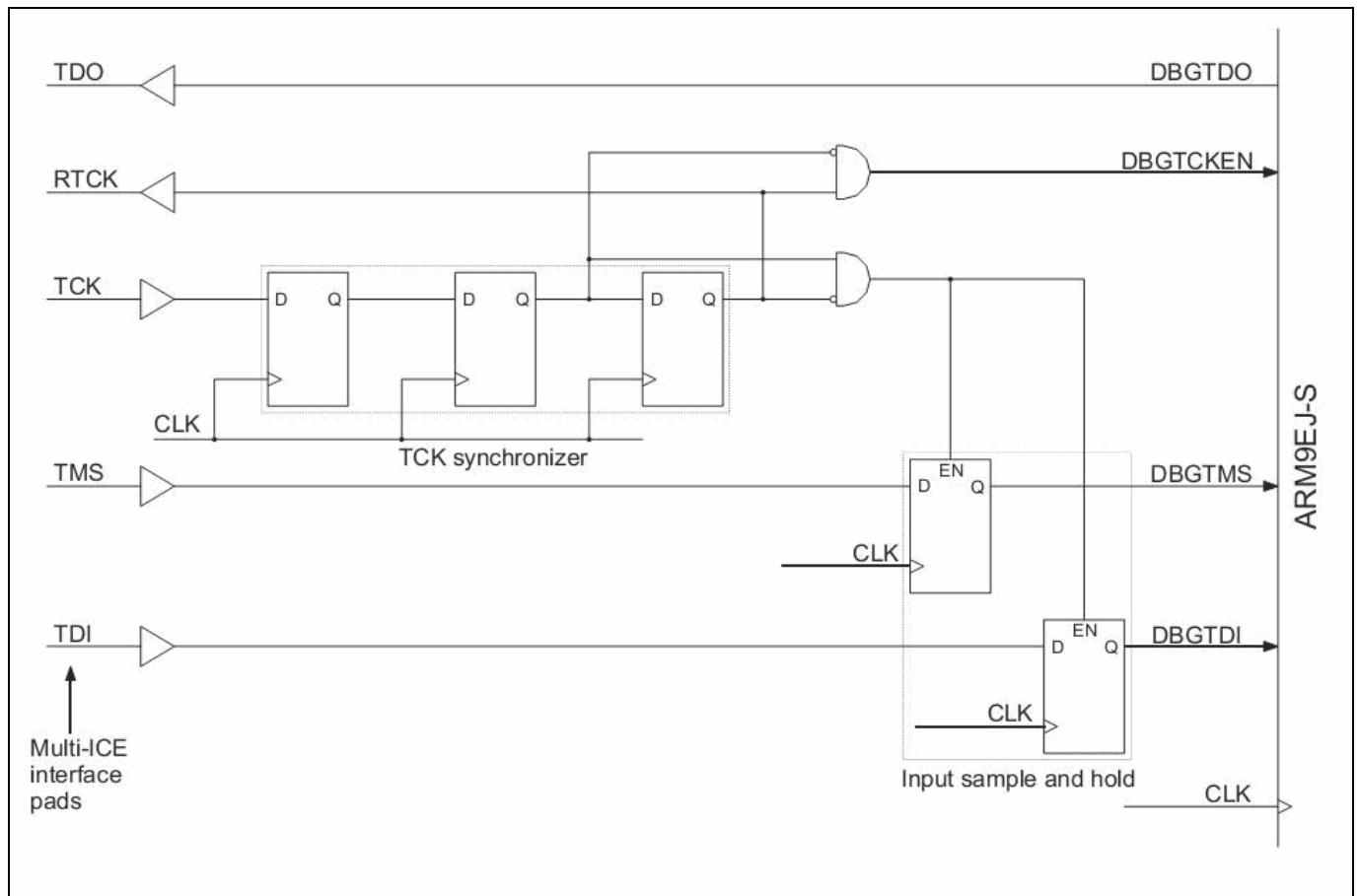
State 는 TMS 의 값에 따라 천이하게 된다.

TDI 나 TDO 의 마지막 bit 에서 다음 state 로 천이하기 위한 TMS 값을 넣어야 한다. 그렇지 않으면 TDI 나 TDO 에 원하지 않는 bit 가 하나 추가된다.

어느 state 에서나 TMS 를 1 로하고 TCK 를 5 번 주면 Test-Logic-Reset 상태에 도착하게 된다.

19.3. Using JTAG

Figure 19-4 JTAG interface 회로



Jtag_sync.v 에서 Figure 18-3 과 같은 회로를 추가하여 JTAG interface 를 구성해야 한다.

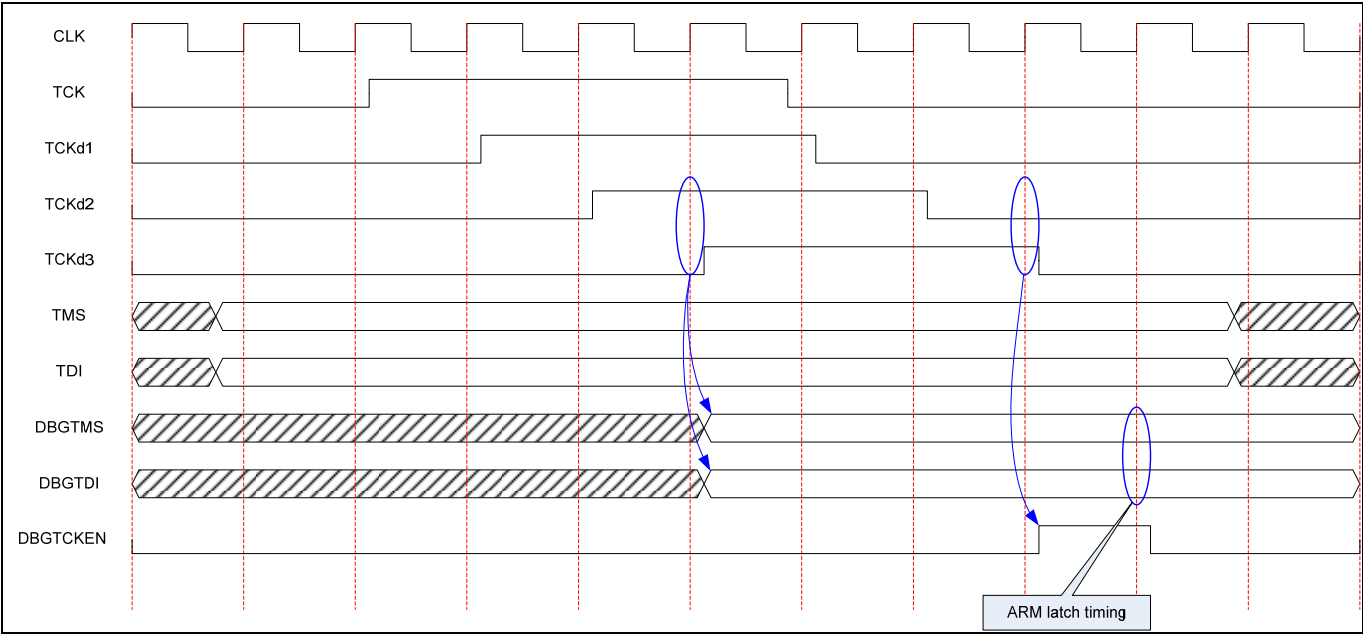
Figure 1-1 에서 보는 바와 같이 TMS 와 TDI 를 F.F.으로 TCK 의 rising edge 직후 latch 한 뒤, ARM 내부에서는 TCK 의 falling edge 에서 사용한다.

따라서 TCK 의 high 구간 만큼 delay 되어 사용되는 현상이 발생한다. 그러므로 TCK 는 TDI 의 invalid data 구간에서는 low 를 유지하다가 TDI 에 data 를 실으면 TCK 를 high 로 올렸다가 바로 low 로 내리는 것이 좋다.

Figure 18-4 은 Figure 18-3 의 waveform 을 보여준다.

TDI 나 TMS 의 sample timing 과 ARM 내부에서 sampling 한 값을 사용하는 timing 과는 4 system clock 만큼 차이가 난다.

Figure 19-5 ARM JTAG Waveform



20. ARM JTAG

20.1. Instruction Register

JTAG 의 동작을 결정한다. 동작은 Table 26-1 에 기술되어 있는 것 중 하나를 선택할 수 있다.

Table 20-1 JTAG Instructions

Instruction	Binary code	Hex code
EXTEST	0000b	0h
SAMPLE/PRELOAD	0011b	3h
SCAN_N	0010b	2h
INTEST	1100b	Ch
IDCODE	1110b	Eh
BYPASS	1111b	Fh
RESTART	0100b	4h

20.1.1. EXTEST

Chip boundary scan chain 을 이용하여 chip 외부로 직접 control 할 수 있는 모드.
DBGSDIN 으로 JTAG 에서 받은 serial data 를 pad 로 출력하고, DBGSDOUT 으로 마지막 pad 의 serial data 를 받는다.

20.1.2. SAMPLE / PRELOAD

INTEST 나 EXTEST 시 JTAG 의 state 천이에 의해서 sampling 이나 update 가 불가능하기 때문에 별도의 JTAG instruction 을 둔다.

20.1.3. SCAN_N

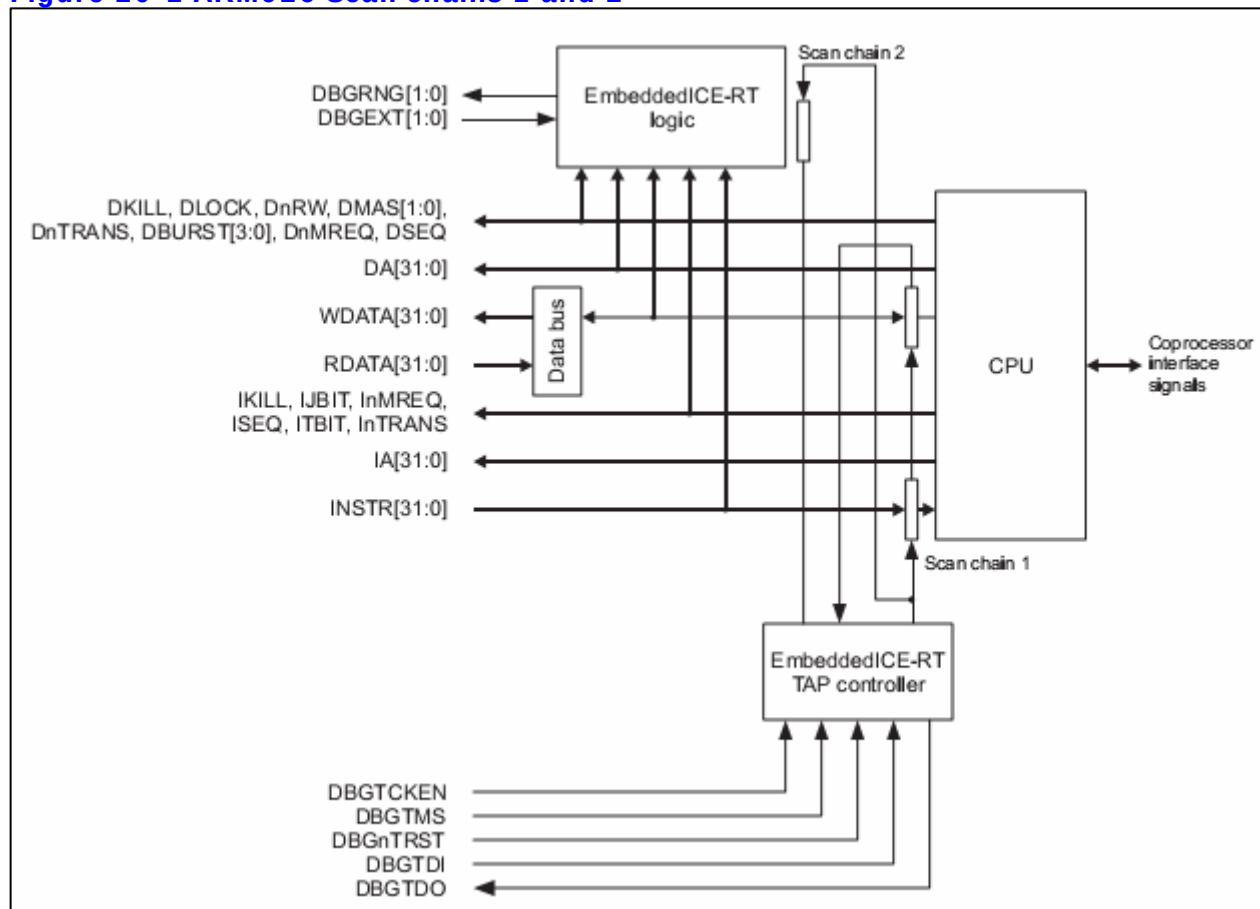
Scan chain 을 선택.
5bit register

20.2. Scan Chains

Table 20-2 Scan chains

Scan chain number	Function
0	Reserved
1	Debug
2	EmbeddedICE-RT programming
3	External boundary scan
4-15	Reserved
15-31	Unassigned

20.2.1. Scan chain 1

Figure 20-1 ARM926 Scan chains 1 and 2

ARM9 TRM page 31 참조.

Table 20-3 Bit description of the ARM926 Scan chain 1

Bit number	Function	R / W
[66:35]	RDATA/WDATA[0:31]	RW
[34]	Unused	
[33]	WPTANDBKPT	W
[32]	SYSSPEED	W
[31:0]	Instruction[31:0]	W

20.2.2. Scan chain 2

Table 20-4 Bit description of the ARM926 Scan chain 2

Bit number	Function	R / W
[37]	Read/Write control	W
[36:32]	Address[4:0]	W
[31:0]	Data[31:0]	RW

20.2.3. Scan chain 6

Table 20-5 Bit description of the ARM926 Scan chain 6

Bit number	Function	R / W
[39]	Read/Write control	W
[38:32]	Address[6:0]	W
[31:0]	Data[31:0]	RW

20.2.4. Scan chain 15

Access CP15 registers

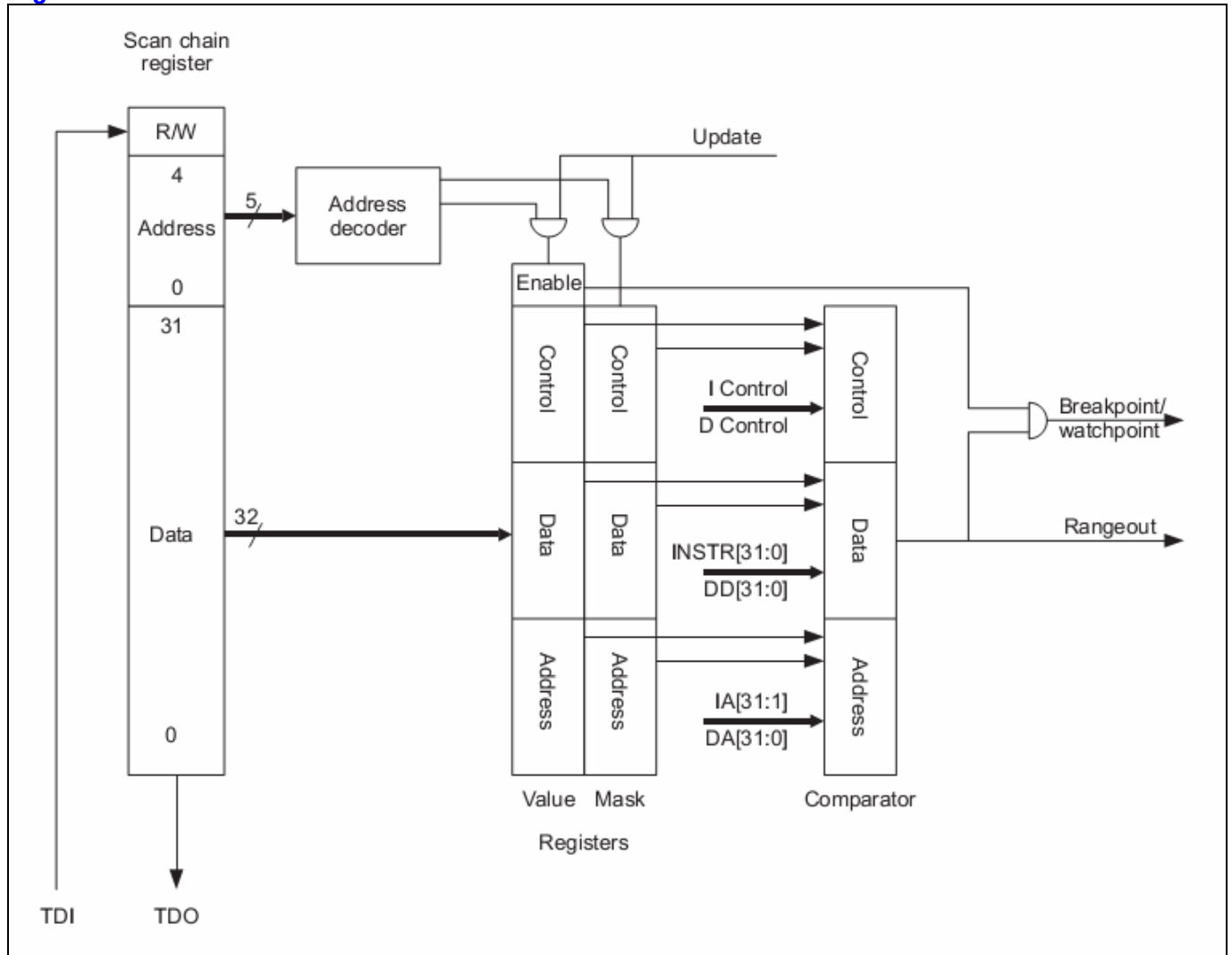
Bits	Function
[47]	1 = Write 0 = Read
[46:44]	Opcode_1 of MRC/MCR instruction.
[43:41]	Opcode_2 of MRC/MCR instruction.
[40:37]	CRn of MRC/MCR instruction.
[36:33]	CRm of MRC/MCR instruction.
[32]	When written : 1 = initiate new access 0 = NOP When read : 1 = access complete 0 = access incomplete
[31:0]	Data value

21. EmbeddedICE-RT

- Two Breakpoint or Watchpoint
- 각 watchpoint는 mask register로 조건을 선택할 수 있다.
- Instruction memory interface나 data memory interface를 감시

21.1. Overview

Figure 21-1 EmbeddedICE-RT macrocell overview



Scan chain 은 총 38bit 이며 다음과 같이 구성되어 있다.

- R/W : read or write 선택
- Address : target register를 선택
- Data : target register의 data

21.2. Registers

Table 21-1 EmbeddedICE-RT Registers

Address	Width	Function	Type
00000b	6	Debug control	Read/Write
00001b	5	Debug status	Read-only
00010b	8	Vector catch control	Read/Write
00100b	6	Debug communications control	Read-only
00101b	32	Debug communications data	Read/Write
01000b	32	Watchpoint 0 address value	Read/Write
01001b	32	Watchpoint 0 address mask	Read/Write
01010b	32	Watchpoint 0 data value	Read/Write
01011b	32	Watchpoint 0 data mask	Read/Write
01100b	9	Watchpoint 0 control value	Read/Write
01101b	8	Watchpoint 0 control mask	Read/Write
10000b	32	Watchpoint 1 address value	Read/Write
10001b	32	Watchpoint 1 address mask	Read/Write
10010b	32	Watchpoint 1 data value	Read/Write
10011b	32	Watchpoint 1 data mask	Read/Write
10100b	9	Watchpoint 1 control value	Read/Write
10101b	8	Watchpoint 1 control mask	Read/Write

21.2.1. Debug control register

Figure 21-2 Debug control register format

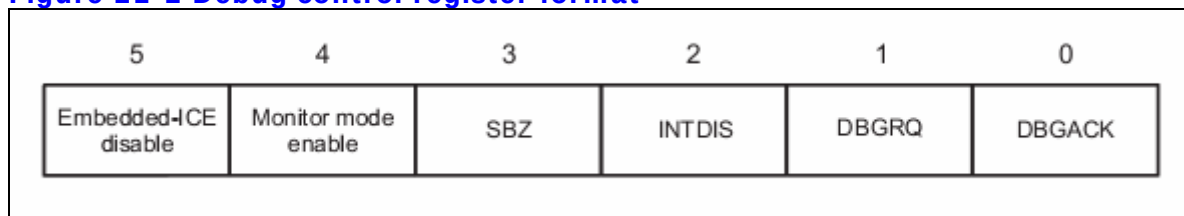


Table 21-2 Debug control register bit functions

Bit number	Name	Function
[5]	EmbeddedICE disable	if 1, enable address and data comparison logic. If 0, disabled.
[4]	Monitor mode enable	if 1, monitor mode debug, else Halt mode debug.
[3]	Reserved	Must be zero
[2]	INTDIS	Interrupt disable control signal
[1:0]	DBGRQ, DBGACK	CPU 의 DBGRQ 와 DBGACK 를 강제로 enable 시킨다.

Interrupt 는 INTDIS 이나 DBGACK 가 active 되면 disable 된다.

21.2.2. Debug status Register

Figure 21-3 Debus status register format

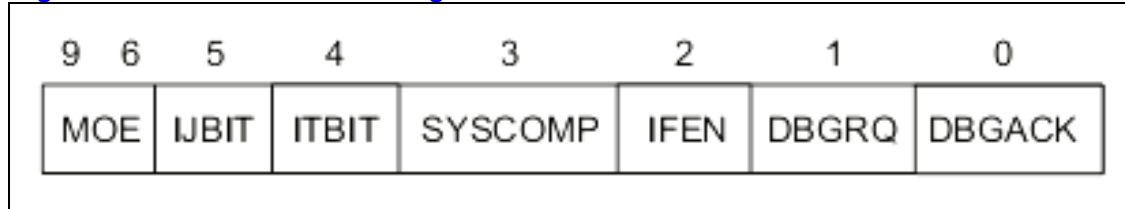


Table 21-3 Bit description of the Debus status register

Bit number	Name	Function
[9:6]	MOE	Method of Entry information. Debug mode 로 전환된 원인의 종류를 알려준다.
[5]	IJBIT	IJBIT status of processor status
[4]	ITBIT	ITBIT status of processor status
[3]	SYSCOMP	Memory access 가 완료됨을 인식.
[2]	IFEN	Interrupt enable status
[1:0]	DBGQRQ, DBGACK	EDBGRQ, DBGACK status

Table 21-4 Means of the MOE

MOE[3:0]	Meaning
0000b	No debug entry(since last restart or debug reset)
0001b	Breakpoint from EICE unit 0
0010b	Breakpoint from EICE unit 1
0011b	Soft breakpoint(BKPT instruction)
0100b	Vector catch breakpoint
0101b	External breakpoint
0110b	Watchpoint from EICE unit 0
0111b	Watchpoint from EICE unit 1
1000b	External watchpoint
1001b	Internal debug request
1010b	External debug request
1011b	Debug re-entry from system speed access

Figure 21-4 Debug control and debug status register structure

